

Physique dans le jeu vidéo

Table des matières

Session 1 : Géométrie et Calcul Vectoriel	4
Session 2 : Cinématique 2D et Mouvement de Projectile	11
Position en 2D	12
Vitesse en 2D	13
Accélération en 2D	15
Session 3 : Les lois de Newton – La méthode physique	18
Objectifs de la session	18
La démarche physique : modéliser l'univers	18
Session 4 : Les forces de la nature	20
Objectifs de la session	20
Session 5 : Intégration numérique 101 - Méthode d'Euler	22
Objectifs de la session	22
Session 6 : L'impulsion, les collisions	30
Objectifs de la session	30
Session 7 : Broad et Narrow phases – Part 1	34
Objectifs de la session	34
Session 8 : Intégration de Verlet et Position Based Dynamics	43
Objectifs de la session	43
Session 9 : TP – Moteur de Particules (VFX)	50
Objectifs du TP	50
Session 10 : Introduction à la Physique des Rotations	61
Objectifs de la session	61
Partie 1 : Du Point au Corps Rigide	61
Partie 2 : Le Dictionnaire Linéaire → Angulaire	61
Partie 3 : Le Couple (Torque)	62
Partie 4 : Le Moment d'Inertie	64
Partie 5 : Vitesse d'un Point sur le Corps	66
Partie 6 : Friction – Glissement vs Roulement	67
Partie 7 : Intégration de la Rotation	70
Partie 8 : La Démo – Billard (billiard.html)	71
Partie 9 : TP – Corps Rigide 2D	74
Partie 10 : Récapitulatif – Le Corps Rigide Complet	77
Quiz Mi-Parcours – Sessions 1 à 10	78
Informations	78
Partie A – Géométrie et Vecteurs (Session 1)	79
Partie B – Cinématique 2D (Session 2)	81
Partie C – Lois de Newton (Session 3)	82
Partie D – Forces de la Nature (Session 4)	83
Partie E – Intégration d'Euler (Session 5)	84
Partie F – Impulsion et Collisions (Session 6)	86
Partie G – Détection de Collision : Broad et Narrow Phase (Session 7)	88
Partie H – Verlet et PBD (Session 8)	90

Partie I — Moteur de Particules (Session 9)	92
Partie J — Rotations et Corps Rigides (Session 10)	94
Partie K — Synthèse	97
Session 11 : Narrow Phase — SAT, GJK et Génération de Contact	99
Objectifs de la session	99
Partie 1 : Le Pipeline — Où se situe la Narrow Phase	99
Partie 2 : Hiérarchie des Tests Narrow Phase	100
Partie 3 : SAT — Separating Axis Theorem	103
Partie 4 : GJK — Gilbert-Johnson-Keerthi	108
Partie 5 : EPA — Expanding Polytope Algorithm	111
Partie 6 : Génération du Contact	112
Partie 7 : SAT vs GJK — Quand utiliser quoi	114
Partie 8 : Le Pipeline Complet	115
Partie 9 : Code — SAT en JavaScript (2D)	115
Partie 10 : Code — GJK en JavaScript (2D)	116
Synthèse	117
Session 12 : TP — Système Solaire & Intégration RK4	118
Objectifs du TP	118
Partie 1 : La Gravitation Universelle	118
Partie 2 : La Vitesse Orbitale	119
Partie 3 : Pourquoi Euler ne Suffit Pas	120
Partie 4 : Runge-Kutta 4 (RK4)	121
Partie 5 : Le Sub-Stepping	123
Partie 6 : Le Système Solaire — Données	124
Partie 7 : TP — Missions	124
Partie 8 : Bonus	126
Synthèse	127
Session 13 : Les moteurs physiques commerciaux	128
Objectifs de la session	128
Partie 1 : Pourquoi utiliser un moteur physique ?	128
Partie 2 : Concepts partagés par tous les moteurs	129
Partie 3 : Le Paysage des Moteurs	132
Partie 4 : Déterminisme	134
Partie 5 : Architecture ECS	134
Partie 6 : Choisir son moteur — le guide pratique	136
Synthèse	136
Session 14 : Pratique des moteurs — Rigid Bodies & Colliders	137
Objectifs de la session	137
Partie 1 : Deux philosophies, une même scène	137
Partie 2 : Three.js + Rapier — Fondamentaux & Setup	137
Partie 3 : Three.js + Rapier — Rigid Bodies & Colliders pas à pas	139
Partie 4 : Godot + Jolt — Fondamentaux & Setup	143
Partie 5 : Godot + Jolt — Rigid Bodies & Colliders pas à pas	144
Partie 6 : Tableau comparatif des API	149
Partie 7 : Les Colliders en détail	151
Synthèse	152

Session 15 : Joints, Moteurs & Raycasts	154
Objectifs de la session	154
Partie 1 : Qu'est-ce qu'un joint ?	154
Partie 2 : Les quatre types de joints	155
Partie 3 : Les moteurs — animer un joint	157
Partie 4 : Les limites — borner la course	158
Partie 5 : Les raycasts — interroger le monde	158
Partie 6 : Exploration — Le Joint Zoo (Rapier + Three.js)	159
Partie 7 : Godot + Jolt — Joints & Moteurs	163
Partie 8 : Tableau comparatif des joints	165
Partie 9 : Raycasts en Godot	165
Synthèse	166
Session 16 : Character Controller — FK et IK	167
Objectifs de la session	167
Partie 1 : Pourquoi pas un rigid body ?	167
Partie 2 : Anatomie du contrôleur	168
Partie 3 : Forward vs Inverse Kinematics	169
Partie 4 : CCD — Cyclic Coordinate Descent	170
Partie 5 : FABRIK — Forward And Backward Reaching IK	171
Partie 6 : Applications de l'IK	173
Partie 7 : Le pont vers les véhicules	174
Partie 8 : Démonstration — le bras IK	174
Exercices intégrés	175
Synthèse	175
Session 17 : TP — Véhicules (Raycast Vehicle)	177
Objectifs du TP	177
Partie 1 : Le modèle Raycast Vehicle	177
Partie 2 : Phase A — la suspension à la main	178
Partie 3 : Piloter	179
Partie 4 : Phase B — le vehicle controller de Rapier	179
Partie 5 : Missions	181
Session 18 : Personnages	182
Objectifs de la session	182
Session 19 : Physique, GPU et IA	184
Objectifs de la session	184
Session 20 : Résumé - Quiz final	186
Objectifs de la session	186
Concepts clés à retenir	186
Quiz final	186

Session 1 : Géométrie et Calcul Vectoriel

Définitions

Un vecteur est une grandeur ayant une direction et une magnitude.

On dit qu'il est de dimension n si il a n composantes.

Dans le plan, $n = 2$, donc un vecteur a 2 composantes, 2 dimensions.

🔗 Représentation

Un vecteur est représenté par une flèche. Il peut être placé n'importe où dans le plan. On le place habituellement par rapport au contexte de ce qu'il représente.



Fig. 1. – Un vecteur

💡 Notation vectorielle

Même si le vecteur est défini par ses magnitude et direction, on utilise une notation vectorielle, qui décompose le vecteur selon les vecteurs $\hat{i}$ et $\hat{j}$ du plan cartésien. On retrouve ses 2 dimensions représentées en colonne.

Exemple.

$$\vec{v} = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = 2\hat{i} + 3\hat{j}$$

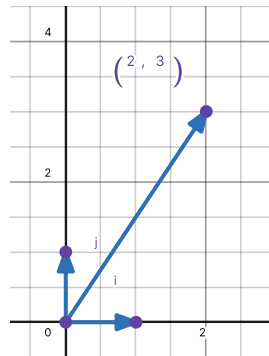


Fig. 2. – Le vecteur $\vec{v}$

🔗 Déplacement

Un déplacement peut se décrire par le vecteur reliant le point A (départ) au point B (arrivée) :

$$\overrightarrow{AB} = \begin{pmatrix} x_B - x_A \\ y_B - y_A \end{pmatrix}$$

🔗 Des vecteurs célèbres en physique

- **Le vecteur position**

Souvent noté $\vec{r}$ (radius), représente la position d'un point dans l'espace en partant de l'origine.

- **Le vecteur vitesse**

Noté $\vec{v}$, représente la vitesse d'un objet, souvent représenté partant du centre de l'objet.

- **Le vecteur accélération**

Noté $\vec{a}$, représente l'accélération d'un objet, souvent représenté partant du centre de l'objet.

Magnitude et direction

La longueur du vecteur, aussi appelée magnitude, est notée $\|\vec{v}\|$, et parfois simplement $|\vec{v}|$.

En 2D : On utilise le théorème de Pythagore pour calculer la norme (autre nom de la magnitude) :

$$\|\vec{v}\| = \sqrt{v_x^2 + v_y^2}$$

Vecteur Unitaire ($\hat{u}$) : Vecteur de longueur 1 direction $\vec{v}$:

$$\hat{u} = \frac{\vec{v}}{\|\vec{v}\|}$$

Un vecteur unitaire est un vecteur de longueur 1, il est souvent utilisé pour représenter une direction sans avoir à se soucier de la magnitude.

Opérations de Base

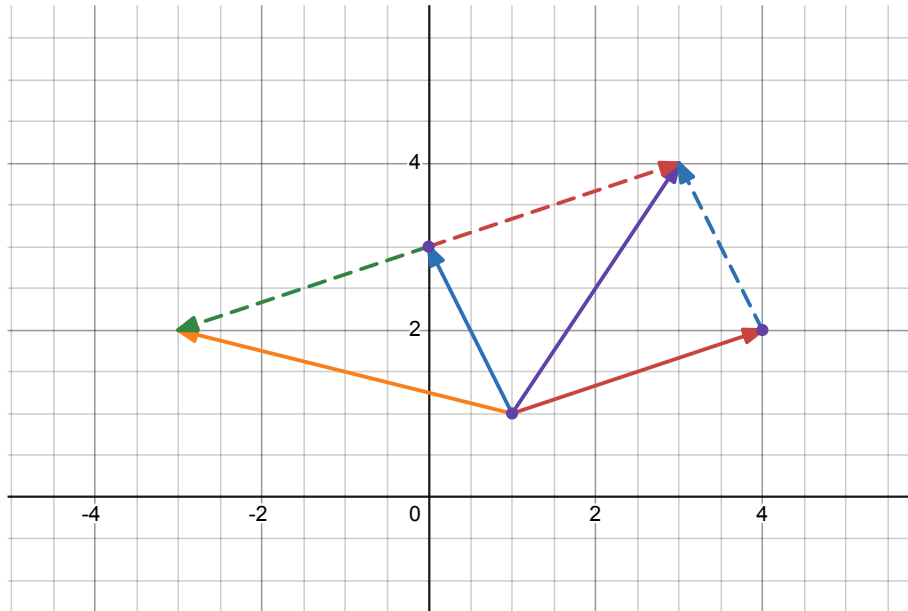
Soit $\vec{u} = \begin{pmatrix} u_x \\ u_y \end{pmatrix}$ et $\vec{v} = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$.

Addition :

$$\vec{u} + \vec{v} = \begin{pmatrix} u_x + v_x \\ u_y + v_y \end{pmatrix}$$

Soustraction :

$$\vec{u} - \vec{v} = \begin{pmatrix} u_x - v_x \\ u_y - v_y \end{pmatrix}$$

Fig. 3. – Le vecteur $\vec{u}$ (rouge) et le vecteur $\vec{v}$ (bleu)

Multiplication par scalaire (k) :

$$k \cdot \vec{u} = \begin{pmatrix} ku_x \\ ku_y \end{pmatrix}$$

Produit Scalaire

Résultat : un **nombre** (scalaire).

Formule algébrique :

$$\vec{u} \cdot \vec{v} = u_x v_x + u_y v_y + u_z v_z$$

Formule géométrique :

$$\vec{u} \cdot \vec{v} = \|\vec{u}\| \cdot \|\vec{v}\| \cdot \cos(\theta)$$

1. Si $\vec{u} \cdot \vec{v} = 0$, alors $\vec{u} \perp \vec{v}$.
2. Si $\vec{u} \cdot \vec{v} > 0$, alors l'angle entre les vecteurs est inférieur à 90° . (devant)
3. Si $\vec{u} \cdot \vec{v} < 0$, alors l'angle entre les vecteurs est supérieur à 90° . (derrière)

Angle entre 2 vecteurs

On isole $\cos(\theta)$. :

$$\cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|}$$

$$\theta = \arccos\left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|}\right)$$

Projections

Projection de $\vec{u}$ sur $\vec{v}$. On utilise le produit scalaire, dans la direction sur laquelle on projette.

$$\text{proj}_{\vec{v}}(\vec{u}) = \left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{v}\|^2} \right) \cdot \vec{v}$$

Et si $\vec{v}$ est normalisé (direction de la droite sur laquelle on projète), on peut simplifier :

$$\text{proj}_{\vec{v}}(\vec{u}) = (\vec{u} \cdot \vec{v}) \cdot \vec{v}$$

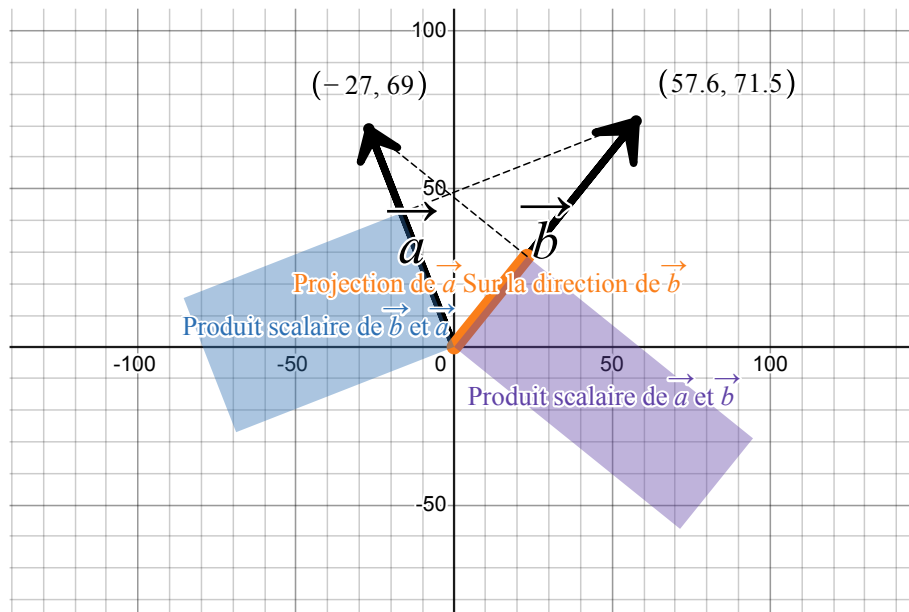


Fig. 4. – Produit scalaire, projection

Produit Vectoriel (3D)

Résultat : un **vecteur** perpendiculaire.

$$\vec{w} = \vec{a} \times \vec{b}$$

Commutativité : Attention, le produit vectoriel n'est pas commutatif ! On a un changement de signe :

$$\vec{a} \times \vec{b} = -\vec{b} \times \vec{a}$$

Magnitude :

$$\|\vec{a} \times \vec{b}\| = \|\vec{a}\| \cdot \|\vec{b}\| \cdot \sin(\theta)$$

Direction : Le produit vectoriel est perpendiculaire aux deux vecteurs :

$$\vec{a} \times \vec{b} \perp \vec{a}$$

$$\vec{a} \times \vec{b} \perp \vec{b}$$

On utilise la règle de la main droite pour trouver la direction du produit vectoriel.

- Le pouce pointe sur $\vec{a}$

- l'index sur b
- le majeur sur $a \times b$.

Calcul (Déterminant) : Le déterminant est un outil algébrique qui permet de trouver les solutions d'un problème de n équations avec n inconnues. Cette solution représente, en quelque sorte une direction émergente (perpendiculaire) des équations.

$$a = \begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} \text{ et } b = \begin{pmatrix} b_x \\ b_y \\ b_z \end{pmatrix}$$

$$\vec{a} \times \vec{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$

Exercice : Le parallélogramme

Soit deux vecteurs dans l'espace 3D :

$$\vec{a} = \begin{pmatrix} 2 \\ 1 \\ 0 \end{pmatrix} \text{ et } \vec{b} = \begin{pmatrix} 0 \\ 3 \\ 2 \end{pmatrix}$$

1. Calculez le produit vectoriel $\vec{w} = \vec{a} \times \vec{b}$ en utilisant la méthode du déterminant.
2. Vérifiez que $\vec{w}$ est bien perpendiculaire à $\vec{a}$ et à $\vec{b}$ à l'aide du produit scalaire.
3. Calculez l'aire du parallélogramme formé par $\vec{a}$ et $\vec{b}$.

Solution : Produit vectoriel par le déterminant. On pose la matrice avec $\hat{i}, \hat{j}, \hat{k}$ en première ligne :

$$\vec{a} \times \vec{b} = \begin{pmatrix} \hat{i} & \hat{j} & \hat{k} \\ 2 & 1 & 0 \\ 0 & 3 & 2 \end{pmatrix}$$

Développement par la première ligne (cofacteurs) :

$$\vec{w} = \hat{i} \cdot \begin{pmatrix} 1 & 0 \\ 3 & 2 \end{pmatrix} - \hat{j} \cdot \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} + \hat{k} \cdot \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$$

Calcul des sous-déterminants 2x2 :

$$\vec{w} = \hat{i} \cdot (1 \cdot 2 - 0 \cdot 3) - \hat{j} \cdot (2 \cdot 2 - 0 \cdot 0) + \hat{k} \cdot (2 \cdot 3 - 1 \cdot 0)$$

$$\vec{w} = \hat{i} \cdot (2) - \hat{j} \cdot (4) + \hat{k} \cdot (6)$$

$$\vec{w} = \begin{pmatrix} 2 \\ -4 \\ 6 \end{pmatrix}$$

Solution : Vérification de perpendicularité.

$$\vec{w} \cdot \vec{a} = (2)(2) + (-4)(1) + (6)(0) = 4 - 4 + 0 = 0 \quad \text{checkmark}$$

$$\vec{w} \cdot \vec{b} = (2)(0) + (-4)(3) + (6)(2) = 0 - 12 + 12 = 0 \quad \text{checkmark}$$

Les deux produits scalaires sont nuls : $\vec{w}$ est bien perpendiculaire à $\vec{a}$ et $\vec{b}$.

Solution : Aire du parallélogramme. L'aire du parallélogramme formé par deux vecteurs est égale à la norme de leur produit vectoriel :

$$\text{Aire} = \|\vec{w}\| = \sqrt{2^2 + (-4)^2 + 6^2} = \sqrt{4 + 16 + 36} = \sqrt{56} \approx 7.48$$

Exercice : Le food truck

Un food truck vend trois types de repas : des burgers à 8, des tacos à 5, et des salades à 6.

Samedi, il a vendu 100 repas pour un total de 620 dollars.

On sait de plus que le nombre de tacos vendus est égal au nombre de burgers plus le double du nombre de salades.

Combien de chaque repas a-t-il vendu ?

Mise en équation :

Soit x le nombre de burgers, y le nombre de tacos, z le nombre de salades.

1. « Total des repas » : $x + y + z = 100$
2. « Total des ventes » : $8x + 5y + 6z = 620$
3. « Relation tacos » : $y = x + 2z$, soit $-x + y - 2z = 0$

Système :

$$x + y + z = 100$$

$$8x + 5y + 6z = 620$$

$$-x + y - 2z = 0$$

Solution : Règle de Cramer. Si $\Delta \neq 0$, alors $x = \frac{\Delta_x}{\Delta}$, $y = \frac{\Delta_y}{\Delta}$, $z = \frac{\Delta_z}{\Delta}$.

Déterminant principal Δ :

$$\Delta = \begin{pmatrix} 1 & 1 & 1 \\ 8 & 5 & 6 \\ -1 & 1 & -2 \end{pmatrix}$$

Développement par la première ligne :

$$\Delta = 1 \cdot \begin{pmatrix} 5 & 6 \\ 1 & -2 \end{pmatrix} - 1 \cdot \begin{pmatrix} 8 & 6 \\ -1 & -2 \end{pmatrix} + 1 \cdot \begin{pmatrix} 8 & 5 \\ -1 & 1 \end{pmatrix}$$

$$\Delta = 1 \cdot (-10 - 6) - 1 \cdot (-16 + 6) + 1 \cdot (8 + 5)$$

$$\Delta = -16 - (-10) + 13 = -16 + 10 + 13 = 7$$

Solution : Calcul de Δ_x , Δ_y , Δ_z . Δ_x (remplacer la colonne 1 par les constantes) :

$$\Delta_x = \begin{pmatrix} 100 & 1 & 1 \\ 620 & 5 & 6 \\ 0 & 1 & -2 \end{pmatrix}$$

$$\Delta_x = 100 \cdot (-16) - 1 \cdot (-1240) + 1 \cdot (620) = -1600 + 1240 + 620 = 260$$

Δ_y (remplacer la colonne 2 par les constantes) :

$$\Delta_y = \begin{pmatrix} 1 & 100 & 1 \\ 8 & 620 & 6 \\ -1 & 0 & -2 \end{pmatrix}$$

$$\Delta_y = 1 \cdot (-1240) - 100 \cdot (-10) + 1 \cdot (620) = -1240 + 1000 + 620 = 380$$

Δ_z (remplacer la colonne 3 par les constantes) :

$$\Delta_z = \begin{pmatrix} 1 & 1 & 100 \\ 8 & 5 & 620 \\ -1 & 1 & 0 \end{pmatrix}$$

$$\Delta_z = 1 \cdot (-620) - 1 \cdot (620) + 100 \cdot (13) = -620 - 620 + 1300 = 60$$

Solution : Résultats et vérification.

$$x = \frac{\Delta_x}{\Delta} = \frac{260}{7} \approx 37 \text{ burgers}$$

$$y = \frac{\Delta_y}{\Delta} = \frac{380}{7} \approx 54 \text{ tacos}$$

$$z = \frac{\Delta_z}{\Delta} = \frac{60}{7} \approx 9 \text{ salades}$$

Vérification :

1. $37 + 54 + 9 = 100$ checkmark
2. $8(37) + 5(54) + 6(9) = 296 + 270 + 54 = 620$ checkmark
3. $54 = 37 + 2(9) = 37 + 18 = 55 \approx 54$ checkmark (arrondi)

Note

Les valeurs exactes sont $x = \frac{260}{7}$, $y = \frac{380}{7}$, $z = \frac{60}{7}$. Dans un cas réel, on ajusterait les données pour obtenir des nombres entiers. L'important ici est la méthode.

Session 2 : Cinématique 2D et Mouvement de Projectile

🔗 Introduction à la cinématique

La cinématique est la branche de la mécanique qui décrit le mouvement des objets sans considérer les causes du mouvement (les forces). Contrairement à la dynamique, qui relie le mouvement aux forces.

Notre objectif dans ce bloc est de développer les outils mathématiques pour décrire précisément *comment* les objets se déplacent.

Position, Vitesse et Accélération

La position, la vitesse et l'accélération sont des concepts qui permettent de décrire précisément comment un objet se comporte.

- La **position** est le vecteur qui indique la localisation d'un point dans l'espace.
- La **vitesse** est le vecteur qui indique la vitesse du point, définie comme la variation (dérivée) de la position par rapport au temps.
- L'**accélération** est le vecteur qui indique l'accélération du point, définie comme la variation (dérivée) de la vitesse par rapport au temps.

🔗 Cas particuliers

- Lorsque la position est constante, le point est au repos, immobile. Sa vitesse et son accélération sont nulles.
- Lorsque la vitesse est constante, le point se déplace à vitesse constante en ligne droite. Son accélération est nulle.
- Lorsque l'accélération est constante, la vitesse augmente constamment, le point se déplace de plus en plus vite.

1. Introduction et Rappels

Rappels sur les vecteurs

Rappel de la définition d'un vecteur comme une quantité possédant une magnitude (longueur, norme) et une direction.

Représentation d'un vecteur en 2D à l'aide de ses composantes :

$$\vec{v} = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$$

.

Opérations vectorielles de base :

$$\vec{a} + \vec{b} = \begin{pmatrix} a_x + b_x \\ a_y + b_y \end{pmatrix}$$

$$\vec{a} - \vec{b} = \begin{pmatrix} a_x - b_x \\ a_y - b_y \end{pmatrix}$$

Multiplication par un scalaire :

$$k\vec{a} = \begin{pmatrix} ka_x \\ ka_y \end{pmatrix}$$

Position en 2D

Vecteur Position

Pour décrire la localisation d'un point dans un plan bidimensionnel, nous utilisons le **vecteur position**, noté $\vec{r}$ (ou parfois $\vec{s}$ ou $\vec{x}$).

Avec un système de coordonnées cartésiennes (x, y) , le vecteur position d'un point P de coordonnées (x, y) est :

$$\vec{r} = \begin{pmatrix} x \\ y \end{pmatrix} = x\hat{i} + y\hat{j}$$

où $\hat{i} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ et $\hat{j} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ sont les vecteurs unitaires.

- Le vecteur position pointe de l'origine vers la position de l'objet.
- La **magnitude** $|\vec{r}| = \sqrt{x^2 + y^2}$ représente la distance à l'origine.
- La **direction** est donnée par l'angle θ avec $\tan(\theta) = \frac{y}{x}$.

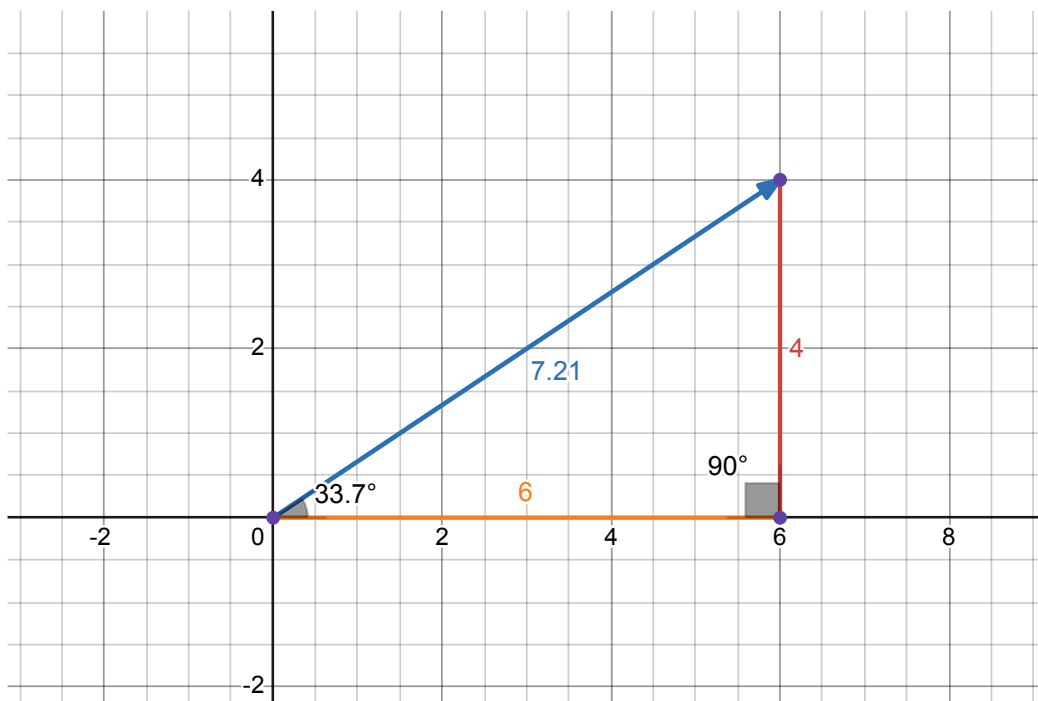


Fig. 5. – Plan 2D avec vecteurs position et vitesse

💡 Trajectoire

Si la position d'un objet change au cours du temps, on décrit son mouvement par :

$$\vec{r}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$$

L'ensemble des points atteints forme la **trajectoire** — une courbe dans l'espace. $x(t)$ et $y(t)$ sont des fonctions du temps.

Exemples de trajectoires :

• **Mouvement rectiligne uniforme :**

$$\vec{r}(t) = \begin{pmatrix} x_0 + v_{0x}t \\ y_0 + v_{0y}t \end{pmatrix}$$

• **Mouvement circulaire uniforme :**

$$\vec{r}(t) = \begin{pmatrix} R \cos(\omega t) \\ R \sin(\omega t) \end{pmatrix}$$

• **Mouvement parabolique (projectile) :**

$$\vec{r}(t) = \begin{pmatrix} x_0 + v_{0x}t \\ y_0 + v_{0y}t - \frac{1}{2}gt^2 \end{pmatrix}$$

Vitesse en 2D

Vitesse Moyenne

Le **déplacement** d'un objet pendant un intervalle $\Delta t = t_f - t_i$ est :

$$\Delta \vec{r} = \vec{r}_f - \vec{r}_i = \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix}$$

Le **vecteur vitesse moyenne** $\vec{v}_{\text{moy}}$ est le rapport du déplacement au temps écoulé :

$$\vec{v}_{\text{moy}} = \frac{\Delta \vec{r}}{\Delta t} = \begin{pmatrix} \frac{\Delta x}{\Delta t} \\ \frac{\Delta y}{\Delta t} \end{pmatrix} = \begin{pmatrix} v_{\text{moy},x} \\ v_{\text{moy},y} \end{pmatrix}$$

La vitesse moyenne a la même direction que le déplacement, et sa magnitude est le déplacement total divisé par le temps écoulé.

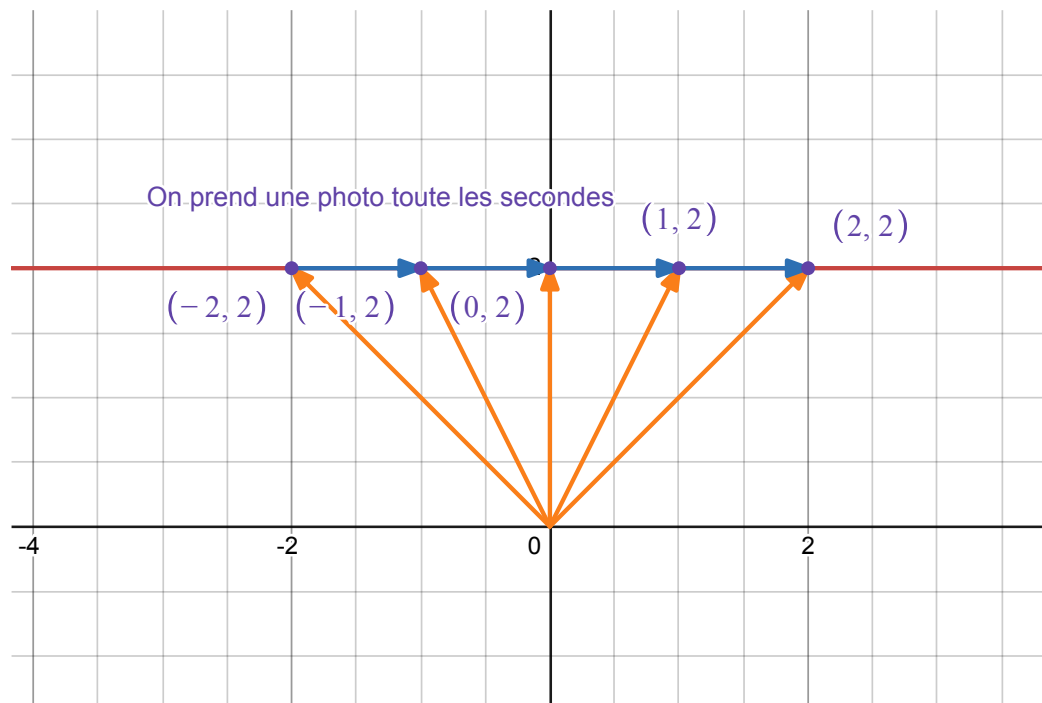


Fig. 6. – Plan 2D avec vecteurs position et vitesse

Exemple. Si une voiture se déplace de $\vec{r}_i = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ à $\vec{r}_f = \begin{pmatrix} 10 \\ 5 \end{pmatrix}$ en $\Delta t = 5$ secondes, alors le déplacement est $\Delta \vec{r} = \begin{pmatrix} 10 \\ 5 \end{pmatrix}$ et la vitesse moyenne est $\vec{v}_{\text{moy}} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ m/s.

Vitesse Instantanée

La **vitesse instantanée** $\vec{v}(t)$ est la limite de la vitesse moyenne lorsque $\Delta t \rightarrow 0$:

$$\vec{v}(t) = \lim_{\Delta t \rightarrow 0} \frac{\Delta \vec{r}}{\Delta t} = \frac{d\vec{r}}{dt}$$

En composantes :

$$\vec{v}(t) = \begin{pmatrix} \frac{dx(t)}{dt} \\ \frac{dy(t)}{dt} \end{pmatrix} = \begin{pmatrix} v_x(t) \\ v_y(t) \end{pmatrix}$$

- La **magnitude** $|\vec{v}(t)| = \sqrt{v_{x(t)}^2 + v_{y(t)}^2}$ est la **vitesse scalaire**.
- La **direction** est tangente à la trajectoire au point considéré.

🔗 Application aux types de trajectoire

1. Mouvement rectiligne uniforme :

$$\vec{v}(t) = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$$

— Le vecteur vitesse est **constant**.

2. Mouvement circulaire uniforme :

$$\vec{v}(t) = \begin{pmatrix} -R\omega \sin(\omega t) \\ R\omega \cos(\omega t) \end{pmatrix}$$

Magnitude constante : $|\vec{v}| = R\omega$. La vitesse est tangente au cercle.

3. Mouvement parabolique (projectile) :

$$\vec{v}(t) = \begin{pmatrix} v_{0x} \\ v_{0y} - gt \end{pmatrix}$$

La composante horizontale v_x reste constante, la verticale v_y diminue linéairement.

Accélération en 2D

Accélération Moyenne

Le **vecteur accélération moyenne** $\vec{a}_{\text{moy}}$ pendant un intervalle $\Delta t = t_f - t_i$ est le rapport du changement de vitesse au temps écoulé :

$$\vec{a}_{\text{moy}} = \frac{\Delta \vec{v}}{\Delta t} = \begin{pmatrix} \frac{\Delta v_x}{\Delta t} \\ \frac{\Delta v_y}{\Delta t} \end{pmatrix} = \begin{pmatrix} a_{\text{moy},x} \\ a_{\text{moy},y} \end{pmatrix}$$

L'accélération moyenne a la direction du changement de vitesse.

Accélération Instantanée

L'**accélération instantanée** $\vec{a}(t)$ est la dérivée de la vitesse par rapport au temps :

$$\vec{a}(t) = \frac{d\vec{v}}{dt} = \begin{pmatrix} \frac{dv_x(t)}{dt} \\ \frac{dv_y(t)}{dt} \end{pmatrix} = \begin{pmatrix} \frac{d^2x(t)}{dt^2} \\ \frac{d^2y(t)}{dt^2} \end{pmatrix}$$

L'accélération peut changer la magnitude de la vitesse (accélérer/décélérer), sa direction, ou les deux.

Cas particulier : Accélération constante

Si l'accélération $\vec{a}$ est constante, on peut **intégrer** pour obtenir la vitesse et la position :

$$\vec{v}(t) = \vec{v}_0 + \vec{a}t = \begin{pmatrix} v_{0x} + a_x t \\ v_{0y} + a_y t \end{pmatrix}$$

$$\vec{r}(t) = \vec{r}_0 + \vec{v}_0 t + \frac{1}{2} \vec{a} t^2 = \begin{pmatrix} x_0 + v_{0x} t + \frac{1}{2} a_x t^2 \\ y_0 + v_{0y} t + \frac{1}{2} a_y t^2 \end{pmatrix}$$

où $\vec{v}_0 = \begin{pmatrix} v_{0x} \\ v_{0y} \end{pmatrix}$ est la vitesse initiale et $\vec{r}_0 = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$ est la position initiale.

C'est ce cas particulier qui est crucial pour l'étude du mouvement de projectile sous l'effet de la gravité.

🔗 Exemples par type de trajectoire

1. Mouvement rectiligne uniforme :

- Position : $\vec{r}(t) = \begin{pmatrix} x_0 + v_x t \\ y_0 + v_y t \end{pmatrix}$
- Vitesse : $\vec{v}(t) = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$
- Accélération : $\vec{a}(t) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

La dérivée d'un vecteur constant est nulle. Aucune accélération n'est nécessaire pour maintenir le mouvement rectiligne uniforme.

2. Mouvement circulaire uniforme :

- Position : $\vec{r}(t) = \begin{pmatrix} R \cos(\omega t) \\ R \sin(\omega t) \end{pmatrix}$
- Vitesse : $\vec{v}(t) = \begin{pmatrix} -R\omega \sin(\omega t) \\ R\omega \cos(\omega t) \end{pmatrix}$
- Accélération : $\vec{a}(t) = \begin{pmatrix} -R\omega^2 \cos(\omega t) \\ -R\omega^2 \sin(\omega t) \end{pmatrix}$

On observe que $\vec{a}(t) = -\omega^2 \vec{r}(t)$: l'accélération est dirigée vers le centre du cercle.

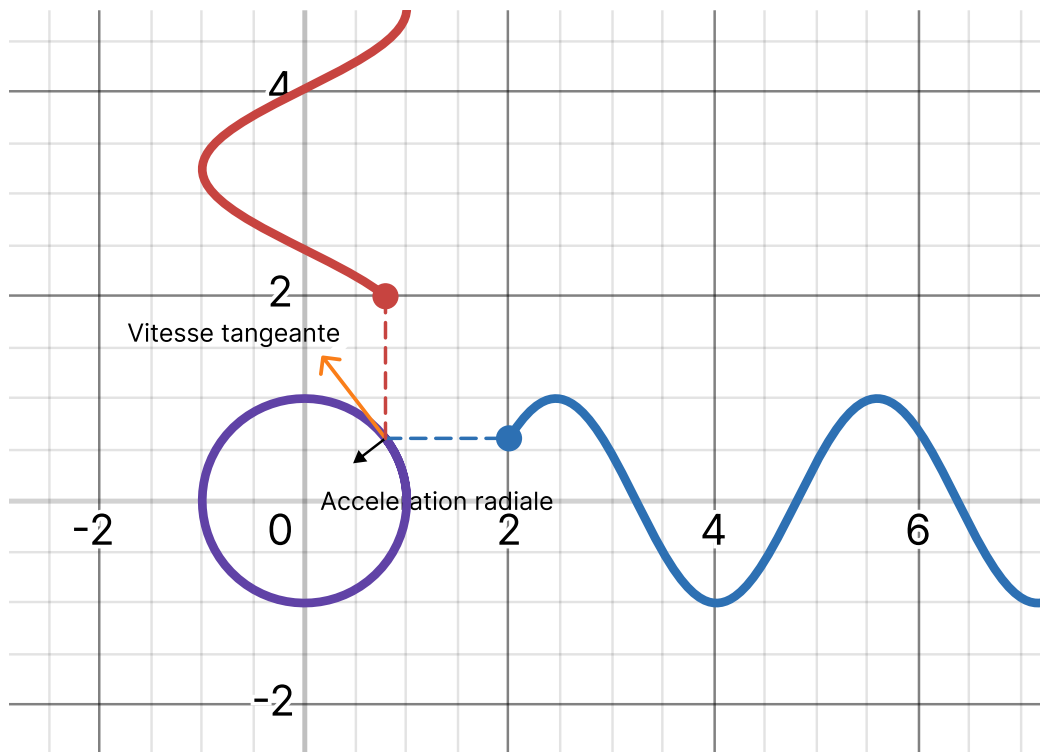


Fig. 7. – Le mouvement circulaire uniforme

3. Mouvement parabolique (projectile) :

- Position : $\vec{r}(t) = \begin{pmatrix} v_{0x} t \\ y_0 + v_{0y} t - \frac{1}{2} g t^2 \end{pmatrix}$
- Vitesse : $\vec{v}(t) = \begin{pmatrix} v_{0x} \\ v_{0y} - g t \end{pmatrix}$
- Accélération : $\vec{a}(t) = \begin{pmatrix} 0 \\ -g \end{pmatrix}$

L'accélération est **constante** et dirigée vers le bas. Avec $g = 9.81 \text{ m/s}^2$, $\vec{a} = \begin{pmatrix} 0 \\ -9.81 \end{pmatrix} \text{ m/s}^2$. Seule la composante verticale de la vitesse change.

Session 3 : Les lois de Newton – La méthode physique

Objectifs de la session

- Réviser la cinématique vue précédemment.
- Introduire les trois lois de Newton comme fondement du moteur physique.
- Comprendre la démarche scientifique : modéliser, prédire, confronter à l'expérience.
- Préparer le lien entre forces et mouvement pour la simulation.

La démarche physique : modéliser l'univers

💡 Physique du Jeu : Comment programmer l'univers

Isaac Newton n'a jamais codé en JavaScript, mais sans lui, aucun jeu vidéo moderne n'existerait. Les lois de Newton sont des modèles mathématiques basés sur l'observation. On les utilise tant qu'aucune expérience ne les invalide.

1. La Première Loi : L'Inertie

La Loi

Un objet au repos reste au repos, et un objet en mouvement continue en ligne droite à vitesse constante, **sauf** si une force extérieure agit sur lui.

Ce que ça veut dire : Les objets résistent au changement. Pour modifier leur vitesse (accélérer, freiner, tourner), il faut appliquer une force.

Exemples :

- Bus qui freine : le corps continue d'avancer.
- **Space Invaders** : le personnage s'arrête instantanément quand on lâche la manette — physiquement faux, comme s'il y avait une friction infinie.
- **Asteroids** : le vaisseau continue en ligne droite sans friction — plus réaliste.

2. La Deuxième Loi : La Dynamique

💬 La Formule

$$\vec{F} = m \cdot \vec{a}$$

En pratique, on cherche l'accélération :

$$\vec{a} = \frac{\vec{F}}{m}$$

Ce que ça veut dire : L'accélération dépend de la force appliquée et de la masse de l'objet.

Exemples :

- Caddie de supermarché : vide, il part vite ; plein, il part lentement avec la même force.
- **Balancing FPS** : un Scout léger (50 kg) accélère à 10 m/s² avec 500 N, tandis qu'un Tank lourd (200 kg) n'accélère qu'à 2.5 m/s².

3. La Troisième Loi : Action-Réaction

La Loi

Pour toute action (force), il y a une réaction égale et opposée.

$$\vec{F}_{A \rightarrow B} = -\vec{F}_{B \rightarrow A}$$

Exemples :

- **Skateboard** : on pousse le sol vers l'arrière, le sol nous pousse vers l'avant.
- **Recul d'arme** : la balle part vers l'avant, l'arme recule vers l'arrière.
- **Rocket jump** : la roquette pousse le sol vers le bas, le joueur est propulsé vers le haut.
- **Collisions** : la boule blanche de billard transmet son impulsion à la boule rouge.

Résumé pour le développeur

Loi	Concept	Implémentation Code
1. Inertie	Les objets gardent leur vitesse.	Ne remettez pas velocity à 0 à chaque frame !
2. F = ma	Force -> Accélération.	<code>acc = forces / mass</code>
3. Action-Réaction	Les interactions sont doubles.	Collision : A repousse B, B repousse A.

Tableau 1. – Aide-mémoire des lois de Newton

Annexe : Pourquoi des objets de masses différentes tombent-ils avec la même vitesse ?

La force de pesanteur est le poids : $\vec{P} = m \cdot \vec{g}$.

La deuxième loi donne : $\vec{a} = \frac{\vec{F}}{m} = \frac{m \cdot \vec{g}}{m} = \vec{g}$.

La masse s'annule. L'accélération gravitationnelle est donc la même pour tous les objets.

Session 4 : Les forces de la nature

Objectifs de la session

- Identifier les forces de la nature agissant sur un objet dans un jeu.
- Comprendre le poids, la normale, le frottement sec et le frottement visqueux.
- Savoir quand et comment modéliser chaque force pour un gameplay réaliste.

Introduction

Dans le vide spatial, un objet lancé garde sa vitesse pour l'éternité (1ère loi de Newton). Sur Terre, tout finit par s'arrêter. Pourquoi ? À cause des interactions avec l'environnement, modélisées par les **forces de la nature**.

1. La Gravité et le Poids

La Gravité

La force d'attraction gravitationnelle attire les objets vers le centre de la Terre. Son accélération est notée $\vec{g}$.

$$\vec{P} = m \cdot \vec{g}$$

Application en jeu : La gravité est une force constante appliquée chaque frame à tous les objets dynamiques.

2. La Force Normale

La Force Normale (N)

C'est la force avec laquelle une surface repousse un objet qui appuie contre elle. Elle est perpendiculaire à la surface.

- Sur un sol plat horizontal : $N = mg$.
- Sur une pente : $N = mg \cos(\theta)$.

Application en jeu : La normale détermine si un objet est au sol et calcule la friction.

3. Le Frottement Sec (Modèle de Coulomb)

Frottement Sec

Frottement de contact entre deux solides. Il dépend du coefficient de friction μ et de la normale N .

$$\|\vec{f}\| = \mu \cdot \|\vec{N}\|$$

Statique vs Cinétique

- **Frottement statique (μ_s) :** l'objet est immobile. La force s'adapte jusqu'à un seuil critique $F_{\max} = \mu_s N$.

- **Frottement cinétique (μ_k)** : l'objet glisse. La force est constante $F_k = \mu_k N$.
- En général : $\mu_s > \mu_k$.

4. Le Frottement Visqueux (Fluide)

Frottement Visqueux

Résistance de l'air ou de l'eau. Il dépend de la vitesse et de la forme de l'objet.

- Vitesse faible : $\vec{f} = -k \cdot \vec{v}$.
- Vitesse élevée : $\vec{f} = -C \cdot \|v\|^2 \cdot \hat{v}$.

Conséquence : la vitesse terminale. Quand la gravité et le drag s'annulent, l'objet ne peut plus accélérer.

Exemple. Comparaison pour le Gameplay :

- **Frottement sec** : l'objet ralentit linéairement et s'arrête net.
- **Frottement fluide** : l'objet ralentit de moins en moins vite, approche asymptotique de 0.

Session 5 : Intégration numérique 101 - Méthode d'Euler

Objectifs de la session

- Comprendre pourquoi l'ordinateur discrétise le temps (frames, pas Δt).
- Apprendre la méthode d'Euler pour intégrer position et vitesse.
- Introduire la série de Taylor comme fondement des méthodes d'intégration.
- Identifier les limites d'Euler et quand l'utiliser malgré tout.

💡 Comment l'ordinateur prédit le futur

Dans nos simulations, le temps n'est pas continu. L'ordinateur calcule le monde image par image (souvent 60 FPS, soit $\Delta t \approx 0.016$ s). Les équations physiques continues (dérivées) doivent devenir des instructions discrètes (additions).

1. La Méthode d'Euler

L'Algorithme

Pour chaque pas de temps Δt :

1. Calculer l'accélération : $\vec{a}_n = \frac{\vec{F}}{m}$
2. Mettre à jour la vitesse : $\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \cdot \Delta t$
3. Mettre à jour la position : $\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \cdot \Delta t$

Intuition : On suppose que la vitesse est constante pendant tout le pas de temps.

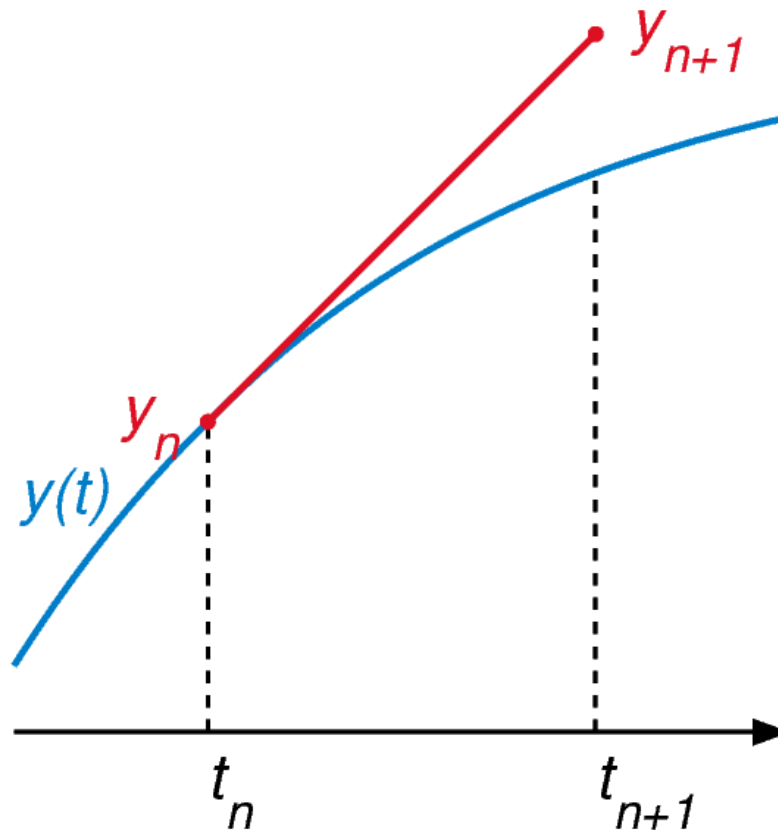


Fig. 8. – La méthode d'Euler : on avance pas à pas en supposant la vitesse constante pendant chaque Δt .

2. D'où ça vient ? La dérivée discrète

La vitesse est la dérivée de la position :

$$\vec{v}(t) = \frac{d\vec{r}}{dt} \approx \frac{\vec{r}(t + \Delta t) - \vec{r}(t)}{\Delta t}$$

En isolant le terme futur :

$$\underbrace{\vec{r}(t + \Delta t)}_{\text{Futur}} \approx \underbrace{\vec{r}(t)}_{\text{Présent}} + \underbrace{\vec{v}(t) \cdot \Delta t}_{\text{Pas}}$$

3. La Série de Taylor

Approximation d'une fonction

La série de Taylor dit qu'une trajectoire peut être reconstruite en additionnant ses dérivées successives :

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}(t)\Delta t + \frac{1}{2}\vec{a}(t)\Delta t^2 + \frac{1}{6}\vec{j}(t)\Delta t^3 + \dots$$

Euler ne garde que le premier terme. C'est une méthode du **premier ordre**.

Pourquoi Euler est une approximation ?

$$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t + \underbrace{O(\Delta t^2)}_{\text{Erreur}}$$

4. Conséquences pratiques

- Euler ignore la courbure de la trajectoire (l'accélération) à l'intérieur du pas.
- Il tend à « dériver » vers l'extérieur des virages ou à gagner de l'énergie (voir Fig. 8).
- Malgré cela, il reste simple et rapide, et fonctionne quand l'accélération change (vent, ressorts, collisions).

Euler (Ordre 1)	Intégration exacte (Ordre 2)
$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t$ <i>Prend la pente au début et trace une ligne droite.</i>	$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t + \frac{1}{2} \vec{a} \Delta t^2$ <i>Prend en compte la courbure.</i>
Facile à coder. Rapide.	Exact pour la gravité constante.

Tableau 2. – Comparaison pour un pas de temps Δt

5. Le code : euler_101.js

Nous allons maintenant décortiquer le fichier `examples/euler_101.js` : une simulation 3D d'une balle qui tombe et rebondit, propulsée par la méthode d'Euler. Le code est divisé en quatre couches : **l'état physique**, **l'initialisation Three.js**, **le cœur du moteur physique**, et **la boucle d'animation**.

5.1. L'état physique (variables globales)

Pourquoi des vecteurs mutables ?

Three.js fournit la classe `Vector3`. Au lieu d'en recréer un à chaque frame (`new THREE.Vector3(...)`), on en crée **un seul** qu'on modifie en place avec `.copy()`, `.add()`, `.set()`. C'est beaucoup plus doux pour le **garbage collector** du navigateur — un réflexe essentiel en boucle de jeu.

Paramètres et état initial.

```
// --- Paramètres Physique ---
const mass = 1.0;
const radius = 0.5;
const gravity = new THREE.Vector3(0, -9.81, 0);

// État initial de la balle (Position, Vitesse)
const startPosition = new THREE.Vector3(-5, 5, 0); // En haut à gauche
const startVelocity = new THREE.Vector3(3, 0, 0); // Lance vers la droite
const position = startPosition.clone();
const velocity = startVelocity.clone();
const force = new THREE.Vector3();
const acceleration = new THREE.Vector3();
```

- `mass`, `radius` : constantes de l'objet. La masse apparaît dans la 2^e loi de Newton $\vec{a} = \frac{\vec{F}}{m}$.

- gravity : le vecteur d'accélération gravitationnelle $\vec{g} = (0, -9.81, 0)$ « m/s »².
- startPosition / startVelocity : l'état à $t=0$, conservé pour pouvoir réinitialiser la simulation.
- position, velocity, force, acceleration : les **quatre vecteurs vivants** qui évoluent à chaque frame. On les clone pour ne pas écraser les valeurs de départ.

🔗 Les quatre vecteurs fondamentaux

Toute simulation newtonienne repose sur ces quatre quantités, mises à jour dans cet ordre exact à chaque pas de temps :

$$\vec{F} \rightarrow \vec{a} \rightarrow \vec{v} \rightarrow \vec{r}$$

Force $\vec{F}$, accélération $\vec{a}$, vitesse $\vec{v}$, position $\vec{r}$. C'est la **chaîne d'Euler**.

5.2. Initialisation Three.js (init)

La fonction `init()` construit la scène une seule fois au démarrage. Elle ne contient **aucune** physique — uniquement de la plomberie graphique.

Scène, caméra, rendu, lumières et sol.

```
function init() {
  scene = new THREE.Scene();
  scene.background = new THREE.Color(0xbfd1e5);
  camera = new THREE.PerspectiveCamera(50, window.innerWidth / window.innerHeight, 0.1, 100);
  camera.position.set(0, 5, 15);

  renderer = new THREE.WebGLRenderer({ antialias: true });
  renderer.setSize(window.innerWidth, window.innerHeight);
  renderer.shadowMap.enabled = true;
  document.body.appendChild(renderer.domElement);

  scene.add(new THREE.HemisphereLight(0xffeeee, 0xaaccaa));
  const light = new THREE.DirectionalLight(0xFFFF22, 1);
  light.position.set(5, 10, 7);
  light.castShadow = true;
  scene.add(light);

  const floor = new THREE.Mesh(
    new THREE.PlaneGeometry(20, 20),
    new THREE.MeshStandardMaterial({ color: 0xaaaaaa })
  );
  floor.rotation.x = -Math.PI / 2;
  floor.receiveShadow = true;
  scene.add(floor);
  scene.add(new THREE.GridHelper(20, 20));

  createBall();
  setupGUI();
  new OrbitControls(camera, renderer.domElement);
  window.addEventListener('resize', onWindowResize, false);
  renderer.setAnimationLoop(animate);
}
```

- **Scène & caméra** : la caméra PerspectiveCamera avec un champ de vision de 50° , placée en $(0, 5, 15)$ pour voir la scène de biais.
- **Render** : WebGLRenderer dessine sur la page. antialias: true lisse les arêtes. shadowMap.enabled active les ombres portées.
- **Lumières** : une HemisphereLight (ciel + sol) pour l'ambiance, et une DirectionalLight (le « soleil ») qui projette des ombres.
- **Sol** : un PlaneGeometry couché par rotation.x = $-\text{Math.PI} / 2$ (rotation de -90° autour de l'axe X pour passer du plan XY au plan XZ). receiveShadow lui permet de recevoir l'ombre de la balle.
- **Lancement** : renderer.setAnimationLoop(animate) demande au navigateur d'appeler animate à chaque frame (≈ 60 FPS).

5.3. La balle et ses flèches (createBall)

Maillage + flèches de débogage.

```
function createBall() {
  ball = new THREE.Mesh(
    new THREE.SphereGeometry(radius, 32, 32),
    new THREE.MeshStandardMaterial({ color: 0x0077ff })
  );
  ball.castShadow = true;
  scene.add(ball);

  velArrow = new THREE.ArrowHelper(new THREE.Vector3(1, 0, 0), position, 1, 0x333333);
  scene.add(velArrow);

  accArrow = new THREE.ArrowHelper(new THREE.Vector3(0, 1, 0), position, 1, 0xAA8800);
  scene.add(accArrow);
}
```

- ball : un Mesh = une SphereGeometry (le rayon vient de notre variable physique) + un matériau bleu. castShadow projette son ombre sur le sol.
- velArrow / accArrow : deux ArrowHelper qui affichent en temps réel la **direction** et la **magnitude** de la vitesse (noir) et de l'accélération (jaune). Indispensables pour **voir** la physique — sans eux, Euler est une boîte noire.

5.4. Le cœur physique : updatePhysics(dt)

C'est ici que la méthode d'Euler prend vie. La fonction suit **exactement** les trois étapes de l'algorithme vu en section 1, plus une quatrième pour gérer le rebond.

Étape 1 — Calcul des forces.

```
function updatePhysics(dt) {
  // 1. Calcul des Forces
  force.set(0, 0, 0); // Reset

  // Ajout de la gravité (P = m * g)
  const gravityForce = gravity.clone().multiplyScalar(mass);
  force.add(gravityForce);
}
```

On **remet à zéro** la force à chaque frame (force.set(0,0,0)) — sinon les forces s'accumuleraient d'une frame à l'autre et la balle s'envolerait. Puis on ajoute le poids $\vec{P} = m \cdot \vec{g}$. Ici, seule la gravité agit ; dans un vrai jeu, on empilerait ici ressorts, vent, frottements, etc.

Étape 2 — Deuxième loi de Newton.

```
// 2. Deuxième Loi de Newton : a = F / m
acceleration.copy(force).divideScalar(mass);
```

On calcule $\vec{a} = \frac{\vec{F}}{m}$. `.copy(force)` recopie la force dans `acceleration`, puis `.divideScalar(mass)` divise par la masse. Avec $m = 1$, on a $\vec{a} = \vec{g}$, ce qui est cohérent : en chute libre, tout tombe à la même accélération.

Étape 3 — Intégration d'Euler.

```
// 3. Intégration d'Euler (Premier ordre)
// v_nouveau = v_actuel + a * dt
velocity.addScaledVector(acceleration, dt);

// p_nouveau = p_actuel + v * dt
position.addScaledVector(velocity, dt);
```

Ce sont **exactement** les deux formules de la définition de la section 1 :

$$\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \cdot \Delta t$$

$$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_{n+1} \cdot \Delta t$$

 Subtilité : la vitesse utilisée pour la position

Remarquez qu'on met à jour la vitesse **avant** la position, et qu'on utilise la **nouvelle** vitesse $\vec{v}_{n+1}$ pour intégrer la position. C'est la variante **semi-implicite** (ou **symplectique**) d'Euler. Elle est légèrement plus stable que l'Euler classique (qui utiliserait $\vec{v}_n$) et conserve mieux l'énergie sur le long terme. C'est le choix par défaut dans la plupart des moteurs de jeu.

Étape 4 — Collisions avec le sol.

```
// 4. Gestion des Collisions (Sol)
if (position.y < radius) {
    position.y = radius;
    velocity.y = -velocity.y * params.restitution;
    velocity.x *= 0.95;
    velocity.z *= 0.95;
}
```

- **Test** : si le centre de la balle descend sous l'altitude `radius`, le bas de la sphère touche le sol.
- **Correction de position** : on remonte la balle à `position.y = radius` pour éviter qu'elle ne s'enfonce ou tremble sous le plan.
- **Rebond** : on inverse la composante verticale de la vitesse et on la multiplie par le coefficient de restitution e (`params.restitution`). Avec $e = 1$ la balle rebondit à la même hauteur ; avec $e = 0$ elle s'écrase ; à $e = 0.8$ elle perd 20 % d'énergie à chaque rebond.
- **Frottement tangentiel** : on amortit légèrement les composantes horizontales ($*0.95$) pour simuler le frottement au contact — sinon la balle glisserait indéfiniment.

💡 Pourquoi un test simple suffit ici

On teste uniquement $\text{position.y} < \text{radius}$ parce que le sol est un plan infini à $y = 0$. Pour des collisions entre **deux sphères**, le test deviendrait $\|\vec{r}_A - \vec{r}_B\| < r_A + r_B$ — c'est-à-dire que la distance entre les centres est inférieure à la somme des rayons. On verra ça en Session 6.

5.5. Synchronisation visuelle (updateVisuals)

La physique calcule des nombres ; Three.js affiche des objets. `updateVisuals` fait le pont.

Copier la physique vers le visuel.

```
function updateVisuals() {
  ball.position.copy(position);

  velArrow.position.copy(position);
  velArrow.setDirection(velocity.clone().normalize());
  velArrow.setLength(velocity.length() * 0.3, 0.2, 0.1);

  accArrow.position.copy(position);
  if (acceleration.lengthSq() > 0.0001) {
    accArrow.setDirection(acceleration.clone().normalize());
    accArrow.setLength(acceleration.length() * 0.3, 0.2, 0.1);
  }
}
```

- `ball.position.copy(position)` : on **copie** le vecteur physique dans le mesh — jamais d'affectation directe (`ball.position = position` casserait la référence interne de Three.js).
- **Flèche vitesse** : `.normalize()` donne la direction (vecteur unitaire), `.length()` donne la magnitude. La longueur de la flèche est proportionnelle à $\|\vec{v}\|$, donc la flèche **grandit** quand la balle accélère.
- **Flèche accélération** : on vérifie `lengthSq() > 0.0001` avant de normaliser pour éviter une division par zéro (un vecteur nul n'a pas de direction). Ici l'accélération est constante (gravité), donc la flèche pointe toujours vers le bas avec la même longueur.

5.6. La boucle d'animation (animate)

Le pouls de la simulation.

```
function animate() {
  const dt = clock.getDelta();
  const safeDt = Math.min(dt, 0.1) * params.timeScale;
  updatePhysics(safeDt);
  updateVisuals();
  renderer.render(scene, camera);
}
```

💡 Le `dt` est le cœur d'Euler

`clock.getDelta()` renvoie le temps écoulé **en secondes** depuis la dernière frame — c'est notre Δt . Sans lui, la simulation tournerait à des vitesses différentes selon la machine (144 Hz vs 60 Hz vs 30 Hz). Avec `dt`, la physique est **indépendante du framerate** : la balle met le même temps à tomber sur un vieux portable que sur un écran de gamer.

- **Clamp** : `Math.min(dt, 0.1)` plafonne Δt à 0,1 s. Si l'utilisateur change d'onglet, le navigateur suspend la boucle ; au retour, `getDelta()` renverrait un Δt énorme (plusieurs secondes) qui ferait **exploser** la simulation (la balle traverserait le sol). Le clamp évite ce piège classique.
- **timeScale** : `params.timeScale` permet de ralentir (0,5×), accélérer (2×) ou mettre en pause (0×) la simulation sans toucher au code physique — utile pour observer un phénomène.
- **Ordre** : `updatePhysics` → `updateVisuals` → `render`. On calcule l'état, on le reflète sur les objets, puis on dessine. Cet ordre garantit que l'image affichée correspond **toujours** à l'état physique le plus récent.

6. Résumé : la chaîne complète

De la force à l'image, en une frame

À chaque image, `animate` déclenche cette cascade :

1. `clock.getDelta()` → mesure Δt réel.
2. `updatePhysics(dt)` :
 - Calcule $\vec{F}$ (gravité).
 - $\vec{a} = \frac{\vec{F}}{m}$.
 - $\vec{v}_{n+1} = \vec{v}_n + \vec{a} \cdot \Delta t$ (Euler).
 - $\vec{r}_{n+1} = \vec{r}_n + \vec{v}_{n+1} \cdot \Delta t$.
 - Teste et résout la collision avec le sol.
3. `updateVisuals()` → copie $\vec{r}$ et $\vec{v}$ vers le mesh et les flèches.
4. `render.render()` → dessine la frame à l'écran.

C'est exactement le schéma de la Fig. 8, traduit en JavaScript. Tout l'art d'un moteur de jeu tient dans cette boucle, exécutée 60 fois par seconde.

Session 6 : L'impulsion, les collisions

Objectifs de la session

- Comprendre l'impulsion comme changement de quantité de mouvement.
- Apprendre la loi de restitution et la vitesse relative.
- Calculer l'impulsion d'une collision entre deux objets.
- Implémenter un laboratoire de chocs 1D.

1. La Quantité de Mouvement

Quantité de mouvement (p)

La quantité de mouvement est le produit de la masse par la vitesse d'un objet :

$$\vec{p} = m \cdot \vec{v}$$

- C'est une grandeur **vectorielle** : elle a la même direction que la vitesse.
- Unité SI : $\text{kg} \cdot \text{m/s}$.
- Elle mesure « combien de mouvement » porte un objet — un camion lent et une balle rapide peuvent avoir la même quantité de mouvement.

💡 Conservation de la quantité de mouvement

Dans un système isolé (sans forces externes), la quantité de mouvement **totale** est conservée :

$$\vec{p}_{\text{totale}} = \vec{p}_A + \vec{p}_B = \text{constante}$$

C'est la loi fondamentale qui gouverne les collisions : ce qui est perdu par un objet est gagné par l'autre.

2. D'où vient l'Impulsion ?

Théorème de l'Impulsion

La force est liée à la variation de la vitesse (si la masse est constante) :

$$\vec{F} = m \cdot \frac{d\vec{v}}{dt}$$

Comme $\vec{p} = m \cdot \vec{v}$, cela s'écrit aussi :

$$\vec{F} = \frac{d\vec{p}}{dt}$$

En intégrant sur un choc instantané ($\Delta t \approx 0$) :

$$\vec{J} = \Delta\vec{p} = m \cdot \Delta\vec{v}$$

Comment un choc bref change-t-il la vitesse si vite ?

D'après $\vec{F} = m \cdot \frac{d\vec{v}}{dt}$, on isole la variation de vitesse :

$$\Delta \vec{v} = \frac{\vec{F} \cdot \Delta t}{m}$$

Si Δt est très petit (un choc quasi instantané), il faut une force **énorme** pour produire un $\Delta \vec{v}$ significatif. Mais le produit $\vec{F} \cdot \Delta t$ reste fini — c'est l'impulsion $\vec{J}$.

- **Exemple :** Une balle de tennis frappée par une raquette. Le contact dure ≈ 5 ms, mais la force atteint ≈ 1000 N. L'impulsion vaut $\vec{J} = 1000 \cdot 0.005 = 5 \text{ kg} \cdot \text{m/s}$ — assez pour faire passer la balle de 0 à 50 m/s.
- L'accélération est bel et bien gigantesque ($a = \frac{F}{m} \approx 20000 \text{ m/s}^2$), mais elle ne dure qu'un instant. Ce qui compte, c'est le **produit** $F \cdot \Delta t$, pas la force seule.

💡 Intuition clé

Ce n'est pas la force seule qui change la vitesse, ni le temps seul. C'est leur **produit** — l'impulsion. Une petite force appliquée longtemps peut produire le même changement de vitesse qu'une force énorme appliquée très brièvement.

💡 Pourquoi on n'utilise pas Euler pour les collisions

La méthode d'Euler (Session 5) calcule $\Delta \vec{v} = \vec{a} \cdot \Delta t$ à chaque pas de temps. Mais lors d'un choc, la force (et donc l'accélération) est énorme et dure un temps négligeable. Avec $\Delta t \approx 0.016$ s, soit on rate complètement le choc (l'objet traverse), soit on surestime grossièrement la force. Au lieu de simuler la force frame par frame, on **saute directement au résultat** : on applique l'impulsion $\vec{J}$ qui change la vitesse instantanément.

3. La Loi de Restitution

Coefficient de restitution (e)

La vitesse à laquelle deux objets s'éloignent après un choc est proportionnelle à la vitesse à laquelle ils se rapprochaient avant le choc.

$$v_{\text{rel}}(\text{après}) = -e \cdot v_{\text{rel}}(\text{avant})$$

- $e = 1$: choc élastique (billes de billard).
- $e = 0$: choc mou (pâte à modeler).

4. Vitesse Relative et Normale

💡 Point de départ : le test de collision

Tout choc commence par un test de collision : détecter que deux objets se chevauchent. Une fois le contact trouvé, seule la vitesse **projetée sur la normale de collision** $\vec{n}$ compte. C'est elle qui détermine si les objets se rapprochent, s'éloignent ou glissent l'un contre l'autre.

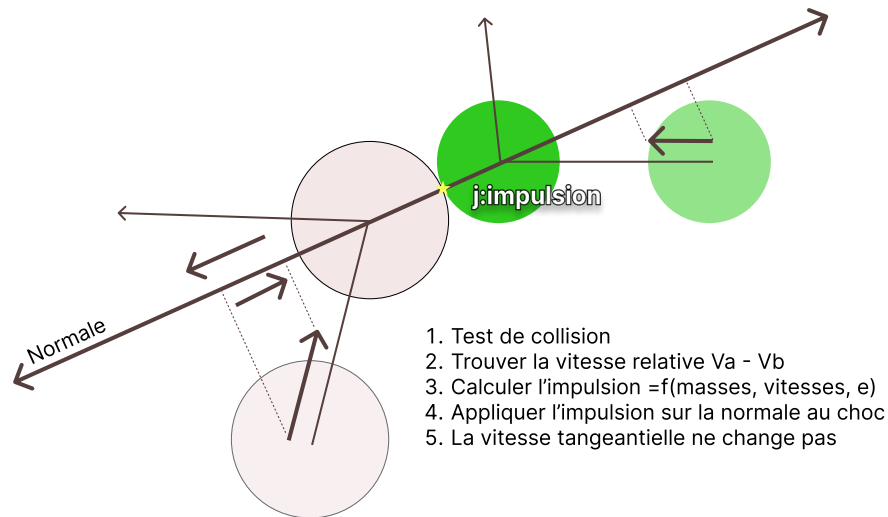


Fig. 9. – Vue schématique d'une collision : normale de contact, vitesses avant et impulsion appliquée.

Vitesse relative scalaire

La vitesse de rapprochement projetée sur la normale de collision :

$$v_{\text{rel}} = (\vec{v}_A - \vec{v}_B) \cdot \vec{n}$$

- $v_{\text{rel}} < 0$: les objets se rapprochent.
- $v_{\text{rel}} > 0$: les objets s'éloignent.
- $v_{\text{rel}} = 0$: ils glissent l'un contre l'autre.

5. Calcul de l'Impulsion

🗨 Formule de l'impulsion

$$j = \underbrace{-(1 + e)v_{\text{rel}}}_{\text{Vitesse à changer}} \cdot \underbrace{\left(\frac{m_A m_B}{m_A + m_B} \right)}_{\text{Masse réduite}}$$

Application aux vitesses :

$$v'_A = v_A + \left(\frac{j}{m_A} \right) \cdot \vec{n}$$

$$v'_B = v_B - \left(\frac{j}{m_B} \right) \cdot \vec{n}$$

💡 Vitesse tangentielle inchangée

L'impulsion $\vec{J}$ est appliquée le long de la normale $\vec{n}$. Seule la composante **normale** de la vitesse est modifiée. La composante **tangentielle** (perpendiculaire à $\vec{n}$) reste inchangée : $\vec{v}_{\text{tangentielle}}' =$

$\vec{v}_{\text{tangentielle}}$

6. TP : Laboratoire de Chocs 1D

🔗 Objectifs du TP

Implémenter la réponse physique d'une collision entre deux boules de masses différentes dans un espace 1D sans gravité ni friction.

Missions

1. Créer deux boules : A ($m = 5$) lancée vers B ($m = 1$).
2. Calculer la normale $\vec{n}$.
3. Calculer la vitesse relative v_{rel} .
4. Si $v_{\text{rel}} > 0$, sortir (les objets s'éloignent).
5. Calculer l'impulsion j avec la masse réduite.
6. Appliquer le changement de vitesse aux deux boules.

Scénarios à observer

- **Pendule de Newton** : masses égales, $e = 1$ — A s'arrête, B repart.
- **Le Mur** : $m_A = 1$, $m_B = 100$ — A rebondit, B bouge à peine.
- **Bulldozer** : $m_A = 100$, $m_B = 1$ — A continue, B repart à environ $2v_A$.
- **Pâte à modeler** : $e = 0$ — les boules se collent.

Session 7 : Broad et Narrow phases – Part 1

Objectifs de la session

- Comprendre le **pipeline de détection de collision** en trois étapes.
- Mesurer le budget temps d'une frame (16 ms) et pourquoi la physique doit tenir en 2 ms.
- Expliquer pourquoi la détection naïve est en $O(N^2)$ et pourquoi c'est inacceptable.
- Maîtriser l'AABB (Axis Aligned Bounding Box) comme fondement de la Broad Phase.
- Détailler trois structures de partitionnement : Grille Uniforme, Quadtree/Octree, Sweep-and-Prune.
- Comparer leurs complexités, avantages et cas d'usage.
- Introduire la Narrow Phase et le test de chevauchement AABB.

1. Le Pipeline de Détection de Collision

Trois étapes pour détecter et résoudre une collision

Un moteur physique moderne ne teste pas toutes les paires d'objets. Il suit un **pipeline en entonnoir** qui élimine progressivement les faux positifs :

1. **Broad Phase** — « Qui pourrait entrer en collision ? »
Réponse rapide avec des boîtes simplifiées (AABB). Élimine 95–99 % des paires.
2. **Narrow Phase** — « Ces deux objets se touchent-ils vraiment ? »
Test géométrique précis (sphère, OBB, GJK, etc.). Coûteux mais appliqué à très peu de paires.
3. **Collision Response** — « Que faire ? »
Impulsion, correction de position, friction (vu en Session 6).

💡 L'entonnoir en chiffres

Pour 1 000 objets, la brute force génère $\approx 500\,000$ paires. La Broad Phase en retient typiquement ≈ 2000 . La Narrow Phase confirme ≈ 200 collisions réelles. La Response n'en traite que les pertinentes. Chaque étage divise le travail par un ordre de grandeur.

2. Le Contexte : Le Budget Temps (16 ms)

Le budget temps d'une frame

À 60 FPS, on dispose de **16.6 millisecondes** pour tout calculer :

- Rendu : 8 ms
- IA / Gameplay : 4 ms
- Audio / Réseau : 2 ms
- **Physique : 2 ms**

Si la détection de collision prend 10 ms, le jeu saccade. L'optimisation est vitale.

! Les 2 ms de la physique

Sur ces 2 ms, la Broad Phase doit prendre **moins de 0.5 ms**. C'est elle qui scale avec N . La Narrow Phase et la Response consomment le reste. Si la Broad Phase est lente, tout le reste cascade — c'est le goulot d'étranglement numéro un des moteurs physiques.

3. Le Problème : La Malédiction $O(N^2)$

Détection naïve (brute force)

Avec N objets, chaque objet teste tous les autres. Le nombre de paires uniques est :

$$C = \frac{N(N-1)}{2} \approx O(N^2)$$

Objets (N)	Paires	Temps à 1 $\mu\text{s}/\text{test}$
10	45	$\approx 0.05 \text{ ms}$
100	4 950	$\approx 5 \text{ ms}$
1 000	499 500	$\approx 500 \text{ ms}$
10 000	49 995 000	$\approx 50 \text{ s}$

Tableau 3. – La croissance quadratique rend la brute force inutilisable au-delà de quelques dizaines d'objets.

Solution : réduire le nombre de paires **avant** les tests coûteux. C'est le rôle de la **Broad Phase**.

4. L'AABB : La Boîte Fondamentale

Axis Aligned Bounding Box (AABB)

Une AABB est le plus petit rectangle (2D) ou pavé (3D) **aligné sur les axes** qui contient un objet. Elle est définie par deux points :

$$\text{AABB} = \{\min(x_{\min}, y_{\min}, z_{\min}), \max(x_{\max}, y_{\max}, z_{\max})\}$$

💡 Pourquoi des boîtes alignées sur les axes ?

Parce que le test de chevauchement entre deux AABB est **trivial** — quelques comparaisons sur chaque axe, pas de produit vectoriel, pas de trigonométrie. C'est le test le plus rapide qui existe en 3D.

Test de chevauchement AABB vs AABB

Deux AABB se chevauchent si et seulement si leurs intervalles se chevauchent sur **tous** les axes :

$$\text{overlap}_x = (x_{\max}^A \geq x_{\min}^B) \wedge (x_{\max}^B \geq x_{\min}^A)$$

Si un seul axe ne se chevauche pas, il n'y a pas de collision. C'est le principe du **Separating Axis**.

Test AABB en JavaScript.

```
function aabbOverlap(a, b) {
  // A et B sont des objets { min: Vector3, max: Vector3 }
  if (a.max.x < b.min.x || b.max.x < a.min.x) return false;
  if (a.max.y < b.min.y || b.max.y < a.min.y) return false;
  if (a.max.z < b.min.z || b.max.z < a.min.z) return false;
  return true; // Chevauchement sur les 3 axes
}
```

Calcul d'une AABB pour une sphère : trivial — la boîte va de $c - r$ à $c + r$ sur chaque axe, où c est le centre et r le rayon.

Calcul d'une AABB pour un mesh complexe : on parcourt tous les sommets et on garde les min et max par axe. Ce calcul se fait **une fois** au chargement, puis on translate la boîte avec l'objet.

5. La Broad Phase : Structures de Partitionnement

Principe de la Broad Phase

On entoure chaque objet par son AABB, puis on utilise une **structure de partitionnement** pour trouver rapidement quelles paires d'AABB **pourraient** se chevaucher. On ne teste précisément que les candidates.

On présente ici les trois structures les plus utilisées en jeu vidéo.

5.A. Grille Uniforme (Spatial Hashing)

Principe

On découpe le monde en cases (cells) de taille fixe S . Chaque objet est rangé dans la case correspondant à sa position. Pour tester les collisions d'un objet, on ne regarde que les objets dans sa case et les cases **adjacentes** (8 en 2D, 26 en 3D).

Spatial Hashing — structure et requête.

```
class SpatialGrid {
  constructor(cellSize) {
    this.cellSize = cellSize;
    this.cells = new Map(); // clé "x,y,z" -> liste d'objets
  }

  getKey(pos) {
    const x = Math.floor(pos.x / this.cellSize);
    const y = Math.floor(pos.y / this.cellSize);
    const z = Math.floor(pos.z / this.cellSize);
    return `${x},${y},${z}`;
  }

  clear() { this.cells.clear(); }

  insert(obj) {
```

```

const key = this.getKey(obj.position);
if (!this.cells.has(key)) this.cells.set(key, []);
this.cells.get(key).push(obj);
}

getNeighbors(pos) {
  const cx = Math.floor(pos.x / this.cellSize);
  const cy = Math.floor(pos.y / this.cellSize);
  const cz = Math.floor(pos.z / this.cellSize);
  const neighbors = [];
  for (let dx = -1; dx <= 1; dx++)
    for (let dy = -1; dy <= 1; dy++)
      for (let dz = -1; dz <= 1; dz++) {
        const cell = this.cells.get(`${cx+dx},${cy+dy},${cz+dz}`);
        if (cell) neighbors.push(...cell);
      }
  return neighbors;
}
}

```

🔗 Choix de la taille de cellule

La taille idéale est $S \approx$ diamètre max des objets. Si S est trop petit, un objet chevauche plusieurs cases (overhead). Si S est trop grand, chaque case contient trop d'objets (le filtrage ne sert à rien).

- **Complexité** : $O(N)$ pour construire la grille, $O(K)$ par objet où K est le nombre d'objets dans les 27 cases voisines. Si la distribution est uniforme, K est constant $\rightarrow$ **total en $O(N)$** .
- **Avantage** : accès $O(1)$, très rapide, facile à coder, excellent pour des objets de taille similaire.
- **Inconvénient** : mauvais si les tailles varient beaucoup (un petit objet et un énorme bâtiment dans la même grille), ou si le monde est infini (il faut un dictionnaire au lieu d'un tableau).
- **Cas d'usage typique** : particules, billes, projectiles, jeux d'arène fermée.

5.B. Arbres Spatiaux (Quadtree / Octree)

Principe

On commence avec une grande boîte contenant tout le monde. Si elle contient plus de T objets (seuil), on la découpe en 4 (2D = Quadtree) ou 8 (3D = Octree) sous-boîtes égales. On recommence récursivement jusqu'à ce que chaque feuille contient au plus T objets.

Octree — insertion et requête.

```

class OctreeNode {
  constructor(min, max, depth = 0, maxDepth = 5, maxObjects = 8) {
    this.min = min; this.max = max;
    this.depth = depth;
    this.maxDepth = maxDepth;
    this.maxObjects = maxObjects;
    this.objects = [];
    this.children = null; // 8 sous-nœuds ou null
  }

  insert(obj) {

```

```

    if (this.children) {
        for (const child of this.children)
            if (child.contains(obj)) child.insert(obj);
        return;
    }
    this.objects.push(obj);
    if (this.objects.length > this.maxObjects
        && this.depth < this.maxDepth) {
        this.subdivide();
        // Redistribuer les objets existants
        for (const o of this.objects)
            for (const child of this.children)
                if (child.contains(o)) child.insert(o);
        this.objects = [];
    }
}

subdivide() {
    const mid = this.min.clone().add(this.max).multiplyScalar(0.5);
    // Créer 8 enfants (2^3 combinaisons de min/max par axe)
    this.children = [];
    for (let i = 0; i < 8; i++) {
        const childMin = new THREE.Vector3(
            (i & 1) ? mid.x : this.min.x,
            (i & 2) ? mid.y : this.min.y,
            (i & 4) ? mid.z : this.min.z
        );
        const childMax = new THREE.Vector3(
            (i & 1) ? this.max.x : mid.x,
            (i & 2) ? this.max.y : mid.y,
            (i & 4) ? this.max.z : mid.z
        );
        this.children.push(new OctreeNode(childMin, childMax, this.depth + 1,
                                           this.maxDepth, this.maxObjects));
    }
}

query(aabb, results = []) {
    if (!this.intersectsAABB(aabb)) return results;
    if (this.children) {
        for (const child of this.children) child.query(aabb, results);
    } else {
        results.push(...this.objects);
    }
    return results;
}
}

```

🗨 Reconstruction vs mise à jour

Dans un jeu où les objets bougent, deux stratégies existent :

- **Rebuild chaque frame** : on détruit et reconstruit l'arbre. Simple mais coûteux ($O(N \log N)$). Acceptable si N est modéré (< 10000).

- **Update incrémental** : on déplace les objets dans l'arbre. Plus complexe, mais nécessaire pour de très grands mondes.

La plupart des moteurs de jeu font un **rebuild** chaque frame : c'est plus simple et souvent plus rapide en pratique grâce au cache.

- **Complexité** : $O(N \log N)$ pour construire, $O(\log N + K)$ pour une requête.
- **Avantage** : s'adapte automatiquement à la densité — zones denses subdivisées davantage, zones vides non subdivisées.
- **Inconvénient** : coûteux à reconstruire/mettre à jour si les objets bougent beaucoup ; overhead mémoire pour les nœuds.
- **Cas d'usage typique** : mondes ouverts, objets de tailles variées, culling de frustum, raycasting.

5.C. Sweep and Prune (SAP)

Principe

On projette les débuts (min) et fins (max) de chaque AABB sur un axe (généralement X). On trie la liste des intervalles. En parcourant la liste triée, on maintient un ensemble « actif » d'intervalles qui n'ont pas encore été fermés. Deux intervalles actifs simultanément se chevauchent sur cet axe.

Sweep and Prune — algorithme 1D.

```
class SweepAndPrune {
  constructor() {
    this.intervals = []; // [{ id, value, isStart }]
  }

  update(objects) {
    this.intervals = [];
    for (const obj of objects) {
      this.intervals.push({ id: obj.id, value: obj.aabb.min.x, isStart: true });
      this.intervals.push({ id: obj.id, value: obj.aabb.max.x, isStart: false });
    }
    // Tri par position croissante
    this.intervals.sort((a, b) => a.value - b.value);
  }

  getPairs() {
    const pairs = [];
    const active = new Set();
    for (const interval of this.intervals) {
      if (interval.isStart) {
        // Cet objet démarre : il chevauche tous les actifs
        for (const id of active)
          pairs.push([id, interval.id]);
        active.add(interval.id);
      } else {
        active.delete(interval.id);
      }
    }
    return pairs;
  }
}
```

```

}
}

```

💡 Cohérence temporelle (temporal coherence)

D'une frame à l'autre, les objets bougent peu. La liste triée est donc **presque** la même. Au lieu d'un tri complet ($O(N \log N)$), on utilise un **tri par insertion** ($O(N)$ sur une liste presque triée). C'est ce qui rend SAP si rapide en pratique : le coût amorti est quasi linéaire.

💬 SAP multi-axe (SAP 3D)

La version 1D génère des **faux positifs** : deux objets peuvent se chevaucher sur X mais pas sur Y. La version 3D exécute SAP sur les trois axes et ne retient que les paires qui se chevauchent sur **tous** les axes. On peut aussi n'utiliser qu'un seul axe (le plus discriminant) et filtrer ensuite avec le test AABB complet.

- **Complexité** : $O(N \log N)$ au pire, $O(N)$ amorti grâce à la cohérence temporelle.
- **Avantage** : exploite la cohérence temporelle, pas de paramètre de taille de cellule à régler, excellent pour des objets de tailles variées.
- **Inconvénient** : dégénère si tous les objets sont alignés sur le même axe (la liste active devient énorme).
- **Cas d'usage typique** : moteurs physiques professionnels (Bullet, Box2D utilise une variante), objets de tailles hétérogènes.

6. Tableau Comparatif des Structures

Critère	Grille Uniforme	Quadtree/Octree	SAP
Complexité construction	$O(N)$	$O(N \log N)$	$O(N \log N) / O(N)$ amorti
Complexité requête	$O(K)$ par objet	$O(\log N + K)$	$O(N + P)$ paires
Tailles hétérogènes	Mauvais	Bon	Bon
Monde infini	Possible (hash)	Difficile	Oui
Objets statiques	Excellent	Excellent	Bon
Objets dynamiques	Très bon	Coûteux (rebuild)	Excellent (cohérence)
Paramètres à régler	Taille de cellule	Seuil, profondeur max	Aucun
Difficulté d'implémentation	Facile	Moyenne	Moyenne

Tableau 4. – Comparaison des trois structures de Broad Phase les plus courantes. K = objets voisins, P = paires candidates.

7. Introduction à la Narrow Phase

Le rôle de la Narrow Phase

La Broad Phase donne une liste de **paires candidates**. La Narrow Phase teste chaque paire avec une géométrie **précise** pour confirmer la collision et calculer la normale de contact.

Tests courants (du moins cher au plus cher) :

- **Sphère vs Sphère** : $\|c_A - c_B\| < r_A + r_B$. Un seul test, $O(1)$.
- **AABB vs AABB** : 6 comparaisons (vu section 4).
- **Sphère vs AABB** : distance du centre au boîtier, clampé sur chaque axe.
- **OBB vs OBB** (Oriented Bounding Box) : théorème des axes séparateurs (SAT), jusqu'à 15 axes en 3D.
- **Maillage vs Maillage** : GJK (Gilbert–Johnson–Keerthi) + EPA pour la pénétration. Coûteux, réservé aux paires confirmées.

💡 Hiérarchie de tests

On applique toujours les tests du **moins cher au plus cher**. Si le test AABB échoue, on s'arrête — inutile de lancer GJK. C'est la même philosophie que l'entonnoir : chaque étage élimine des candidats à moindre coût.

8. TP : Broad Phase — Spatial Hashing

💡 Objectif du TP

Implémenter une Broad Phase par **grille uniforme (Spatial Hashing)** et le test de chevauchement **AABB**, puis comparer les performances avec la brute force $O(N^2)$. Le fichier de départ `session07_Broad_Part_1.js` fournit la scène Three.js, la GUI et le moteur de collision — il ne reste qu'à compléter trois fonctions.

Mise en place

- Ouvrir `session07_Broad_Part_1.html` dans le navigateur (via Live Server).
- La scène contient un cube 3D rempli de billes en mouvement avec rebonds sur les parois.
- La GUI propose : nombre de billes (10–3000), toggle Broad Phase ON/OFF, restitution, vitesse du temps, reset.
- Un compteur **FPS** et **Tests/frame** est affiché en temps réel.
- Au départ, la Broad Phase est activée mais les fonctions sont vides — tout passe en brute force. C'est normal, c'est ce que vous allez corriger.

Missions

Mission 1 — `SpatialGrid.insert(obj)`

Ranger chaque objet dans la bonne cellule de la grille. Utiliser `getKey(obj.position)` pour obtenir la clé "x,y,z", créer le tableau si la cellule n'existe pas, puis ajouter l'objet.

Mission 2 — `SpatialGrid.getNeighbors(pos)`

Récupérer les objets situés dans la cellule de `pos` et les **26 cellules voisines** ($3 \times 3 \times 3$). Boucler sur `dx`, `dy`, `dz` de -1 à +1, construire la clé et récupérer le contenu. Retourner le tableau des voisins.

Mission 3 — `aabbOverlap(aabbA, aabbB)`

Implémenter le test de chevauchement AABB vu en section 4 : 6 comparaisons, une par axe. Renvoyer `false` dès qu'un axe ne se chevauche pas, `true` si les trois se chevauchent.

Mission 4 — Benchmark

Une fois les trois fonctions complétées, comparer Broad Phase ON vs OFF avec différents nombres de billes. Observer le compteur **Tests/frame** : il doit chuter drastiquement quand la Broad Phase est active.

Scénarios à observer

- **Faible densité (50 billes)** : la brute force reste fluide. La Broad Phase est légèrement plus rapide mais la différence est faible.
- **Forte densité (1000+ billes)** : la brute force s'effondre (≈ 500000 tests/frame), la Broad Phase reste fluide.
- **Extrême (3000 billes)** : avec Broad Phase ON, la simulation reste interactive ; avec OFF, elle devient un diaporama. C'est la démonstration concrète de l'entonnoir.
- **Compteur Tests/frame** : avec Broad Phase, le nombre de tests doit être proportionnel au nombre de **voisins réels**, pas à N^2 . Comparer les chiffres affichés.

⚠ Pièges à éviter

- Ne pas oublier d'appeler `grid.clear()` au début de chaque frame avant de réinsérer les objets.
- La fonction `getNeighbors` doit couvrir les **27** cellules ($3 \times 3 \times 3$), pas seulement la cellule courante — un objet à cheval entre deux cellules doit être détecté.
- Le test AABB doit renvoyer `false` **dès le premier axe qui ne se chevauche pas** — ne pas tester les trois axes si le premier échoue (optimisation).
- Faire attention aux **doublons** : le code fourni utilise `ballB.id <= ballA.id` pour éviter de tester chaque paire deux fois. Ne pas retirer cette garde.

Bonus

Ajouter un **Octree** comme troisième option dans la GUI (en plus de Brute Force et Grille). Comparer les performances lorsque les billes sont concentrées dans un coin du cube (distribution non uniforme). L'Octree devrait mieux gérer cette situation car il subdivise les zones denses.

Session 8 : Intégration de Verlet et Position Based Dynamics

Objectifs de la session

- Comprendre **pourquoi** l'intégration d'Euler explose sur les systèmes de particules (ressorts, tissus).
- Maîtriser l'intégration de **Verlet** : vitesse implicite, stabilité, dérivation de la formule.
- Appliquer un **damping** adapté à Verlet pour dissiper l'énergie.
- Découvrir **Position Based Dynamics** (PBD) : résoudre des contraintes de distance en manipulant directement les positions.
- Assembler ces briques dans un TP : un drapeau animé en tissu.

1. Le Problème : Pourquoi Euler Explode

Rappel : intégration d'Euler explicite

À chaque pas de temps, Euler met à jour position et vitesse séparément :

$$x_{n+1} = x_n + v_n \cdot dt \quad ; \quad v_{n+1} = v_n + a_n \cdot dt$$

C'est simple, mais la position et la vitesse sont **décalées** : on intègre la vitesse avec l'accélération **d'avant** le déplacement.

La loi de Hooke (rappel — la source du problème)

Un ressort exerce une force de rappel proportionnelle à l'étirement :

$$\vec{F} = -k \cdot (L - L_0) \cdot \hat{u}$$

- k : raideur, L : longueur actuelle, L_0 : longueur au repos, $\hat{u}$: direction unitaire.

Pour un tissu rigide, on veut un k **élevé**. Mais avec Euler, un grand k rend le système **instable** : l'énergie augmente à chaque pas jusqu'à l'explosion.

*Force de Hooke — ce qu'on *évitera* dans le TP.*

```
// Force de rappel appliquée à la particule `a` par le ressort (a,b)
function springForce(a, b, k, restLength) {
  const delta = b.clone().sub(a);
  const L = delta.length();
  const u = delta.normalize(); // direction unitaire hat(u)
  return u.multiplyScalar(-k * (L - restLength)); // -k * (L - L0) * u
}
```

Ce code fonctionne, mais pour l'utiliser avec Euler sans explosion, il faudrait un dt minuscule ou un k faible (tissu mou). C'est inacceptable pour un jeu à 60 FPS.

! La solution de cette session

Au lieu de **calculer des forces** (Hooke) puis d'intégrer (Euler), on va :

1. Intégrer avec **Verlet** — une méthode **symplectique** qui ne dérive pas en énergie.

2. Maintenir la rigidité avec **PBD** — on corrige directement les positions, sans aucune force ni raideur k .

Résultat : un tissu stable à n'importe quel dt , sans paramètre k à régler.

2. L'Intégration de Verlet

Idée de Verlet

Verlet ne stocke **pas** la vitesse explicitement. La vitesse est **déduite** de la différence entre la position actuelle et la position précédente :

$$v \approx \frac{x_n - x_{n-1}}{dt}$$

La formule d'intégration devient :

$$x_{n+1} = x_n + (x_n - x_{n-1}) + a \cdot dt^2$$

🧐 D'où vient la formule ?

Par développement de Taylor de $x(t)$ autour de t_n :

$$x(t_n + dt) = x_n + v_n \cdot dt + \frac{1}{2}a_n \cdot dt^2 + O(dt^3)$$

$$x(t_n - dt) = x_n - v_n \cdot dt + \frac{1}{2}a_n \cdot dt^2 + O(dt^3)$$

En additionnant les deux, les termes en v_n s'annulent :

$$x(t_n + dt) + x(t_n - dt) = 2x_n + a_n \cdot dt^2 + O(dt^4)$$

D'où la formule de Verlet :

$$x_{n+1} = 2x_n - x_{n-1} + a_n \cdot dt^2$$

qui est équivalente à $x_n + (x_n - x_{n-1}) + a \cdot dt^2$. Notez que l'erreur passe de $O(dt^3)$ (Euler) à $O(dt^4)$ — Verlet est **plus précise**.

Verlet vs Euler

Critère	Euler explicite	Verlet
Vitesse stockée	Oui (v_n)	Non (implicite)
Précision	$O(dt^3)$	$O(dt^4)$
Stabilité énergie	Mauvaise (dérive)	Bonne (symplectique)
Déplacement manuel	Casse la vitesse	Vitesse s'ajuste seule
Coût mémoire	x, v	x, x_{prev}

Tableau 5. – Verlet est plus précise, plus stable, et tolère qu'on déplace les particules à la main — exactement ce dont PBD a besoin.

❗ Pourquoi PBD exige Verlet

En PBD, on **déplace directement** les positions pour satisfaire les contraintes (section 4). Avec Euler, ce déplacement manuel n'affecte pas v — la vitesse devient incohérente avec la position, et le système « explose » à la frame suivante. Avec Verlet, la vitesse étant $x_n - x_{n-1}$, **toute correction de position met à jour automatiquement la vitesse**. C'est le mariage parfait.

Utilisations typiques de Verlet : cordes, tissus, chevelures, tas de billes, ragdolls.

2.A. Implémentation de Verlet

Une particule Verlet possède :

- position : position actuelle x_n .
- prevPosition : position précédente x_{n-1} .
- acceleration : accélération cumulée des forces (gravité, vent, ...).
- pinned : booléen, fixée au décor (mât, clou, ...).

Une étape de Verlet pour une particule. Voici **exactement** la fonction à coller dans le TP (Mission 1).

```
function verletIntegrate(p, dt) {
    if (p.pinned) return;           // fixée : ne bouge pas
    const temp = p.position.clone();
    const velocity = p.position.clone().sub(p.prevPosition);
    p.position
        .add(velocity.multiplyScalar(DAMPING))    // vitesse implicite + damping
        .addScaledVector(p.acceleration, dt * dt); // a·dt²
    p.prevPosition.copy(temp);
}
```

- temp sauvegarde x_n avant de l'écraser, pour devenir x_{n-1} au prochain pas.
- velocity est $x_n - x_{n-1}$ (sans diviser par dt — le dt est absorbé dans $a \cdot dt^2$).
- DAMPING (section 3) est un coefficient ≤ 1 appliqué à la vitesse.

3. Le Damping (Amortissement)

Force d'amortissement (forme classique)

On ajoute une force opposée à la vitesse pour dissiper l'énergie :

$$\vec{F}_{\text{damping}} = -b \cdot \vec{v}$$

Sans damping, un système isolé oscillerait indéfiniment (aucune perte).

Damping en Verlet — forme implicite

En Verlet, la vitesse est **implicite** : $\vec{v} \approx x_n - x_{n-1}$. Au lieu d'ajouter une force $-b \cdot \vec{v}$ puis de réintégrer, on **multiplie directement** la vitesse par un coefficient $d \leq 1$:

$$\vec{v}_{\text{damped}} = d \cdot (x_n - x_{n-1})$$

- $d = 1$: aucun amortissement (oscillations perpétuelles).
- $d < 1$: une fraction de la vitesse est perdue à chaque frame.
- $d = 0$: la particule n'a plus d'inertie (tombe comme un bloc).

C'est cette forme — **un simple coefficient** — que vous utiliserez dans le TP.

Damping en Verlet.

```
const DAMPING = 0.99;    // 1 = aucun amortissement, <1 = dissipation

// Dans la boucle d'intégration :
//   velocity = position - prevPosition
//   position += velocity * DAMPING    // <-- amortissement
```

Le `multiplyScalar(DAMPING)` dans `verletIntegrate` (section 2.A) applique exactement cela.

💡 Choix du coefficient

Pour un drapeau, $d \approx 0.99$ donne un flottement naturel (l'énergie du vent compense les pertes). Pour une corde qui doit s'arrêter, $d \approx 0.9$ à 0.95 . Trop bas (< 0.9), le mouvement devient « pâteux » et artificiel.

4. Position Based Dynamics (PBD)

Principe de PBD

Au lieu de calculer des forces complexes (Hooke) et de les intégrer, on **corrige directement les positions** des particules pour qu'elles respectent des contraintes géométriques : distance, angle, volume, etc.

Une **contrainte** est une règle du type « p_1 et p_2 doivent être à distance L_0 ». Si la règle est violée, on déplace les particules pour la restaurer.

Contrainte de distance — la mathématique

Soient deux particules p_1, p_2 à distance $L = \|p_2 - p_1\|$. On veut $L = L_0$. L'écart est $\Delta L = L - L_0$. On corrige chaque particule de la moitié de l'écart, le long de la direction unitaire $\hat{u} = \frac{p_2 - p_1}{L}$:

$$p_1 \rightarrow p_1 + \frac{1}{2}\Delta L \cdot \hat{u} \quad ; \quad p_2 \rightarrow p_2 - \frac{1}{2}\Delta L \cdot \hat{u}$$

Si une seule particule est mobile, elle absorbe **tout** l'écart (ΔL au lieu de $\Delta \frac{L}{2}$). Si les deux sont fixées, on ne corrige rien.

Algorithme d'une frame PBD

1. **Intégration** — appliquer Verlet à toutes les particules (gravité, vent, damping).
2. **Contraintes** — pour chaque contrainte, corriger les positions (fonction `satisfyConstraint`).
3. **Itérations** — répéter l'étape 2 N fois pour converger (voir ci-dessous).
4. **Rendu** — afficher les positions corrigées.

Contrainte de distance PBD. Voici **exactement** la fonction à coller dans le TP (Mission 2).

```
function satisfyConstraint(p1, p2, restLength) {
  const delta = p2.position.clone().sub(p1.position);
  const dist = delta.length();
  if (dist === 0) return;
  const diff = (dist - restLength) / dist;          // écart relatif
  const correction = delta.multiplyScalar(0.5 * diff);
  if (!p1.pinned) p1.position.add(correction);
  if (!p2.pinned) p2.position.sub(correction);
}
```

- delta est $p_2 - p_1$; dist est L ; diff est $\Delta \frac{L}{L}$ (écart **relatif**).
- correction = delta * 0.5 * diff vaut $\frac{1}{2}\Delta L \cdot \hat{u}$ — exactement la formule ci-dessus.
- Les gardes !pinned gèrent les particules fixées : si une seule est mobile, elle reçoit la correction complète (l'autre add/sub est sautée).

💡 Itérations = raideur (le point clé de PBD)

Une **seule** passe de contraintes ne restaure pas exactement L_0 : corriger une contrainte en déplace les voisines, qui ne sont plus à leur distance de repos. C'est un système **couplé**.

La solution : **itérer**. En répétant `satisfyConstraint` N fois par frame, les corrections se propagent et le système converge vers une solution où **toutes** les contraintes sont proches de L_0 .

- $N = 1$: tissu très élastique, la gravité l'étire.
- $N = 5$: tissu mou mais cohérent.
- $N = 15$ à 20 : tissu rigide, réaliste pour un drapeau.
- $N = 50$: quasi-rigide (corde tendue).

Pas de paramètre k : la raideur émerge du nombre d'itérations. C'est ce qui rend PBD si robuste — et si utilisé dans les moteurs modernes (PhysX, Bullet, Havok).

💡 Pourquoi PBD est inconditionnellement stable

On ne fait que **déplacer** des positions d'une quantité bornée ($\Delta \frac{L}{2}$). Il n'y a aucune force qui puisse diverger, aucun k trop grand, aucun dt critique. Le pire cas (N faible) donne un tissu mou — jamais une explosion. À comparer avec Hooke + Euler, où un k un peu trop grand fait tout sauter.

5. TP : Drapeau en Tissu (Verlet + PBD)

💡 Objectif du TP

Animer un drapeau accroché à un mât. Le tissu est une grille de particules liées par des contraintes de distance. Le fichier de départ `session08_cloth.html` fournit la scène Three.js, le mât, la géométrie du drapeau, le vent et la GUI — il ne reste qu'à compléter **deux fonctions** en copiant-collant le code des sections 2.A et 4.

Mise en place

- Ouvrir `session08_cloth.html` dans le navigateur (via Live Server).
- La scène contient un drapeau rouge accroché à un mât vertical.
- Au départ, les deux fonctions sont vides : le drapeau tombe comme un bloc (la gravité est appliquée mais aucune contrainte ne maintient le tissu). C'est normal.
- La GUI propose : intensité du vent, damping, itérations PBD, fixer/libérer la colonne gauche, reset, FPS.

Missions

Mission 1 — `verletIntegrate(p, dt)`

Copier le code de la section 2.A dans le corps de la fonction. Attention : la particule fixée (`p.pinned`) ne doit pas bouger. La vitesse implicite est `position - prevPosition`, multipliée par `DAMPING`, puis on ajoute `acceleration * dt²`.

Mission 2 — `satisfyConstraint(p1, p2, restLength)`

Copier le code de la section 4. Calculer `delta = p2.position - p1.position`, la distance, l'écart relatif, et corriger chaque particule de la moitié — sauf les particules `pinned`, qui ne bougent pas.

Mission 3 — Observer et régler

Une fois les deux fonctions en place :

- **Itérations PBD** à 1 : le drapeau s'étire et se déchire visuellement (contraintes molles).
- **Itérations PBD** à 15–20 : le drapeau reste rigide et ondule avec le vent.
- **Damping** à 1.0 : oscillations perpétuelles. À 0.95 : le drapeau se stabilise.
- **Vent** à 0 : le drapeau pend sous la gravité. À 3 : il flotte horizontalement.
- Décocher **Fixer colonne gauche** : le drapeau se détache du mât et tombe (les contraintes restent, plus rien ne le tient).

Scénarios à observer

- **Drapeau qui flotte** : avec vent ≈ 2 , damping ≈ 0.99 et 15 itérations, le drapeau ondule de façon réaliste. C'est exactement la technique utilisée pour les drapeaux et capes dans les jeux.
- **Tissu qui se déchire** : avec 1 itération, les contraintes ne convergent pas — la gravité étire le tissu. Cela montre **pourquoi** on itère en PBD (section 4).
- **Stabilité** : aucun k à régler, aucune explosion. Verlet + PBD est inconditionnellement stable — c'est toute la motivation de la session (section 1).

⚠ Pièges à éviter

- Ne pas oublier le `if (p.pinned) return;` dans `verletIntegrate` — sinon le mât s'envole avec le drapeau.
- Dans `satisfyConstraint`, tester `!p1.pinned` et `!p2.pinned` **séparément** : si les deux sont fixées, on ne corrige rien ; si une seule est fixée, l'autre absorbe toute la correction (le code de la section 4 gère ce cas).
- Garder le `if (dist === 0) return;` pour éviter une division par zéro lorsque deux particules se superposent.
- Ne pas modifier l'ordre $p1/p2$ du `delta` : la correction doit aller dans le bon sens (le code de la section 4 est déjà correct).

Bonus

Ajouter des contraintes **diagonales** dans `initCloth` (en plus des horizontales et verticales) avec `restLength = SPACING * Math.sqrt(2)`. Observer : le tissu résiste mieux au cisaillement et ne se déforme pas en losange. C'est ce qu'on appelle la **shear constraint** dans les simulateurs de tissu.

Session 9 : TP — Moteur de Particules (VFX)

Objectifs du TP

- Construire un **moteur de particules** : émettre, mettre à jour, recycler.
- Modéliser les **forces** sur une particule : gravité, drag, flottabilité, turbulence.
- Intégrer avec **Euler** — et comprendre **pourquoi** c'est légitime ici (vs Verlet en session 8).
- Gérer la **collision** avec le sol : impulsion, restitution, perte d'énergie.
- Utiliser un **object pool** pour éviter les allocations par frame.
- Produire trois effets avec le **même** moteur : pluie, feu, fumée.

💡 Contexte

Un système de particules simule un grand nombre de petits objets éphémères : pluie, feu, fumée, étincelles. Contrairement au tissu de la session 8, chaque particule vit peu de temps et est très amortie — l'intégration d'Euler suffit, et la physique (forces, collisions) est le cœur du travail. Le rendu (shader, BufferGeometry) est **fourni**.

1. La Particule et l'Object Pool

Propriétés d'une particule

- position, velocity (Vector3)
- color (Color), size, alpha (transparence)
- life (temps restant), maxLife (durée totale)
- alive (booléen — active ou recyclable)

💡 Object Pooling — le point clé

Créer/détruire des objets à chaque frame provoque des pics de **garbage collection** et des saccades. On alloue donc **une fois** un tableau fixe de MAX_PARTICLES, et on **recycle** les particules mortes en les réinitialisant plutôt qu'en les supprimant. Zéro allocation pendant le jeu.

```
const pool = [];
for (let i = 0; i < MAX_PARTICLES; i++) {
  pool.push({ position: new THREE.Vector3(), velocity: new THREE.Vector3(),
    color: new THREE.Color(), alpha: 0, size: 1,
    life: 0, maxLife: 1, alive: false });
}
```

2. L'Émetteur

Emitter

Génère des particules à un **taux** donné (ex: 300/s) à partir d'une position emitterOrigin. La **forme** de l'émetteur contrôle la position et la direction initiales :

- **Point** : tout part du même endroit, direction fixe.
- **Cône** : direction déviée aléatoirement dans un cône d'ouverture spread.
- **Sphère** : position aléatoire sur une sphère, direction = normale sortante.

💡 Taux précis avec un accumulateur

On ne peut pas émettre un nombre fractionnel de particules par frame. On accumule le reste : `spawnAccumulator += rate * dt`, on émet `floor(spawnAccumulator)`, et on soustrait ce qu'on a émis. Le taux reste exact en moyenne.

3. Le Cycle de Vie

Cycle d'une particule

1. **Naissance** — `spawnParticle(p)` : position, vitesse, `life = maxLife`, couleur de départ.
2. **Update** — `updateParticle(p, dt)` : forces (gravité, drag), intégration d'Euler, `life -= dt`.
3. **Rendu** — `updateBuffers()` : recopie l'état dans la `BufferGeometry`.
4. **Mort / Recyclage** — quand `life <= 0`, `alive = false` puis `recycleParticle(p)` relance la particule à l'émetteur.

💬 Euler, pas Verlet — et pourquoi ici c'est légitime

En session 8, on a **banni** Euler au profit de Verlet pour le tissu : Euler dérive en énergie, et un drapeau qui oscille pendant des heures finit par exploser. Pour la VFX, on **revient** à Euler — délibérément. Trois raisons :

1. **Durée de vie courte.** Une particule de feu vit ~ 1 s, une goutte de pluie ~ 1.5 s. La dérive d'Euler est proportionnelle au temps d'intégration — sur 1 s, elle est **invisible**, quand sur 10 min elle faisait exploser le tissu.
2. **Amortissement fort.** Le drag dissipe l'énergie à chaque frame. Même si Euler en ajoute un peu, le drag en retire plus — le système est **globalement dissipatif**, pas conservatif. C'est l'inverse du tissu, où l'énergie devait être conservée.
3. **Pas de contraintes couplées.** Verlet brillait pour PBD (corriger des positions satisfait les contraintes automatiquement). Une particule VFX est **libre** — aucune contrainte de distance à maintenir. Euler suffit.

Quand même utiliser Verlet pour des particules ? Si on relie les particules entre elles (corde, tissu, ragdoll) — mais alors on retombe sur le cas de la session 8.

Le pas de temps en VFX

On utilise souvent un `dt` **fixe** (ex: $\frac{1}{60}$ s) plutôt que le vrai `dt` variable. Pourquoi ? Un `dt` variable peut donner des comportements différents selon le FPS (une étincelle qui va plus loin à 30 FPS qu'à 60 FPS). Avec un `dt` fixe, la simulation est **reproductible** — important pour déboguer un effet. Le prix : une légère désynchronisation physique/rendu, négligeable pour la VFX.

4. Forces sur une Particule

Le cœur physique du moteur est dans `updateParticle` : à chaque frame, on calcule l'accélération totale, on intègre. La qualité de l'effet dépend entièrement des **forces** qu'on choisit.

4.A. Gravité

Gravité constante

La force la plus simple : $\vec{F}_g = m \cdot \vec{g}$, avec $\vec{g} = (0, -9.81, 0)$ « m/s² ». L'accélération est $\vec{a} = \vec{g}$ (indépendante de la masse — Galilée). Pour la pluie, c'est l'essentiel.

4.B. Drag — linéaire vs quadratique

Drag linéaire (Stokes)

Force opposée à la vitesse, **proportionnelle** à v :

$$\vec{F}_{\text{drag}} = -k \cdot \vec{v}$$

- k : coefficient de frottement (kg/s).
- Valable pour des **petits** objets **lents** dans un fluide visqueux (goutte de brouillard, particule de fumée).
- La vitesse **terminale** est $v_\infty = \frac{g}{k}$: la particule cesse d'accélérer.

Drag quadratique (turbulent)

Pour des objets plus gros ou plus rapides, la traînée dépend du carré de la vitesse :

$$\vec{F}_{\text{drag}} = -\frac{1}{2} \rho C_d A \cdot \|\vec{v}\| \cdot \vec{v}$$

- ρ : densité du fluide, C_d : coefficient de traînée, A : section.
- On simplifie en $\vec{F}_{\text{drag}} = -k \cdot \|\vec{v}\| \cdot \vec{v}$.
- Valable pour une **goutte de pluie** réelle ($v \sim 9$ m/s) ou un projectile.
- La vitesse terminale est $v_\infty = \sqrt{\frac{2mg}{\rho C_d A}}$.

🧐 Lequel choisir ?

Dans le TP, on utilise le drag **linéaire** (plus simple, stable avec Euler). Pour la pluie réaliste, le drag quadratique donne une vitesse terminale plus nette. La différence visuelle : avec le drag linéaire, la particule ralentit **graduellement** ; avec le quadratique, elle **plafonne** brusquement à v_∞ .

Implémentation des deux drags.

```
// Drag linéaire (utilisé dans le TP)
const dragLin = velocity.clone().multiplyScalar(-k);

// Drag quadratique (bonus)
const speed = velocity.length();
const dragQuad = velocity.clone().multiplyScalar(-k * speed);

// L'accélération totale :
// a = g + F_drag / m
```

4.C. Flottabilité — pourquoi le feu monte

Poussée d'Archimède

Un volume V de fluide (air chaud, fumée) subit une force **vers le haut** égale au poids de l'air déplacé :

$$\vec{F}_{\text{buoy}} = -\rho_{\text{air}} \cdot V \cdot \vec{g}$$

(le signe $-$ car $\vec{g}$ pointe vers le bas, la poussée est vers le haut).

Le poids de la particule elle-même est $\vec{F}_g = m \cdot \vec{g}$. La force **nette** verticale est :

$$\vec{F}_{\text{net}} = (m - \rho_{\text{air}} V) \cdot \vec{g} = m \cdot \vec{g} \cdot \left(1 - \frac{\rho_{\text{air}}}{\rho_{\text{particule}}}\right)$$

puisque $\rho_{\text{particule}} = \frac{m}{V}$.

! Le raccourci du TP

Plutôt que de modéliser densité et volume, on **fusionne** la gravité et la flottabilité en une seule « gravité effective » :

$$g_{\text{eff}} = g \cdot \left(1 - \frac{\rho_{\text{air}}}{\rho_{\text{particule}}}\right)$$

- Si $\rho_{\text{particule}} > \rho_{\text{air}}$ (pluie, étincelle) : $g_{\text{eff}} < 0$ — ça tombe.
- Si $\rho_{\text{particule}} < \rho_{\text{air}}$ (feu, fumée) : $g_{\text{eff}} > 0$ — ça monte.

C'est exactement ce que fait le preset fire avec gravity: +2.5 : une gravité **positive** = flottabilité nette vers le haut. Pas un hack — une simplification légitime d'Archimède.

4.D. Turbulence — vent et flicker

Force de turbulence

Pour que la fumée ondule et le feu crépite, on ajoute une force **pseudo-aléatoire** qui varie dans le temps et l'espace. Une approche simple et peu coûteuse : des sinusoïdes déphasées (comme le vent du drapeau en session 8) :

$$\vec{F}_{\text{turb}}(\vec{r}, t) = A \cdot (\sin(\omega_1 t + k_1 x), \sin(\omega_2 t + k_2 y), \sin(\omega_3 t + k_3 z))$$

- A : amplitude, ω_i : pulsations, k_i : nombres d'onde (variations spatiales).
- **Pas** du vrai bruit de Perlin — mais visuellement convaincant et $O(1)$ par particule.

Turbulence dans updateParticle.

```
const t = performance.now() * 0.001;
const turbX = TURBULENCE * Math.sin(t * 3.0 + p.position.y * 1.5);
const turbZ = TURBULENCE * Math.sin(t * 4.0 + p.position.x * 1.2);
// Ajouter à l'accélération (en plus de la gravité et du drag)
```

```
ax += turbX;
az += turbZ;
```

Régler TURBULENCE à 0 pour de la pluie (trajectoires droites), à ~ 3 pour de la fumée (ondulations), à ~ 5 pour un feu qui crépite.

5. Collision avec le Sol

Détection et réponse

Quand une particule traverse le sol ($y < y_{\text{sol}}$) :

1. **Détection** : `if (p.position.y < GROUND_Y)`
2. **Correction de position** : `p.position.y = GROUND_Y` (on remonte à la surface).
3. **Réponse en impulsion** : on inverse la composante normale de la vitesse, atténuée par un coefficient de **restitution** $e \in [0, 1]$:

$$v_{y'} = -e \cdot v_y$$

- $e = 1$: rebond parfait (énergie conservée) — irréaliste.
- $e = 0$: la particule s'écrase (perte totale d'énergie normale).
- $e \sim 0.3$: rebond mou, typique d'une goutte ou d'une étincelle.

🧐 Pourquoi seulement la composante y ?

Le sol est un plan **horizontal**. La normale du plan est $(0, 1, 0)$ — seule la composante v_y est affectée par le rebond. Les composantes v_x, v_z (tangentielles) sont **conservées** (en l'absence de frottement de surface). C'est la décomposition **normale/tangentielle** vue en session 6, appliquée à un plan.

Collision dans `updateParticle`.

```
const GROUND_Y = -2;
const RESTITUTION = 0.3;

// ... après intégration ...
if (p.position.y < GROUND_Y) {
    p.position.y = GROUND_Y;           // correction
    p.velocity.y = -RESTITUTION * p.velocity.y; // rebond
    // Bonus : éclaboussure — accélérer la mort
    if (Math.abs(p.velocity.y) < 0.5) p.life = Math.min(p.life, 0.2);
}
```

La dernière ligne simule l'éclaboussure : une goutte qui touche le sol avec une faible vitesse « meurt » vite (elle s'étale et disparaît).

6. Rendu — Points + Shader

🧐 Cette section est **fournie** — pas au programme du TP

Le rendu (BufferGeometry, shader, blending) est déjà implémenté dans le fichier de départ. Cette section explique **ce qui est en place** pour que vous compreniez le code fourni — vous n'avez **pas** à écrire le shader ni à gérer la BufferGeometry. Concentrez-vous sur la **physique** (sections 4 et 5).

`BufferGeometry` : la géométrie sur le GPU

En Three.js, une `BufferGeometry` est un conteneur de **tableaux de données brutes** (`Float32Array`) envoyés directement à la carte graphique. Pas de hiérarchie de sommets/faces : juste des **attributs** alignés en mémoire, un par propriété. C'est le format le plus proche du GPU — et le seul performant pour des milliers de particules.

Chaque **attribut** est un tableau plat où les données d'une particule i occupent un bloc contigu :

$$\text{position}_i = (\text{array}[3i], \text{array}[3i + 1], \text{array}[3i + 2])$$

Dans notre moteur, quatre attributs décrivent chaque particule :

Attribut	Dimension	Rôle
position	3 (xyz)	Position dans le monde — utilisée par le vertex shader pour placer le point.
color	3 (rgb)	Couleur — interpolée sur la vie (jaune → rouge pour le feu).
aSize	1	Taille en pixels du point sprite — atténuée par la distance dans le shader.
aAlpha	1	Transparence $\alpha \in [0, 1]$ — 0 = invisible (morte ou fondu final).

Tableau 6. – Quatre attributs par particule, stockés dans une `BufferGeometry` et lus par le shader.

🔗 Pourquoi `BufferGeometry` plutôt que des `Mesh` ?

Dessiner 3000 `Mesh` séparés = 3000 **draw calls** (un par sphère) — le CPU devient le goulot d'étranglement. Avec `THREE.Points` + une `BufferGeometry`, les 3000 particules partagent **une seule géométrie** et **un seul matériau** : c'est **un seul draw call**. Le GPU fait tout le travail en parallèle. C'est la différence entre 60 FPS et 5 FPS.

`THREE.Points` — un point par sommet

`THREE.Points` est l'objet qui dit à Three.js : « dessine chaque sommet de la géométrie comme un **point** (un carré de pixels), pas comme un triangle ». La taille du carré est contrôlée par `gl_PointSize` dans le **vertex shader** — d'où l'attribut `aSize`. Le **fragment shader** décide ensuite de la couleur et de la forme finale (disque doux via `discard + smoothstep`).

🔗 Comment le rasterizer sait-il quoi afficher ?

Pour un triangle, le rasterizer remplit l'intérieur des 3 sommets projetés. Mais un point n'a qu'**un** sommet — comment obtient-on des pixels ?

Le GPU a un mode primitif dédié (GL_POINTS) : à partir de la position projetée (gl_Position) et de la taille (gl_PointSize), il génère **automatiquement** un carré de $N \times N$ pixels centré sur le sommet. Pas de triangle, pas de géométrie supplémentaire — la taille **est** la géométrie.

Dans ce carré, chaque pixel reçoit une coordonnée $gl_PointCoord \in [0, 1]^2$ (centre = (0.5, 0.5)), **générée par le rasterizer** — pas un attribut qu'on fournit. C'est ce qui permet au fragment shader de calculer la distance au centre et de dessiner un disque :

```
vec2 uv = gl_PointCoord - 0.5;    // centré
if (length(uv) > 0.5) discard;    // carré -> disque
```

Comment le pipeline sait-il qu'il doit utiliser ce mode ? Ce n'est pas le shader qui décide — c'est le **type primitif du draw call**. Three.js choisit `gl.POINTS` vs `gl.TRIANGLES` selon la classe de l'objet (`THREE.Points` → points, `THREE.Mesh` → triangles). C'est cette constante, passée à `gl.drawArrays(...)`, qui active le mode point du rasterizer. Le **même** vertex shader pourrait tourner dans les deux modes — seul le draw call change.

4.A. Le Vertex Shader — Mesh vs Points

Jusqu'ici, toutes les sessions utilisaient `MeshStandardMaterial` : Three.js générerait le shader pour nous. Pour les particules, on écrit le nôtre — et il est **très différent**. Comparons.

Le shader « habituel » — `MeshStandardMaterial` (simplifié)

Voici l'essentiel de ce que Three.js génère pour un Mesh. On ne l'écrit jamais à la main, mais c'est ce qui tourne sous le capot depuis la session 1.

```
// Vertex shader (Mesh) — un sommet parmi les 3 d'un triangle
attribute vec3 position;    // lieu du sommet dans le modèle
attribute vec3 normal;      // normale pour l'éclairage
attribute vec2 uv;          // coordonnée de texture

uniform mat4 modelViewMatrix; // modèle -> vue (caméra)
uniform mat4 projectionMatrix; // vue -> écran (perspective)
uniform mat3 normalMatrix;    // transpose-inverse pour les normales

varying vec2 vUv;            // UV interpolée vers le fragment
varying vec3 vNormal;        // normale interpolée vers le fragment

void main() {
    vUv = uv;
    vNormal = normalize(normalMatrix * normal);
    gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    // Pas de gl_PointSize : la taille du triangle est déduite des 3 sommets
    // projetés. Le rasterizer remplit l'intérieur du triangle.
}
```

Trois choses à remarquer :

1. `normal` et `normalMatrix` servent à l'**éclairage** (diffuse, spéculaire) — calculé dans le fragment shader à partir de `vNormal`.
2. `uv` est interpolée **à travers le triangle** (varying) pour qu'on puisse sampler une texture par pixel.

3. **Aucune** `gl_PointSize` : la taille du triangle émerge de la projection des 3 sommets, le rasterizer remplit l'intérieur.

Notre shader Points — une particule = un sommet = un carré

Voici le vertex shader que vous trouverez dans `session09_particles.js`. Beaucoup plus court, et conceptuellement différent.

```
attribute vec3 position;    // position de la particule dans le monde
attribute vec3 color;      // couleur (vertexColors: true)
attribute float aSize;     // taille en pixels du sprite
attribute float aAlpha;    // transparence

uniform mat4 modelViewMatrix;
uniform mat4 projectionMatrix;
uniform float uPixelRatio; // retina / non-retina

varying vec3 vColor;
varying float vAlpha;

void main() {
    vColor = color;
    vAlpha = aAlpha;
    vec4 mv = modelViewMatrix * vec4(position, 1.0);
    gl_PointSize = aSize * uPixelRatio * (100.0 / -mv.z); // <-- la clé
    gl_Position = projectionMatrix * mv;
}
```

Ce qui change par rapport au Mesh :

1. **Pas de normal, pas d'uv, pas d'éclairage.** Une particule est un billboard plat face caméra — pas de notion de « surface qui reçoit la lumière ». La couleur vient directement de l'attribut `color`.
2. **`gl_PointSize` est obligatoire.** C'est lui qui décide de la taille du carré de pixels dessiné pour ce sommet. Sans cette ligne, rien ne s'affiche (taille = 0 par défaut).
3. **L'atténuation par la distance** (`100.0 / -mv.z`) : `mv.z` est la profondeur dans l'espace caméra (négative devant la caméra). Plus la particule est loin, plus `|-mv.z|` est grand, plus `gl_PointSize` est petit — comme un objet qui rapetisse avec la distance.
4. **Pas d'interpolation de varying à travers une surface.** `vColor` et `vAlpha` sont fixés **une fois** par le vertex shader ; seul `gl_PointCoord` (généré automatiquement par pixel du sprite) varie dans le fragment shader.

💡 Mesh vs Points en une ligne

Un Mesh projette **3 sommets** et remplit le **triangle** qu'ils forment (avec UVs, normales, éclairage).
 Un Point projette **1 sommet** et dessine un **carré de pixels** centré dessus, dont la taille est explicitement `gl_PointSize`. C'est pourquoi 3000 particules = 3000 invocations du vertex shader + 1 draw call, contre 3000 sphères = 300 000 sommets + 3000 draw calls.

Le pipeline en une frame

1. `updateBuffers()` recopie l'état du pool (CPU) vers les `Float32Array` des attributs.
2. On flaggue `attribut.needsUpdate = true` pour signaler à Three.js qu'il faut re-uploader les données — sinon le GPU garde l'ancienne version.
3. `renderer.render()` lance **un** draw call : le vertex shader tourne pour chaque particule, le fragment shader pour chaque pixel des points.

💡 Pourquoi ``needsUpdate`` ? CPU et GPU ont des mémoires séparées

Le `Float32Array` qu'on modifie en JS vit dans la mémoire **CPU**. Le GPU, lui, garde sa propre copie des attributs dans sa **VRAM**. Quand on mute le tableau JS, le GPU ne « voit » rien — il faut explicitement **uploader** les nouvelles données.

`needsUpdate` est le **flag Three.js** qui déclenche cet upload : au prochain `render()`, Three.js re-copie le tableau vers le GPU puis remet le flag à `false`. C'est une commodité Three.js — en WebGL brut, on appellerait `gl.bufferSubData(...)` à la main. La **contrainte** (mémoires séparées) est matérielle ; le **flag** est un détail d'API.

Trois idées de rendu

- **THREE.Points** + `BufferGeometry` : un seul draw call pour des milliers de particules. Attributs par particule : `position`, `color`, `aSize`, `aAlpha`.
- **Disque doux** : le fragment shader écarte (`discard`) ce qui dépasse un cercle et adoucit le bord avec `smoothstep` — sinon on a des carrés moches.
- **Couleur / taille / alpha sur la vie** : avec $t = \frac{\text{life}}{\text{maxLife}}$, on interpole la couleur (départ → fin), l'alpha (fendu) et la taille. C'est ce qui fait qu'un feu passe du jaune au rouge sombre en s'estompant.

💡 Additive blending pour le feu

`material.blending = THREE.AdditiveBlending` additionne les couleurs des particules superposées : les flammes se cumulent et **brillent**. Indispensable pour le feu, à éviter pour la pluie/fumée (qui doivent rester opaques).

💡 Et avec WebGPU ? (pour culture)

Notre TP utilise WebGL, où la physique tourne sur le **CPU** (en JS) et les données sont **uploadées** vers le GPU à chaque frame (`needsUpdate`). C'est suffisant pour quelques milliers de particules.

WebGPU, l'API moderne qui remplace WebGL, change deux choses :

1. **Compute shaders** : la physique (gravité, drag, collisions) peut tourner **entièrement sur le GPU** dans un programme générique. Les données ne quittent jamais la VRAM — le CPU ne fait rien par frame. C'est ainsi qu'on simule des **millions** de particules.

2. **Fin de GL_POINTS** : WebGPU n'a plus de primitive « point » intégrée. On dessine les particules avec du **instancing** — un quad (2 triangles) répliqué par particule, positionné et dimensionné dans le vertex shader. Plus de `gl_PointSize` ni `gl_PointCoord` — on utilise de vraies UVs.

La **physique** que vous apprenez ici (forces, intégration, collisions) reste identique — seul l'endroit où elle s'exécute change. Le dossier `three.js/examples/` du projet contient des démos WebGPU de particules (`webgpu_compute_particles_snow.html`, `_rain.html`, `_fluid.html`) si vous voulez voir la différence.

7. TP — Moteur de Particules

🔗 Objectif du TP

Compléter le fichier `session09_particles.html` (via Live Server). La scène, le pool, la `BufferGeometry`, le shader, la GUI et la boucle sont fournis — il reste **cinq fonctions** à écrire. Une fois en place, le sélecteur de **preset** (pluie / feu / fumée) donne trois effets très différents avec le même moteur.

Mise en place

- Ouvrir `session09_particles.html` dans le navigateur.
- Au départ, rien ne s'affiche : les 5 fonctions sont vides, aucune particule n'est initialisée. C'est normal.
- La GUI propose : taux d'émission, gravité, drag, lifetime, forme de l'émetteur, spread, preset, reset, FPS.

Missions

Mission 1 — `spawnParticle(p)`

Récupérer `{ pos, dir }` via `sampleEmitter`, placer la particule, donner une vitesse initiale (`ps.speed` avec un peu d'aléa), régler `life = maxLife = LIFETIME` (avec un peu d'aléa), la couleur de départ du preset, `alpha = 1`, `size = ps.size`, et `alive = true`.

Mission 2 — `updateParticle(p, dt)`

Appliquer gravité (section 4.A) + drag linéaire (4.B), intégrer en Euler, décrémenter `life`. Si `life <= 0`, `alive = false`. Calculer $t = \frac{life}{maxLife}$ et interpoler couleur / alpha / taille sur la vie. **Bonus** : ajouter la turbulence (4.D) et la collision avec le sol (section 5).

Mission 3 — `recycleParticle(p)`

Relancer la particule morte en appelant `spawnParticle(p)` — elle retourne à l'émetteur.

Mission 4 — `updateBuffers()`

Recopier `position`, `color`, `aSize`, `aAlpha` du pool vers les attributs de la `BufferGeometry`. Une particule morte a `aSize = aAlpha = 0` (invisible). Flagger `needsUpdate = true`.

Mission 5 — `sampleEmitter(shape, spread)`

Renvoyer `{ pos, dir }` selon la forme. Le preset fournit `emitterPos` et `dir` de base. **Point** : `pos = emitterPos`, `dir = dir` du preset. **Cône** : `dir = dir` du preset + déviation aléatoire dans un cône

d'ouverture spread. **Sphère** : pos = point aléatoire sur une sphère de rayon spread autour de emitterPos, dir = normale sortante. Indices : `Vector3.randomDirection()` pour la sphère ; `Quaternion().setFromUnitVectors(+Z, dir)` pour orienter le cône.

Scénarios à observer

- **Pluie** : gravité négative forte (section 4.A), blending normal, forme point, émetteur en hauteur. Les gouttes tombent en ligne droite et disparaissent au sol. Avec la collision (section 5), elles rebondissent faiblement.
- **Feu** : gravité **positive** = flottabilité (section 4.C), **additive blending**, forme cône, émetteur près du sol. Les particules montent, passent du jaune au rouge sombre et s'estompent. Avec la turbulence (4.D), le feu crépite.
- **Fumée** : gravité positive légère (flottabilité faible), drag élevé (section 4.B), blending normal, forme cône large. Montée lente, gris qui fonce, longue durée de vie. La turbulence la fait onduler.
- **Euler vs Verlet** : observer qu'aucune explosion ne se produit même après des minutes — preuve que le drag dissipe l'énergie qu'Euler pourrait ajouter (section 3).
- **Performance** : avec 3000 particules et un pool, le FPS reste stable — preuve que l'object pooling évite les pics de GC.

⚠ Pièges à éviter

- **Ne pas allouer** de `Vector3/Color` dans `updateParticle` ou `updateBuffers` — c'est justement ce que le pool évite. Réutiliser les champs existants (`p.position.add(...)`, `p.color.lerpColors(...)`).
- Oublier `needsUpdate = true` sur les attributs modifiés → l'écran ne bouge pas.
- Une particule morte doit avoir `aAlpha = 0` (et idéalement `aSize = 0`), sinon elle reste visible à l'origine.
- Tester `life <= 0` **après** intégration, et bien passer `alive = false` pour déclencher le recyclage.
- Le `discard` dans le shader est essentiel : sans lui, `THREE.Points` dessine des carrés.

Bonus

- **Drag quadratique** (section 4.B) : remplacer le drag linéaire par $-k \cdot \|v\| \cdot v$ et observer la vitesse terminale plus nette de la pluie.
- **Turbulence** (section 4.D) : ajouter un slider `TURBULENCE` dans la GUI et l'appliquer dans `updateParticle`. Régler à 0 pour la pluie, 3 pour la fumée, 5 pour le feu.
- **Collision avec le sol** (section 5) : ajouter le rebond avec restitution dans `updateParticle`, plus l'éclaboussure (mort accélérée).
- **Preset étincelles** : gravité forte, lifetime très court (~ 0.5 s), additive blending, couleur blanc → jaune → rouge, turbulence élevée.
- **Attraction** : une force vers un point mobile (la souris) — $\vec{F} = k \cdot (\text{cible} - \text{position})$. Utile pour des étincelles orbitantes ou un trou noir.

Session 10 : Introduction à la Physique des Rotations

Objectifs de la session

- Passer de la **particule** (point sans dimension) au **corps rigide** (objet qui pivote).
- Comprendre le **couple**, le **moment d'inertie** et la **vitesse angulaire**.
- Relier les équations linéaires et angulaires — elles sont **analogues**.
- Modéliser la **friction de glissement vs roulement** — le cas de la boule de billard.
- Implémenter un corps rigide qui se déplace **et** tourne, avec intégration par quaternion.

💡 L'objectif du jour

Depuis le début du cours, on traite les objets comme des **points** — une position, une vitesse, une masse. Mais un vrai objet ne fait pas que se déplacer : il **pivote** autour de son centre de masse. Une boule de billard roule, un cube tombe en basculant, un personnage s'effondre en pivotant. Cette session introduit les outils pour modéliser la rotation — et on le fait avec un cas concret : **la boule de billard**.

Partie 1 : Du Point au Corps Rigide

Particule vs corps rigide

- **Particule** (sessions 1-9) : un point avec une masse m , une position $\vec{r}$, une vitesse $\vec{v}$. Pas d'orientation, pas de rotation. Toute la masse est concentrée en un point.
- **Corps rigide** : un objet avec une **étendue spatiale**. Il a un **centre de masse** (G), une **orientation** (rotation), et une **répartition de masse** qui détermine comment il résiste à la rotation.

Un corps rigide peut faire **deux choses simultanément** :

1. **Translater** — son centre de masse se déplace (comme une particule).
2. **Pivoter** — il tourne autour de son centre de masse.

💬 L'hypothèse rigide

On dit **rigide** parce que les points internes de l'objet ne bougent pas les uns par rapport aux autres — la forme ne se déforme pas. C'est une approximation : un vrai objet se déforme légèrement sous les forces. Mais pour une boule de billard, un cube de bois, un personnage en rigid body — c'est excellent.

Partie 2 : Le Dictionnaire Linéaire → Angulaire

💬 Tout ce que vous savez s'applique

La physique de la rotation est **exactement parallèle** à la physique de la translation. Chaque concept linéaire a un équivalent angulaire. Si vous comprenez $\vec{F} = m\vec{a}$, vous comprenez $\tau = I\alpha$.

Concept	Linéaire	Angulaire
Position	$\vec{r}$ (m)	Angle θ (rad) / Quaternion q
Vitesse	$\vec{v} = d \cdot \vec{r}/dt$ (m/s)	$\vec{\omega} = d \cdot \vec{\theta}/dt$ (rad/s)
Accélération	$\vec{a}$ (m/s ²)	$\vec{\alpha}$ (rad/s ²)
Résistance	Masse m (kg)	Moment d'inertie I (kg·m ²)
Cause du mouvement	Force $\vec{F}$ (N)	Couple $\vec{\tau}$ (N·m)
Loi du mouvement	$\vec{F} = m\vec{a}$	$\vec{\tau} = I\vec{\alpha}$
Quantité de mouvement	$\vec{p} = m\vec{v}$	Moment angulaire $\vec{L} = I\vec{\omega}$
Énergie cinétique	$1/2mv^2$	$1/2I\omega^2$

Tableau 7. – Analogie entre mouvement linéaire et rotation. Chaque équation linéaire a sa contrepartie angulaire — il suffit de remplacer les variables.

🔔 Mémoriser le tableau

La masse m mesure la **résistance à la translation** — plus m est grand, plus il est difficile d'accélérer. Le moment d'inertie I mesure la **résistance à la rotation** — plus I est grand, plus il est difficile de faire tourner. La force $\vec{F}$ cause la translation, le couple $\vec{\tau}$ cause la rotation. **Tout le reste suit.**

Partie 3 : Le Couple (Torque)

Définition

Le couple est l'équivalent angulaire de la force. Mais contrairement à une force, le couple dépend de **deux** choses :

1. La force appliquée $\vec{F}$.
2. Le **point d'application** — ou plus précisément, le **bras de levier** $\vec{r}$, la distance entre le centre de masse et le point où la force est appliquée.

En 3D, le couple est un produit vectoriel :

$$\vec{\tau} = \vec{r} \times \vec{F}$$

En 2D, il se simplifie à un scalaire :

$$\tau = r_x F_y - r_y F_x$$

💡 Pourquoi le point d'application compte

Pousser une porte près de la charnière ne produit presque rien. Pousser la même porte avec la même force **au bout de la poignée** la fait pivoter facilement. La force est la même, mais le **bras de levier** est différent — et le couple est proportionnel au bras de levier.

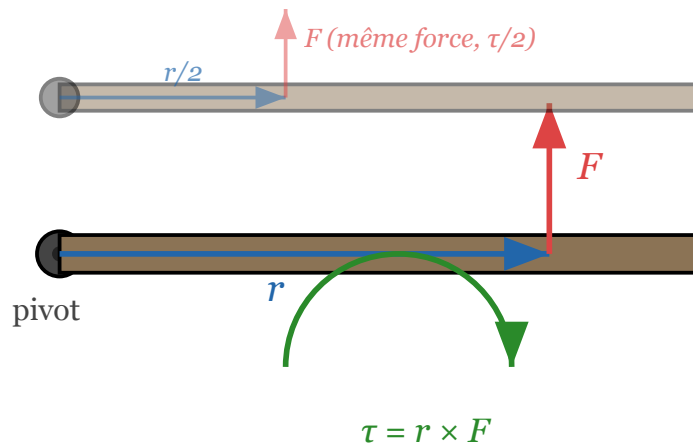


Fig. 10. – Le couple dépend du bras de levier. En haut : force au bout de la porte (r grand, τ grand). En bas : même force au milieu ($r/2$, τ divisé par 2). Pousser près du pivot produit peu de rotation.

Exemples intuitifs

Exemple 1 : La porte

Une porte de 0.8 « m » de large. On pousse avec 10 « N » :

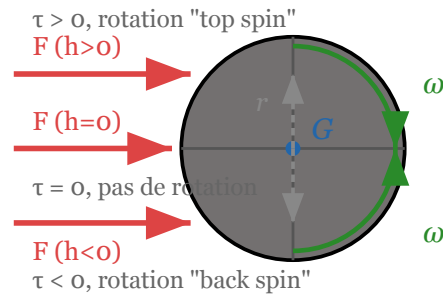
- **Au bout** ($r = 0.8$) : $\tau = 0.8 \times 10 = 8$ « N »·« m ». La porte s'ouvre facilement.
- **Près de la charnière** ($r = 0.1$) : $\tau = 0.1 \times 10 = 1$ « N »·« m ». La porte bouge à peine.
- **Sur la charnière** ($r = 0$) : $\tau = 0$. La porte ne tourne pas, peu importe la force.

Exemple 2 : La clé

Serrer un boulon avec une clé de 0.2 « m » et une force de 50 « N » perpendiculaire : $\tau = 0.2 \times 50 = 10$ « N »·« m ». Si on ajoute un prolongateur de 0.4 « m » : $\tau = 0.4 \times 50 = 20$ « N »·« m » — le double. C'est pourquoi les mécaniciens utilisent des clés longues.

Exemple 3 : La boule de billard

On frappe la boule avec une queue. Si on frappe **au centre** ($h = 0$), le bras de levier est nul — pas de couple, la boule glisse sans tourner. Si on frappe **haut** ($h > 0$), on crée un couple — la boule tourne. C'est le **top spin** (rotation vers l'avant). Si on frappe **bas** ($h < 0$), c'est le **back spin** (rotation vers l'arrière).



$$\tau = \mathbf{r} \times \mathbf{F} \rightarrow \omega = \tau / I$$

Fig. 11. – Coup de queue centré vs décentré sur une boule de billard. La hauteur d'impact h détermine le bras de levier r et donc le couple $\tau = r \times F$. Un coup centré ($h = 0$) ne produit pas de rotation ; un coup haut produit du top spin ; un coup bas produit du back spin.

Partie 4 : Le Moment d'Inertie

Définition

Le moment d'inertie I représente la **résistance d'un objet à la rotation** — exactement comme la masse résiste à la translation. Il dépend de la **répartition de la masse** par rapport à l'axe de rotation :

$$I = \sum_i m_i r_i^2$$

où m_i est la masse de chaque point et r_i sa distance à l'axe. Plus la masse est **loin** de l'axe, plus I est grand — et plus il est difficile de faire tourner.

💡 L'intuition du patineur

Un patineur qui tourne sur lui-même rapproche ses bras de son corps → il tourne **plus vite**.

Pourquoi ? Parce que rapprocher les bras diminue r_i pour les masses des bras, donc diminue I . Or le moment angulaire $L = I\omega$ se **conserv**e (pas de couple externe). Si I diminue, ω augmente — le patineur accélère. C'est la conservation du moment angulaire, l'équivalent de la conservation de quantité de mouvement.

Formules pour les formes courantes

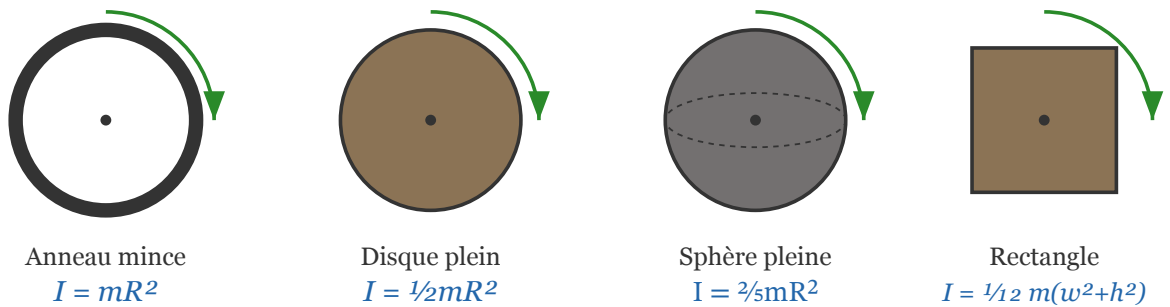


Fig. 12. – Moments d'inertie pour des formes courantes autour de leur centre. Notez comment la masse répartie **loin** de l'axe (anneau) donne un I plus grand que la masse concentrée au centre (disque).

Cas courants (autour du centre de masse)

- **Anneau mince** (masse sur le bord) : $I = mR^2$ — toute la masse est à distance R .
- **Disque plein / cylindre** : $I = \frac{1}{2}mR^2$ — la masse est répartie de 0 à R .
- **Sphère pleine** : $I = \frac{2}{5}mR^2$ — la masse est répartie en 3D.
- **Sphère creuse** : $I = \frac{2}{3}mR^2$ — masse concentrée sur la surface.
- **Rectangle** (autour du centre, dans le plan) : $I = \frac{1}{12}m(w^2 + h^2)$.
- **Tige fine** (autour du centre) : $I = \frac{1}{12}mL^2$.
- **Tige fine** (autour d'une extrémité) : $I = \frac{1}{3}mL^2$.

💡 La boule de billard

Une boule de billard est une **sphère pleine** de masse $m = 0.2$ « kg » et rayon $R = 0.5$ (unités de la démo) :

$$I = \frac{2}{5}mR^2 = \frac{2}{5} \times 0.2 \times 0.25 = 0.02 \text{ kg} \cdot \text{m}^2$$

C'est cette valeur que l'on utilise dans la démo [billiard.html](#).

Le théorème des axes parallèles

Théorème de Huygens-Steiner

Si on connaît I_{cm} autour du centre de masse, on peut calculer I autour de **n'importe quel axe parallèle** à une distance d :

$$I = I_{\text{cm}} + md^2$$

Exemple : une tige de masse m et longueur L a $I_{\text{cm}} = \frac{1}{12}mL^2$ autour de son centre. Autour d'une extrémité ($d = \frac{L}{2}$) :

$$I = \frac{1}{12}mL^2 + m\left(\frac{L}{2}\right)^2 = \frac{1}{12}mL^2 + \frac{1}{4}mL^2 = \frac{1}{3}mL^2$$

On retrouve la formule de la tige autour d'une extrémité. Le théorème **marche**.

Partie 5 : Vitesse d'un Point sur le Corps

Le point crucial

Quand un corps rigide se déplace **et** tourne, la vitesse de **chaque point** sur le corps est différente. Le centre de masse a une vitesse $\vec{v}_G$. Mais un point sur le bord a une vitesse **différente** — il subit la translation **plus** la rotation.

Vitesse d'un point P

Si un corps se déplace à $\vec{v}_G$ et tourne à $\vec{\omega}$, la vitesse d'un point P à la position $\vec{r}$ (relative au centre de masse) est :

$$\vec{v}_P = \vec{v}_G + (\vec{\omega} \times \vec{r})$$

En 2D, avec $\vec{r} = (r_x, r_y)$ et $\vec{\omega} = \omega$ (scalaire, perpendiculaire au plan) :

$$v_{P.x} = v_{G.x} - \omega \cdot r_y$$

$$v_{P.y} = v_{G.y} + \omega \cdot r_x$$

$$v_P = v_G + (\omega \times r)$$

Si $v_P \neq 0$: la boule glisse (friction cinétique)

Si $v_P = 0$: la boule roule sans glisser

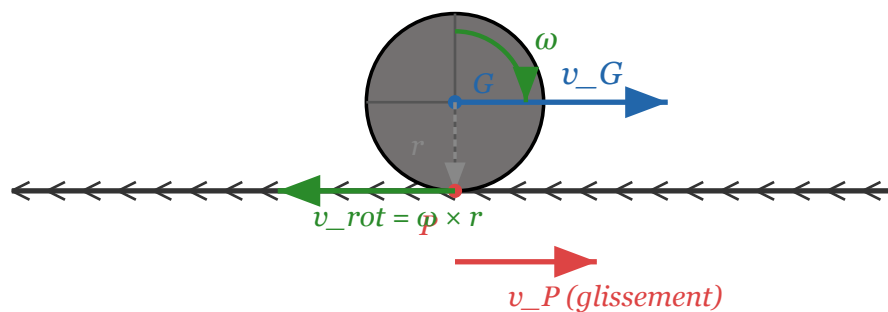


Fig. 13. – Vitesse du point de contact P sur une boule qui roule. Le centre G se déplace à $\vec{v}_G$ (bleu). La rotation $\vec{\omega}$ (vert) crée une vitesse $\vec{\omega} \times \vec{r}$ au point de contact (vert, vers l'arrière). La vitesse **réelle** du point de contact est $\vec{v}_P = \vec{v}_G + (\vec{\omega} \times \vec{r})$ (rouge). Si $\vec{v}_P \neq 0$, la boule **glisse**. Si $\vec{v}_P = 0$, la boule **roule sans glisser**.

Exemple : la boule qui glisse vs roule

Cas 1 : Glissement pur (pas de rotation)

On frappe la boule au centre ($h = 0$) $\rightarrow$ pas de couple $\rightarrow \omega = 0$. La boule se déplace à $\vec{v}_G$ mais ne tourne pas. Au point de contact :

$$\vec{v}_P = \vec{v}_G + (0 \times \vec{r}) = \vec{v}_G$$

Le point de contact **avance** à la même vitesse que le centre. La boule **glisse** sur le tapis — le tapis frotte.

Cas 2 : Roulement pur

La boule tourne **assez vite** pour que le point de contact soit **immobile** par rapport au sol. Condition :

$$\vec{v}_P = \vec{v}_G + (\vec{\omega} \times \vec{r}) = 0$$

$$\vec{v}_G = -(\vec{\omega} \times \vec{r}) = \vec{\omega} \times (-\vec{r})$$

En 2D, avec $\vec{r} = (0, -R)$ (contact en bas) :

$$v_G = \omega R$$

C'est la **condition de roulement sans glissement** : $v_G = \omega R$. La boule avance exactement à la vitesse qu'il faut pour que le point de contact soit immobile.

Cas 3 : Glissement + rotation (cas général)

La boule a $\vec{v}_G$ et ω , mais $v_G \neq \omega R$. Le point de contact a une vitesse **non nulle** — c'est la **vitesse de glissement**. La friction va agir pour **réduire** cette vitesse de glissement jusqu'à atteindre le roulement pur. C'est ce qu'on modélise dans la démo.

Partie 6 : Friction — Glissement vs Roulement

💬 Le cœur de la démo billard

Quand la boule glisse ($v_P \neq 0$), le tapis exerce une **force de friction cinétique** qui s'oppose au glissement. Cette force fait **deux choses** simultanément :

1. Elle **ralentit** la translation (réduit v_G).
2. Elle **accélère** la rotation (augmente ω) — car la friction appliquée au point de contact crée un couple.

La friction **converge** le système vers le roulement pur ($v_G = \omega R$). Une fois le roulement atteint, la friction de glissement s'arrête — il ne reste que la **résistance au roulement** (beaucoup plus faible).

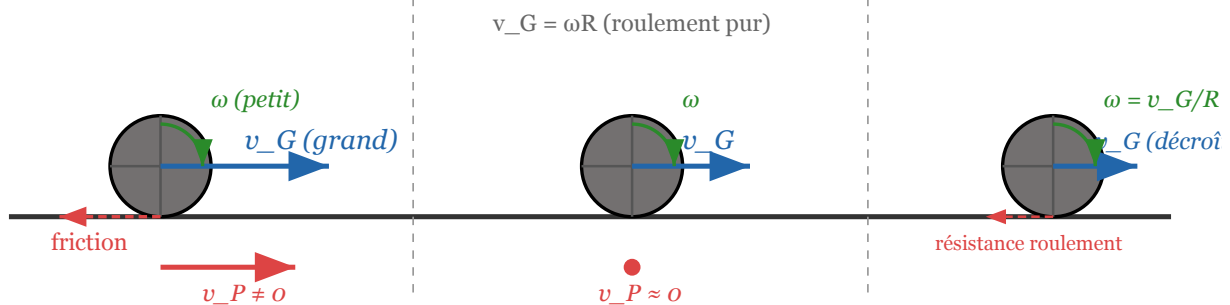
Phase 1 : Glissement pur**Phase 2 : Transition ($v_P \approx 0$)****Phase 3 : Roulement pur**

Fig. 14. – Les trois phases d’une boule de billard. Phase 1 : glissement pur, $v_G \gg \omega R$, friction cinétique forte (ralentit v_G , accélère ω). Phase 2 : transition, $v_G \approx \omega R$, la friction de glissement devient nulle. Phase 3 : roulement pur, seule la résistance au roulement (très faible) décélère lentement la boule.

La force de friction cinétique

Friction de glissement

Quand la boule glisse, le tapis exerce une force de friction cinétique :

$$\vec{F}_{\text{friction}} = -\mu_{\text{slide}} \cdot m \cdot g \cdot \hat{v}_P$$

où :

- μ_{slide} : coefficient de friction cinétique (≈ 0.2 pour un tapis de billard).
- $m \cdot g$: poids de la boule (force normale du sol).
- $\hat{v}_P$: direction de la vitesse de glissement (normalisée).

La force est **opposée** à la vitesse de glissement — elle tend à **annuler** $\vec{v}_P$.

Effet sur la translation

La friction décélère le centre de masse :

$$\vec{a}_G = \frac{\vec{F}_{\text{friction}}}{m} = -\mu_{\text{slide}} \cdot g \cdot \hat{v}_P$$

L’accélération est **constante** en magnitude ($\mu_{\text{slide}} \cdot g$) et opposée au glissement. La vitesse linéaire diminue.

Effet sur la rotation

La même force de friction, appliquée au point de contact, crée un **couple** :

$$\vec{\tau} = \vec{r} \times \vec{F}_{\text{friction}}$$

où $\vec{r} = (0, -R, 0)$ (du centre vers le contact). Ce couple **accélère** la rotation :

$$\vec{\alpha} = \frac{\vec{\tau}}{I}$$

Pour une sphère pleine ($I = \frac{2}{5}mR^2$) avec friction horizontale F :

$$\alpha = \frac{R \cdot F}{\frac{2}{5}mR^2} = \frac{5F}{2mR}$$

La rotation augmente — la boule se met à tourner **plus vite**.

! La friction fait deux choses opposées

Paradoxalement, la **même** force de friction :

- **Ralentit** la translation (v_G diminue).
- **Accélère** la rotation (ω augmente).

C'est exactement ce qu'il faut pour converger vers $v_G = \omega R$: v_G descend, ω monte, jusqu'à se rencontrer. C'est la beauté de la physique du roulement.

La résistance au roulement

Friction de roulement

Une fois le roulement pur atteint ($v_P = 0$), il n'y a plus de glissement — la friction cinétique s'annule. Mais la boule finit quand même par s'arrêter. Pourquoi ?

La **résistance au roulement** est un effet différent : la déformation microscopique du tapis sous la boule crée une force **très faible** qui décélère la boule. On la modélise simplement :

$$\vec{v} * = (1 - \mu_{\text{roll}} \cdot dt \cdot k)$$

où μ_{roll} est très petit (≈ 0.02) et k un facteur empirique. C'est un **amortissement** simple, pas une vraie force de friction.

💡 Dans la démo [billiard.html](#)

Le code distingue les deux modes :

```
if (slideSpeed > 0.05) {
  // Mode glissement : friction cinétique
  // - Ralentit v_G (linéaire)
  // - Accélère omega (couple)
} else {
  // Mode roulement : résistance simple
  // - Amortit v et omega ensemble
}
```

Le seuil 0.05 est la tolérance — en dessous, on considère que $v_P \approx 0$ et on passe en mode roulement.

Partie 7 : Intégration de la Rotation

Le problème de l'orientation

En translation, on intègre : $\vec{v} += \vec{a} \cdot dt$, puis $\vec{r} += \vec{v} \cdot dt$. Simple.

En rotation, c'est pareil pour la vitesse angulaire : $\vec{\omega} += \vec{\alpha} \cdot dt$. Mais pour l'**orientation** — comment on met à jour l'angle ? En 2D, $\theta += \omega \cdot dt$ suffit. En 3D, c'est plus subtil.

2D : angle scalaire

Rotation 2D

En 2D, la rotation est un simple angle scalaire θ :

```
angularVelocity += (torque / inertia) * dt;
angle += angularVelocity * dt;
```

On peut utiliser `ctx.rotate(angle)` pour le rendu. Simple et suffisant pour un jeu 2D.

3D : quaternions

❗ Pourquoi pas les angles d'Euler ?

En 3D, on **pourrait** utiliser trois angles (Euler : lacet, tangage, roulis). Mais les angles d'Euler souffrent du **gimbal lock** — une perte d'un degré de liberté quand deux axes s'alignent. De plus, interpoler entre deux orientations Euler donne des rotations **non naturelles** (le chemin le plus court n'est pas pris).

Les **quaternions** résolvent ces problèmes. Un quaternion $q = (w, x, y, z)$ représente une rotation de w radians autour de l'axe (x, y, z) normalisé. Pas de gimbal lock, interpolation naturelle (slerp).

Intégration par quaternion

Pour intégrer la rotation en 3D :

1. Calculer l'axe et l'angle de rotation ce frame : $\vec{axis} = \hat{\vec{\omega}}$, $\angle = |\vec{\omega}| \cdot dt$.
2. Créer un quaternion de rotation : $q_{\text{delta}} = \text{setFromAxisAngle}(\vec{axis}, \angle)$.
3. Multiplier avec l'orientation actuelle : $q_{\text{new}} = q_{\text{delta}} \cdot q_{\text{old}}$.

```
const axis = angularVelocity.clone().normalize();
const angle = angularVelocity.length() * dt;
if (angle > 1e-6) {
    const dq = new THREE.Quaternion().setFromAxisAngle(axis, angle);
    quaternion.premultiply(dq); // q_new = dq * q_old
}
```

Le `premultiply` applique la rotation dans le **repère monde** — si on voulait la rotation dans le repère **local** de l'objet, on utiliserait `multiply`.

💡 Euler explicite pour la rotation

Comme pour la translation, on utilise **Euler explicite** pour la rotation :

$$\vec{\omega}_{\text{new}} = \vec{\omega} + \left(\frac{\vec{\tau}}{I} \right) \cdot dt$$

$$q_{\text{new}} = \text{quat}(\hat{\vec{\omega}}, |\vec{\omega}| \cdot dt) \cdot q_{\text{old}}$$

Les mêmes problèmes de stabilité s'appliquent — Euler dérive en énergie. Mais pour une boule de billard qui s'arrête en quelques secondes (forte dissipation), c'est parfaitement adapté. Pour un satellite en orbite pendant des heures (pas de dissipation), il faudrait un intégrateur symplectique.

Partie 8 : La Démo — Billard (billiard.html)

🔗 La démo

Ouvrir `examples/billiard.html` dans un navigateur. La démo simule une boule de billard sur un tapis, avec :

- Un coup de queue paramétrable (force + hauteur d'impact).
- La distinction glissement vs roulement.
- La friction cinétique qui ralentit v_G et accélère ω .
- La résistance au roulement qui arrête lentement la boule.
- Des flèches de visualisation : vitesse $\vec{v}_G$ (bleu) et vitesse de glissement $\vec{v}_P$ (rouge).

Anatomie du code

État physique

```
const physicsState = {
  pos: new THREE.Vector3(-4, BALL_RADIUS, 0), // position du centre de masse
  vel: new THREE.Vector3(0, 0, 0),           // v_G (vitesse linéaire)
  angVel: new THREE.Vector3(0, 0, 0),         // ω (vitesse angulaire)
  quat: new THREE.Quaternion(),              // orientation (quaternion)
  isSliding: false                           // mode : glissement vs roulement
};
```

C'est le **corps rigide** — position, vitesse linéaire, vitesse angulaire, orientation. Exactement le dictionnaire de la Partie 2.

Le coup de queue (impulsion)

```
function shootBall() {
  resetGame();
  syncFromParams();

  // 1. Impulsion linéaire : v_G = F / m
  const direction = new THREE.Vector3(1, 0, 0);
  const linearSpeed = IMPULSE_FORCE / BALL_MASS;
  physicsState.vel.copy(direction.multiplyScalar(linearSpeed));

  // 2. Impulsion angulaire : τ = r × F → ω = τ / I
  // r = (0, h, 0) est le bras de levier (hauteur d'impact)
  const r = new THREE.Vector3(0, IMPACT_HEIGHT, 0);
```

```

const F = new THREE.Vector3(IMPULSE_FORCE, 0, 0);
const torqueImpulse = new THREE.Vector3().crossVectors(r, F);
physicsState.angVel.copy(torqueImpulse.divideScalar(INERTIA));
}

```

Deux choses se passent :

1. La force donne une vitesse linéaire $v = \frac{F}{m}$.
2. Le couple $\tau = r \times F$ donne une vitesse angulaire $\omega = \frac{\tau}{I}$.

Le paramètre `impactHeight` (h) contrôle le bras de levier — c'est la **hauteur** où la queue frappe la boule. À $h = 0$ (centre), pas de rotation. À $h > 0$, top spin. À $h < 0$, back spin.

Le calcul du glissement

```

// Vecteur du centre vers le point de contact : r = (0, -R, 0)
const rVector = new THREE.Vector3(0, -BALL_RADIUS, 0);

// Vitesse du point de contact : v_P = v_G + (ω × r)
const rotationalVelAtPoint = new THREE.Vector3()
    .crossVectors(omega, rVector);
const slideVel = new THREE.Vector3()
    .addVectors(v, rotationalVelAtPoint);
slideVel.y = 0; // on ignore la composante verticale

const slideSpeed = slideVel.length();

```

C'est **exactement** la formule de la Partie 5 : $\vec{v}_P = \vec{v}_G + (\vec{\omega} \times \vec{r})$. Si `slideSpeed > SLIDE_THRESHOLD`, la boule glisse. Sinon, elle roule.

La friction (mode glissement)

```

if (slideSpeed > SLIDE_THRESHOLD) {
    // Force de friction : opposée à la vitesse de glissement
    const frictionDir = slideVel.clone().normalize().negate();
    const frictionMag = MU_SLIDE * BALL_MASS * GRAVITY;
    const frictionForce = frictionDir.multiplyScalar(frictionMag);

    // A. Effet linéaire : ralentit v_G
    const linearAcc = frictionForce.clone().divideScalar(BALL_MASS);
    v.addScaledVector(linearAcc, dt);

    // B. Effet angulaire : τ = r × F → accélère ω
    const torque = new THREE.Vector3()
        .crossVectors(rVector, frictionForce);
    const angularAcc = torque.divideScalar(INERTIA);
    omega.addScaledVector(angularAcc, dt);
}

```

La friction fait **deux choses** :

1. **A.** Décélère v_G (la boule ralentit linéairement).
2. **B.** Crée un couple $\tau = r \times F$ qui **accélère** ω (la boule tourne plus vite).

C'est la convergence vers $v_G = \omega R$ — la boule passe du glissement au roulement.

La résistance (mode roulement)

```

} else {
  // Décroissance exponentielle : frame-rate independent
  const dragFactor = Math.exp(-MU_ROLL * dt);
  v.multiplyScalar(dragFactor);

  // Friction statique à basse vitesse : garantit l'arrêt
  // (l'exponentielle seule n'atteint jamais zéro)
  const speed = v.length();
  if (speed > 0) {
    const staticDrag = 0.2 * dt; // décélération constante (m/s²)
    const newSpeed = Math.max(0, speed - staticDrag);
    v.multiplyScalar(newSpeed / speed);
  }

  // Contraindre  $\omega$  à la condition de roulement :  $v_G = \omega \times r$ 
  // (évite l'oscillation glissement→roulement)
  omega.set(v.z / BALL_RADIUS, 0, -v.x / BALL_RADIUS);

  // Seuil d'arrêt
  if (v.lengthSq() < 0.02) {
    v.set(0, 0, 0);
    omega.set(0, 0, 0);
  }
}

```

Trois choses se passent en mode roulement :

1. **Décroissance exponentielle** — la résistance au roulement ralentit v_G de façon continue ($\exp(-\mu_r \cdot dt)$). Frame-rate independent.
2. **Friction statique** — une décélération **constante** qui garantit l'arrêt (l'exponentielle seule n'atteint jamais zéro, seulement asymptotiquement).
3. **Contrainte de roulement** — on recalcule ω depuis v_G pour maintenir $v_G = \omega R$ **exactement**. Sans cela, les erreurs de virgule flottante font osciller la boule entre glissement et roulement.

L'intégration (position + orientation)

```

// Position : r += v * dt (Euler)
physicsState.pos.addScaledVector(v, dt);

// Orientation : quaternion (Partie 7)
const axis = omega.clone().normalize();
const angle = omega.length() * dt;
if (angle > 1e-6) {
  const q = new THREE.Quaternion()
    .setFromAxisAngle(axis, angle);
  physicsState.quat.premultiply(q); // q_new = dq * q_old
}

```

Translation en Euler, rotation en quaternion. Le seuil 1e-6 évite les calculs inutiles quand $\omega \approx 0$.

Collisions avec les murs

```
let hitWall = false;
if (pos.x > halfX) { pos.x = halfX; v.x *= -0.8; hitWall = true; }
if (pos.x < -halfX) { pos.x = -halfX; v.x *= -0.8; hitWall = true; }
if (pos.z > halfZ) { pos.z = halfZ; v.z *= -0.8; hitWall = true; }
if (pos.z < -halfZ) { pos.z = -halfZ; v.z *= -0.8; hitWall = true; }

// Après un rebond, recalculer  $\omega$  pour maintenir la condition de roulement
// (sinon  $v_G$  et  $\omega$  sont désynchronisés → re-glissement parasite)
if (hitWall) {
    omega.set(v.z / BALL_RADIUS, 0, -v.x / BALL_RADIUS);
}
```

Le rebond inverse la vitesse ($\times -0.8$: 20% de perte d'énergie), mais **sans** mettre à jour ω , la condition $v_G = \omega R$ est cassée — la boule re-entre en mode glissement au frame suivant. On resynchronise ω immédiatement après le rebond.

Expériences à essayer

Manipulations

1. **Coup centré** ($h = 0$) : la boule glisse sans tourner. Observer la flèche rouge (vitesse de glissement) — elle est égale à v_G . La friction met du temps à faire rouler la boule.
2. **Coup haut** ($h = +0.3$) : top spin. La boule tourne vers l'avant. Si $\omega R > v_G$, le point de contact va **vers l'arrière** — la friction **accélère** la boule ! (Effet de « rattrapage » du top spin.)
3. **Coup bas** ($h = -0.3$) : back spin. La boule tourne vers l'arrière. Le point de contact va **vers l'avant** — la friction **décélère** la boule plus vite. Observer la boule qui recule après s'être arrêtée (effet de « retour » du back spin).
4. **Force faible** ($F = 2$) vs **force forte** ($F = 15$) : la force change v_G mais pas ω (si h est le même). Observer comment la **durée** du glissement change — plus de glissement = plus de temps avant le roulement.
5. **Friction glissement** ($\mu_s = 0$) : la boule glisse indéfiniment sans jamais se mettre à rouler (surface de glace). À $\mu_s = 2$, elle se met à rouler presque instantanément.
6. **Friction roulement** ($\mu_r = 0$) : une fois en roulement, la boule ne s'arrête jamais (roulement parfait). À $\mu_r = 0.5$, elle s'arrête très vite.
7. **Vitesse temps** (0.1) : ralentir la simulation pour observer la transition glissement → roulement en détail. La flèche rouge (glissement) rétrécit jusqu'à disparaître, puis la boule roule.

Partie 9 : TP — Corps Rigide 2D

🔗 Objectif du TP

Implémenter un **corps rigide 2D** — un rectangle qui se déplace **et** tourne quand on clique dessus. Le principe est le même que la boule de billard, mais en 2D avec un angle scalaire au lieu d'un quaternion.

Étape 1 : Classe Rigidbody

Ajouter les propriétés angulaires à un objet qui a déjà position, velocity, mass :

```
class Rigidbody {
    constructor(width, height, mass) {
        // --- Linéaire (déjà vu) ---
        this.pos = new Vector2(0, 0);
        this.vel = new Vector2(0, 0);
        this.mass = mass;
        this.force = new Vector2(0, 0); // accumulateur

        // --- Angulaire (nouveau) ---
        this.angle = 0; // θ (radians)
        this.angularVelocity = 0; // ω (rad/s)
        this.torque = 0; // τ (accumulateur)
        // Moment d'inertie d'un rectangle : I = (1/12) m (w² + h²)
        this.inertia = (1/12) * mass * (width*width + height*height);
    }
}
```

Étape 2 : ApplyForce(force, worldPoint)

La méthode clé

```
applyForce(force, worldPoint) {
    // 1. Force linéaire : F_total += F
    this.force.x += force.x;
    this.force.y += force.y;

    // 2. Couple : τ = r_x * F_y - r_y * F_x
    const r = {
        x: worldPoint.x - this.pos.x,
        y: worldPoint.y - this.pos.y
    };
    this.torque += r.x * force.y - r.y * force.x;
}
```

On accumule les forces et les couples pendant la frame, puis on les applique dans l'intégrateur. Le bras de levier $\vec{r}$ est la distance entre le centre de masse et le point d'application.

Étape 3 : L'intégrateur

Euler explicite — linéaire + angulaire

```
update(dt) {
    // --- Linéaire ---
    const ax = this.force.x / this.mass;
    const ay = this.force.y / this.mass;
    this.vel.x += ax * dt;
    this.vel.y += ay * dt;
    this.pos.x += this.vel.x * dt;
    this.pos.y += this.vel.y * dt;

    // --- Angulaire ---
```

```

const alpha = this.torque / this.inertia;
this.angularVelocity += alpha * dt;
this.angle += this.angularVelocity * dt;

// --- Reset accumulateurs ---
this.force.x = 0; this.force.y = 0;
this.torque = 0;
}

```

Exactement le même schéma qu'Euler en translation — juste **dupliqué** pour la rotation. C'est la beauté de l'analogie linéaire/angularaire.

Étape 4 : Interaction souris

Cliquer pour pousser

```

canvas.addEventListener('click', (e) => {
  const mousePos = getMousePos(e);           // position souris (world)
  const forceDir = {
    x: mousePos.x - body.pos.x,               // direction : vers la souris
    y: mousePos.y - body.pos.y
  };
  const magnitude = 500;                      // force arbitraire
  const len = Math.hypot(forceDir.x, forceDir.y);
  const force = {
    x: (forceDir.x / len) * magnitude,
    y: (forceDir.y / len) * magnitude
  };
  body.applyForce(force, mousePos);           // applique au point cliqué
});

```

Le point d'application est la **position de la souris** — si on clique loin du centre, on crée un **couple** et le rectangle tourne. Si on clique près du centre, il se déplace sans tourner. C'est l'équivalent du `impactHeight` de la démo billard.

Étape 5 : Rendu

Dessiner le rectangle pivoté

```

function draw() {
  ctx.save();
  ctx.translate(body.pos.x, body.pos.y);      // aller au centre
  ctx.rotate(body.angle);                     // pivoter
  ctx.fillRect(-w/2, -h/2, w, h);             // dessiner centré
  ctx.restore();
}

```

En Canvas 2D, on utilise `ctx.translate` + `ctx.rotate` pour positionner et orienter le rectangle. L'ordre est important : **d'abord** traduire, **ensuite** pivoter.

Partie 10 : Récapitulatif — Le Corps Rigide Complet

Phase	Linéaire	Angulaire	Code
Accumuler	$\vec{F} += \vec{F}_{\text{appliquée}}$	$\tau += r \times F$	<code>applyForce()</code>
Intégrer	$\vec{v} += (\vec{F}/m) \cdot dt$	$\vec{\omega} += (\tau/I) \cdot dt$	<code>update()</code>
Déplacer	$\vec{r} += \vec{v} \cdot dt$	$\theta += \omega \cdot dt / q$	<code>update()</code>
Reset	$\vec{F} = 0$	$\tau = 0$	<code>update()</code> fin

Tableau 8. – Le cycle complet d’un corps rigide. Chaque frame : accumuler les forces et couples, intégrer (Euler), mettre à jour position et orientation, réinitialiser les accumulateurs.

 Ce qu'on retient

- 1. Un corps rigide a **deux** dynamiques — linéaire et angulaire — parfaitement **analogues**.
- 2. Le couple $\tau = r \times F$ dépend du **point d’application** — pas seulement de la force.
- 3. Le moment d’inertie I dépend de la **répartition de masse** — pas seulement de la masse totale.
- 4. La vitesse d’un point $\vec{v}_P = \vec{v}_G + (\vec{\omega} \times \vec{r})$ est la clé du glissement vs roulement.
- 5. La friction de glissement **converge** vers le roulement en ralentissant v_G et en accélérant ω simultanément.
- 6. L’intégration de la rotation utilise des **quaternions** en 3D (pas de gimbal lock) et un **angle scalaire** en 2D.

Quiz Mi-Parcours — Sessions 1 à 10

Informations

Déroulement

- Durée : **3 heures**.
- Ce quiz couvre les sessions 1 à 10 : géométrie vectorielle, cinématique, lois de Newton, forces, intégration d'Euler, impulsion et collisions, broad/narrow phase, Verlet + PBD, moteur de particules, et rotations.
- Le quiz est **interactif** : on le traverse ensemble, question par question, avec discussion.
- Format : choix multiples (QCM). Pour chaque question, la bonne réponse **est** l'explication.
- Documents autorisés : vos notes de cours et le PDF du cours. Pas d'internet.

Partie A — Géométrie et Vecteurs (Session 1)

A1. Produit scalaire — QCM

Soit $\vec{u} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$ et $\vec{v} = \begin{pmatrix} -4 \\ 3 \end{pmatrix}$. Que vaut $\vec{u} \cdot \vec{v}$, et que dit ce résultat sur l'angle entre $\vec{u}$ et $\vec{v}$?

- 0 — le produit scalaire est nul, ce qui signifie que les vecteurs sont **perpendiculaires** (angle de 90°). C'est le test le plus rapide pour vérifier une orthogonalité.
- 25 — c'est $\|\vec{u}\| \cdot \|\vec{v}\| = 5 \times 5$, mais le produit scalaire réel est $3 \times (-4) + 4 \times 3 = 0$. On a confondu norme et produit scalaire.
- 25 — c'est l'opposé du produit des normes, pas le produit scalaire.
- 7 — c'est $3 + 4$, la somme des composantes de $\vec{u}$, pas le produit scalaire.

A2. Vecteur unitaire — QCM

Soit $\vec{v} = \begin{pmatrix} 6 \\ 8 \\ 0 \end{pmatrix}$. Le vecteur unitaire $\hat{u} = \frac{\vec{v}}{\|\vec{v}\|}$ dans la direction de $\vec{v}$ vaut :

- $\begin{pmatrix} 0.6 \\ 0.8 \\ 0 \end{pmatrix}$ — on divise chaque composante par la norme $\|\vec{v}\| = 10$. Le résultat est de longueur 1. Un vecteur unitaire représente une **direction pure**, sans magnitude.
- $\begin{pmatrix} 6 \\ 8 \\ 0 \end{pmatrix}$ — c'est le vecteur original, pas normalisé ($\|\vec{v}\| = 10$, pas 1).
- $\begin{pmatrix} 3 \\ 4 \\ 0 \end{pmatrix}$ — on a divisé par 2, mais ce n'est pas unitaire ($\|\vec{u}\| = 5$).
- $\begin{pmatrix} 0.8 \\ 0.6 \\ 0 \end{pmatrix}$ — les composantes sont inversées ; ce n'est plus dans la direction de $\vec{v}$.

A3. Produit vectoriel — QCM

Vous programmez un personnage qui marche sur un terrain 3D. Vous connaissez deux vecteurs non parallèles qui définissent une face du terrain. Vous avez besoin de la **normale** de cette face (un vecteur perpendiculaire à la surface). Quelle opération utilisez-vous ?

- Le **produit vectoriel** ($\vec{a} \times \vec{b}$) — il produit toujours un vecteur **perpendiculaire** à ses deux entrées. C'est l'outil standard pour calculer une normale de surface.
- Le **produit scalaire** ($\vec{a} \cdot \vec{b}$) — il produit un **scalaire**, pas un vecteur. On ne peut pas obtenir une normale avec.
- La **soustraction** ($\vec{a} - \vec{b}$) — elle donne un vecteur dans le **plan** des deux entrées, pas perpendiculaire.
- La **normalisation** ($\hat{u} = \frac{\vec{v}}{\|\vec{v}\|}$) — elle donne un vecteur **unitaire** mais dans la **même direction** que l'entrée, pas perpendiculaire.

A4. Produit scalaire — QCM

Lors d'une collision, vous avez la vitesse $\vec{v}$ d'un objet et la normale $\hat{n}$ de la surface. Vous devez **décomposer** la vitesse en composante **normale** (vers la surface) et **tangentielle** (le long de la surface). Quelle opération vous donne la composante normale scalaire ?

1. Le **produit scalaire** ($\vec{v} \cdot \hat{n}$) — il projette $\vec{v}$ sur $\hat{n}$ et donne un **scalaire** : la magnitude de la composante normale. C'est exactement ce qu'on utilise dans le calcul d'impulsion ($v_{\text{rel}} = \vec{v} \cdot \hat{n}$).
2. Le **produit vectoriel** ($\vec{v} \times \hat{n}$) — il donne un vecteur **perpendiculaire** aux deux, pas une composante projetée.
3. La **soustraction** ($\vec{v} - \hat{n}$) — elle n'a pas de sens physique ici ; on ne soustrait pas un vecteur unitaire à une vitesse.
4. La **normalisation** ($\frac{\vec{v}}{\|\vec{v}\|}$) — elle donne la **direction** de la vitesse, pas sa composante sur la normale.

Partie B — Cinématique 2D (Session 2)

B1. Projectile — QCM

Dans un mouvement de projectile **sans résistance de l'air**, que peut-on dire de la composante horizontale de la vitesse v_x ?

1. Elle reste **constante** — il n'y a aucune force horizontale, donc aucune accélération horizontale. La gravité n'affecte que la composante verticale v_y .
2. Elle diminue linéairement avec le temps — la gravité est verticale, elle n'affecte pas v_x .
3. Elle augmente avec l'accélération gravitationnelle — la gravité est verticale, pas horizontale.
4. Elle devient nulle au sommet de la trajectoire — au sommet, c'est v_y qui est nulle, pas v_x .

B2. Vitesse comme dérivée — QCM

La vitesse est la **dérivée** de la position. Un objet a une position $\vec{r}(t) = \begin{pmatrix} 2t \\ t^2 \end{pmatrix}$. Que vaut sa vitesse $\vec{v}(t)$?

1. $\vec{v}(t) = \begin{pmatrix} 2 \\ 2t \end{pmatrix}$ — on dérive chaque composante : $\frac{d}{dt}(2t) = 2$, $\frac{d}{dt}(t^2) = 2t$. La vitesse est le **taux de variation** de la position.
2. $\vec{v}(t) = \begin{pmatrix} 2t \\ t^2 \end{pmatrix}$ — c'est la position, pas la vitesse. On n'a pas dérivé.
3. $\vec{v}(t) = \begin{pmatrix} 2 \\ t \end{pmatrix}$ — on a dérivé t^2 en t au lieu de $2t$ (oubli du facteur 2).
4. $\vec{v}(t) = \begin{pmatrix} 2t \\ 2t \end{pmatrix}$ — on a dérivé le mauvais terme ($2t$ au lieu de le garder).

B3. Mouvement circulaire — QCM

Un personnage tourne sur un manège à vitesse **constante**. Que peut-on dire de son accélération ?

1. Elle pointe **vers le centre** du cercle — c'est l'accélération **centripète**. Même si la vitesse scalaire est constante, la **direction** change, donc il y a une accélération. Sans elle, le personnage irait en ligne droite (1ère loi de Newton).
2. Elle est **nulle** — la vitesse étant constante, il n'y a pas d'accélération. Faux : la **direction** change, pas seulement la magnitude.
3. Elle pointe **vers l'extérieur** (centrifuge) — c'est une force **ressentie** dans le référentiel tournant, pas l'accélération réelle.
4. Elle est **tangente** au cercle — ce serait le cas si la vitesse augmentait, pas si elle est constante.

Partie C — Lois de Newton (Session 3)

C1. Deuxième loi — QCM

La deuxième loi de Newton dit $\vec{F} = m \cdot \vec{a}$. Si on **double la masse** d'un objet tout en gardant la **même force** appliquée, que devient son accélération ?

1. Elle est **divisée par 2** — car $a = \frac{F}{m}$, donc si m double, a diminue de moitié. Plus de masse = plus d'inertie = moins d'accélération pour la même force.
2. Elle est **doublee** — la masse donne plus d'inertie, donc **moins** d'accélération, pas plus.
3. Elle **ne change pas** — l'accélération dépend de la force **et** de la masse ($a = \frac{F}{m}$).
4. Elle est **multipliée par 4** — il n'y a aucune raison que la masse et la force se multiplient.

C2. Troisième loi — QCM

Une balle de 0.5 kg entre en collision avec un mur fixe. La balle exerce une force de 100 N sur le mur. Quelle force le mur exerce-t-il sur la balle ?

1. 100 N dans le sens opposé (vers la balle) — la troisième loi dit que l'action et la réaction sont **égales et opposées**, indépendamment des masses. Le mur ne bouge pas parce qu'il est ancré, mais il exerce quand même 100 N.
2. 0 N — le mur est fixe, il ne bouge pas. Faux : ne pas bouger ne signifie pas ne pas exercer de force.
3. 100 N dans le même sens que la balle — la réaction est **opposée** à l'action, pas dans le même sens.
4. 50 N — la masse de la balle n'entre **pas** en compte dans la troisième loi. La réaction est toujours égale à l'action.

C3. Chute libre — QCM

Pourquoi deux objets de masses différentes (1 kg et 100 kg) tombent-ils avec la même accélération dans le vide ?

1. Parce que la masse s'annule : $a = \frac{F}{m} = \frac{mg}{m} = g$ — la force gravitationnelle est **proportionnelle** à la masse, donc quand on divise par la masse pour obtenir l'accélération, elle s'annule. L'accélération est g , indépendante de m .
2. Parce que la gravité ne dépend pas de la masse — faux, la force de gravité **dépend** de la masse ($F = mg$). C'est l'accélération qui n'en dépend pas.
3. Parce que les objets ont le même poids — faux, le 100 kg est 100 fois plus lourd. Mais il est aussi 100 fois plus difficile à accélérer.
4. Parce qu'il n'y a pas d'air — l'absence d'air supprime le **drag**, mais la raison de l'égalité est l'annulation de la masse dans $a = \frac{F}{m}$.

Partie D — Forces de la Nature (Session 4)

D1. Force normale — QCM

La force normale exercée par une surface sur un objet posé dessus :

1. Est toujours **perpendiculaire** à la surface — c'est la surface qui « pousse » l'objet pour l'empêcher de traverser. Sur un plan incliné, elle vaut $mg \cos(\theta)$, ni verticale ni égale au poids.
2. Est toujours **égale au poids** de l'objet — faux sur un plan incliné : $N = mg \cos(\theta) < mg$.
3. Est toujours **verticale**, dirigée vers le haut — faux sur un plan incliné : elle est perpendiculaire au plan, pas verticale.
4. Dépend du **coefficient de friction** de la surface — faux : la normale dépend du poids et de l'angle, pas de μ . C'est la **friction** qui dépend de la normale ($f = \mu N$).

D2. Frottement statique vs cinétique — QCM

Pourquoi est-il plus difficile de **déplacer** un objet immobile que de le **maintenir** en mouvement ?

1. Le frottement **statique** (μ_s) est supérieur au frottement **cinétique** (μ_k) — il faut plus de force pour **décoller** l'objet que pour le garder en mouvement. C'est l'expérience quotidienne : pousser un meuble lourd est difficile au début, puis plus facile une fois qu'il glisse.
2. Le frottement cinétique est supérieur au frottement statique — c'est l'inverse : $\mu_s > \mu_k$.
3. L'objet est plus lourd quand il est immobile — la masse ne change pas avec le mouvement.
4. La force normale augmente avec la vitesse — la normale dépend du poids et de l'angle, pas de la vitesse.

D3. Vitesse terminale — QCM

Qu'est-ce que la **vitesse terminale** ?

1. La vitesse à laquelle la force de **drag** **compense exactement la gravité** — l'accélération devient nulle et l'objet cesse d'accélérer. Il tombe à vitesse **constante**. Pour un drag linéaire : $v_\infty = m \frac{g}{k}$.
2. La vitesse maximale qu'un objet peut atteindre avant de se briser — c'est une limite matérielle, pas physique.
3. La vitesse à laquelle un objet touche le sol — c'est la vitesse d'impact, pas la vitesse terminale.
4. La vitesse initiale qu'il faut donner à un objet pour qu'il tombe — la vitesse terminale est atteinte **naturellement** par le drag, pas donnée initialement.

Partie E — Intégration d'Euler (Session 5)

E1. La chaîne d'Euler — QCM

Dans quel ordre doit-on mettre à jour les quantités à chaque frame avec la méthode d'Euler (semi-implicite) ?

1. Force $\rightarrow$ Accélération $\rightarrow$ Vitesse $\rightarrow$ Position — c'est la **chaîne d'Euler** : on calcule les forces, on dérive l'accélération via $F = ma$, puis on intègre la vitesse, puis la position. $\vec{F} \rightarrow \vec{a} \rightarrow \vec{v} \rightarrow \vec{r}$.
2. Position $\rightarrow$ Vitesse $\rightarrow$ Accélération $\rightarrow$ Force — c'est l'ordre **inverse** ; on ne peut pas calculer la position avant la vitesse.
3. Accélération $\rightarrow$ Force $\rightarrow$ Position $\rightarrow$ Vitesse — la force **cause** l'accélération, pas l'inverse.
4. Vitesse $\rightarrow$ Position $\rightarrow$ Force $\rightarrow$ Accélération — la force doit venir **en premier** ; tout découle d'elle.

E2. Tracer le code — QCM

Voici un extrait de `updatePhysics(dt)` :

```
force.set(0, 0, 0); // ligne 1
force.add(gravity.clone().multiplyScalar(mass)); // ligne 2
acceleration.copy(force).divideScalar(mass); // ligne 3
velocity.addScaledVector(acceleration, dt); // ligne 4
position.addScaledVector(velocity, dt); // ligne 5
```

E2a. Pourquoi remet-on force à zéro (ligne 1) au début de chaque frame ?

1. Pour éviter que les forces s'accumulent d'une frame à l'autre — sans cela, la gravité de la frame n s'ajouterait à celle de la frame $n - 1$, et la force grandirait indéfiniment. La balle s'envolerait.
2. Pour libérer la mémoire du vecteur force — `set(0, 0, 0)` ne libère rien, il écrase les valeurs.
3. Pour remettre la position à l'origine — force n'a rien à voir avec la position.
4. C'est optionnel, ça ne change rien — sans le reset, les forces s'accumulent et la simulation explose.

E2b. Avec $m = 2 \text{ kg}$ et $\vec{g} = \begin{pmatrix} 0 \\ -9.81 \\ 0 \end{pmatrix}$, que vaut `acceleration` après la ligne 3 ?

1. $\vec{a} = \vec{g} = \begin{pmatrix} 0 \\ -9.81 \\ 0 \end{pmatrix}$ — car $\vec{a} = \frac{\vec{F}}{m} = \frac{m\vec{g}}{m} = \vec{g}$. La masse s'annule : en chute libre, l'accélération est toujours $\vec{g}$, indépendamment de m .
2. $\vec{a} = \begin{pmatrix} 0 \\ -19.62 \\ 0 \end{pmatrix}$ — on a oublié que la masse s'annule. $F = mg = 2 \times 9.81$, mais $a = \frac{F}{m} = \frac{19.62}{2} = 9.81$.
3. $\vec{a} = \begin{pmatrix} 0 \\ -4.905 \\ 0 \end{pmatrix}$ — on a divisé deux fois par la masse.
4. $\vec{a} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$ — la force et la masse ne s'annulent pas ; $a = \frac{F}{m} = g$.

E2c. La ligne 5 utilise la vitesse **déjà mise à jour** (ligne 4) pour intégrer la position. Comment appelle-t-on cette variante ?

1. **Euler semi-implicite** (symplectique) — elle conserve mieux l'énergie que l'Euler classique et est le choix par défaut dans les moteurs de jeu. On utilise la **nouvelle** vitesse pour intégrer la position.
2. **Euler classique** — l'Euler classique utilise l'ancienne vitesse v_n pour la position, pas la nouvelle.
3. **Verlet** — Verlet ne stocke pas la vitesse explicitement, c'est la méthode de la session 8.
4. **Runge-Kutta** — c'est une méthode d'ordre 4, beaucoup plus complexe.

E3. Stabilité d'Euler — QCM

E3a. Pourquoi la méthode d'Euler « dérive » en énergie sur les systèmes conservatifs (ressorts, orbites) ?

1. Euler ignore la **courbure** de la trajectoire (l'accélération) à l'intérieur du pas — l'énergie ajoutée par l'erreur n'est jamais dissipée et s'accumule jusqu'à l'explosion. C'est pourquoi on a banni Euler pour le tissu (session 8).
2. Euler calcule l'accélération trop précisément, ce qui crée des oscillations — c'est le contraire : Euler est **imprécis**.
3. Euler utilise un pas de temps trop grand par défaut — le pas n'est pas le problème, c'est la **méthode** elle-même.
4. Euler ne conserve pas la quantité de mouvement — Euler conserve la quantité de mouvement, c'est l'énergie qui dérive.

E3b. Pourquoi Euler est-il malgré tout acceptable pour une balle qui rebondit avec restitution < 1 ?

1. Chaque rebond **dissipe** de l'énergie — le système est globalement **dissipatif**, pas conservatif. L'erreur d'Euler est invisible car la dissipation est plus forte que la dérive.
2. La balle est trop légère pour que l'erreur soit visible — la masse n'a rien à voir avec la dérive d'Euler.
3. Euler est exact pour les rebonds — Euler n'est jamais exact, il est de premier ordre.
4. La gravité compense exactement l'erreur d'Euler — la gravité n'a aucun lien avec l'erreur de troncature.

E3c. Quel est le rôle du `Math.min(dt, 0.1)` dans la boucle d'animation ?

1. Il **plafonne** le pas de temps — si l'utilisateur change d'onglet, `getDelta()` renvoie un dt énorme (plusieurs secondes) qui ferait exploser la simulation. Le clamp évite ce piège classique.
2. Il accélère la simulation en limitant le dt — le clamp **ralentit** le dt quand il est trop grand, il ne l'accélère pas.
3. Il limite le FPS à 10 images par seconde — le clamp agit sur dt , pas sur le FPS.
4. Il empêche la balle de tomber trop vite — le clamp limite le **pas de temps**, pas la vitesse de la balle.

Partie F — Impulsion et Collisions (Session 6)

F1. Conservation — QCM

Dans une collision entre deux objets (système isolé, pas de forces externes), quelle quantité est **toujours** conservée, élastique ou non ?

1. La **quantité de mouvement totale** ($\vec{p}_A + \vec{p}_B$) — c'est la loi fondamentale des collisions : ce qui est perdu par un objet est gagné par l'autre. Valable pour **toutes** les collisions.
2. L'**énergie cinétique totale** — seulement conservée dans les collisions **élastiques** ($e = 1$). Perdue dans les collisions inélastiques.
3. La **vitesse totale** — les vitesses changent individuellement ; il n'y a pas de « conservation de vitesse ».
4. La **masse totale** — oui, mais c'est trivial. La quantité de mouvement est la quantité physique **pertinente**.

F2. Restitution — QCM

Le coefficient de restitution e est défini par :

$$e = -\frac{v_{\text{relative après}}}{v_{\text{relative avant}}}$$

Si $e = 0.5$ et la vitesse relative avant l'impact est 10 m/s, que vaut la vitesse relative après l'impact ?

1. -5 m/s — la formule donne $v_{\text{rel après}} = -e \times v_{\text{rel avant}} = -0.5 \times 10 = -5$. Le signe négatif signifie que les objets se **séparent** (sens opposé), et la magnitude est divisée par 2.
2. 10 m/s — cela correspondrait à $e = 1$ (élastique parfait), pas $e = 0.5$.
3. 5 m/s (même sens) — le signe est faux. Après l'impact, les objets se **séparent**, le sens s'inverse.
4. 0 m/s — cela correspondrait à $e = 0$ (les objets collent), pas $e = 0.5$.

F3. Impulsion — QCM

Dans le code de résolution de collision, on trouve cette garde :

```
const vRel = relVel.dot(normal);
if (vRel > 0) return; // <-- on arrête tout
```

F3a. Que signifie $v_{\text{rel}} > 0$?

1. Les deux objets **s'éloignent** déjà l'un de l'autre — la composante normale de leur vitesse relative est positive, ils se séparent. Il n'y a pas de collision à résoudre.
2. Les deux objets **se rapprochent** — s'ils se rapprochent, $v_{\text{rel}} < 0$ (négatif), pas positif.
3. Les deux objets sont **immobiles** — si $v_{\text{rel}} = 0$, ils sont immobiles relativement, pas si $v_{\text{rel}} > 0$.
4. La collision est **élastique** — v_{rel} ne dit rien sur le type de collision, seulement sur la direction.

F3b. Que se passerait-il si on retirait cette garde (`if (vRel > 0) return;`) ?

1. On appliquerait une impulsion à des objets qui **se séparent** — ils s'éloigneraient encore plus. C'est une impulsion « fantôme » qui **ajoute** de l'énergie au lieu de résoudre la collision.
2. Rien — la garde est optionnelle — sans la garde, le code applique l'impulsion même quand elle ne devrait pas.

3. Les objets passeraient **à travers** le sol — la garde n'a rien à voir avec le sol, elle gère les collisions entre objets.
4. La simulation s'arrêterait — la simulation continue, mais avec des impulsions incorrectes.

Partie G — Détection de Collision : Broad et Narrow Phase (Session 7)

G1. Complexité de la brute force — QCM

Avec $N = 1000$ objets, combien de paires la brute force génère-t-elle ?

1. 499500 — la formule est $\frac{N(N-1)}{2} = 1000 \times \frac{999}{2}$, car chaque paire n'est testée qu'une fois. C'est le $O(N^2)$ qui rend la brute force inutilisable au-delà de quelques dizaines d'objets.
2. 1000000 — c'est N^2 , mais ça compte chaque paire **deux fois** (A-B et B-A). La vraie formule divise par 2.
3. 1000 — une seule paire par objet, mais chaque objet doit tester **tous** les autres.
4. 999000 — c'est $N(N-1)$, mais ça double les paires. On divise par 2 pour éviter les doublons.

G2. AABB — QCM

Pourquoi utilise-t-on des boîtes **alignées sur les axes** (AABB) pour la Broad Phase ?

1. Parce que le test de chevauchement est **trivial** — quelques comparaisons par axe, pas de trigonométrie ni de produit vectoriel. C'est le test le plus rapide qui existe en 3D. Les objets n'ont pas besoin d'être alignés ; c'est la **boîte englobante** qui l'est.
2. Parce que les objets sont toujours alignés sur les axes dans un jeu — les objets peuvent tourner, mais leur **boîte englobante** reste alignée pour la performance.
3. Parce que c'est la seule façon de calculer une boîte englobante — on peut aussi utiliser des OBB (Oriented Bounding Box), mais le test est plus coûteux.
4. Parce que la rotation des objets n'a pas d'importance en physique — la rotation importe en Narrow Phase, mais la Broad Phase la ignore volontairement pour la performance.

G3. Structures de partitionnement — QCM

Associez chaque structure à son cas d'usage idéal :

1. **Grille uniforme** →
2. **Quadtree/Octree** →
3. **Sweep and Prune** →

Options (une par structure) : a. **Monde ouvert**, zones denses et vides, culling de frustum — l'arbre s'adapte à la densité en subdivisant les zones denses. b. **Particules de taille similaire** dans une arène fermée — accès $O(1)$, très rapide, facile à coder. c. **Objets de tailles hétérogènes**, cohérence temporelle exploitée — tri presque gratuit d'une frame à l'autre car les objets bougent peu.

G4. Budget temps — QCM

À 60 FPS, on dispose de 16.6 ms par frame. Pourquoi la Broad Phase est-elle le **goulot d'étranglement numéro un** des moteurs physiques ?

1. Parce qu'elle **scale avec N** — c'est elle qui détermine combien de paires sont testées. Si elle est lente, tout le reste cascade (Narrow Phase, Response). Elle doit consommer **moins de 0.5 ms** sur les 2 ms budget physique.
2. Parce que c'est l'étape la plus précise géométriquement — c'est la **moins** précise (AABB simplifiées). La précision est en Narrow Phase.
3. Parce qu'elle calcule les impulsions de collision — c'est la **Response** qui calcule les impulsions, pas la Broad Phase.
4. Parce qu'elle est la dernière étape du pipeline — c'est la **première** étape. C'est justement parce qu'elle est première qu'elle est le goulot.

Partie H — Verlet et PBD (Session 8)

H1. Verlet vs Euler — QCM

Quelle est la principale différence entre Verlet et Euler ?

1. Verlet **déduit** la vitesse de $x_n - x_{n-1}$ (vitesse implicite), Euler **stocke** v explicitement — c'est la différence fondamentale. Verlet n'a pas de variable `velocity` ; la vitesse émerge de la différence des positions.
2. Verlet **stocke** la vitesse explicitement, Euler non — c'est l'inverse : Euler stocke v , Verlet la déduit.
3. Verlet est du **premier ordre**, Euler du deuxième — c'est l'inverse : Verlet est $O(dt^4)$, Euler $O(dt^3)$.
4. Il n'y a pas de différence, c'est la même méthode — les deux méthodes ont des propriétés très différentes (stabilité, précision, gestion des contraintes).

H2. Verlet et PBD — QCM

H2a. Pourquoi Verlet est-elle **plus précise** qu'Euler ?

1. Verlet est $O(dt^4)$ tandis qu'Euler est $O(dt^3)$ — la dérivation par Taylor montre qu'en additionnant $x(t + dt)$ et $x(t - dt)$, les termes en v_n s'annulent, laissant une erreur d'ordre supérieur.
2. Verlet est $O(dt^3)$ comme Euler — faux, Verlet est d'un ordre supérieur.
3. Verlet utilise des pas de temps plus petits — le pas de temps est le même, c'est la **méthode** qui est plus précise.
4. Verlet ne utilise pas l'accélération — Verlet utilise bien $a \cdot dt^2$ dans sa formule.

H2b. Pourquoi PBD **exige** Verlet (et ne fonctionne pas bien avec Euler) ?

1. PBD **déplace directement** les positions pour satisfaire les contraintes. Avec Verlet, la vitesse étant $x_n - x_{n-1}$, **toute correction de position met à jour automatiquement la vitesse**. Avec Euler, le déplacement manuel n'affecte pas v — la vitesse devient incohérente et le système explose.
2. Parce qu'Euler est moins précis — la précision n'est pas le problème ici, c'est la **cohérence** position/vitesse.
3. Parce que Verlet est plus rapide — la vitesse n'est pas la question, c'est la **gestion des contraintes**.
4. Parce qu'Euler ne supporte pas les forces — Euler supporte parfaitement les forces, c'est le déplacement **manuel** qui pose problème.

H3. Contrainte PBD — QCM

Voici la fonction `satisfyConstraint` du TP :

```
function satisfyConstraint(p1, p2, restLength) {
  const delta = p2.position.clone().sub(p1.position);
  const dist = delta.length();
  if (dist === 0) return;
  const diff = (dist - restLength) / dist;
  const correction = delta.multiplyScalar(0.5 * diff);
  if (!p1.pinned) p1.position.add(correction);
}
```

```
if (!p2.pinned) p2.position.sub(correction);
}
```

H3a. Que représente $\text{diff} = (\text{dist} - \text{restLength}) / \text{dist}$?

1. L'**écart relatif** — on divise par dist pour normaliser. La correction sera proportionnelle à delta (la direction), pas à la distance absolue. C'est un pourcentage d'étirement.
2. L'écart **absolu** — l'écart absolu est $\text{dist} - \text{restLength}$, sans la division.
3. La distance au repos — c'est restLength , pas diff .
4. La direction de la correction — la direction est delta , pas diff .

H3b. Pourquoi corrige-t-on de $0.5 * \text{diff}$ et non diff ?

1. Parce qu'on répartit l'écart **équitablement** entre les deux particules — chacune absorbe la **moitié** de la correction. Si on corrigeait de diff , chaque particule se déplacerait de l'écart **total**, doublant la correction.
2. Parce que 0.5 est un coefficient de damping — le damping est une autre chose (section 3 du cours).
3. Parce que la contrainte est **semi-rigide** — la rigidité vient des **itérations**, pas du facteur 0.5.
4. C'est un choix arbitraire — non, c'est la **moitié** de l'écart pour chaque particule, ce qui donne la correction **totale** exacte.

H3c. Pourquoi itère-t-on cette fonction N fois par frame ? Que se passe-t-il avec $N = 1$ vs $N = 20$?

1. Corriger une contrainte **déplace ses voisines** — il faut itérer pour propager les corrections jusqu'à convergence. $N = 1$ donne un tissu **élastique** (s'étire), $N = 20$ donne un tissu **rigide**. Les itérations **sont** la raideur en PBD.
2. Plus d'itérations = plus de précision numérique — ce n'est pas une question de précision, c'est la **convergence** d'un système couplé.
3. La formule l'exige mathématiquement — une seule passe suffit mathématiquement pour une contrainte isolée, mais les contraintes sont **couplées**.
4. Pour éviter la division par zéro — la garde `if (dist === 0)` gère déjà ce cas, les itérations n'ont rien à voir.

H4. Damping — QCM

En Verlet, le damping est appliqué comme un coefficient $d \leq 1$ sur la vitesse implicite. Si $d = 0.99$:

1. La particule perd 1% de sa vitesse par frame — elle conserve 99% de sa vitesse. C'est une **dissipation douce**, typique pour un drapeau qui flotte.
2. La particule perd 99% de sa vitesse par frame — c'est l'inverse : $d = 0.99$ signifie qu'on **garde** 99%, pas qu'on perd 99%.
3. La particule ne perd **aucune** énergie — c'est seulement si $d = 1$ (aucun amortissement).
4. La particule s'arrête en **une frame** — c'est seulement si $d = 0$ (plus d'inertie).

Partie I — Moteur de Particules (Session 9)

I1. Object Pool — QCM

Pourquoi utilise-t-on un **object pool** plutôt que de créer/détruire des particules à chaque frame ?

1. Pour éviter les pics de **garbage collection** — créer/détruire des objets chaque frame provoque des pauses du GC et des saccades. Un pool alloue **une fois** un tableau fixe et **recycle** les particules mortes. Zéro allocation pendant le jeu.
2. Pour **sauver de la mémoire** — un pool utilise **autant** de mémoire qu'un système dynamique, mais de façon **fixe** et prévisible.
3. Pour rendre les particules **plus rapides** à calculer — la physique des particules est la même ; c'est l'**allocation** qui est évitée.
4. Parce que JavaScript **exige** un object pool — c'est une **optimisation**, pas une obligation du langage.

I2. Euler vs Verlet pour les particules — QCM

Pourquoi l'intégration d'Euler est-elle **légitime** pour un moteur de particules VFX, alors qu'elle était bannie pour le tissu (session 8) ?

1. Les particules vivent **peu de temps**, sont très **amorties** (drag fort), et n'ont pas de **contraintes couplées** — la dérive d'Euler est invisible sur 1 s, le drag dissipe plus que ce qu'Euler ajoute, et il n'y a pas de contraintes de distance à maintenir.
2. Les particules sont **plus légères**, donc Euler est plus précis — la masse n'affecte pas la précision de l'intégrateur.
3. Euler est **toujours** meilleur que Verlet — Euler a été banni pour le tissu précisément parce qu'il dérive.
4. On utilise **Verlet** pour les particules, pas Euler — le cours dit explicitement qu'on **revient** à Euler pour la VFX.

I3. Forces du feu — QCM

Pour produire un effet de **feu** (les particules **montent**), que doit valoir la « gravité effective » $g_{\text{eff}} = g \cdot \left(1 - \frac{\rho_{\text{air}}}{\rho_{\text{particule}}}\right)$?

1. **Positive** (vers le haut) — le feu est moins dense que l'air ($\rho_{\text{particule}} < \rho_{\text{air}}$), donc la flottabilité (Archimède) **dépasse** la gravité. Le preset fire utilise gravity: +2.5 : la particule **monte**.
2. **Négative** (vers le bas) — cela produirait de la **pluie**, pas du feu. Les particules tomberaient.
3. **Zéro** — les particules flotteraient immobiles, sans monter ni descendre.
4. Ça dépend de la **taille** des particules — la flottabilité dépend de la **densité**, pas de la taille.

I4. Collision avec le sol — QCM

Quand une particule touche le sol ($y < y_{\text{sol}}$), que doit-on faire à sa vitesse verticale v_y ?

1. **Inverser et multiplier par la restitution** : $v_{y'} = -e \cdot v_y$ — cela donne un rebond amorti. Seule la composante **normale** (y) est affectée ; les composantes tangentielles (v_x, v_z) sont **conservées** (pas de friction de surface).

2. **Mettre à zéro** — la particule s'arrêterait net, sans rebond. C'est le cas $e = 0$.
3. **Inverser sans amortissement** : $v_{y'} = -v_y$ — c'est le cas $e = 1$ (élastique parfait), la particule rebondit indéfiniment.
4. **Multiplier par e sans inverser** : $v_{y'} = e \cdot v_y$ — la particule **accélérerait** vers le bas, ce qui n'a aucun sens.

Partie J — Rotations et Corps Rigides (Session 10)

J1. Analogie linéaire → angulaire — QCM

Associez chaque concept linéaire à son équivalent angulaire :

1. Force $\vec{F} \rightarrow$
2. Masse $m \rightarrow$
3. Vitesse $\vec{v} \rightarrow$
4. Quantité de mouvement $\vec{p} = m\vec{v} \rightarrow$

Options : a. Moment d'inertie I — résistance à la rotation (comme m résiste à la translation). b. Couple $\vec{\tau}$ — cause la rotation (comme $\vec{F}$ cause la translation). c. Vitesse angulaire $\vec{\omega}$ — taux de rotation (comme $\vec{v}$ est le taux de déplacement). d. Moment angulaire $\vec{L} = I\vec{\omega}$ — quantité de mouvement de rotation.

J2. Couple — QCM

Vous poussez une porte. Que détermine le **couple** (et donc la rotation de la porte) ?

1. La force **ET** le bras de levier ($\tau = r \times F$) — pousser **au bout** de la porte (r grand) produit beaucoup de rotation ; pousser **près de la charnière** (r petit) produit peu de rotation ; pousser **sur la charnière** ($r = 0$) ne produit **aucune** rotation, peu importe la force.
2. **Uniquement la force** — la même force produit des couples très différents selon le point d'application.
3. **Uniquement le bras de levier** — sans force, il n'y a pas de couple.
4. La **masse** de la porte — la masse détermine l'inertie (I), pas le couple (τ).

J3. Moment d'inertie — QCM

Une sphère **pleine** a $I = \frac{2}{5}mR^2$. Une sphère **creuse** (même m , même R) a $I = \frac{2}{3}mR^2$. Laquelle est plus difficile à faire tourner ?

1. La sphère **creuse** — sa masse est concentrée **sur le bord** (plus loin de l'axe), donc I est plus grand. Plus la masse est loin de l'axe, plus I est grand, plus il est difficile de faire tourner. (Intuition : le patineur qui écarte les bras tourne plus lentement.)
2. La sphère **pleine** — sa masse est répartie **partout**, donc plus proche de l'axe en moyenne. I est **plus petit**.
3. Elles sont **identiques** — même m et même R , mais la **répartition** de la masse diffère, et c'est ça qui compte.
4. Ça dépend de la **vitesse** — le moment d'inertie est une propriété **géométrique** de l'objet, indépendante de la vitesse.

J4. Roulement sans glissement — QCM

J4a. Quelle est la condition de roulement sans glissement ?

1. $v_G = \omega R$ — la vitesse du centre égale la vitesse tangentielle de rotation. Le point de contact est **immobile** par rapport au sol ($\vec{v}_P = 0$).
2. $v_G = 0$ — la boule ne bouge pas, ce n'est pas du roulement.

3. $\omega = 0$ — la boule ne tourne pas, c'est du glissement pur.
4. $v_G = \omega R$ — il manque le rayon R ; les unités ne correspondent pas (m/s vs rad/s).

J4b. Pendant la phase de glissement d'une boule de billard, que fait la friction cinétique ?

1. Elle **ralentit** la translation (v_G diminue) ET **accélère** la rotation (ω augmente) — la même force de friction fait les deux : elle décélère le centre de masse tout en créant un couple qui accélère la rotation. Les deux convergent vers $v_G = \omega R$.
2. Elle **ralentit** uniquement la translation — elle crée aussi un couple, donc elle accélère aussi la rotation.
3. Elle **accélère** uniquement la rotation — elle décélère aussi la translation via $\vec{a} = \frac{\vec{F}}{m}$.
4. Elle **accélère** les deux — la friction **s'oppose** au glissement, elle ne l'accélère pas.

J5. Quaternions — QCM

Pourquoi utilise-t-on des quaternions plutôt que des angles d'Euler en 3D ?

1. Les angles d'Euler souffrent du **gimbal lock** (perte d'un degré de liberté quand deux axes s'alignent) et ne s'interpolent pas **naturellement** (le chemin le plus court n'est pas pris) — les quaternions résolvent ces deux problèmes.
2. Les quaternions sont **plus rapides** à calculer — la vitesse n'est pas la raison principale, c'est la **qualité** de la rotation.
3. Les quaternions sont **plus intuitifs** pour les artistes — c'est l'inverse : les angles d'Euler sont plus intuitifs mais bugués.
4. Les angles d'Euler ne peuvent **pas** représenter toutes les rotations — ils le peuvent, mais avec des problèmes (gimbal lock, interpolation).

J6. Intégration de la rotation — QCM

```
const axis = angularVelocity.clone().normalize();
const angle = angularVelocity.length() * dt;
if (angle > 1e-6) {
    const dq = new THREE.Quaternion().setFromAxisAngle(axis, angle);
    quaternion.premultiply(dq);
}
```

J6a. Pourquoi utilise-t-on premultiply plutôt que multiply ?

1. premultiply applique la rotation dans le **repère monde** ($q_{\text{new}} = dq \cdot q_{\text{old}}$) — pour un corps rigide, on veut généralement la rotation dans le repère monde. multiply l'appliquerait dans le **repère local** de l'objet.
2. premultiply est **plus rapide** — la vitesse n'est pas la raison, c'est le **repère** de la rotation.
3. premultiply **évite le gimbal lock** — le gimbal lock est un problème des angles d'Euler, pas des quaternions.
4. Il n'y a **aucune différence** — premultiply et multiply donnent des résultats **différents** (repère monde vs local).

J6b. À quoi sert la garde `if (angle > 1e-6)` ?

1. À éviter de créer un quaternion depuis un vecteur **nul** — si `angularVelocity` est nul, `normalize()` produit un vecteur invalide (division par zéro, NaN). La garde saute la mise à jour quand la rotation est négligeable.
2. À **optimiser** les performances — c'est une **correction** de robustesse, pas une optimisation.
3. À empêcher le quaternion de **grandir** indéfiniment — les quaternions sont normalisés, ils ne grandissent pas.
4. À **limiter** la vitesse de rotation — la garde ne limite rien, elle saute juste le cas trivial.

Partie K — Synthèse

K1. Choix de la méthode d'intégration — QCM

Associez chaque scénario à la meilleure méthode d'intégration :

1. a) Pluie : 3000 gouttes éphémères () →
2. b) Drapeau qui flotte pendant des heures →
3. c) Boule de billard qui s'arrête en quelques secondes →
4. d) Corde suspendue avec interaction du joueur →

Options (une par scénario) : a. **Euler** — la boule est très dissipative (friction), la dérive est invisible sur quelques secondes. b. **Verlet + PBD** — le drapeau oscille pendant des heures, Euler dériverait en énergie. PBD maintient les contraintes sans calcul de forces. c. **Euler** — les gouttes vivent quelques secondes, sont très amorties (drag), pas de contraintes couplées. La dérive est invisible. d. **Verlet + PBD** — une corde est un système de contraintes de distance couplées. Verlet tolère les déplacements manuels du joueur.

K2. Le pipeline de collision — QCM

Pourquoi le pipeline de collision utilise-t-il un **entonnoir** (Broad → Narrow → Response) plutôt que de tester toutes les paires ?

1. Parce que la brute force est en $O(N^2)$ — inutilisable au-delà de quelques dizaines d'objets. L'entonnoir **élimine** des candidats à chaque étage (95–99% à la Broad Phase), divisant le travail par un ordre de grandeur à chaque fois. Chaque étage est **moins cher** que le suivant.
2. Parce que c'est **plus précis** géométriquement — c'est l'inverse : chaque étage est **moins** précis mais **plus** rapide que le suivant.
3. Parce que c'est **requis** par la physique — l'entonnoir est une **optimisation** d'ingénierie, pas une nécessité physique.
4. Parce que ça utilise **moins de mémoire** — l'entonnoir vise la **vitesse**, pas la mémoire.

K3. Debug — QCM

Un étudiant a écrit ce code pour un drapeau Verlet + PBD. Le drapeau « explose ». Identifiez les bugs :

```
function verletIntegrate(p, dt) {
  const velocity = p.position.clone().sub(p.prevPosition);
  p.position.add(velocity);
  p.position.addScaledVector(p.acceleration, dt); // ligne A
  p.prevPosition.copy(p.position);                // ligne B
}

function satisfyConstraint(p1, p2, restLength) {
  const delta = p1.position.clone().sub(p2.position);
  const dist = delta.length();
  const diff = (dist - restLength) / restLength; // ligne C
  const correction = delta.multiplyScalar(diff);
  p1.position.add(correction);
}
```

```

    p2.position.add(correction); // ligne D
}

```

K3a. Bug sur la ligne A : `addScaledVector(p.acceleration, dt)`

1. Devrait être $dt * dt$ — en Verlet, le terme d'accélération est $a \cdot dt^2$, pas $a \cdot dt$. Avec dt au lieu de dt^2 , l'accélération est 60 fois trop forte (à 60 FPS), ce qui fait exploser les particules.
2. Devrait être $dt / 2$ — il n'y a pas de facteur $\frac{1}{2}$ dans le terme d'accélération de Verlet.
3. Le code est correct — non, Verlet utilise dt^2 , pas dt .
4. Il faut **retirer** la ligne — l'accélération est nécessaire (gravité, vent).

K3b. Bug sur la ligne B : `p.prevPosition.copy(p.position)`

1. Il faut sauvegarder l'ancienne position **avant** de la modifier — on copie la **nouvelle** position dans `prevPosition`, mais elle a déjà été modifiée. La vitesse implicite ($x_n - x_{n-1}$) devient nulle ou incohérente. Il faut cloner `temp` au début, puis `p.prevPosition.copy(temp)` à la fin.
2. Le code est correct — non, on écrase `prevPosition` avec la position **déjà modifiée**.
3. Il faut retirer la ligne — sans cette ligne, la vitesse implicite ne se met pas à jour.
4. Il faut copier **après** la fonction — la copie doit se faire **à la fin** de la fonction, mais sur l'ancienne position sauvegardée.

K3c. Bug sur la ligne D : `p2.position.add(correction)`

1. Devrait être `p2.position.sub(correction)` — les deux particules doivent être corrigées en **sens opposé** (l'une rapprochée, l'autre éloignée). Avec `add` pour les deux, elles se déplacent dans la **même** direction — la contrainte n'est jamais satisfaite et le système diverge.
2. Le code est correct — non, les deux particules se déplacent dans le **même** sens, ce qui est physiquement faux.
3. Devrait être `p2.position.add(correction.clone().multiplyScalar(2))` — cela aggrave le problème.
4. Il faut retirer la ligne — sans corriger `p2`, seule une particule est déplacée.

Session 11 : Narrow Phase — SAT, GJK et Génération de Contact

Objectifs de la session

- Comprendre le rôle exact de la **Narrow Phase** dans le pipeline de collision.
- Maîtriser les **tests simples** : Sphère-Sphère, Sphère-AABB, AABB-AABB.
- Apprendre le **SAT (Separating Axis Theorem)** pour les polygones et boîtes orientées (OBB).
- Apprendre le **GJK (Gilbert-Johnson-Keerthi)** pour les formes convexes quelconques.
- Découvrir l'**EPA (Expanding Polytope Algorithm)** pour calculer la profondeur de pénétration.
- Générer un **contact complet** (point, normale, profondeur) prêt pour la Response.

Rappel de la Session 7

La Broad Phase a filtré les paires candidates avec des boîtes simplifiées (AABB). La Narrow Phase répond à la question : « **Ces deux objets se touchent-ils vraiment ?** » — et si oui, **comment** (point, normale, profondeur).

Partie 1 : Le Pipeline — Où se situe la Narrow Phase

Le pipeline en entonnoir

Un moteur physique moderne suit un **pipeline en entonnoir** qui élimine progressivement les faux positifs. Chaque étage divise le travail par un ordre de grandeur :

1. **Broad Phase** — « Qui pourrait entrer en collision ? » → 500 000 paires → 2 000 candidates.
2. **Narrow Phase** — « Se touchent-ils vraiment ? » → 2 000 candidates → 200 confirmées.
3. **Response** — « Que faire ? » → Impulsion + correction (vu en Session 6).

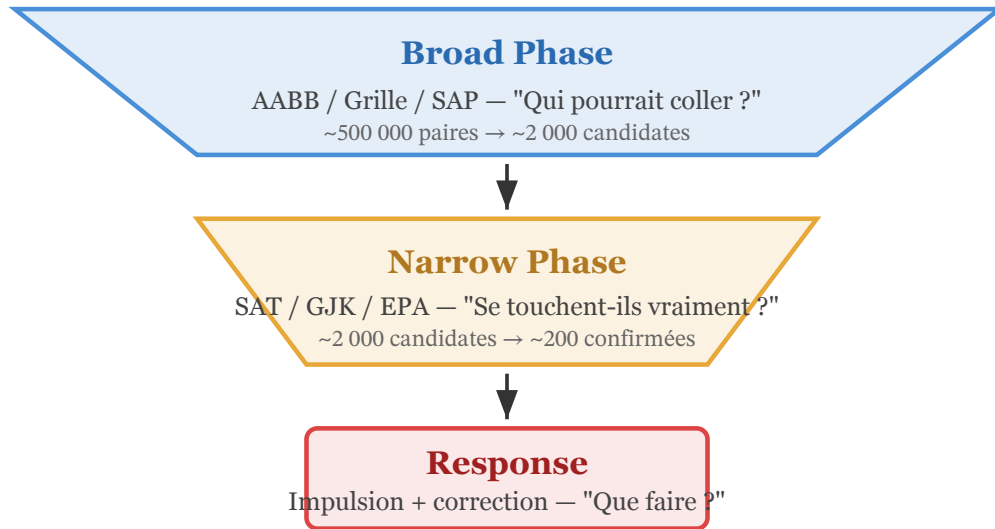


Fig. 15. – Le pipeline de détection de collision en entonnoir. La Broad Phase élimine 95–99 % des paires avec des tests triviaux. La Narrow Phase confirme les collisions avec des tests géométriques précis mais coûteux. La Response ne traite que les collisions réelles.

Le budget temps

À 60 FPS, la physique dispose de **2 ms** par frame. La Broad Phase consomme < **0.5 ms**. La Narrow Phase et la Contact Generation ensemble doivent tenir dans le reste. C'est pourquoi on n'applique les tests coûteux (SAT, GJK) qu'aux paires **candidates** — pas aux N^2 paires.

Partie 2 : Hiérarchie des Tests Narrow Phase

Du moins cher au plus cher

On applique **toujours** les tests du moins cher au plus cher. Si un test simple échoue (pas de collision), on s'arrête — inutile de lancer GJK. C'est la même philosophie que l'entonnoir : chaque étage élimine des candidats à moindre coût.

Hiérarchie des tests Narrow Phase (du moins cher au plus cher)

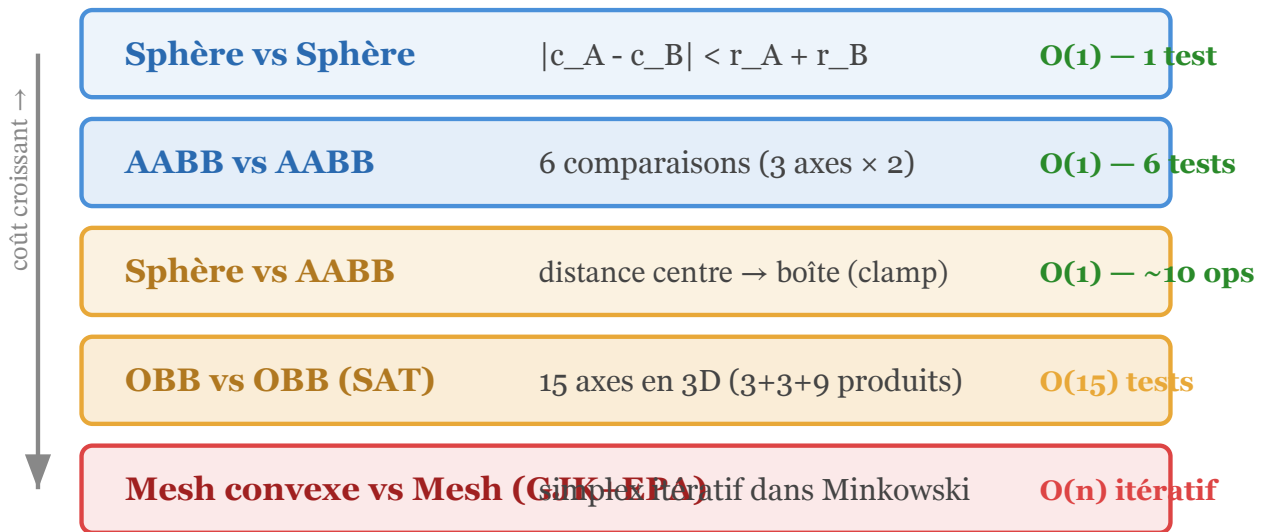


Fig. 16. – Hiérarchie des tests Narrow Phase, du moins cher (Sphère-Sphère, 1 comparaison) au plus cher (GJK+EPA pour meshes convexes). On remonte la hiérarchie uniquement si le test précédent n'a pas pu conclure.

Test 1 : Sphère vs Sphère — le plus simple

Principe

Deux sphères se touchent si la distance entre leurs centres est inférieure à la somme de leurs rayons :

$$\vec{d} = \vec{c}_B - \vec{c}_A$$

$$\text{collision} \Leftrightarrow \|\vec{d}\| < r_A + r_B$$

Pour éviter le calcul de racine carrée, on compare les carrés :

$$\vec{d} \cdot \vec{d} < (r_A + r_B)^2$$

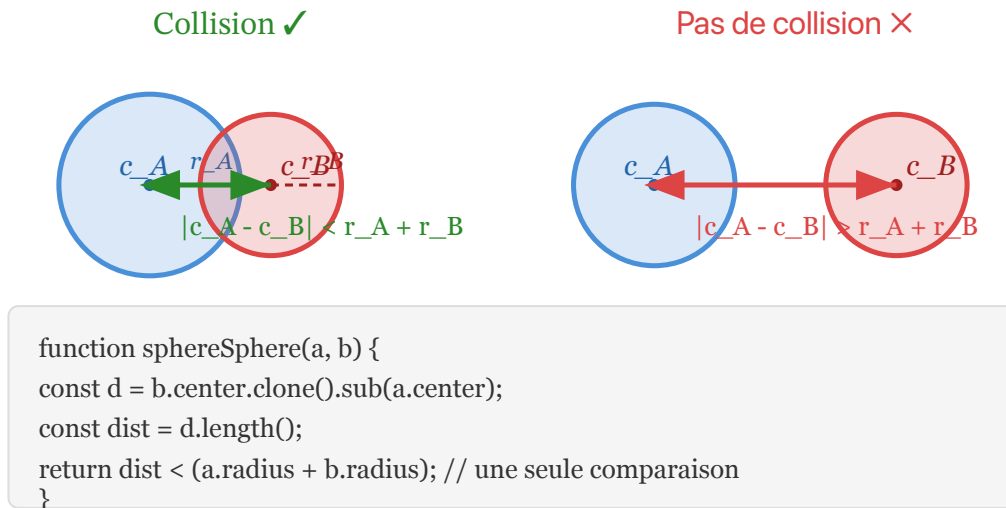


Fig. 17. – Test Sphère vs Sphère. À gauche : collision — la distance entre centres est inférieure à la somme des rayons. À droite : pas de collision — la distance est supérieure. Un seul test, $O(1)$.

💡 Optimisation : comparer les carrés

$\text{dist}^2 = dx*dx + dy*dy + dz*dz$ est beaucoup plus rapide que $\text{dist} = \text{sqrt}(dx*dx + dy*dy + dz*dz)$. On compare donc $\text{dist}^2 < (r_A + r_B)^2$. La racine carrée est une instruction lente — on l'évite toujours quand possible.

Test 2 : AABB vs AABB — déjà vu

Rappel (Session 7)

Deux AABB se chevauchent si et seulement si leurs intervalles se chevauchent sur **tous** les axes :

$$\text{overlap}_x = (x_{\max}^A \geq x_{\min}^B) \wedge (x_{\max}^B \geq x_{\min}^A)$$

Si un seul axe ne se chevauche pas, il n'y a pas de collision. **6 comparaisons en 3D.**

Test 3 : Sphère vs AABB — le clamp

Principe

On trouve le point de l'AABB le plus proche du centre de la sphère en **clampant** les coordonnées du centre sur les bornes de l'AABB :

$$\text{closest}_x = \text{clamp}(c_x, x_{\min}, x_{\max})$$

$$\text{closest}_y = \text{clamp}(c_y, y_{\min}, y_{\max})$$

$$\text{closest}_z = \text{clamp}(c_z, z_{\min}, z_{\max})$$

Puis on mesure la distance entre le centre et ce point :

$$\text{collision} \Leftrightarrow \|\vec{c} - \text{closest}\| < r$$

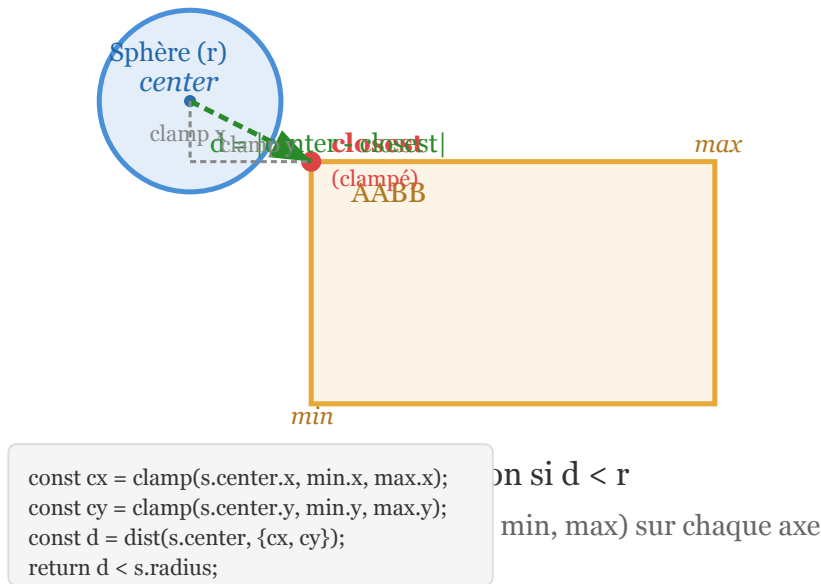


Fig. 18. – Test Sphère vs AABB. Le point le plus proche de l'AABB est obtenu en clampant le centre de la sphère sur chaque axe. Si la distance entre le centre et ce point clampé est inférieure au rayon, il y a collision.

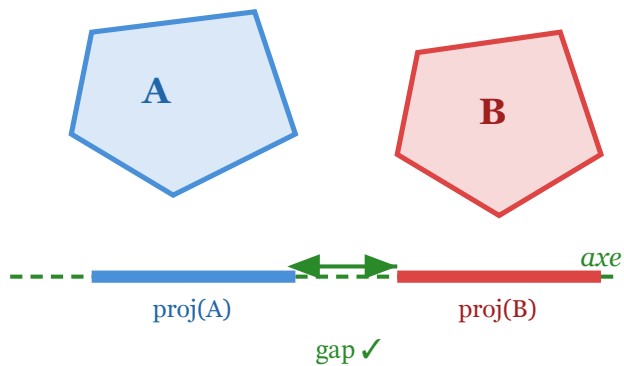
Partie 3 : SAT — Separating Axis Theorem

Le théorème

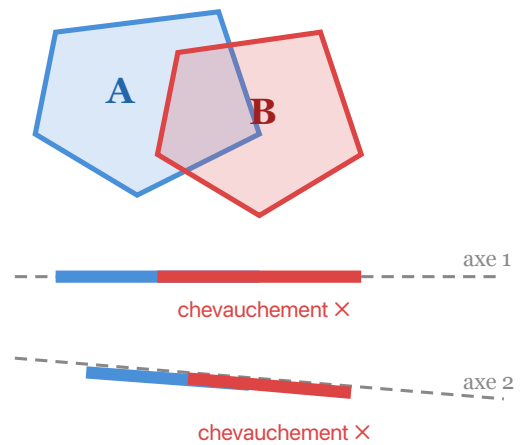
Si deux **polygones convexes** ne se touchent pas, alors il existe un axe (une droite) sur lequel leurs **projections** ne se chevauchent pas. Cet axe est appelé **axe séparateur**.

Inversement : si on teste tous les axes candidats et qu'aucun ne sépare les deux formes, alors **elles se touchent**.

Axe séparateur trouvé → pas de collision



Aucun axe séparateur → collision

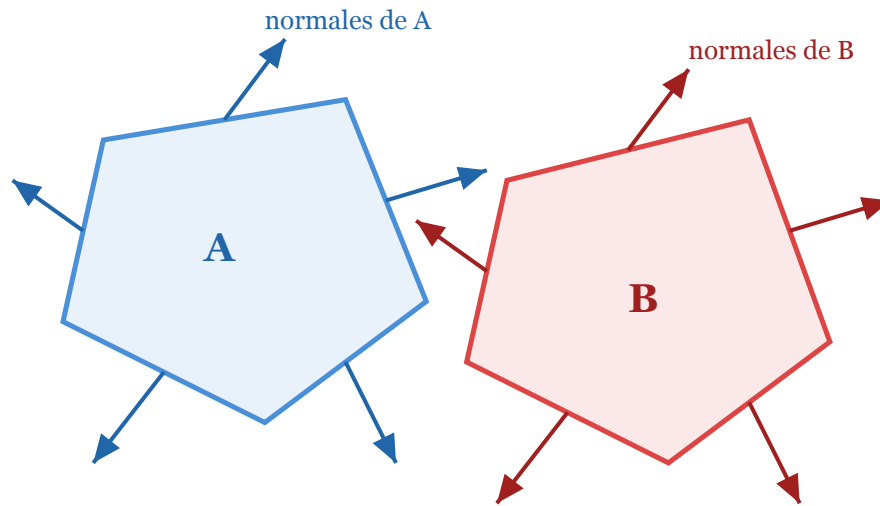


Tous les axes chevauchent → collision confirm

Fig. 19. – Principe du SAT. À gauche : un axe séparateur est trouvé — les projections de A et B sur cet axe ont un **gap** → pas de collision. À droite : tous les axes testés montrent un chevauchement → collision confirmée.

❗ Quels axes tester ?

On ne teste pas **tous** les axes possibles (il y en a une infinité). On teste uniquement les **normales des faces** des deux polygones. Pour deux polygones convexes en 2D, cela fait $n_A + n_B$ axes (où n est le nombre de côtés). Si un seul de ces axes sépare les projections, c'est fini — pas de collision.



On teste chaque normale de face comme axe de projection. Si une seule sépare → pas de collision

Fig. 20. – Les axes à tester pour SAT sont les normales des faces des deux polygones (en bleu pour A, en rouge pour B). On projette tous les sommets sur chaque axe et on regarde si les intervalles se chevauchent.

Comment projeter sur un axe

Projection d'un sommet

Pour projeter un sommet P sur un axe $\hat{n}$ (vecteur unitaire), on calcule le produit scalaire :

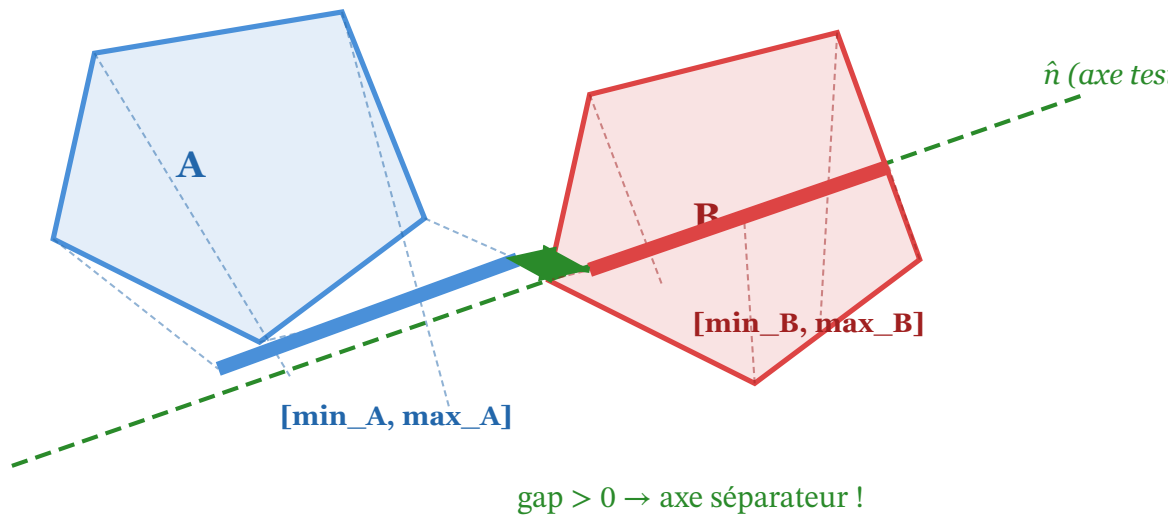
$$\text{proj}(P) = P \cdot \hat{n}$$

Pour un polygone entier, on projette **tous** ses sommets et on garde l'intervalle :

$$\text{intervalle} = [\min(\text{proj}(P_i)), \max(\text{proj}(P_i))]$$

Deux intervalles $[a_{\min}, a_{\max}]$ et $[b_{\min}, b_{\max}]$ se chevauchent si :

$$a_{\max} \geq b_{\min} \wedge b_{\max} \geq a_{\min}$$



Projection: $\text{proj}(P) = P \cdot \hat{n}$. Intervalle = $[\min(\text{proj}), \max(\text{proj})]$ sur tous les sommets.

Fig. 21. – Détail d’une projection SAT. Tous les sommets de A et B sont projetés sur l’axe $\hat{n}$. On obtient deux intervalles. Si un **gap** existe entre les intervalles, cet axe est séparateur — pas de collision.

L’algorithme SAT en résumé

SAT — algorithme

1. Collecter les axes candidats : normales de toutes les faces de A et B.
2. Pour chaque axe $\hat{n}$:
 - Projeter tous les sommets de A sur $\hat{n} \rightarrow$ intervalle $[a_{\min}, a_{\max}]$.
 - Projeter tous les sommets de B sur $\hat{n} \rightarrow$ intervalle $[b_{\min}, b_{\max}]$.
 - Si les intervalles **ne se chevauchent pas** $\rightarrow$ **axe séparateur trouvé** $\rightarrow$ pas de collision, on s’arrête.
3. Si **aucun** axe ne sépare $\rightarrow$ collision confirmée.

L’axe avec le **plus petit chevauchement** donne la **normale de contact** et la **profondeur de pénétration** — gratuitement !

💡 Bonus : contact gratuit

SAT ne dit pas juste « oui/non ». L’axe avec le **plus petit chevauchement** de projection est la **normale de contact**, et la valeur de ce chevauchement est la **profondeur de pénétration**. Pas besoin d’EPA — SAT donne le contact directement.

SAT en 3D : OBB vs OBB

Oriented Bounding Box (OBB)

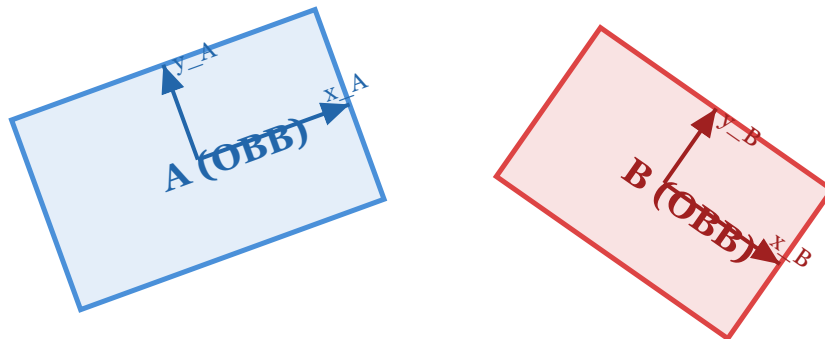
Une OBB est comme une AABB, mais **orientée** — elle peut être tournée. Elle suit les axes **locaux** de l'objet, pas les axes du monde. Plus précise qu'une AABB pour les objets allongés et tournés (un fusil, une poutre).

! 15 axes en 3D

Pour deux OBB en 3D, SAT doit tester **15 axes** :

- **3 normales de faces de A** (x_A, y_A, z_A — les axes locaux de A).
- **3 normales de faces de B** (x_B, y_B, z_B — les axes locaux de B).
- **9 produits vectoriels** ($x_A \times x_B, x_A \times y_B, \dots$ — une pour chaque combinaison d'arêtes).

Si un seul de ces 15 axes sépare les projections $\rightarrow$ pas de collision. Sinon $\rightarrow$ collision, et l'axe avec le plus petit chevauchement donne le contact.



En 3D, SAT teste 15 axes pour OBB vs OBB :

- 3 normales de faces de A (x_A, y_A, z_A) • 9 produits vectoriels ($x_A \times x_B, x_A \times y_B, \dots$)
- 3 normales de faces de B (x_B, y_B, z_B) Total = 3 + 3 + 9 = 15 axes à tester

Fig. 22. – OBB vs OBB : deux boîtes orientées différemment. SAT teste 15 axes en 3D — les 3 normales de faces de chaque boîte, plus 9 produits vectoriels d'arêtes. C'est plus coûteux que l'AABB (6 comparaisons) mais reste direct et non itératif.

Limites de SAT

⚠ Quand SAT ne suffit pas

SAT fonctionne uniquement pour des formes **convexes**. Un polygone concave (en L, en U, en étoile) ne peut pas être testé avec SAT directement — il faut décomposer la forme en sous-formes convexes (convex decomposition), ce qui est coûteux. Pour les formes convexes **quelconques** (meshes convexes, capsules, cônes), on utilise **GJK** à la place.

Partie 4 : GJK — Gilbert-Johnson-Keerthi

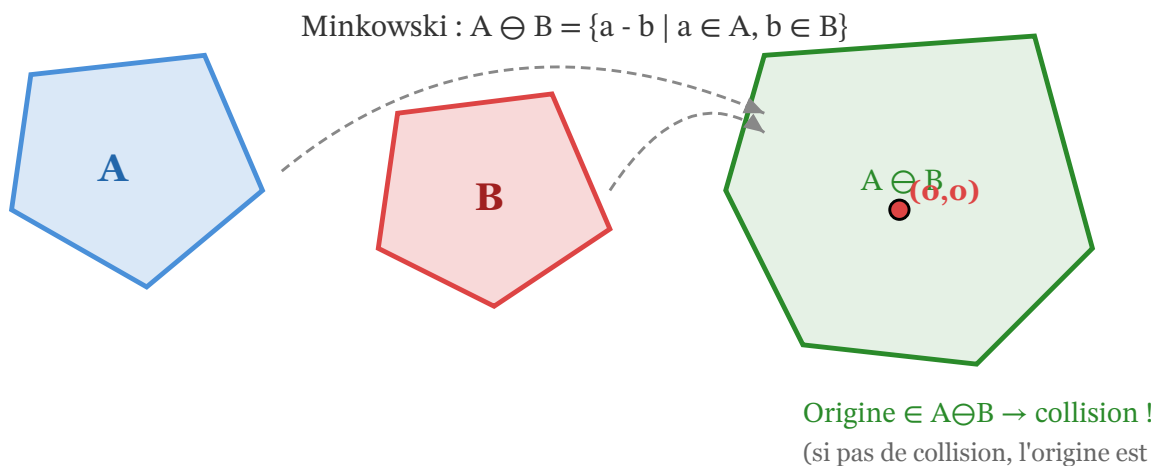
Le principe de GJK

GJK travaille dans l'**espace de Minkowski**. Au lieu de comparer directement deux formes A et B, on construit la **différence de Minkowski** :

$$A \ominus B = \{ \vec{a} - \vec{b} \mid \vec{a} \in A, \vec{b} \in B \}$$

Propriété clé : A et B se touchent si et seulement si l'origine (0, 0, 0) est **à l'intérieur** de $A \ominus B$.

GJK ne construit pas $A \ominus B$ explicitement (ce serait trop coûteux). Il l'**échantillonne** itérativement avec la **fonction de support**.



GJK ne construit pas $A \ominus B$ explicitement — il échantillonne avec la fonction de support.

Fig. 23. – La différence de Minkowski $A \ominus B$. Si l'origine est à l'intérieur de cette forme, A et B sont en collision. GJK n'a pas besoin de construire toute la forme — il l'échantillonne avec la fonction de support.

La fonction de support

Fonction de support

La fonction de support $S_A(\vec{d})$ retourne le point de A le plus extrême dans la direction $\vec{d}$:

$$S_A(\vec{d}) = \operatorname{argmax}_{P \in A} (P \cdot \vec{d})$$

C'est-à-dire : « **Quel est ton point le plus loin dans la direction $\vec{d}$?** »

Pour la différence de Minkowski :

$$S_{A \ominus B}(\vec{d}) = S_A(\vec{d}) - S_B(-\vec{d})$$

On n'a besoin que des fonctions de support de A et B séparément — jamais de construire $A \ominus B$.

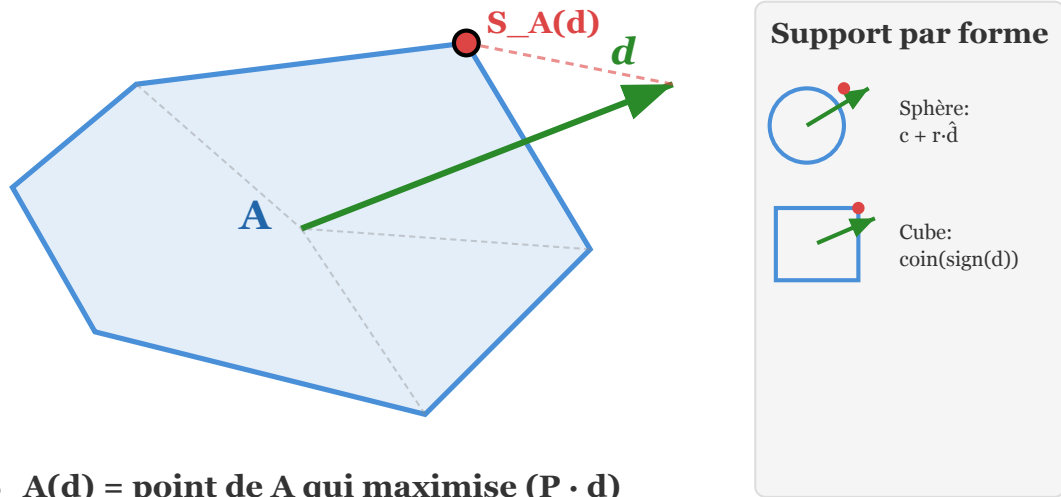


Fig. 24. – La fonction de support. Pour une direction $\vec{d}$ donnée, $S_{A(\vec{d})}$ est le sommet de A qui maximise le produit scalaire avec $\vec{d}$. Pour une sphère, c'est $\vec{c} + r \cdot \hat{\vec{d}}$. Pour un cube, c'est le coin correspondant au signe des composantes de $\vec{d}$.

💡 Support par forme

La fonction de support est **triviale** pour les formes primitives :

- **Sphère** : $S(\vec{d}) = \vec{c} + r \cdot \hat{\vec{d}}$ — le centre plus le rayon dans la direction.
- **AABB / OBB** : le coin correspondant au signe des composantes de $\vec{d}$.
- **Mesh convexe** : parcourir tous les sommets et garder celui avec le plus grand produit scalaire — $O(n)$ mais on peut précalculer.

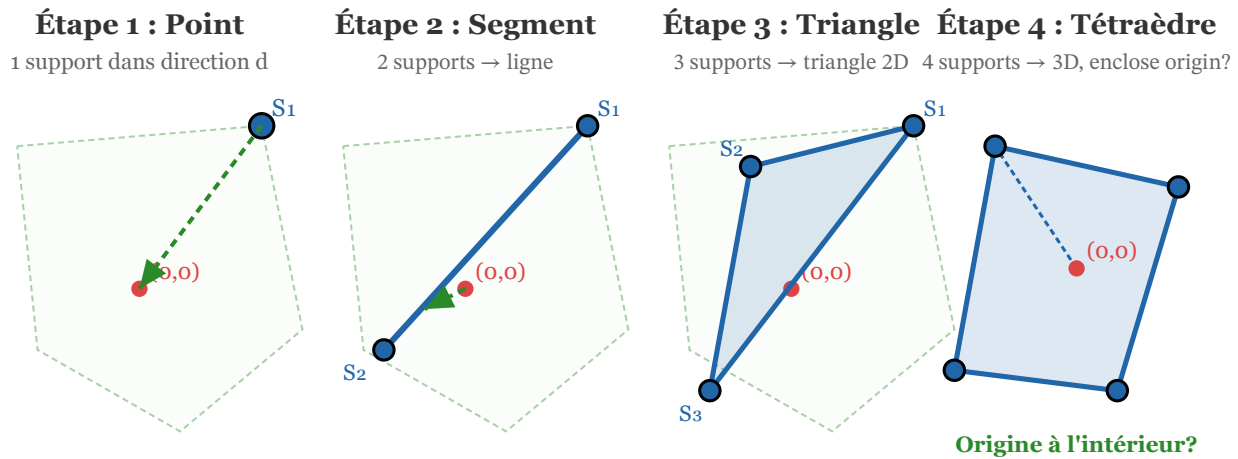
L'algorithme GJK — construire un simplex

Simplex

Un **simplex** est la généralisation d'un triangle en toute dimension :

- 0D : un **point**.
- 1D : un **segment** (2 points).
- 2D : un **triangle** (3 points).
- 3D : un **tétraèdre** (4 points).

GJK construit itérativement un simplex dans $A \ominus B$ qui **encercle l'origine**. Si le simplex finit par contenir l'origine → collision. Si on ne peut plus l'étendre → pas de collision.



Algorithme GJK :

1. Support initial $\rightarrow$ 2. Étendre le simplex vers l'origine $\rightarrow$ 3. L'origine est-elle contenue ?
Si oui $\rightarrow$ collision. Si non (le simplex ne peut plus grandir) $\rightarrow$ pas de collision.

Fig. 25. – Évolution du simplex dans GJK. Étape 1 : un point de support. Étape 2 : un segment. Étape 3 : un triangle (2D). Étape 4 : un tétraèdre (3D). À chaque étape, on étend le simplex **vers l'origine**. Si l'origine finit à l'intérieur $\rightarrow$ collision.

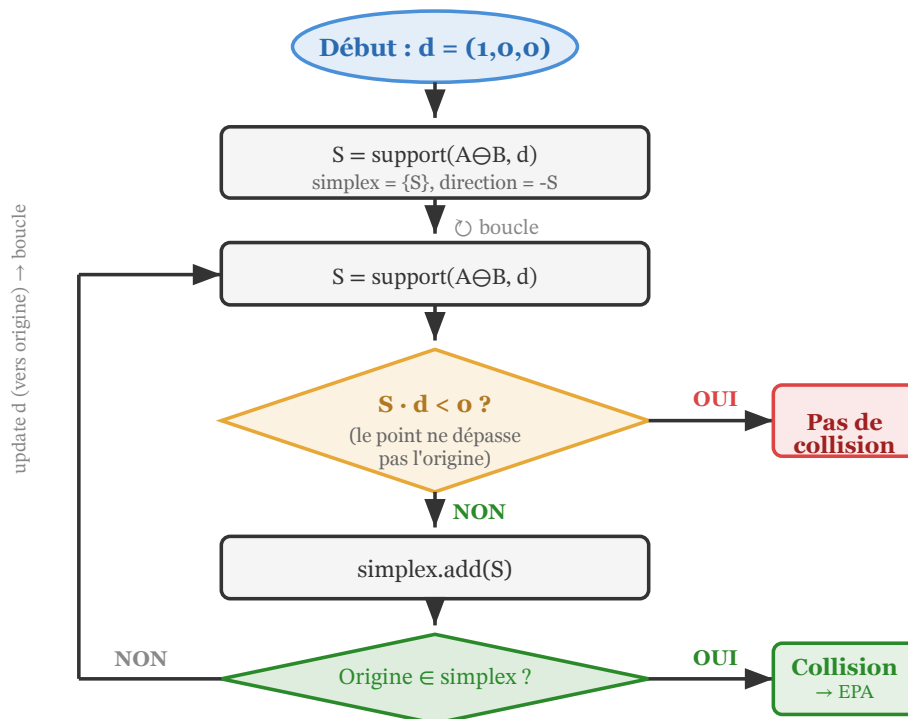


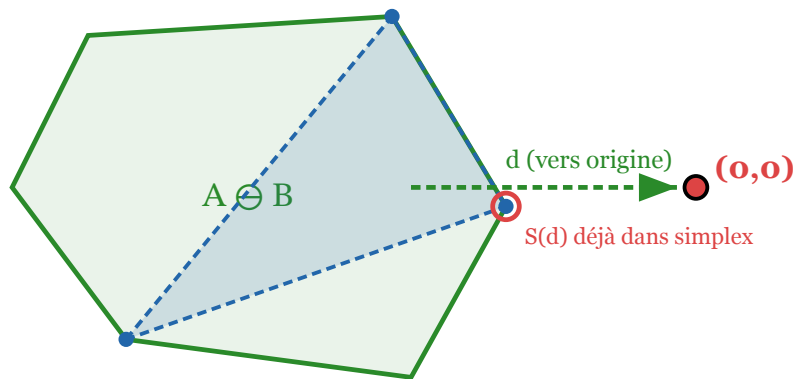
Fig. 26. – Organigramme de l'algorithme GJK. On obtient un point de support, on vérifie s'il dépasse l'origine (sinon, pas de collision), on l'ajoute au simplex, on vérifie si l'origine est contenue (si oui, collision), sinon on met à jour la direction et on boucle.

GJK : cas sans collision

Condition d'arrêt — pas de collision

À chaque itération, on demande le support dans la direction de l'origine. Si ce nouveau point **ne dépasse pas l'origine** sur cette direction (c'est-à-dire $S \cdot \vec{d} < 0$), cela signifie que l'origine est **hors de portée** — il n'y a **pas de collision**.

C'est la beauté de GJK : il peut conclure « pas de collision » dès qu'il n'y a plus de point à échantillonner dans la bonne direction.



Support dans la direction de l'origine est déjà dans le simplex → l'origine est hors $A \oplus B$ → pas de collision

Fig. 27. — GJK sans collision. L'origine est hors de $A \oplus B$. Le simplex ne peut plus grandir vers l'origine — le support dans la direction de l'origine est déjà dans le simplex. GJK conclut : pas de collision.

GJK : cas avec collision → EPA

! GJK dit oui, mais pas combien

GJK répond à la question « **Est-ce qu'il y a collision ?** » — oui ou non. Mais il ne dit pas **la profondeur de pénétration** ni **la normale de contact**. Pour obtenir ces informations, on enchaîne avec **EPA (Expanding Polytope Algorithm)**.

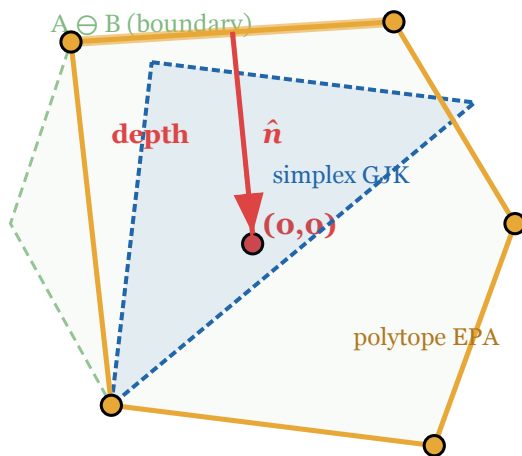
Partie 5 : EPA — Expanding Polytope Algorithm

Principe d'EPA

Une fois que GJK a confirmé la collision (le simplex contient l'origine), EPA **étend** ce simplex vers la vraie frontière de $A \oplus B$:

1. On prend le simplex final de GJK (triangle en 2D, tétraèdre en 3D).
2. On trouve l'arête/face la plus proche de l'origine.
3. On demande le support dans la direction de la normale de cette arête/face.
4. Si le nouveau point est plus loin que l'arête/face, on l'ajoute au polytope (qui grandit).

5. On répète jusqu'à ne plus pouvoir grandir — l'arête/face la plus proche de l'origine donne la **normale de contact** et la **profondeur de pénétration**.



EPA — après GJK

1. GJK dit "collision" (origine dans simplex)
2. EPA étend le simplex vers la vraie frontière
3. Trouve l'arête la plus proche de l'origine

→ normale + profondeur

Fig. 28. — EPA après GJK. Le simplex de GJK (bleu pointillé) contient l'origine. EPA étend le polytope (orange) vers la vraie frontière de $A \oplus B$. L'arête la plus proche de l'origine donne la normale $\hat{n}$ (rouge) et la profondeur de pénétration.

💡 EPA = le complément de GJK

GJK et EPA travaillent en tandem :

- **GJK** : « Y a-t-il collision ? » → OUI/NON.
- **EPA** : « Si oui, quelle est la normale et la profondeur ? »

Ensemble, ils forment le duo standard pour la détection de collision sur des formes convexes quelconques dans les moteurs professionnels (Bullet, PhysX, Box2D).

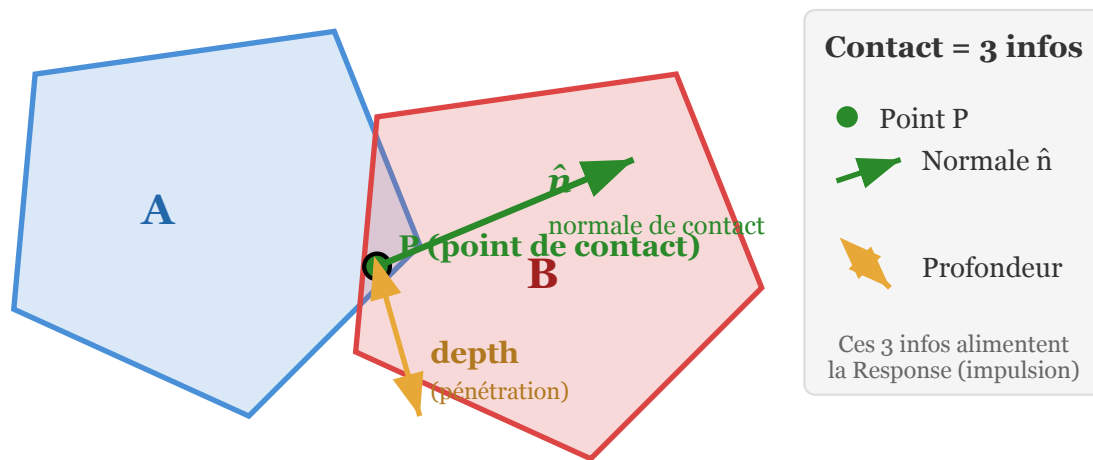
Partie 6 : Génération du Contact

Les trois ingrédients d'un contact

Une fois la collision confirmée (par SAT, GJK+EPA, ou un test simple), la Narrow Phase doit produire un **contact** avec trois informations :

1. **Point de contact** P — où les deux objets se touchent (en coordonnées du monde).
2. **Normale de collision** $\hat{n}$ — direction dans laquelle séparer les objets (de A vers B).
3. **Profondeur de pénétration** d — de combien les objets se chevauchent.

Ces trois valeurs alimentent directement la **Response** (Session 6) : l'impulsion est calculée le long de $\hat{n}$, la correction de position déplace les objets de d .



La Narrow Phase ne dit pas juste "oui/non" — elle calcule le contact complet.

Sans ces 3 valeurs, la Response (Session 6) ne peut pas appliquer d'impulsion.

Fig. 29. — Un contact complet : le point P (vert), la normale $\hat{n}$ (vert, de A vers B), et la profondeur de pénétration (orange). Sans ces trois valeurs, la Response ne peut pas appliquer d'impulsion ni corriger la position.

Contact selon la méthode

Méthode	Point	Normale	Profondeur
Sphère-Sphère	milieu du segment	direction centres	somme rayons - distance
Sphère-AABB	closest point	centre - closest	rayon - distance
AABB-AABB	centre du chevauchement	axe de pénétration min	chevauchement min
SAT	sur la face	axe séparateur min	chevauchement min
GJK + EPA	calculé par EPA	EPA arête closest	distance arête-origine

Tableau 9. — Comment chaque méthode produit le contact (point, normale, profondeur). Les tests simples donnent le contact directement. SAT donne la normale et la profondeur via l'axe de chevauchement minimal. GJK a besoin d'EPA pour obtenir ces informations.

Partie 7 : SAT vs GJK — Quand utiliser quoi

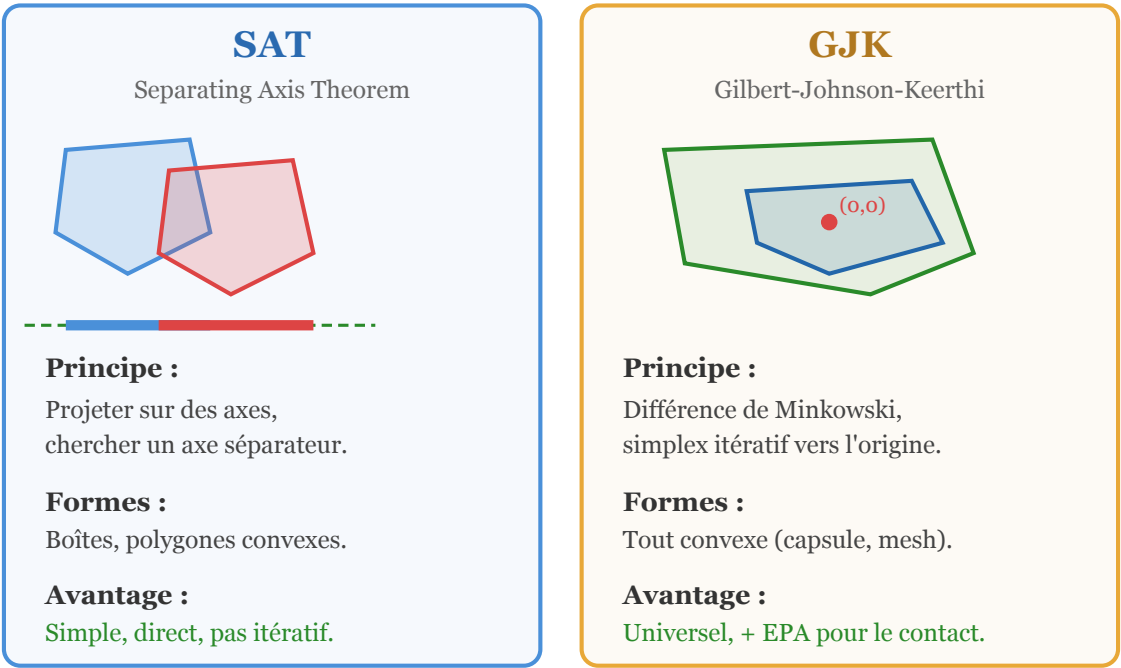


Fig. 30. – Comparaison SAT vs GJK. SAT projette sur des axes fixes et cherche un séparateur — simple et direct. GJK construit un simplex itératif dans l’espace de Minkowski — universel mais itératif.

Critère	SAT	GJK
Formes supportées	Boîtes, polygones convexes	Tout convexe (mesh, capsule, cône)
Dimension	2D : $n_A + n_B$ axes, 3D : 15 axes (OBB)	Itératif, 2–4 itérations typique
Coût par paire	$O(n)$ (nombre de sommets)	$O(k)$ itérations $\times$ coût support
Contact direct	Oui (axe min chevauchement)	Non (besoin d’EPA)
Convexes quelconques	Non (axes fixes)	Oui
Implementation	Simple	Plus complexe
Cas d’usage	OBB vs OBB, polygones 2D	Mesh convexe, capsule, formes custom

Tableau 10. – Comparaison détaillée SAT vs GJK. SAT est plus simple et donne le contact directement, mais limité aux boîtes et polygones. GJK est universel pour toute forme convexe mais nécessite EPA pour le contact.

💡 En pratique dans les moteurs

La plupart des moteurs physiques utilisent **les deux** :

- **SAT** pour les OBB (boîtes orientées) — c'est le cas le plus fréquent.
- **GJK + EPA** pour les meshes convexes et formes custom (capsules, cylindres, cônes).
- **Tests simples** (sphère-sphère, sphère-AABB) pour les cas triviaux — la majorité des collisions dans un jeu.

On commence toujours par le test le moins cher. Si on a des sphères, on teste sphère-sphère. Si on a des boîtes orientées, on teste OBB-OBB avec SAT. Si on a des meshes convexes, on passe à GJK.

Partie 8 : Le Pipeline Complet

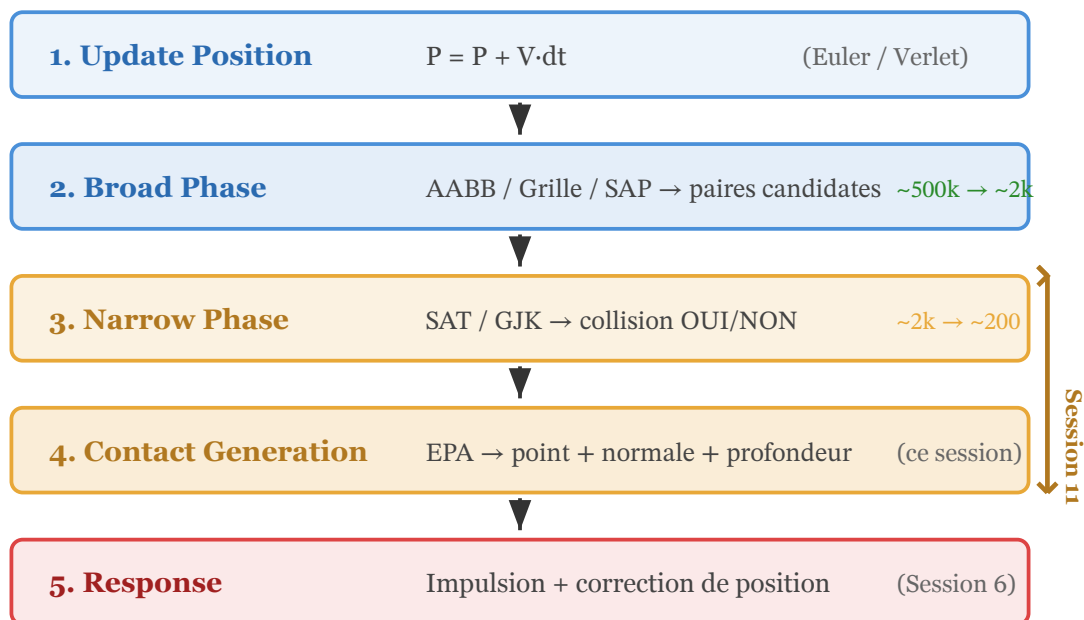


Fig. 31. – Le pipeline complet de détection et résolution de collision. La Session 11 couvre les étapes 3 (Narrow Phase) et 4 (Contact Generation). L'étape 5 (Response) a été vue en Session 6.

Résumé du pipeline

- 1. Update Position** : $P = P + V \cdot dt$ (Euler ou Verlet).
- 2. Broad Phase** : AABB / Grille / SAP → paires candidates (500k → 2k).
- 3. Narrow Phase** : Sphère-Sphère / AABB / SAT / GJK → collision OUI/NON (2k → 200).
- 4. Contact Generation** : point + normale + profondeur (EPA ou axe SAT minimal).
- 5. Response** : impulsions pour séparer les objets + correction de position (Session 6).

Partie 9 : Code — SAT en JavaScript (2D)

SAT pour deux polygones convexes 2D.

```
function sat(polyA, polyB) {
  const axes = [...getNormals(polyA), ...getNormals(polyB)];
```

```

let minOverlap = Infinity;
let minAxis = null;

for (const axis of axes) {
  // Projeter tous les sommets de A sur l'axe
  const [minA, maxA] = project(polyA, axis);
  // Projeter tous les sommets de B sur l'axe
  const [minB, maxB] = project(polyB, axis);

  // Y a-t-il un gap ?
  if (maxA < minB || maxB < minA) {
    // Axe séparateur trouvé → pas de collision
    return null;
  }

  // Mesurer le chevauchement sur cet axe
  const overlap = Math.min(maxA - minB, maxB - minA);
  if (overlap < minOverlap) {
    minOverlap = overlap;
    minAxis = axis;
  }
}

// Aucun axe séparateur → collision
// minAxis = normale de contact, minOverlap = profondeur
return { normal: minAxis, depth: minOverlap };
}

function project(poly, axis) {
  let min = Infinity, max = -Infinity;
  for (const v of poly.vertices) {
    const proj = v.dot(axis);
    min = Math.min(min, proj);
    max = Math.max(max, proj);
  }
  return [min, max];
}

```

Partie 10 : Code — GJK en JavaScript (2D)

GJK pour deux formes convexes (support function requise).

```

function gjk(supportA, supportB) {
  let d = new Vector2(1, 0); // direction initiale arbitraire
  let simplex = [support(supportA, supportB, d)];
  d = simplex[0].clone().negate(); // vers l'origine

  for (let i = 0; i < 50; i++) { // limite d'itérations
    const newPoint = support(supportA, supportB, d);

    // Le point ne dépasse pas l'origine → pas de collision
    if (newPoint.dot(d) < 0) return null;

    simplex.push(newPoint);
  }
}

```

```

// L'origine est-elle dans le simplex ?
if (containsOrigin(simplex)) {
    return { simplex }; // collision → passer à EPA
}

// Mettre à jour d : direction vers l'origine
// depuis le point/segment/triangle le plus proche
d = closestDirectionToOrigin(simplex);
// Retirer les points du simplex qui ne contribuent pas
simplex = reduceSimplex(simplex, d);
}
return null; // trop d'itérations → abandon
}

// Support de A⊗B = S_A(d) - S_B(-d)
function support(supportA, supportB, d) {
    return supportA(d).sub(supportB(d.clone().negate()));
}

```

⚠ GJK est subtil à implémenter

Les parties difficiles de GJK sont `containsOrigin` (vérifier si l'origine est dans le simplex) et `closestDirectionToOrigin` (mettre à jour la direction). En 2D, c'est gérable avec un triangle. En 3D avec un tétraèdre, les cas à traiter sont nombreux. C'est pourquoi la plupart des jeux utilisent une bibliothèque physique (Bullet, Cannon.js, Rapier) plutôt que de réimplémenter GJK.

Synthèse

💡 Ce qu'il faut retenir

- La **Narrow Phase** confirme les collisions entre paires candidates et génère le **contact** (point, normale, profondeur).
- On applique les tests du **moins cher au plus cher** : Sphère-Sphère → AABB → SAT → GJK.
- **SAT** projette sur les normales des faces — simple, direct, donne le contact gratuitement. Limité aux boîtes et polygones convexes.
- **GJK** construit un simplex dans l'espace de Minkowski — universel pour toute forme convexe, mais itératif et ne donne pas le contact directement.
- **EPA** étend le simplex de GJK pour trouver la normale et la profondeur de pénétration.
- Le **contact** (point + normale + profondeur) alimente la **Response** (impulsion + correction, Session 6).

Session 12 : TP — Système Solaire & Intégration RK4

Objectifs du TP

- Simuler un **système solaire** complet avec la gravité de Newton.
- Comprendre **pourquoi** l'intégration d'Euler ne suffit pas pour les orbites.
- Implémenter **Runge-Kutta 4 (RK4)** pour une intégration stable sur les orbites.
- Découvrir le **sub-stepping** — diviser le pas de temps pour gagner en précision.
- Trouver la **vitesse initiale** pour une orbite circulaire parfaite.
- **Bonus** : ajouter la gravité mutuelle entre planètes (N-body).

🔗 Contexte

Depuis la Session 1, on intègre les équations du mouvement avec **Euler** : $v \leftarrow v + a \cdot dt$, puis $x \leftarrow x + v \cdot dt$. C'est simple, rapide, et suffisant pour des particules amorties (Session 9) ou du tissu (Session 8). Mais pour les **orbites planétaires**, Euler accumule de l'erreur à chaque pas — les planètes dérivent de leur orbite et finissent par s'échapper. Ce TP introduit **RK4**, une méthode d'intégration qui « sonde » le pas de temps à 4 endroits pour estimer la trajectoire avec une précision $O(dt^4)$.

Partie 1 : La Gravitation Universelle

Loi de gravitation de Newton

Deux masses m_1 et m_2 séparées par une distance r s'attirent avec une force :

$$\vec{F} = G \cdot \frac{m_1 m_2}{r^2} \cdot \hat{u}$$

où G est la constante gravitationnelle, r la distance entre les centres, et $\hat{u}$ la direction de l'attraction.

L'accélération subie par un objet de masse m sous l'attraction d'une masse M est :

$$\vec{a} = G \cdot \frac{M}{r^2} \cdot \hat{u}$$

Notez que la masse m de l'objet **s'annule** — l'accélération ne dépend que de la masse attractive M et de la distance r .

💡 Pourquoi on ne ressent pas l'attraction entre deux personnes ?

Parce que $G = 6.674 \times 10^{-11} \text{N} \cdot \text{m}^2/\text{kg}^2$ est **minuscule**. Deux personnes de 70 kg à 1 m l'une de l'autre s'attirent avec une force de $\sim 3 \times 10^{-7} \text{N}$ — moins qu'un grain de poussière. Il faut une masse **planétaire** pour que la gravité devienne perceptible.

Le Soleil comme source unique

Dans ce TP, on fait une approximation : le Soleil est **fixe** au centre et est la seule source de gravité. Les planètes sont attirées par le Soleil mais ne s'attirent pas entre elles (sauf bonus).

Accélération vers le Soleil

Pour une planète à la position $\vec{r}$ (le Soleil est à l'origine), l'accélération est :

$$\vec{a} = -G \cdot \frac{M_{\text{soleil}}}{\|\vec{r}\|^2} \cdot \hat{r} = -G \cdot \frac{M_{\text{soleil}}}{\|\vec{r}\|^3} \cdot \vec{r}$$

En code :

```
function getAcceleration(position) {
  const r2 = position.lengthSq(); // ||r||²
  if (r2 < 5) return new THREE.Vector3(0,0,0); // singularité évitée
  const fMag = (G * SUN_MASS) / r2; // G·M / r²
  return position.clone().negate().normalize().multiplyScalar(fMag);
}
```

💡 Pourquoi le test $r^2 < 5$?

Si une planète atteint le centre du Soleil ($r = 0$), l'accélération devient infinie ($\frac{1}{r^2} \rightarrow \infty$). On ajoute un **seuil de sécurité** : si la distance est trop petite, on annule l'accélération. C'est une rustine — dans un vrai moteur, on gérerait la collision avec la surface du Soleil.

Partie 2 : La Vitesse Orbitale

Orbite circulaire — équilibre force gravitationnelle / force centripète

Pour qu'une planète orbite en **cercle parfait**, il faut que la force gravitationnelle exactement compense la force centripète :

$$G \cdot \frac{M}{r^2} = \frac{v^2}{r}$$

En simplifiant :

$$v = \sqrt{G \cdot \frac{M}{r}}$$

C'est la **vitesse orbitale** — la vitesse tangentielle qu'il faut donner à une planète à la distance r pour qu'elle orbite en cercle.

💬 Direction de la vitesse initiale

La vitesse doit être **perpendiculaire** à la direction du Soleil. Si la planète est sur l'axe x (position $(r, 0, 0)$), la vitesse doit être sur l'axe z ou y :

```
pos: new THREE.Vector3(distWorld, 0, 0),
vel: new THREE.Vector3(0, 0, -vOrbit), // perpendiculaire, sens trigonométrique
```

Les trois cas selon la vitesse initiale

Vitesse	Trajectoire	Forme
$v < v_{\text{orb}}$	La planète tombe vers le Soleil	Ellipse décadente / crash
$v = v_{\text{orb}}$	Orbite circulaire parfaite	Cercle
$v > v_{\text{orb}}$	La planète s'éloigne	Ellipse ou hyperbole (évasion)

Tableau 11. – Trois régimes selon la vitesse initiale. En dessous de v_{orb} , la planète spirale vers le Soleil. Au-dessus, elle s'échappe. À v_{orb} exactement, l'orbite est circulaire.

💡 Tester dans le TP

Dans la solution, changez manuellement la vitesse de la Terre : multipliez v_{orb} par 0.8 (la Terre spirale vers le Soleil) ou par 1.3 (la Terre s'échappe en ellipse large). C'est une excellente façon de **sentir** la physique.

Partie 3 : Pourquoi Euler ne Suffit Pas

Le problème d'Euler sur les orbites

L'intégration d'Euler **explicite** met à jour la vitesse puis la position :

$$v_{n+1} = v_n + a \cdot dt$$

$$x_{n+1} = x_n + v_{n+1} \cdot dt$$

Le problème : sur une orbite, la force **change constamment de direction**. Euler suppose que la vitesse est **constante** sur tout le pas dt . À chaque pas, il « dérape » légèrement tangentiellement — l'erreur s'accumule et l'orbite **grandit** progressivement. La planète gagne de l'énergie à chaque révolution.

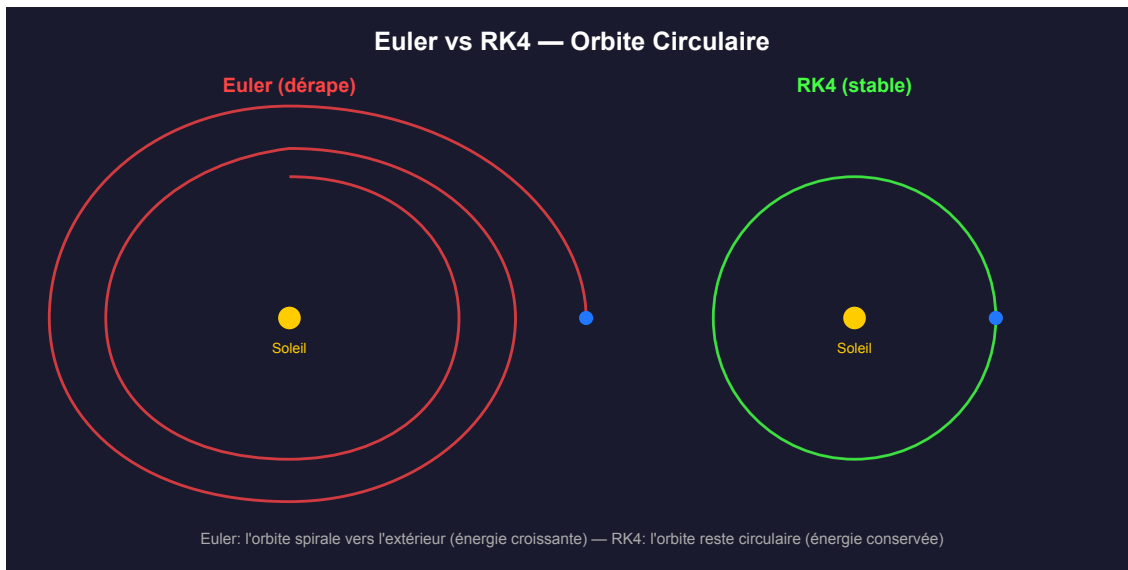


Fig. 32. — Euler vs RK4 sur une orbite circulaire. Avec Euler (rouge), l'orbite spirale vers l'extérieur — l'énergie augmente à chaque tour. Avec RK4 (vert), l'orbite reste stable même après des dizaines de révolutions.

⚠ Euler n'est pas mauvais — il est mal adapté

Euler fonctionne parfaitement pour des systèmes **amortis** (particules avec drag, tissu) où l'énergie se dissipe. Mais pour un système **conservatif** comme les orbites (pas de friction), l'erreur d'Euler ne se dissipe jamais — elle s'accumule. C'est pourquoi on a besoin d'une méthode qui **respecte l'énergie**.

Partie 4 : Runge-Kutta 4 (RK4)

L'idée de RK4

Au lieu d'évaluer l'accélération **une seule fois** par pas (comme Euler), RK4 l'évalue à **quatre endroits** du pas de temps et fait une moyenne pondérée :

1. k_1 : évaluer au début du pas (t)
2. k_2 : évaluer au milieu du pas ($t + dt/2$), en utilisant k_1
3. k_3 : évaluer au milieu du pas ($t + dt/2$), en utilisant k_2
4. k_4 : évaluer à la fin du pas ($t + dt$), en utilisant k_3

La mise à jour finale :

$$y_{n+1} = y_n + \frac{k_1 + 2k_2 + 2k_3 + k_4}{6}$$

💡 L'analogie du GPS

Euler, c'est conduire en regardant uniquement le rétroviseur : on suppose qu'on va dans la même direction qu'il y a dt secondes, puis on corrige la vitesse. RK4, c'est regarder la route à 4 endroits devant soi et faire une moyenne — beaucoup plus précis.

RK4 appliqué à la gravité

État et dérivée

Pour la gravité, l'état d'une planète est la paire $(\vec{r}, \vec{v})$ (position + vitesse). La dérivée de cet état est $(\vec{v}, \vec{a})$ — la vitesse dérive la position, l'accélération dérive la vitesse.

RK4 calcule donc 4 paires (k_x, k_v) :

1. $k_1 = (\vec{v}, \vec{a}(\vec{r}))$ — état actuel
2. $k_2 = (\vec{v} + k_{1_v} dt/2, \vec{a}(\vec{r} + k_{1_x} dt/2))$ — milieu avec k_1
3. $k_3 = (\vec{v} + k_{2_v} dt/2, \vec{a}(\vec{r} + k_{2_x} dt/2))$ — milieu avec k_2
4. $k_4 = (\vec{v} + k_{3_v} dt, \vec{a}(\vec{r} + k_{3_x} dt))$ — fin avec k_3

Mise à jour :

$$\vec{r} + = \frac{k_{1_x} + 2k_{2_x} + 2k_{3_x} + k_{4_x}}{6}$$

$$\vec{v} + = \frac{k_{1_v} + 2k_{2_v} + 2k_{3_v} + k_{4_v}}{6}$$

⚠ Ne pas modifier l'état pendant le calcul !

Les 4 étapes de RK4 évaluent l'accélération à des positions **intermédiaires** — il ne faut pas modifier `p.pos` et `p.vel` pendant le calcul. Utilisez des `.clone()` pour créer des copies des positions/vitesses intermédiaires. La mise à jour finale se fait **à la fin**, une seule fois.

RK4 en JavaScript.

```
function updatePhysicsRK4(p, dt) {
  const x0 = p.pos.clone();
  const v0 = p.vel.clone();

  // k1 - évaluer à t
  const k1_v = getAcceleration(x0).multiplyScalar(dt);
  const k1_x = v0.clone().multiplyScalar(dt);

  // k2 - évaluer à t + dt/2
  const x2 = x0.clone().addScaledVector(k1_x, 0.5);
  const v2 = v0.clone().addScaledVector(k1_v, 0.5);
  const k2_v = getAcceleration(x2).multiplyScalar(dt);
  const k2_x = v2.multiplyScalar(dt);

  // k3 - évaluer à t + dt/2
  const x3 = x0.clone().addScaledVector(k2_x, 0.5);
  const v3 = v0.clone().addScaledVector(k2_v, 0.5);
  const k3_v = getAcceleration(x3).multiplyScalar(dt);
  const k3_x = v3.multiplyScalar(dt);

  // k4 - évaluer à t + dt
  const x4 = x0.clone().add(k3_x);
  const v4 = v0.clone().add(k3_v);
  const k4_v = getAcceleration(x4).multiplyScalar(dt);
```

```

const k4_x = v4.multiplyScalar(dt);

// Mise à jour finale
p.vel.add(k1_v.addScaledVector(k2_v, 2)
        .addScaledVector(k3_v, 2)
        .add(k4_v).divideScalar(6));
p.pos.add(k1_x.addScaledVector(k2_x, 2)
        .addScaledVector(k3_x, 2)
        .add(k4_x).divideScalar(6));
}

```

Partie 5 : Le Sub-Stepping

Pourquoi le sub-stepping ?

Même avec RK4, un **trop grand pas de temps** provoque de l'erreur. Sur une orbite, la planète parcourt une portion significative de cercle en un pas — l'accélération change beaucoup entre le début et la fin du pas.

Le **sub-stepping** consiste à diviser le dt d'une frame visuelle en plusieurs sous-pas plus petits :

$$dt_{\text{sub}} = d \frac{t_{\text{frame}}}{N}$$

On calcule la physique N fois par frame, chaque fois avec un petit dt_{sub} . Le rendu visuel ne se fait qu'une fois — c'est la physique qui est affinée.

! Sub-stepping vs RK4 — les deux se complètent

- **RK4** améliore la **qualité** de chaque pas (4 évaluations au lieu d'1).
- **Sub-stepping** améliore la **taille** du pas (plus de pas plus petits).
- Ensemble, ils donnent une précision excellente même avec un `timeScale` élevé.

Sub-stepping en JavaScript.

```

function updatePhysics(dtFrame) {
  const dtSubStep = dtFrame / params.subSteps;
  for (let i = 0; i < params.subSteps; i++) {
    for (const p of planets) {
      if (params.integrator === 'Euler')
        updatePhysicsEuler(p, dtSubStep);
      else
        updatePhysicsRK4(p, dtSubStep);
    }
    totalPhysicsTime += dtSubStep;
  }
}

```

💡 Combien de sub-steps ?

Avec 20 sub-steps et RK4, les orbites restent stables même à `timeScale = 1.5`. Sans sub-stepping (`subSteps = 1`), même RK4 dérape à haute vitesse. Testez dans le TP : baissez `subSteps` à 1 et augmentez `timeScale` — vous verrez les orbites se dégrader en temps réel.

Partie 6 : Le Système Solaire — Données

Les planètes

On utilise des unités **simulées** (pas les vraies unités SI) pour garder des nombres manipulables :

- $G = 100$ (conste gravitationnelle simulée)
- $M_{\text{soleil}} = 3330$ (masse du Soleil, en unités de masse terrestre $\times 10$)
- 1 UA = 18 unités de monde (pour que tout tienne à l'écran)

Planète	Masse ($M_{\oplus}$)	Distance (UA)	Couleur
Mercure	0.055	0.39	gris
Vénus	0.815	0.72	jaune
Terre	1.000	1.00	bleu
Mars	0.107	1.52	rouge
Jupiter	317.8	5.20	ocre
Saturne	95.2	9.54	beige

💡 Pourquoi les masses ne servent à rien (sans N-body)

Sans la gravité mutuelle entre planètes, la masse de chaque planète **n'intervient pas** dans le calcul — l'accélération $a = GM_{\text{soleil}}/r^2$ ne dépend que de la masse du Soleil. Les masses sont incluses pour le **bonus N-body**.

Partie 7 : TP — Missions

💡 Mise en place

Ouvrir `session12_solar.html` via Live Server. La scène, le Soleil, les planètes (meshes), les trails, le HUD et la GUI sont fournis — il reste **cinq fonctions** à compléter. Au départ, les planètes ne bougent pas : la vitesse orbitale renvoie 0 et l'accélération est nulle. C'est normal.

Mission 1 — `computeOrbitalVelocity(distWorld)`

Calculer la vitesse pour une orbite circulaire à la distance `distWorld` du Soleil :

$$v = \sqrt{G \cdot M_{\text{soleil}} / r}$$

Utiliser `G`, `SUN_MASS` et `distWorld`. **Indices** : `Math.sqrt()`, une seule ligne.

Test : les planètes doivent apparaître sur l'axe x et commencer à orbiter dans le sens trigonométrique (vers $-z$).

Mission 2 — ``getAcceleration(position)``

Calculer l'accélération gravitationnelle du Soleil sur un objet à `position` (le Soleil est à l'origine) :

1. Direction : $-\vec{r}$ (vers le Soleil)
2. Distance : $r = \text{position.length}()$
3. Magnitude : $a = G \cdot M / r^2$
4. Retourner : direction normalisée $\times$ magnitude

Sécurité : si $r^2 < 5$, retourner un vecteur nul (éviter la singularité au centre du Soleil).

Indices : `position.lengthSq()`, `position.clone().negate().normalize()`, `multiplyScalar()`.

Mission 3 — ``updatePhysicsEuler(p, dt)``

Implémenter l'intégration d'Euler explicite :

1. Calculer l'accélération à la position actuelle
2. $v += a \cdot dt$
3. $x += v \cdot dt$ (avec la **nouvelle** vitesse)

Test : sélectionner « Euler » dans la GUI. Les planètes orbitent mais **dérangent** — l'orbite grandit à chaque révolution. C'est le comportement attendu, la preuve qu'Euler ne convient pas.

Mission 4 — ``updatePhysicsRK4(p, dt)``

Implémenter Runge-Kutta 4 (voir Partie 4). Les 4 étapes k_1, k_2, k_3, k_4 avec les positions/vitesses intermédiaires, puis la combinaison finale.

Pièges :

- Ne **pas** modifier `p.pos` et `p.vel` pendant le calcul — utiliser `.clone()`.
- L'accélération ne dépend que de la **position** (pas de la vitesse) — c'est `getAcceleration(x_intermédiaire)` à chaque étape.
- Les k_v et k_x sont déjà multipliés par dt — la combinaison finale divise par 6, pas par $6 \cdot dt$.

Test : sélectionner « RK4 » dans la GUI. Les orbites restent **stables** même après 10+ révolutions. Comparer avec Euler — la différence est frappante.

Mission 5 — ``updatePhysics(dtFrame)``

Implémenter la boucle de sub-stepping :

1. Calculer $dt_{\text{sub}} = dt_{\text{frame}} / \text{subSteps}$
2. Boucler `subSteps` fois : pour chaque sous-pas, intégrer toutes les planètes
3. Incrémenter `totalPhysicsTime` à chaque sous-pas
4. Choisir Euler ou RK4 selon `params.integrator`

Test : baisser subSteps à 1 et monter timeScale à 0.5 — les orbites se dégradent. Remonter subSteps à 20 — tout redevient stable.

Scénarios à observer

Expériences recommandées

Euler vs RK4 : avec subSteps = 20, basculer entre Euler et RK4. Euler dérape en quelques révolutions, RK4 reste stable indéfiniment.

Sub-steps : avec RK4, baisser subSteps de 20 à 1. À timeScale = 0.05, peu de différence. À timeScale = 0.5, RK4 sans sub-steps dérape — le sub-stepping est essentiel à haute vitesse.

Vitesse initiale : dans le code, multiplier vOrbit par 0.7 — la planète spirale vers le Soleil. Par 1.3 — la planète s'échappe en ellipse large. Par 1.0 — cercle parfait.

TimeScale extrême : monter timeScale à 1.5 avec RK4 + 20 sub-steps. Les années défilent en quelques secondes — on voit Mercure faire 10 tours pendant que Jupiter en fait 1.

Partie 8 : Bonus

Bonus 1 — Gravité mutuelle (N-body)

Activer params.nBody = true dans la GUI. Dans getAcceleration, ajouter l'attraction de chaque autre planète :

$$\vec{a}_{\text{total}} = \vec{a}_{\text{soleil}} + \sum_i G \cdot \frac{m_i}{r_i^2} \cdot \hat{u}_i$$

Observation : les orbites ne sont plus parfaitement circulaires — les planètes se perturbent mutuellement. Jupiter (la plus massive) dévie Mars et la Terre. C'est ainsi que Neptune a été **découvert** en 1846 — par les perturbations qu'il causait sur Uranus, avant même d'être observé au télescope !

Bonus 2 — Anneaux de Saturne

Ajouter un anneau à Saturne avec THREE.RingGeometry. Faire tourner l'anneau à une vitesse légèrement différente de Saturne (les anneaux ne sont pas solides — chaque particule orbite à sa propre vitesse).

Bonus 3 — Lune de la Terre

Ajouter une Lune qui orbite autour de la Terre. La Lune subit la gravité du Soleil **et** celle de la Terre. Il faut calculer l'accélération totale et intégrer la Lune comme une planète supplémentaire — mais sa position initiale est relative à la Terre, pas au Soleil.

Bonus 4 — Comparaison énergétique

Calculer l'énergie totale de chaque planète ($E = 1/2v^2 - GM/r$) et l'afficher dans le HUD. Avec Euler, l'énergie **augmente** progressivement. Avec RK4, elle reste **constante** — c'est la preuve que RK4 respecte la conservation d'énergie.

Bonus 5 — Post-processing et rendu

La solution utilise trois techniques de rendu pour un look « space sim » :

- **Bloom** (UnrealBloomPass) : les éléments lumineux (Soleil, trails) débordent en halo doux. Toggle dans la GUI.
- **Trails à dégradé** : au lieu d'une ligne uniforme, chaque trail utilise des `vertexColors` avec un alpha qui croît du tail (invisible) vers le head (vif). Combiné avec l'AdditiveBlending, les trails « glow ».
- **Halo du Soleil** : un Sprite avec une texture de gradient radial additif donne un halo diffus autour du Soleil.

Pour aller plus loin : ajouter un `FilmPass` (grain de film), un `Vignette` via shader, ou des étoiles scintillantes dans le fond HDR.

Synthèse

🗨 Ce qu'il faut retenir

- **Euler** est simple mais **accumule de l'erreur** sur les systèmes conservatifs (orbites) — l'énergie dérive.
- **RK4** évalue l'accélération à 4 endroits du pas de temps — précision $O(dt^4)$ au lieu de $O(dt)$.
- **Sub-stepping** divise le pas de temps en N sous-pas — indispensable à haute vitesse même avec RK4.
- La **vitesse orbitale** $v = \sqrt{GM/r}$ donne une orbite circulaire parfaite.
- **RK4 + sub-stepping** est le standard pour les simulations orbitales en temps réel.
- Pour des simulations scientifiques (NASA, ESA), on utilise des méthodes encore plus précises (symplectiques, Verlet leapfrog) qui **garantissent** la conservation d'énergie sur des millions de pas.

💡 Et dans les jeux ?

Les jeux n'ont généralement **pas besoin** de RK4 — les orbites sont rares et le sub-stepping d'Euler suffit pour la plupart des effets. RK4 est pertinent pour les simulations spatiales (Kerbal Space Program, Elite Dangerous) où la précision orbitale est le cœur du gameplay.

Session 13 : Les moteurs physiques commerciaux

Objectifs de la session

- Comprendre **pourquoi** on utilise un moteur physique existant en production plutôt que le sien.
- Découvrir les moteurs majeurs (**PhysX, Havok, Jolt, Box2D, Rapier**) et leurs cas d'usage.
- Découvrir les concepts qu'ils partagent : **CCD, sleeping, joints, query** (raycast).
- Comprendre le **déterminisme** et pourquoi il est difficile à obtenir.
- Introduire l'architecture **ECS** (Entity-Component-System) qui structure les moteurs modernes.
- **Préparer la pratique** (Session 14) : installer **Rapier** avec Three.js et activer **Jolt** dans Godot.

🔗 Le moment charnière du cours

Depuis la Session 1, vous construisez **votre propre moteur** : intégrateurs, collisions, impulsions, ressorts, particules. Vous savez maintenant **pourquoi c'est difficile**. Cette session répond à la question : « **Et en production, on refait tout ça ?** » — Non. On utilise un moteur éprouvé. Mais maintenant, vous **comprenez ce qu'il fait sous le capot**, et c'est ce qui distingue un programmeur de jeux qui **utilise** un moteur d'un programmeur qui le **comprend**.

Partie 1 : Pourquoi utiliser un moteur physique ?

Le problème de la complexité

Notre moteur « maison » couvre des cas simples : quelques dizaines d'objets, formes convexes, impulsions basiques. Un jeu réel exige :

- **Des centaines d'objets** en interaction simultanée (empilements, destructions).
- **Des formes complexes** : convexes, concaves (meshes), composées.
- **De la stabilité** : un empilement de cubes ne doit pas trembler ni exploser.
- **Des joints** : charnières, ressorts, moteurs (ragdolls, véhicules).
- **Des queries** : raycasts pour les tirs, shapecasts pour les personnages.
- **De la performance** : tout ça en moins de 2 ms par frame.

Avantages des moteurs existants

- **Complexité** : gestion des contacts multiples, stabilité des empilements, solveurs itératifs optimisés.
- **Temps** : des décennies de R&D cumulées (PhysX a plus de 20 ans).
- **Performance** : SIMD, multithreading, broad phase optimisée — des milliers d'objets à 60 FPS.
- **Outils** : debug rendering, serialization, détermination des scènes.
- **Communauté** : bugs connus, solutions documentées, intégrations prêtes.

💡 Mais pourquoi on a fait notre moteur alors ?

Parce qu'un moteur physique est une **boîte noire dangereuse** si on ne comprend pas ce qu'elle contient. Sans les Sessions 1–12, un moteur commercial est magique : « **pourquoi mon personnage traverse le sol ?** », « **pourquoi mon empilement tremble ?** », « **pourquoi**

mes objets flottent ? ». Avec elles, vous savez que c'est une question de **timestep**, de **solver iterations**, de **sleeping threshold** — et vous savez **où regarder**.

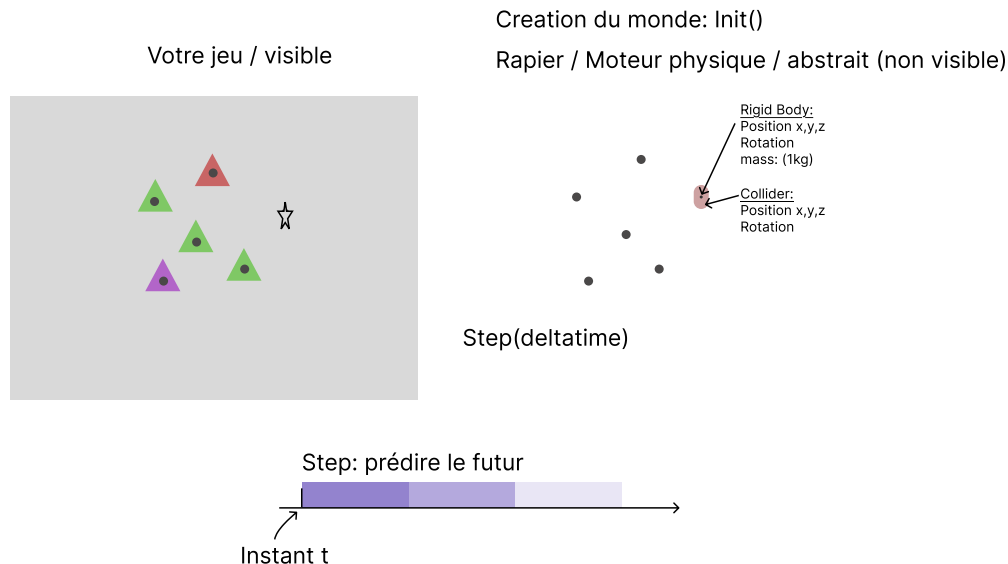


Fig. 33. – Un moteur physique commercial : on fournit les colliders et les forces, il retourne les positions et rotations. Toute la machinerie interne — broad phase, narrow phase, solveur de contraintes — est invisible mais fonctionne sur les principes vus en cours.

Partie 2 : Concepts partagés par tous les moteurs

Rigid Body vs Soft Body

Rigid Body vs Soft Body

- **Rigid Body** : objet **indéformable** (boîte, sphère, capsule). La distance entre deux points internes ne change jamais. C'est ce que tous les moteurs font nativement — et tout ce dont un jeu a besoin dans 95 % des cas.
- **Soft Body** : objet **déformable** (tissu, gelée, ballon). La forme change en fonction des forces appliquées. Beaucoup plus coûteux — souvent simulé par un système masse-ressort comme notre tissu de la Session 8, ou par des techniques spécialisées (FEM, position-based dynamics).

💡 La stratégie classique

Les jeux utilisent des **rigid bodies** presque partout, et simulent la déformation **visuellement** : un ragdoll est un ensemble de rigid bodies reliés par des joints ; une destruction est un échange d'un mesh statique contre plusieurs rigid bodies. La déformation « vraie » (soft body) est réservée aux cas où elle est le gameplay (World of Goo, JellyCar).

Continuous Collision Detection (CCD)

Le problème du tunneling

Un objet rapide peut **traverser** un mur mince entre deux frames. Si une balle va à 100 m/s avec un dt de $1/60$ s, elle se déplace de **1,67 m par pas**. Un mur de 0,2 m d'épaisseur ? La balle est devant le mur à t , derrière à $t + dt$ — aucune des deux positions n'est **dans** le mur. Le test discret (Session 6) ne voit jamais la collision.

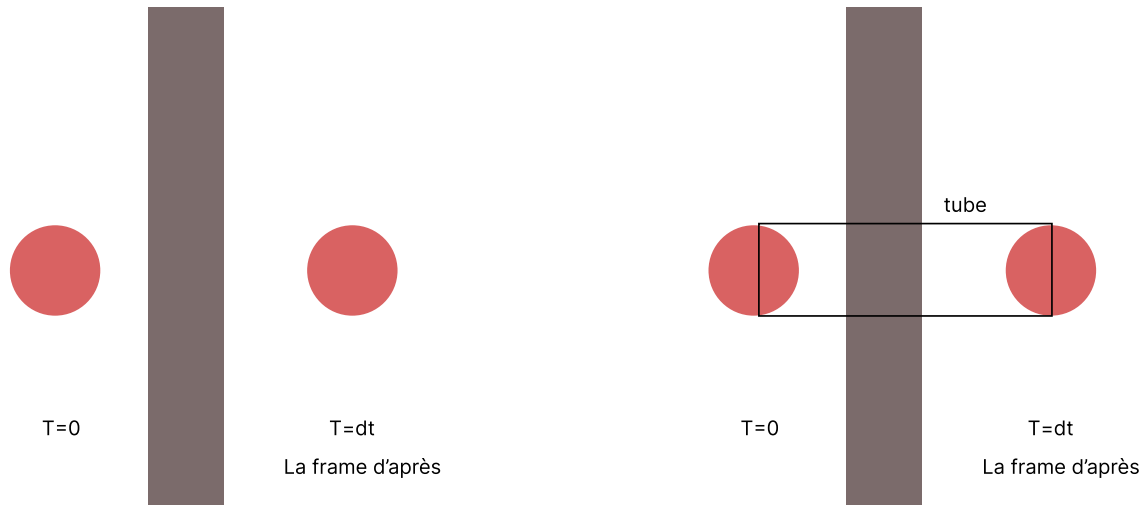


Fig. 34. – Tunneling : la balle rapide passe de l'autre côté du mur entre deux frames. La détection discrète (positions à t et $t + dt$) rate la collision. Le CCD teste la **trajectoire complète** entre t et $t + dt$.

La solution : CCD

Le **Continuous Collision Detection** teste la collision le long de la **trajectoire** entre t et $t + dt$, pas seulement aux positions discrètes :

- **Swept test** : on « balaie » la forme le long de son déplacement et on teste le volume couvert.
- **Raycast** : pour une sphère, on raycaste le centre le long de $\vec{v} \cdot dt$ avec un rayon élargi.
- **Conservative advancement** : on avance par petits pas adaptatifs jusqu'au premier contact.

Coût : le CCD est **plus cher** que la détection discrète. C'est pourquoi les moteurs l'activent **par objet** — uniquement pour les projectiles rapides, pas pour tous les objets.

CCD dans les moteurs.

- **PhysX / Unity** : `rigidbody.enableCcd = true` — pour les projectiles.
- **Rapier** : `RigidBodyDesc.dynamic().setCcdEnabled(true)`.
- **Godot** : propriété `continuous_cd` sur les `RigidBody`.
- **Notre moteur** : le sub-stepping de la Session 12 est une **rustine approximative** de CCD — on réduit dt pour que le déplacement par pas soit petit devant les obstacles.

Sleeping

Sleeping (mise en sommeil)

Si un objet est **immobile** (vitesse et vitesse angulaire sous un seuil pendant un certain temps), on arrête de le simuler. Il « dort » :

- Il ne consomme plus de temps de calcul.
- Il ne réagit plus aux forces tant qu'il dort.
- Il se **réveille** si un autre objet le touche, ou si on lui applique une force programmatique.

Pourquoi c'est essentiel : dans un niveau, 90 % des objets physiques sont au sol et immobiles. Sans sleeping, on simule inutilement des milliers de corps stables — le broad phase tourne, le solveur itère, et tout ça pour rien.

! Le piège du sleeping

Un objet endormi ne détecte plus les collisions **provoquées par lui**. Si un personnage marche sur un pont d'objets endormis, il faut que le premier contact **réveille** la chaîne. Les moteurs gèrent ça automatiquement — mais un objet au-dessus du sol qui « flotte » endormi est un bug classique quand on téléporte des objets sans les réveiller.

Les types de corps

Les trois types de rigid bodies

Tous les moteurs distinguent :

- **Dynamic** : subit les forces et les collisions. Masse finie. C'est l'objet physique standard.
- **Fixed (Static)** : ne bouge **jamais**. Masse infinie. Le sol, les murs, le décor.
- **Kinematic** : contrôlé **par le code** (position ou vitesse), ignore les forces, mais pousse les dynamic bodies. Plateformes mobiles, ascenseurs.

La distinction existe parce que simuler un mur comme un dynamic body de masse énorme coûte cher et cause de l'instabilité numérique. Un fixed body est **gratuit** et **parfaitement stable**.

Type	Subit les forces ?	Pousse les dyn. ?	Usage
Dynamic	Oui	Oui	Objets du jeu, débris, projectiles
Fixed	Non	Oui (mur)	Sol, murs, niveau
Kinematic	Non (code !)	Oui	Plateformes, ascenseurs, portes

Tableau 12. – Les trois types de corps. Le kinematic est le plus subtil : le code contrôle sa trajectoire, mais les dynamic bodies réagissent quand il les pousse.

Les queries (raycast, shapecast)

Scene queries

Au-delà de la simulation, les moteurs offrent des **questions directes** sur l'état du monde :

- **Raycast** : « qu'est-ce que cette droite touche, et à quelle distance ? » — essentiel pour les tirs (hitscan), la ligne de vue, les IA.
- **Shapecast** : « si je déplace cette forme de A à B, que touche-t-elle en premier ? » — utilisé par les character controllers pour se déplacer sans traverser les murs.
- **Overlap** : « quels corps chevauchent ce volume ? » — zones de dégâts, pickups, déclencheurs.

Les queries **ne modifient pas** la simulation — elles l'interrogent.

💡 Le raycast, outil universel

Le personnage ne « marche » presque jamais avec un rigid body — il fait un **shapecast vers le bas** pour trouver le sol, puis le code déplace le personnage et résout les collisions manuellement. C'est pour ça que les moteurs ont des **CharacterBody** (Godot, Unity) : des corps non simulés qui utilisent les queries pour se déplacer de façon contrôlée. On les verra en Session 18.

Partie 3 : Le Paysage des Moteurs

Les moteurs natifs (C/C++)

PhysX — le standard historique

Développé par NVIDIA (originellement NovodeX, 2001). Open-source depuis 2016. C'est le moteur **par défaut** de Unity et Unreal Engine. On le trouve dans la plupart des AAA. Points forts : maturité, outils, GPU acceleration (pour la simulation massive). C'est un **standard de facto**.

Havok — le moteur premium

Moteur commercial fermé (Irlande, 2000). Utilisé par Half-Life 2, Zelda TotK, l'ensemble des grosses productions Sony/Microsoft. Réputé pour sa **stabilité** sur les empilements massifs et sa destruction avancée. Coûteux — réservé aux studios avec un budget.

Jolt Physics — le challenger moderne

Open-source (par Jorrit Rouwe, 2021), écrit en C++ moderne. Rapide, multithreadé, déterministe. Utilisé pour **Horizon Forbidden West** (démonstration de puissance), et il est devenu le moteur **par défaut** de Godot 4.4+. Le succès récent le plus notable du domaine.

Box2D — le roi du 2D

D'Erin Catto (2006), open-source. La référence 2D : Angry Birds, Terraria, et des centaines de jeux 2D. Le solveur de contraintes de Box2D (Sequential Impulse) est **le** solveur que tous les autres ont copié. Le moteur « maison 2D » de ce cours est architecturé comme un mini-Box2D.

Les moteurs web (JavaScript / WASM)

Rapier — le standard web moderne

Écrit en **Rust**, compilé en **WebAssembly**. Rapide, déterministe, API moderne. Le moteur **recommandé** pour Three.js. Développé par Dimforge. Nous allons l'utiliser aujourd'hui. Points forts : performance proche du natif, WASM (le code binaire tourne à pleine vitesse dans le navigateur), API propre.

Cannon.js / Ammo.js — la génération précédente

- **Cannon.js** : moteur en pur JavaScript (2011), simple mais lent, peu maintenu. Cannon-es est un fork maintenu.
- **Ammo.js** : portage Emscripten de Bullet Physics vers WASM. Performant mais API datée, gros bundle (2 Mo), moins pratique.

Ces moteurs ont précédé Rapier — aujourd'hui, pour un nouveau projet Three.js, **Rapier est le choix par défaut**.

Moteur	Langage	Licence	Usage notable
PhysX	C++	Open source	Unity/Unreal par défaut, AAA
Havok	C++	Commerciale	Half-Life 2, Zelda TotK
Jolt Physics	C++	Open source	Horizon FW, Godot 4.4+
Box2D	C++	Open source	Angry Birds, Terraria (2D)
Bullet	C++	Open source	Blender, GTA IV
Rapier	Rust→WASM	Open source	Web (Three.js), Bevy
Cannon-es	JavaScript	Open source	Three.js (simple, legacy)
Ammo.js	C++→WASM	Open source	Three.js (Bullet port)
Godot Physics	C++	Open source	Godot (Godot Physics/Jolt)

Tableau 13. – Paysage des moteurs physiques. Notez Godot en bas : le moteur de jeu **intègre** son propre moteur physique (ou Jolt) — pas besoin d'en choisir un.

🧠 Le retour de Jolt — une leçon d'humilité

Jolt a été écrit **par une seule personne** (Jorrit Rouwe) en quelques années, et il rivalise avec PhysX et Havok, développés par des équipes entières depuis 20 ans. Il a été adopté par Godot comme moteur par défaut. Preuve que le domaine n'est pas figé — et qu'une **compréhension profonde** des principes (les vôtres maintenant) permet de construire des outils de classe mondiale.

Partie 4 : Déterminisme

Déterminisme

Un moteur est **déterministe** si les mêmes inputs produisent **toujours exactement les mêmes outputs** — même sur des machines différentes, même après des millions de pas.

💡 Pourquoi c'est important

- **Multijoueur lockstep** : chaque client simule la même physique et n'échange que les inputs. Si la physique diverge d'un bit, les joueurs voient des mondes **différents** — le jeu casse. StarCraft II, Age of Empires IV fonctionnent comme ça.
- **Replays** : enregistrer seulement les inputs et rejouer la simulation requiert un moteur déterministe.
- **Tests de régression** : un bug de physique se reproduit exactement — on peut le déboguer.
- **Rollback netcode** : rembobiner la physique et rejouer (fighting games) exige le déterminisme.

Pourquoi c'est difficile

Le déterminisme se casse par :

- **Ordre des opérations** : si les paires de collision sont traitées dans un ordre différent (hash map, multithreading), les résultats divergent.
- **Précision flottante** : le format IEEE 754 est déterministe pour $+$, $-$, $\times$, $\div$ — mais pas pour sqrt , sin , cos (implémentations différentes selon le CPU/libm).
- **Multithreading** : deux threads qui traitent des corps en interaction dans un ordre non fixé $\rightarrow$ divergence.

Solutions : ordre de traitement **fixe et trié** (par handle, pas par adresse), maths à précision **fixée** (fixed-point ou entiers), fonctions trigonométriques **réimplémentées** (deterministic sin/cos), thread scheduling **contraint**.

⚠️ En pratique

Rapier est déterministe **localement** (même machine, même binaire) mais pas **cross-platform** par défaut. Pour du lockstep multijoueur en production, on utilise des moteurs spécifiquement conçus pour ça (ou on fixe la précision manuellement). Godot **n'est pas** déterministe — les replays Godot enregistrent les états, pas les inputs.

Partie 5 : Architecture ECS

Entity-Component-System

L'architecture dominante des moteurs modernes (Unity DOTS, Bevy, Flecs, EnTT) :

- **Entity** : un simple **ID** (ex: entity_42). Pas de données, pas de logique. Juste une étiquette.
- **Component** : des **données pures** (ex: Position, Rigidbody, Mesh). Un composant ne contient **aucune logique** — juste des champs.

- **System** : de la **logique pure** qui opère sur toutes les entités possédant certains composants (ex: PhysicsSystem traite tout ce qui a Rigidbody + Position).

L'idée en code.

```
// Entity: juste un ID
const entity = world.createEntity();      // entity = 42

// Components: des données pures
world.addComponent(entity, Position, { x: 0, y: 10, z: 0 });
world.addComponent(entity, Velocity, { x: 0, y: 0, z: 0 });
world.addComponent(entity, Rigidbody, { mass: 1.0 });

// System: de la logique qui traite toutes les entités
// possédant Position + Velocity
world.addSystem((dt) => {
    for (const e of world.query(Position, Velocity)) {
        e.position.x += e.velocity.x * dt;    // ...
    }
});
```

❗ Pourquoi ECS plutôt que l'héritage ?

L'approche orientée objet classique (class Projectile extends PhysiqueObject extends GameObject) crée des **hiérarchies rigides** : un objet « posable ET physique ET dégâts » force des contorsions d'héritage.

ECS compose : un baril = Position + Rigidbody + Mesh + Pickup. Une flèche = Position + Rigidbody + Mesh + Projectile. On **ajoute et enlève des composants à la volée** — un ennemi mort devient un ragdoll : on retire AI, on ajoute Rigidbody à ses membres.

Bonus majeur : les **systems** itèrent sur des tableaux de données contiguës (**cache-friendly**) — c'est ce qui rend les moteurs ECS si rapides.

💡 Le lien avec la physique

Rapier, en interne, ressemble à un ECS : chaque Rigidbody est une entité avec des composants (position, velocity, collider handles). Quand on écrit world.step(), un PhysicsSystem itère sur tous les corps. Vous avez **déjà** écrit ce code — votre boucle updatePhysics(dtFrame) de la Session 12 est un PhysicsSystem !

Partie 6 : Choisir son moteur — le guide pratique

Contexte	Choix recommandé	Pourquoi
Jeu web Three.js	Rapier	WASM rapide, API moderne, le standard
Jeu web 2D simple	Rapier 2D ou Planck.js	Rapide et simple
Jeu Godot	Intégré (Jolt)	Zéro intégration, outils inclus
Jeu Unreal	Chaos (intégré)	Intégré + destruction
Jeu Unity	PhysX (défaut)	Intégré, mature
Simulation massive	PhysX GPU / Jolt	Multithread, GPU
Multijoueur lockstep	Moteur déterministe dédié	Floats cross-platform

Tableau 14. – Guide de choix rapide. Dans tous les cas : les concepts sont les mêmes, seule l'API change.

Synthèse

Ce qu'il faut retenir

- **Un moteur physique commercial** économise des années de R&D — mais sans comprendre les principes (intégration, collisions, solveur), on ne peut pas le déboguer ni le pousser dans ses retranchements.
- Tous les moteurs partagent les mêmes concepts : **rigid bodies** (dynamic/fixed/kinematic), **colliders**, **CCD** anti-tunneling, **sleeping**, **queries** (raycast).
- **Le pattern d'intégration** est universel : créer le monde, créer mesh + body par objet, `step()`, synchroniser. La physique est la source de vérité.
- **Le déterminisme** (mêmes inputs → mêmes outputs) est essentiel pour le multijoueur lockstep et les replays, mais difficile (ordre des opérations, flottants, threads).
- **L'ECS** (Entity-Component-System) est l'architecture des moteurs modernes : composition par données, logique dans les systems, cache-friendly.
- **Rapier** est le standard pour Three.js ; **Godot** intègre sa propre physique (Jolt) — deux philosophies qu'on mettra en pratique en Session 14.

La suite — la pratique

Cette session a posé le **paysage** et les **concepts**. La **Session 14** est la partie **pratique** : on installe et configure **Rapier** avec Three.js et **Jolt** avec Godot, puis on construit **pas à pas** des rigid bodies et des colliders dans chaque moteur — setup, boucle de simulation, synchronisation, et le tableau comparatif des API. Tout ce qu'on a vu en théorie ici devient du code là-bas.

Session 14 : Pratique des moteurs — Rigid Bodies & Colliders

Objectifs de la session

- **Installer et configurer** deux moteurs physiques : **Rapier** avec Three.js, et **Jolt** intégré à Godot.
- Comprendre le **setup** de chaque moteur : `importmap` et `WASM` côté web, `project.godot` côté Godot.
- Créer **pas à pas** des **rigid bodies** (fixed, dynamic, kinematic) et des **colliders** dans chaque moteur.
- Maîtriser la **boucle de simulation** : `world.step()` + synchronisation manuelle (Rapier) vs `_physics_process` automatique (Godot).
- Démontrer le **CCD** anti-tunneling et le **sleeping** sur la même scène dans les deux moteurs.
- Établir le **tableau comparatif** des API pour passer de l'un à l'autre sans friction.

🔗 Le passage à la pratique

La Session 13 a posé le **paysage** et les **concepts** partagés. Cette session est son **pendant pratique** : on met les mains dans le code. Deux moteurs, une même scène — un sol, une pyramide de cubes, un boulet de canon — pour comparer **concrètement** les deux philosophies : la **bibliothèque** (Rapier, qu'on intègre à Three.js) et le **moteur intégré** (Godot + Jolt, où la physique est un nœud de l'arbre).

Partie 1 : Deux philosophies, une même scène

Bibliothèque vs moteur intégré

- **Rapier + Three.js** : la physique est une **bibliothèque externe**. On crée un monde, on crée des corps, on appelle `world.step()`, et on **recopie** manuellement les positions du moteur vers les meshes. La physique et le rendu sont **déconnectés** — c'est nous qui faisons le pont.
- **Godot + Jolt** : la physique est **intégrée au moteur de jeu**. Un `RigidBody3D` est à la fois un corps physique **et** un nœud de scène. Godot gère le `step()` et la synchronisation automatiquement. On n'écrit **aucune** boucle de copie.

🔗 Pourquoi comparer les deux ?

Parce qu'en production, le choix se pose. Si vous faites un jeu web avec Three.js, vous **devez** intégrer Rapier. Si vous faites un jeu Godot, la physique est **déjà là**. Comprendre les deux approches vous permet de choisir en connaissance de cause — et de transférer vos connaissances d'un moteur à l'autre, car les **concepts** (rigid body, collider, CCD, sleeping) sont identiques.

Partie 2 : Three.js + Rapier — Fondamentaux & Setup

Documentation & ressources

Liens utiles

- **Rapier (site officiel)** : rapier.rs — documentation, exemples, et le **playground** interactif.

- **Rapier JS (dimforge)** : github.com/dimforge/rapier — les packages `@dimforge/rapier3d-compat` (WASM) et `@dimforge/rapier3d` (Node).
- **Three.js** : threejs.org/docs — la référence de l'API rendu.
- **Exemple du cours** : `examples/session14_rapier.html` + `session14_rapier.js` — la scène complète (sol, pyramide, boulet, HUD sleeping).

Setup — l'importmap

Pourquoi un importmap ?

Les modules ES ne savent pas résoudre les noms nus (`import ... from 'three'`). L'importmap déclare la correspondance entre ces noms et les CDN. Pour Rapier, on ajoute une entrée vers le package `-compat` qui embarque le binaire WASM en base64 :

```
<script type="importmap">
{
  "imports": {
    "three": "https://unpkg.com/three@0.185.1/build/three.module.js",
    "jsm/": "https://unpkg.com/three@0.185.1/examples/jsm/",
    "rapier": "https://cdn.skypack.dev/@dimforge/rapier3d-compat@0.19.3"
  }
}
</script>
```

C'est le **seul** changement côté HTML. Le JS fait ensuite `import RAPIER from 'rapier'` — le nom nu est résolu par l'importmap.

💡 En production (Vite, Webpack)

Pour un vrai projet, on installe le package via npm (`npm install @dimforge/rapier3d-compat`) et l'importmap devient inutile — le bundler résout les noms. L'importmap CDN est pratique pour **prototyper** et pour ce cours : zéro build, un seul fichier à ouvrir.

Le chargement WASM — l'étape asynchrone

Rapier est un binaire, pas du JavaScript

Rapier est écrit en **Rust** et compilé en **WebAssembly** (WASM). Le package `-compat` embarque le binaire en base64. Conséquence : le chargement est **asynchrone** — impossible de créer quoi que ce soit avant `RAPIER.init()`. Toute la fonction d'initialisation est donc `async` :

```
import RAPIER from 'rapier';          // résolu par l'importmap

async function init() {
  await RAPIER.init();                 // ← le WASM se décode ici
  // seulement APRÈS : world, bodies, colliders...
}
init().catch(err => console.error(err));
```

⚠ Le piège classique

Si on oublie le `await RAPIER.init()`, toute création de `World` ou de `RigidBody` lève une erreur du type « **RAPIER is not initialized** ». C'est l'équivalent d'oublier d'allumer le moteur avant de conduire.

Le monde physique

L'équivalent de notre « univers » maison

```
const FIXED_DT = 1 / 60;
const world = new RAPIER.World({ x: 0, y: -9.81, z: 0 });
world.timestep = FIXED_DT; // pas fixe – pattern de la Session 5
```

Le monde est le **registre** de tous les corps + la gravité + le pas de temps. C'est l'équivalent de notre tableau `planets` + la constante `G` — mais il gère aussi la *broad phase*, le solveur, le *sleeping*... tout ce qu'on a écrit à la main.

Partie 3 : Three.js + Rapier — Rigid Bodies & Colliders pas à pas

Étape 1 — Le sol : un corps FIXE

RigidBodyDesc.fixed() + ColliderDesc.cuboid()

```
// --- Rendu : le mesh avec la texture grille ---
const mesh = new THREE.Mesh(
  new THREE.BoxGeometry(60, 0.2, 60), gridMaterial);

// --- Physique : corps FIXE (masse infinie, jamais simulé) ---
const body = world.createRigidBody(
  RAPIER.RigidBodyDesc.fixed().setTranslation(0, -0.1, 0));
world.createCollider(
  RAPIER.ColliderDesc.cuboid(30, 0.1, 30) // ← DEMI-tailles !
    .setFriction(params.friction)
    .setRestitution(params.restitution),
  body);
```

Deux objets distincts : le **rigid body** (le comportement — fixe, dynamique...) et le **collider** (la forme — cuboïde, boule...). Un body peut porter **plusieurs** colliders ; un collider sans body n'existe pas.

💡 Le piège des demi-tailles

`ColliderDesc.cuboid(hx, hy, hz)` prend des **demi-dimensions** (half-extents), alors que `BoxGeometry(w, h, d)` de Three.js prend des dimensions **complètes**. Un cube de 1 m = `BoxGeometry(1,1,1)` côté rendu, `cuboid(0.5, 0.5, 0.5)` côté physique. Se tromper ici double ou réduit la taille du collider — et la physique se comporte bizarrement sans erreur apparente.

Étape 2 — La pyramide : des corps DYNAMIQUES

Le pattern « pairs » — le lien rendu ↔ physique

```
const pairs = []; // { mesh, body } pour CHAQUE objet physique

for (let row = 0; row < 5; row++) {
  for (let i = 0; i < 5 - row; i++) {
    // Rendu
    const mesh = new THREE.Mesh(boxGeo, cubeMaterial);
    scene.add(mesh);
    // Physique : corps DYNAMIQUE
    const body = world.createRigidBody(
      RAPIER.RigidBodyDesc.dynamic()
        .setTranslation(x, 0.5 + row * 1.05, 0));
    world.createCollider(
      RAPIER.ColliderDesc.cuboid(0.5, 0.5, 0.5), body);
    // Le lien croisé — c'est LUI qu'on itère dans animate()
    pairs.push({ mesh, body });
  }
}
```

Rapier ne sait rien de Three.js, et Three.js ne sait rien de Rapier. Chaque objet physique existe donc **en double** : un mesh (ce qu'on voit) et un body (ce qui est simulé). Le tableau `pairs` maintient la correspondance — c'est le **cœur de toute intégration** moteur physique + moteur de rendu.

Étape 3 — Un corps KINEMATIC (plateforme)

RigidBodyDesc.kinematicPositionBased()

```
// Plateforme mobile : contrôlée par le code, pousse les dynamic bodies
const platform = world.createRigidBody(
  RAPIER.RigidBodyDesc.kinematicPositionBased()
    .setTranslation(0, 1.5, -5));
world.createCollider(
  RAPIER.ColliderDesc.cuboid(3, 0.2, 3), platform);

// Dans la boucle : on déplace la plateforme par le code
function animate() {
  // ...
  const t = performance.now() * 0.001;
  platform.setNextKinematicTranslation({ x: Math.sin(t) * 4, y: 1.5, z: -5 });
  // ...
}
```

Le kinematic **ignore les forces** mais **pousse** les dynamic bodies. On contrôle sa position via `setNextKinematicTranslation()` — la position est appliquée au **prochain** `step()`. C'est l'outil pour les ascenseurs, portes automatiques, plateformes de jeu.

💡 Kinematic vs Dynamic — quand choisir ?

Un ascenseur **Dynamic** rebondirait sous le poids des personnages et oscillerait. Un **Kinematic** suit exactement la trajectoire qu'on lui impose — il pousse les corps qui le touchent sans jamais être

dévié. La règle : si le mouvement est **scripté** (sinus, spline, input joueur), c'est kinematic. S'il doit **réagir** aux forces, c'est dynamic.

Étape 4 — La boucle : accumulateur, step, synchronisation

Le fixed timestep de la Session 5, appliqué à Rapier

```
function animate() {
  const dt = Math.min(clock.getDelta(), 0.1);
  accumulator += dt;
  while (accumulator >= FIXED_DT) {
    world.step(); // ← TOUTE la physique ici
    accumulator -= FIXED_DT;
  }
  // - Synchronisation : rendu ← physique -
  for (const { mesh, body } of pairs) {
    const t = body.translation(); // { x, y, z }
    const r = body.rotation(); // quaternion
    mesh.position.set(t.x, t.y, t.z);
    mesh.quaternion.set(r.x, r.y, r.z, r.w);
  }
  renderer.render(scene, camera);
}
```

La physique tourne à **pas fixe** (60 Hz), indépendamment du framerate. Si une frame dure 32 ms, la boucle while fait **deux pas**. Puis on **recopie** les positions et rotations du moteur vers les meshes.

❗ Règle d'or : la physique est la source de vérité

On ne déplace **jamais** le mesh directement — on pousse le body (force, impulsion), et le mesh suit à la frame suivante. Inverser cette relation casse la simulation : le body et le mesh se désynchronisent, les collisions se produisent au mauvais endroit.

Étape 5 — Le boulet : impulsion + CCD

applyImpulse + setCcdEnabled — la démo tunneling

```
function fireCannonball(origin, dir) {
  // 1. Rendu
  const mesh = new THREE.Mesh(
    new THREE.SphereGeometry(0.45, 24, 24),
    new THREE.MeshStandardMaterial({ color: 0x2c3e50, metalness: 0.9 }));
  scene.add(mesh);

  // 2. Physique : CCD activé, collider lourd
  const body = world.createRigidBody(
    RAPIER.RigidBodyDesc.dynamic()
    .setTranslation(origin.x, origin.y, origin.z)
    .setCcdEnabled(params.ccd)); // ← CCD ici !
  world.createCollider(
    RAPIER.ColliderDesc.ball(0.45)
```

```

        .setDensity(10), // lourd – 10× la densité par défaut
        body);

// 3. Impulsion : Session 6 en une ligne
body.applyImpulse(
  { x: dir.x * params.cannonSpeed, y: dir.y * params.cannonSpeed, z: dir.z *
params.cannonSpeed },
  true); // true = réveille le corps
pairs.push({ mesh, body });
}

```

Le boulet est le cas d'école du **tunneling** : à 120 unités/s, il avance de **2 unités par pas** — plus qu'un cube entier. Désactivez le CCD dans la GUI et tirez à haute vitesse : il **traverse** la pyramide. Réactivez-le : il la démolit.

Étape 6 — Le HUD : observer le sleeping

isSleeping() — la preuve que le moteur optimise

```

function updateHUD() {
  let sleeping = 0, dynamic = 0;
  for (const { body } of pairs) {
    if (body.isDynamic()) dynamic++;
    if (body.isSleeping()) sleeping++;
  }
  hud.textContent = `Corps : ${dynamic} | Endormis : ${sleeping}`;
}

```

Laissez la scène se stabiliser : le compteur d'**endormis** grimpe vers le total. Le moteur a **arrêté de simuler** les cubes immobiles — zéro coût CPU pour eux. Touchez la pyramide : les cubes touchés **se réveillent** en cascade.

⚠ Le piège du sleeping

Quand la GUI change la gravité, il faut **réveiller tout le monde** (`body.wakeUp()`) — sinon les corps endormis continuent d'ignorer la nouvelle gravité ! C'est un bug classique : on change un paramètre global et rien ne bouge parce que les objets dorment.

Étape 7 — Le ménage

removeRigidBody + reset

```

function removePair(pair) {
  scene.remove(pair.mesh);
  pair.mesh.geometry.dispose();
  pair.mesh.material.dispose();
  world.removeRigidBody(pair.body); // supprime aussi ses colliders
  pairs = pairs.filter(p => p !== pair);
}

```

Comme toujours : on libère la mémoire GPU (geometry/material) et on retire le body du monde. Les boulets sont plafonnés à 12 pour la performance — un vrai moteur gère des **milliers** de corps, mais chaque `world.step()` a un coût.

Partie 4 : Godot + Jolt — Fondamentaux & Setup

Documentation & ressources

Liens utiles

- **Godot (site officiel)** : godotengine.org — téléchargement, documentation.
- **Godot Physics (docs)** : docs.godotengine.org/.../physics — introduction à la physique 3D.
- **Jolt dans Godot** : github.com/godot-jolt/godot-jolt — le plugin (intégré depuis 4.4).
- **RigidBody3D (API)** : docs.godotengine.org/.../class_rigidbody3d — référence de la classe.
- **Projet du cours** : le dossier `demo-physics` — scène `session13_demo.tscn` + script `session13_demo.gd` (construit la scène au runtime).

Setup — le `project.godot`

Trois lignes qui comptent

```
[physics]
3d/physics_engine="Jolt Physics"    ← active Jolt (Partie 3 de la S13)

[application]
run/main_scene="res://scenes/session13_demo.tscn"
```

On **choisit** le moteur physique du projet comme un paramètre. Jolt remplace le moteur par défaut (Godot Physics) depuis la 4.4. Même architecture de cours (broad phase, solveur), autre implémentation — plus rapide, plus stable sur les empilements.

🔗 Jolt intégré dans Godot 4.4+

Depuis Godot 4.4, Jolt Physics est disponible comme option de projet : `physics/3d/physics_engine = "Jolt Physics"`. Le projet `demo-physics` l'utilise. Avantages : stabilité des empilements, performance multithreadée — même architecture, meilleures maths.

Pas de WASM, pas de synchronisation

La différence fondamentale

Contrairement à Rapier, **aucune** étape de chargement n'est nécessaire. La physique démarre avec le moteur. Et contrairement à Three.js + Rapier, **aucune** boucle de synchronisation n'est à écrire : un `RigidBody3D` est à la fois le corps physique **et** le nœud de scène. Godot met à jour la transformation de toute la hiérarchie quand le moteur avance le body.

Partie 5 : Godot + Jolt — Rigid Bodies & Colliders pas à pas

🔗 L'arborescence du projet

```
demo-physics/
├─ project.godot          ← moteur Jolt, scène principale
├─ scenes/
│   └─ session13_demo.tscn ← racine Node3D + le script attaché
├─ scripts/
│   └─ session13_demo.gd   ← tout le code (construit la scène au runtime)
└─ textures/
    └─ grid.png            ← texture du sol (la même que Three.js !)
```

La scène .tscn est minimale : une racine Node3D avec le script attaché. Tout le reste (caméra, lumière, sol, pyramide) est construit **par code** dans `_ready()`, pour tenir dans un seul fichier comparable à la version Rapier.

Étape 1 — `_ready()` : construire la scène par code

La méthode d'initialisation

```
extends Node3D

var _cannon_speed := 60.0
var _ccd_enabled := true
var _camera: Camera3D
var _hud: Label
var _dynamic_bodies: Array[RigidBody3D] = []

func _ready() -> void:
    _setup_camera()
    _setup_light()
    _setup_ground()
    _setup_hud()
    _spawn_stack()
```

`_ready()` est l'équivalent de la `init()` du JS : elle s'exécute une fois au démarrage. Elle appelle cinq fonctions qui créent les sous-systèmes. **Aucune scène n'est préfabriquée dans l'éditeur** — c'est un choix pédagogique pour montrer que Godot permet de tout faire par code.

Étape 2 — Le sol : `StaticBody3D`

Corps FIXE avec enfant `CollisionShape3D` + `MeshInstance3D`

```
func _setup_ground() -> void:
    var ground := StaticBody3D.new()
    ground.position = Vector3(0, -0.1, 0)

    # --- Forme de collision ---
    var col := CollisionShape3D.new()
    var box := BoxShape3D.new()
    box.size = Vector3(60, 0.2, 60) # ← taille COMPLÈTE
    col.shape = box
```



```

ground.add_child(col)

# --- Rendu ---
var mesh := MeshInstance3D.new()
var box_mesh := BoxMesh.new()
box_mesh.size = Vector3(60, 0.2, 60)
mesh.material_override = gridMaterial
mesh.mesh = box_mesh
ground.add_child(mesh)

add_child(ground)

```

Un `StaticBody3D` est le correspondant du `RigidBodyDesc.fixed()` de Rapier. Deux enfants : `CollisionShape3D` (la physique) et `MeshInstance3D` (le rendu). Ils sont **sous le même nœud** : Godot sait qu'il doit synchroniser la forme et le mesh.

Le piège inverse de Rapier

Dans Godot, `BoxShape3D.size` et `BoxMesh.size` prennent la taille **complète** du cube (pas les demi-tailles). C'est plus intuitif, mais il faut le savoir pour ne pas faire un sol de 30 m au lieu de 60 m en transposant naïvement du code Rapier.

Étape 3 — La pyramide : `RigidBody3D` par code

Chaque cube est un `RigidBody3D` avec deux enfants

```

func _spawn_stack() -> void:
    var cube_mesh := BoxMesh.new()
    cube_mesh.size = Vector3.ONE

    for row: int in 5:
        var count := 5 - row
        for i in count:
            var cube := RigidBody3D.new()
            cube.position = Vector3((i - (count - 1) / 2.0) * 1.05, 0.5 + row * 1.05, 0)

            var col := CollisionShape3D.new()
            var shape := BoxShape3D.new()
            shape.size = Vector3.ONE
            col.shape = shape
            cube.add_child(col)

            var mesh := MeshInstance3D.new()
            var mat := StandardMaterial3D.new()
            mat.albedo_color = CUBE_COLORS[row % CUBE_COLORS.size()]
            mesh.material_override = mat
            mesh.mesh = cube_mesh
            cube.add_child(mesh)

        add_child(cube)
    _dynamic_bodies.append(cube)

```

Le `RigidBody3D` est à la fois le corps dynamique et le **conteneur de la hiérarchie**. On lui ajoute un `CollisionShape3D` pour la physique et un `MeshInstance3D` pour le rendu. Godot **oriente automatiquement** le mesh quand le body bouge : **aucune boucle de synchronisation** à écrire.

`_dynamic_bodies` est le pendant du tableau `pairs` de la version Rapier, mais il ne contient que les corps — le lien avec le mesh est implicite (enfant de chaque body).

Étape 4 — Un corps KINEMATIC (plateforme)

AnimatableBody3D — le kinematic de Godot

```
func _setup_platform() -> void:
    var platform := AnimatableBody3D.new()
    platform.position = Vector3(0, 1.5, -5)

    var col := CollisionShape3D.new()
    var box := BoxShape3D.new()
    box.size = Vector3(6, 0.4, 6)
    col.shape = box
    platform.add_child(col)

    var mesh := MeshInstance3D.new()
    var box_mesh := BoxMesh.new()
    box_mesh.size = Vector3(6, 0.4, 6)
    mesh.material_override = _make_platform_material()
    mesh.mesh = box_mesh
    platform.add_child(mesh)

    add_child(platform)
    _platform = platform

# Dans _physics_process : on déplace la plateforme
func _physics_process(delta: float) -> void:
    var t := Time.get_ticks_msec() * 0.001
    _platform.position.x = sin(t) * 4.0
```

`AnimatableBody3D` (Godot 4) est le successeur du `KinematicBody` — un corps contrôlé par le code qui pousse les dynamic bodies sans subir les forces. On modifie directement `position` ; Godot applique le déplacement au prochain pas physique.

💡 AnimatableBody3D vs RigidBody3D en mode kinematic

Un `RigidBody3D` peut aussi être mis en mode kinematic via `body.mode = RigidBody3D.MODE_KINEMATIC`. `AnimatableBody3D` est plus léger et conçu pour les **plateformes animées** — il gère le transport des corps qui se trouvent dessus. Pour un ascenseur simple, c'est le bon choix.

Étape 5 — Le tir : rayon caméra + CCD + vitesse

`\_fire\_cannonball()` — le même tunneling, la même démo

```
func _fire_cannonball(screen_pos: Vector2) -> void:
    var from := _camera.project_ray_origin(screen_pos)
    var dir := _camera.project_ray_normal(screen_pos)

    var ball := _make_ball(from + dir * 2.0, 0.45, Color(0.17, 0.24, 0.32), 5.0)
    ball.continuous_cd = _ccd_enabled      # ← CCD ici (propriété du nœud)
    ball.linear_velocity = dir * _cannon_speed # ← vitesse directe
    _ball_count += 1
    _dynamic_bodies.append(ball)
```

La caméra fournit `project_ray_origin()` et `project_ray_normal()` — le raycast écran → monde. On crée un `RigidBody3D`, on active `continuous_cd`, et on donne une vitesse initiale. Pas d'appel à `apply_impulse` ici : **linear_velocity est équivalent** quand la vitesse initiale est connue.

Étape 6 — Les entrées : _unhandled_input

Un seul callback pour tout

```
func _unhandled_input(event: InputEvent) -> void:
    if event is InputEventMouseButton \
        and event.pressed and event.button_index == MOUSE_BUTTON_LEFT:
        _fire_cannonball(event.position)
    elif event is InputEventKey and event.pressed and not event.echo:
        match event.keycode:
            KEY_R: _reset()
            KEY_C: _ccd_enabled = not _ccd_enabled
            KEY_UP: _cannon_speed = minf(_cannon_speed + 10.0, 150.0)
            KEY_DOWN: _cannon_speed = maxf(_cannon_speed - 10.0, 10.0)
```

Un seul gestionnaire remplace les écouteurs de clic + clavier. Godot propose des `InputEvent` typés. Le `match` GDScript remplace le `switch/if-else`.

Étape 7 — Le HUD sleeping

`\_process` lit le moteur à chaque frame

```
func _process(_delta: float) -> void:
    var sleeping := 0
    for body in _dynamic_bodies:
        if body.is_sleeping():
            sleeping += 1
    _hud.text = "Corps dynamiques : %d | Endormis : %d\n" % [ ... ]
    _hud.text += "Vitesse du boulet : %d (↑/↓) | CCD : %s (C)\n" % [ ... ]
```

`_process(delta)` est appelé à **chaque frame rendue** (framerate variable). C'est le bon endroit pour le HUD. Le moteur physique tourne dans `_physics_process(delta)` (60 Hz fixe), mais on n'en a pas besoin ici : Godot gère le `step()` en interne.

Étape 8 — Pas de synchronisation, pas de `step()`

Ce que Godot cache — et pourquoi c'est plus rapide à prototyper

Dans le projet Godot, **aucune boucle** ne copie `body.position` vers `mesh.position`. Le `RigidBody3D` est le parent, le `CollisionShape3D` et le `MeshInstance3D` sont des enfants : Godot met à jour la transformation de toute la hiérarchie quand le moteur avance le `body`.

Le `world.step()` est lui aussi caché. C'est `_physics_process(delta)` et les paramètres `physics/physics_ticks_per_second` qui font le travail, exactement comme l'accumulateur de la Session 5.

La contrepartie : on a **moins de contrôle** sur le moment exact du pas physique. Pour des jeux classiques c'est un avantage ; pour des réseaux à pas prédictifs ou des simulations très spécifiques, c'est une contrainte.

Partie 6 : Tableau comparatif des API

Concept	Three.js + Rapier (JS)	Godot 4 (GDScript)	Unity 6 (C#, PhysX)
Monde physique	<code>new RAPIER.World(gravity)</code>	Automatique — <code>ProjectSettings / Jolt</code>	Automatique — <code>Physics.scene (PhysX)</code>
Pas fixe	<code>while(accumulator >= FIXED_DT) world.step()</code>	<code>_physics_process(delta)</code> — 60 Hz	<code>FixedUpdate()</code> — 50 Hz par défaut
Chargement	<code>await RAPIER.init()</code> (WASM asynchrone)	Aucune — démarrage immédiat	Aucune — démarrage immédiat
Corps fixe	<code>RigidBodyDesc.fixed()</code>	<code>StaticBody3D</code>	Collider seul (static collider)
Corps dynamique	<code>RigidBodyDesc.dynamic()</code>	<code>RigidBody3D</code>	Rigidbody (défaut)
Corps kinematic	<code>RigidBodyDesc.kinematicPositionBased()</code>	<code>AnimatableBody3D</code>	<code>Rigidbody.isKinematic = true</code>
Forme cubique	<code>ColliderDesc.cuboid(0.5, 0.5, 0.5)</code> (demi-tailles)	<code>BoxShape3D.new(); shape.size = Vector3.ONE</code> (taille complète)	<code>BoxCollider; size = Vector3.one</code> (taille complète)
Forme sphérique	<code>ColliderDesc.ball(0.45)</code>	<code>SphereShape3D.new(); shape.radius = 0.45</code>	<code>SphereCollider; radius = 0.45</code>
Plusieurs colliders	<code>world.createCollider(desc, body) × N</code>	Plusieurs <code>CollisionShape3D</code> enfants	Plusieurs composants Collider
Synchronisation	Manuelle : <code>mesh.position.copy(body.translation())</code>	Automatique : nœud enfant	Automatique : Rigidbody pilote le Transform
CCD	<code>body.setCcdEnabled(true)</code>	<code>ball.continuous_cd = true</code>	<code>rb.collisionDetectionMode = Continuous</code>
Vitesse initiale	<code>body.setLinvel(v)</code>	<code>ball.linear_velocity = v</code>	<code>rb.linearVelocity = v</code>
Impulsion	<code>body.applyImpulse(v, true)</code>	<code>ball.apply_impulse(v)</code>	<code>rb.AddForce(v, ForceMode.Impulse)</code>
Sleeping	<code>body.isSleeping()</code>	<code>body.is_sleeping()</code>	<code>rb.IsSleeping()</code>
Réveil	<code>body.wakeUp()</code>	<code>body.wake_up()</code>	<code>rb.WakeUp()</code>
Gravité	<code>world.gravity = { ... }</code>	<code>ProjectSettings</code> ou <code>Area3D</code>	<code>Physics.gravity</code> ou <code>rb.useGravity</code>
Suppression	<code>world.removeRigidBody(body)</code> + ménage mesh	<code>body.queue_free()</code>	<code>Destroy(gameObject)</code>
Téléportation	À éviter — utiliser forces/impulsions	À éviter — préférer <code>_integrate_forces()</code>	<code>rb.MovePosition()</code> / <code>MoveRotation()</code>

Tableau 15. – Même physique, trois façons de la piloter. Les concepts sont identiques ; seules l'API et l'architecture changent. Unity, comme Godot, intègre son moteur (PhysX) — la synchronisation est automatique.

Partie 7 : Les Colliders en détail

Les formes primitives

Formes disponibles dans les deux moteurs

Les deux moteurs offrent les mêmes primitives — seules les signatures diffèrent :

Forme	Rapier	Godot	Unity
Sphère	<code>ColliderDesc.ball(radius)</code>	<code>SphereShape3D.new(); shape.radius = r</code>	<code>SphereCollider; radius = r</code>
Cuboïde	<code>ColliderDesc.cuboid(hx, hy, hz) (demi)</code>	<code>BoxShape3D.new(); shape.size = v (complet)</code>	<code>BoxCollider; size = v (complet)</code>
Capsule	<code>ColliderDesc.capsule(halfHeight, radius)</code>	<code>CapsuleShape3D.new(); shape.radius / height</code>	<code>CapsuleCollider; radius / height</code>
Cylindre	<code>ColliderDesc.cylinder(halfHeight, radius)</code>	<code>CylinderShape3D.new(); shape.radius / height</code>	<code>MeshCollider (pas de primitive)</code>
Convexe	<code>ColliderDesc.convexHull(points)</code>	<code>ConvexPolygonShape3D.new(); shape.points</code>	<code>MeshCollider; convex = true</code>
Mesh (statique)	<code>ColliderDesc.trimesh(vertices, indices)</code>	<code>ConcavePolygonShape3D.new(); shape.faces</code>	<code>MeshCollider; convex = false</code>

Tableau 16. – Les primitives. Les formes convexes (sphère, cuboïde, capsule, convexe) sont simulées pour les dynamic bodies. Les trimeshes/concaves ne sont que **statiques** — un dynamic body ne peut pas être un mesh concave (le moteur ne sait pas résoudre les collisions sur une forme concave en mouvement). Notons qu'Unity n'a pas de primitive cylindre : on utilise un MeshCollider convexe ou une capsule approchée.

Colliders composés

Plusieurs colliders par body

Un rigid body peut porter **plusieurs** colliders — utile pour les formes complexes (une table = 5 cuboïdes : le plateau + 4 pieds) :

```
// Rapier : on crée plusieurs colliders pour le même body
const table = world.createRigidBody(
    RAPIER.RigidBodyDesc.dynamic().setTranslation(0, 1, 0));
world.createCollider(RAPIER.ColliderDesc.cuboid(1.0, 0.05, 0.6), table); // plateau
world.createCollider(RAPIER.ColliderDesc.cuboid(0.05, 0.5, 0.05)
    .setTranslation(-0.9, -0.5, -0.5), table); // pied 1
world.createCollider(RAPIER.ColliderDesc.cuboid(0.05, 0.5, 0.05)
    .setTranslation(0.9, -0.5, -0.5), table); // pied 2
// ... pieds 3 et 4

# Godot : on ajoute plusieurs CollisionShape3D enfants
var table := RigidBody3D.new()
table.position = Vector3(0, 1, 0)
```

```

var top := CollisionShape3D.new()
var top_shape := BoxShape3D.new()
top_shape.size = Vector3(2.0, 0.1, 1.2)
top.shape = top_shape
table.add_child(top)

var leg1 := CollisionShape3D.new()
leg1.position = Vector3(-0.9, -0.5, -0.5)
var leg_shape := BoxShape3D.new()
leg_shape.size = Vector3(0.1, 1.0, 0.1)
leg1.shape = leg_shape
table.add_child(leg1)
# ... pieds 2, 3, 4

```

Le moteur calcule le moment d'inertie **global** à partir de tous les colliders — la table tourne de façon réaliste quand on la pousse.

Matériaux : friction & restitution

Les propriétés de surface

Chaque collider porte des propriétés de **matériau** qui correspondent exactement à ce qu'on a codé à la main en Session 6 :

```

// Rapier
RAPIER.ColliderDesc.cuboid(0.5, 0.5, 0.5)
    .setFriction(0.7)           // 0 = glace, 1 = caoutchouc
    .setRestitution(0.3)       // 0 = pas de rebond, 1 = rebond parfait
    .setFrictionCombineRule(RAPIER.CoefficientCombineRule.Average)
    .setRestitutionCombineRule(RAPIER.CoefficientCombineRule.Max);

# Godot – via un PhysicsMaterial
var mat := PhysicsMaterial.new()
mat.friction = 0.7
mat.bounce = 0.3
col.material = mat

```

Les **règles de combinaison** (combine rules) déterminent comment les propriétés de deux colliders en contact se combinent : Average (moyenne), Min, Max, Multiply. Par défaut : friction = moyenne, restitution = max. C'est pourquoi une balle en caoutchouc (restitution 0.9) rebondit haut même sur un sol dur (restitution 0.1) — le Max l'emporte.

Synthèse

💡 Ce qu'il faut retenir

- **Deux philosophies** : Rapier est une **bibliothèque** qu'on intègre à Three.js (synchronisation manuelle) ; Godot **intègre** Jolt (synchronisation automatique via la hiérarchie de nœuds).
- **Le setup** : Rapier nécessite un `importmap` + `await RAPIER.init()` (WASM asynchrone) ; Godot nécessite une ligne dans `project.godot` pour activer Jolt.

- **Le pattern d'intégration** est universel : créer le monde, créer mesh + body par objet, `step()`, synchroniser. En Rapier on l'écrit ; en Godot il est implicite.
- **Les trois types de corps** (fixed, dynamic, kinematic) existent dans les deux moteurs avec des noms différents mais un comportement identique.
- **Les colliders** sont des objets **séparés** du rigid body en Rapier (attachés après création), et des **nœuds enfants** en Godot. Les formes primitives sont les mêmes ; attention au piège des demi-tailles (Rapier) vs tailles complètes (Godot).
- **Le CCD, le sleeping, les matériaux** (friction, restitution) sont des propriétés uniques à configurer — les concepts de la Session 13 appliqués en une ligne.

💡 La suite

Session 15 : les **joints** — charnières, ressorts, moteurs, ragdolls. On réutilisera les rigid bodies d'aujourd'hui pour les relier entre eux. Puis contraintes et IK (S16), et le TP véhicules (S17) — où tout ce qu'on voit aujourd'hui servira en pratique.

Session 15 : Joints, Moteurs & Raycasts

Objectifs de la session

- Comprendre ce qu'est un **joint** (articulation) et comment il **contraint** le mouvement relatif de deux rigid bodies.
- Maîtriser les **quatre types fondamentaux** : fixed, revolute, prismatic, spherical — et choisir le bon selon le cas.
- Configurer des **moteurs** sur les joints pour les animer activement (vitesse ou position cible).
- Ajouter des **limites** (min/max) pour borner la course d'une articulation.
- Découvrir les **raycasts** comme outil de requête du moteur physique — détection du sol, visée, capteurs.
- Construire pas à pas une **catapulte fonctionnelle** dans Rapier + Three.js, puis la transposer en Godot.

💡 Le passage à l'articulation

La Session 14 a posé les **corps** — fixed, dynamic, kinematic — et les **colliders**. Mais jusqu'ici, chaque corps était **isolé** : un cube tombe, rebondit, s'empile. Aucun n'était **relié** à un autre. Les joints changent tout : ils permettent de **souder**, **charnière**, **glisser** et **pivoter** des corps entre eux. C'est ce qui transforme une boîte qui tombe en un bras de catapulte, un ragdoll, une porte, un véhicule.

Partie 1 : Qu'est-ce qu'un joint ?

Joint (articulation)

Un **joint** est une contrainte qui relie **deux** rigid bodies et **limite leurs degrés de liberté** relatifs. Sans joint, deux bodies ont 6 degrés de liberté relatifs (3 translations + 3 rotations). Un joint en **supprime** certains :

- **Fixed** : supprime les 6 — les deux corps sont soudés.
- **Revolute** : supprime 5 — il ne reste qu'une **rotation** autour d'un axe.
- **Prismatic** : supprime 5 — il ne reste qu'une **translation** le long d'un axe.
- **Spherical** : supprime 3 — il reste les 3 **rotations** (rotule).

Le moteur physique **résout** ces contraintes à chaque `step()`, au même titre que les contacts. Un joint est donc une **équation** ajoutée au solveur — et le solveur trouve les vitesses qui satisfont **à la fois** les contacts et les joints.

💡 Joints vs contraintes maison

En Session 7, on a codé un ressort à la main : une force $F = -k \cdot x$ qui rapproche deux points. Un joint est **l'équivalent rigide** : au lieu d'une force **proportionnelle** à l'écart, le solveur impose **zéro écart** — c'est une contrainte **dure**. Le ressort est **mou** (il oscille), le joint est **dur** (il ne cède pas). En pratique, on utilise des joints pour les articulations mécaniques et des ressorts pour les suspensions, tissus, objets souples.

Partie 2 : Les quatre types de joints

Fixed — la soudure

Fixed Joint

Le joint le plus simple : les deux corps sont **rigidement liés**. La distance et l'orientation relatives sont **figées**. C'est l'équivalent de souder deux pièces.

```
// Rapier — JointData.fixed(anchor1, frame1, anchor2, frame2)
const joint = world.createImpulseJoint(
  RAPIER.JointData.fixed(
    { x: 0, y: 0.5, z: 0 }, // ancre sur A (local)
    { x: 0, y: 0, z: 0, w: 1 }, // frame A (quaternion identité)
    { x: 0, y: -0.5, z: 0 }, // ancre sur B (local)
    { x: 0, y: 0, z: 0, w: 1 } // frame B
  ),
  bodyA, bodyB, true // true = réveille les corps
);
```

Les **frames** (quaternions) définissent l'orientation relative **souhaitée** entre les deux ancres. Le identity signifie « alignées ». On peut les tourner pour souder deux corps avec un angle précis.

💡 Fixed joint vs un seul body multi-colliders

Pourquoi souder deux bodies plutôt que d'en faire un seul avec plusieurs colliders ? Parce qu'un joint permet de **casser** la liaison à l'exécution (`world.removeJoint(joint)`) — un pont qui s'effondre, un véhicule qui perd une roue. Un body multi-colliders est **insécable**. Le fixed joint est l'outil de la **destruction** et des **assemblages dynamiques**.

Revolute — la charnière

Revolute Joint

Le joint le plus utile en mécanique : les deux corps ne peuvent que **tourner** l'un par rapport à l'autre autour d'un **axe** défini. C'est la charnière d'une porte, l'axe d'une roue, le coude d'un bras.

```
// Rapier — JointData.revolute(anchor1, anchor2, axis)
const joint = world.createImpulseJoint(
  RAPIER.JointData.revolute(
    { x: 0, y: -1, z: 0 }, // ancre sur A (pied du bras)
    { x: 0, y: 1, z: 0 }, // ancre sur B (sommet de la base)
    { x: 0, y: 0, z: 1 } // axe de rotation (Z = avant/arrière)
  ),
  bodyA, bodyB, true
);
```

L'**axe** est exprimé dans le **repère local de A**. Le moteur aligne l'ancre de B sur l'ancre de A et autorise uniquement la rotation autour de cet axe. En 3D, l'axe compte : { x: 0, y: 1, z: 0 } = charnière verticale (porte), { x: 0, y: 0, z: 1 } = charnière horizontale (bras de catapulte).

Cas d'usage du revolute.

- **Porte** : un body fixe (mur) + un body dynamique (porte) + revolute axe Y.

- **Roue** : un body châssis + un body roue + revolute axe X.
- **Catapulte** : un body base + un body bras + revolute axe Z + moteur (Partie 3).
- **Ragdoll** : un body torse + un body bras + revolute axe Z + limites (Partie 4).

Prismatic — le tiroir

Prismatic Joint

Le pendant du revolute pour la translation : les deux corps ne peuvent que **glisser** l'un par rapport à l'autre le long d'un **axe**. C'est le tiroir d'une commode, le piston d'un vérin, la platine d'une presse.

```
// Rapier - JointData.prismatic(anchor1, anchor2, axis)
const joint = world.createImpulseJoint(
  RAPIER.JointData.prismatic(
    { x: 0, y: 0, z: 0 }, // ancre sur A
    { x: 0, y: 0, z: 0 }, // ancre sur B
    { x: 0, y: 1, z: 0 }  // axe de translation (Y = haut/bas)
  ),
  bodyA, bodyB, true
);
```

Le prismatic **bloque toute rotation** — contrairement au revolute qui bloque toute translation. C'est la contrainte **complémentaire** : l'un tourne, l'autre glisse.

Spherical — la rotule

Spherical Joint

Le joint le plus libre : les deux ancres coïncident, mais les corps peuvent **tourner librement** dans les trois axes. C'est l'épaule, la hanche, la rotule d'une suspension.

```
// Rapier - JointData.spherical(anchor1, anchor2)
const joint = world.createImpulseJoint(
  RAPIER.JointData.spherical(
    { x: 0, y: 0, z: 0 }, // ancre sur A
    { x: 0, y: 0, z: 0 }  // ancre sur B — pas d'axe !
  ),
  bodyA, bodyB, true
);
```

Pas d'axe — c'est la différence clé avec le revolute. Le spherical **ne contraint que la position** des ancres ; les rotations sont libres. On l'utilise pour les **chaînes** (maillons), les **ragdolls** (épaules), les **pendules** (balancier libre).

⚠ Spherical vs Revolute — le piège

Un débutant qui veut faire une porte utilise parfois un spherical joint « parce que ça tourne ». Mais le spherical autorise **toutes** les rotations — la porte se mettrait à pivoter sur tous ses axes comme un drapeau. Le revolute **restreint** à un seul axe. La règle : si le mouvement doit être **guidé** (un seul axe), c'est revolute. S'il doit être **libre** (rotule), c'est spherical.

Partie 3 : Les moteurs — animer un joint

Motor (moteur d'articulation)

Un joint **contraint** — un **moteur pousse**. On ajoute un moteur à un joint revolute ou prismatic pour qu'il **cherche activement** une cible : une **vitesse** (tourner à 5 rad/s) ou une **position** (aller à 30°). Le moteur applique un **couple** (torque) ou une **force** le long de l'axe libre du joint.

```
// Rapier — moteur en VITESSE (tourne à 5 rad/s)
joint.configureMotorVelocity(
    5.0,      // targetVel : vitesse cible (rad/s pour revolute, m/s pour prismatic)
    1.0       // factor : "force" du moteur (0 = mou, ∞ = rigide)
);

// Rapier — moteur en POSITION (va à π/4 rad)
joint.configureMotorPosition(
    Math.PI / 4, // targetPos : angle cible
    1000.0,      // stiffness : raideur (≈ k du ressort de la S7)
    100.0        // damping : amortissement (≈ c du ressort)
);
```

Le moteur en **vitesse** est un **régulateur** : il maintient la vitesse quoi qu'il arrive (une roue qui tourne). Le moteur en **position** est un **servo** : il vise un angle précis (un bras robotique). Les deux paramètres stiffness et damping sont **exactement** le ressort de la Session 7 — k et c — mais résolu comme une **contrainte** au lieu d'une force.

🔗 Le lien avec la Session 7

En Session 7, le ressort était $F = -k \cdot x - c \cdot v$. Le moteur de position d'un joint est **la même équation**, mais intégrée dans le solveur de contraintes au lieu d'être une force externe.

Conséquence : il est **plus stable** (pas d'oscillation explosive si k est grand) et **plus précis** (le solveur itère). C'est pourquoi on ne code plus les ressorts à la main quand on a un moteur physique — on utilise les moteurs de joints.

configureMotor — le mode combiné

Pour un contrôle fin, on peut combiner **position** et **vitesse** cibles :

```
joint.configureMotor(
    Math.PI / 4, // targetPos : angle cible
    0.0,        // targetVel : vitesse cible *en atteignant* la position
    1000.0,     // stiffness
    100.0       // damping
);
```

Le moteur vise l'angle $\pi/4$ **en se déplaçant à** la vitesse 0 (il ralentit en arrivant). C'est le mode **asservi** — l'équivalent d'un PID. On l'utilise pour les bras robotiques, les portes automatiques, les mécanismes précis.

Partie 4 : Les limites — borner la course

Limits (butées)

Un joint revolute ou prismatic peut être **borné** : on définit un angle (ou une translation) **minimum** et **maximum** au-delà desquels le corps ne peut pas aller. Le moteur ajoute une **butée** — l'équivalent du butoir mécanique.

```
// Rapier — limites sur un revolute (bras de catapulte)
joint.setLimits(
    -Math.PI / 2, // min : -90° (bras vers le bas)
    Math.PI / 3   // max : +60° (bras vers le haut)
);
```

Les limites sont **activées par défaut** après setLimits. On peut les désactiver avec joint.setLimits(0, -1) (convention Rapier : min > max = désactivé). En pratique, on les laisse activées — un joint sans limites peut tourner indéfiniment, ce qui est rarement souhaité.

! Limites + moteur = mécanisme complet

Une catapulte, c'est : un **revolute joint** + un **moteur en position** (qui tire le bras vers l'arrière) + des **limites** (qui empêchent le bras de dépasser la verticale). Quand le projectile quitte le bras, le moteur **relâche** (vitesse cible = 0) et le bras s'arrête aux limites. C'est exactement le TP de cette session.

Partie 5 : Les raycasts — interroger le monde

Raycast

Un **raycast** projette un **rayon** (un segment) dans le monde physique et retourne le **premier collider touché**, avec la **distance** et la **normale** du point d'impact. C'est l'outil de **requête** le plus utilisé — il ne **modifie** pas la simulation, il **lit** l'état du monde.

```
// Rapier — raycast vertical depuis y=10 vers le bas
const ray = new RAPIER.Ray(
    { x: 0, y: 10, z: 0 }, // origine
    { x: 0, y: -1, z: 0 }  // direction (NORMALISÉE !)
);
const hit = world.castRay(ray, 20.0, true);
//                               maxDist ↑      ↑ solid (true = premier hit)
if (hit) {
    const point = ray.pointAt(hit.timeOfImpact); // { x, y, z }
    const normal = hit.normal;                   // { x, y, z }
    console.log("Touché à", hit.timeOfImpact, "m");
}
```

Le timeOfImpact est la **distance** (pas le temps — le rayon est normalisé). true = solid : on s'arrête au **premier** collider. false = on retourne **tous** les colliders touchés (via castRayAndGetNormal).

Cas d'usage des raycasts.

- **Détection du sol** : raycast vers le bas depuis le personnage → est-il au sol ? À quelle distance ?
- **Visée FPS** : raycast depuis la caméra dans la direction du regard → quel ennemi est touché ?

- **Capteur de véhicule** : raycast vers l'avant → obstacle à quelle distance ?
- **Placement d'objet** : raycast depuis le curseur → où poser un objet sur le sol ?
- **Suspension de roue** : raycast vers le bas depuis la roue → compression de l'amortisseur.

🔗 Raycast physique vs raycast rendu

En Session 14, on a utilisé `THREE.Raycaster` pour convertir un clic souris en direction de tir. C'est un raycast **rendu** — il intersecte les **meshes** (visuels). Le raycast **physique** intersecte les **colliders** (simulés). Les deux sont utiles : le rendu pour « sur quoi clique le joueur », le physique pour « où est le sol sous le personnage ». En pratique, on utilise presque toujours le **physique** — les colliders sont plus simples et plus stables que les meshes.

Partie 6 : Exploration — Le Joint Zoo (Rapier + Three.js)

🔗 L'exemple du cours

exemples/session15_rapier.html + session15_rapier.js — quatre stations indépendantes, côte à côte, une par type de joint. Chaque station est **isolée** (pas de collision entre corps articulés via `setContactsEnabled(false)`) pour garantir la stabilité. La GUI contrôle les moteurs et limites de chaque station en temps réel.

⚠️ Le piège universel : contacts entre corps articulés

Quand deux corps reliés par un joint se touchent **au niveau de l'axe d'articulation**, le solveur lutte entre la contrainte du joint et la réponse de collision — d'où un mouvement **erratique**. La solution : `joint.setContactsEnabled(false)` désactive les collisions entre les deux bodies du joint. C'est la **règle d'or** des articulations : on ne fait **jamais** collisionner deux corps reliés par un joint au niveau de leur point d'attache.

Station 1 — Le berce de Newton (Revolute, 5 balles)

Cinq pendules côte à côte, sans moteur

```
// --- Support fixe (barre horizontale) ---
const anchorBody = world.createRigidBody(
  RAPIER.RigidBodyDesc.fixed().setTranslation(-8, 5, 0)
);

// --- 5 balles, chacune suspendue par un revolute joint ---
for (let i = 0; i < 5; i++) {
  const x = -8 + (i - 2) * 0.72; // espacées de ~2x le rayon

  const ballBody = world.createRigidBody(
    RAPIER.RigidBodyDesc.dynamic()
      .setTranslation(x, 2.5, 0) // 2.5 = 5 - 2.5 (longueur du fil)
  );
  world.createCollider(
    RAPIER.ColliderDesc.ball(0.35)
      .setDensity(3.0) // acier - lourd
  );
}
```

```

        .setRestitution(0.95), // quasi-élastique
    ballBody
);

// --- JOINT REVOLUTE : balle suspendue au support ---
const joint = world.createImpulseJoint(
    RAPIER.JointData.revolute(
        { x: (i - 2) * 0.72, y: 0, z: 0 }, // ancre sur le support
        { x: 0, y: 2.5, z: 0 },           // ancre sur la balle (haut)
        { x: 0, y: 0, z: 1 }              // axe Z = pendule
    ),
    anchorBody, ballBody, true
);
joint.setContactsEnabled(false); // ← RÈGLE D'OR
}

```

Le berce de Newton est la démonstration la plus **élégante** du revolute : cinq pendules identiques, **sans moteur**, qui se balancent librement. Le bouton « Tirer » téléporte la première balle à un angle donné et la relâche — l'énergie traverse la rangée et la dernière balle s'envole. Les balles ont une **restitution** élevée (0.95) pour que les collisions soient quasi-élastiques. Pas de limites ici : les balles peuvent penduler librement.

🔗 Pourquoi pas de moteur ?

Le berce de Newton est un système **passif** — l'énergie se conserve à travers les collisions. Ajouter un moteur **injecterait** de l'énergie et détruirait la démo. C'est l'inverse du pendule motorisé : ici, le revolute joint sert uniquement à **contraindre** le mouvement (un seul axe), pas à l'entraîner. Le moteur est un **composant optionnel** du joint, pas une obligation.

Station 2 — Le piston (Prismatic + moteur + limites)

Un piston glisse sur une pente et pousse une boîte

```

// --- Une seule pente inclinée à 20° (FIXE) ---
const slopeAngle = 20 * Math.PI / 180;
const slopeQuat = {
    x: 0, y: 0,
    z: Math.sin(slopeAngle / 2),
    w: Math.cos(slopeAngle / 2)
};
const slopeBody = world.createRigidBody(
    RAPIER.RigidBodyDesc.fixed()
    .setTranslation(-2, 2, 0)
    .setRotation(slopeQuat) // rotation Z = inclinaison
);
world.createCollider(
    RAPIER.ColliderDesc.cuboid(3, 0.15, 0.5)
    .setFriction(0.1), // glissant — la boîte redescend
    slopeBody
);

```



```
// --- Le piston (DYNAMIQUE, glisse le long de la pente) ---
const pistonBody = world.createRigidBody(
  RAPIER.RigidBodyDesc.dynamic()
  .setTranslation(pistonX, pistonY, 0)
  .setRotation(slopeQuat) // aligné avec la pente
  .setAngularDamping(5.0)
);

// --- LE JOINT PRISMATIC le long de la pente ---
// L'axe X local du slopeBody = direction de la pente (inclinée à 20°)
const pistonJoint = world.createImpulseJoint(
  RAPIER.JointData.prismatic(
    { x: -2, y: 0.5, z: 0 }, // ancre sur la pente (local)
    { x: 0, y: 0, z: 0 }, // ancre sur le piston (local)
    { x: 1, y: 0, z: 0 } // axe X local = direction de la pente
  ),
  slopeBody, pistonBody, true
);
pistonJoint.setContactsEnabled(false);

// --- LIMITES + MOTEUR (oscille : pousse puis retracts) ---
pistonJoint.setLimits(-1.5, 2.0);
pistonJoint.configureMotorVelocity(2.0, 200);

// --- La boîte à pousser (corps LIBRE, pas de joint) ---
const boxBody = world.createRigidBody(
  RAPIER.RigidBodyDesc.dynamic()
  .setTranslation(boxX, boxY, 0)
  .setRotation(slopeQuat)
);
world.createCollider(
  RAPIER.ColliderDesc.cuboid(0.35, 0.35, 0.35)
  .setFriction(0.1), // glissant comme la pente
  boxBody
);
```

Le piston est le **complément** du berce : au lieu de **tourner**, il **glisse**. Ici, le prismatic est aligné avec la **pente** — l'axe X **local** du slopeBody, incliné à 20°, devient la direction de glissement dans le monde. Le piston et la boîte sont tous deux **rotés** par le même quaternion que la pente, pour rester alignés avec sa surface. Le moteur oscille (pousse 2 s, retracts 2 s) → la boîte monte la pente, puis redescend par gravité. La friction est volontairement **faible** (0.1) pour que la boîte glisse facilement.

Station 3 — La chaîne (Spherical, 4 maillons)

Une chaîne de sphères reliées par des rotules

```
// --- Anneau d'attache (FIXE, en haut) ---
const anchorBody = world.createRigidBody(
  RAPIER.RigidBodyDesc.fixed().setTranslation(4, 6, 0)
);

let prevBody = anchorBody;
let prevAnchor = { x: 0, y: -0.3, z: 0 }; // bas de l'anneau
```

```

for (let i = 0; i < 4; i++) {
    const y = 6 - (i + 1) * 0.6;

    // --- Maillon (DYNAMIQUE) ---
    const linkBody = world.createRigidBody(
        RAPIER.RigidBodyDesc.dynamic()
        .setTranslation(4, y, 0)
    );
    world.createCollider(
        RAPIER.ColliderDesc.ball(0.25), linkBody
    );

    // --- JOINT SPHERICAL entre ce maillon et le précédent ---
    const joint = world.createImpulseJoint(
        RAPIER.JointData.spherical(
            prevAnchor, // ancre sur le précédent
            { x: 0, y: 0.25, z: 0 } // ancre sur ce maillon (haut)
        ),
        prevBody, linkBody, true
    );
    joint.setContactsEnabled(false);

    prevBody = linkBody;
    prevAnchor = { x: 0, y: -0.25, z: 0 }; // bas du maillon
}

```

La chaîne montre le joint **le plus libre** : chaque maillon peut **tourner dans tous les axes** — c’est une rotule. Pas de moteur ici (le spherical n’en a pas en 3D), pas de limites : la chaîne tombe, oscille, s’enroule. Le bouton « Pousser » applique une impulsion latérale à tous les maillons — observez comment l’oscillation se propage le long de la chaîne, maillon par maillon. C’est le principe d’un **ragdoll** : un corps articulé sans contrôle moteur.

Station 4 — Le capteur raycast

Un rayon tournant qui scanne la scène

```

// Le rayon tourne autour de l'axe Y
let rayAngle = 0;

function updateRaycast() {
    rayAngle += raycastSpeed * FIXED_DT;

    // Direction dans le plan XZ, légèrement vers le bas
    const dir = {
        x: Math.cos(rayAngle),
        y: -0.3,
        z: Math.sin(rayAngle)
    };
    // Normaliser
    const len = Math.hypot(dir.x, dir.y, dir.z);
    dir.x /= len; dir.y /= len; dir.z /= len;
}

```

```
// --- LE RAYCAST ---
const ray = new RAPIER.Ray(origin, dir);
const hit = world.castRay(ray, maxDist, true);

if (hit) {
  const point = ray.pointAt(hit.timeOfImpact);
  // Dessiner le rayon jusqu'au point d'impact
  // Placer un marqueur rouge au point d'impact
}
}
```

Le capteur raycast est la seule station **sans joint** — c’est une **requête** du moteur physique, pas une contrainte. Un rayon vert tourne depuis un émetteur élevé et scanne les obstacles autour de lui. À chaque frame, `world.castRay()` retourne le premier collider touché, sa **distance** (`timeOfImpact`) et sa **normale**. Le marqueur rouge se place au point d’impact — c’est exactement le principe d’un **radar**, d’un **capteur de proximité**, ou d’un **lidar**. Aucune collision n’est générée : le raycast **lit** le monde, il ne le **modifie** pas.

💡 Le raycast en pratique

Dans un jeu, le raycast est partout : détection du sol sous le personnage (`castRay` vers le bas), visée FPS (vers l’avant), capteur de véhicule (vers l’avant), placement d’objet (depuis la souris). La station 4 montre le cas le plus **pur** : un rayon qui tourne et marque les impacts. En production, on ne dessine pas le rayon — on lit juste `hit.timeOfImpact` pour savoir « à quelle distance est l’obstacle ».

Partie 7 : Godot + Jolt — Joints & Moteurs

Les joints en Godot — des nœuds de scène

En Godot, un joint est un **nœud** (`Joint3D`) placé dans la scène, avec deux propriétés : `node_a` et `node_b` — les `NodePath` vers les deux bodies à relier. Le joint se positionne **automatiquement** entre les deux.

```
# Godot – HingeJoint3D (le revolute de Godot)
var hinge := HingeJoint3D.new()
hinge.node_a = base_body.get_path()
hinge.node_b = arm_body.get_path()
hinge.position = Vector3(0, 0.5, 0) # point d'attache

# Limites (en degrés, pas en radians !)
hinge.set_flag(HingeJoint3D.FLAG_USE_LIMIT, true)
hinge.set_param(HingeJoint3D.PARAM_LIMIT_LOWER, -80.0) # -80°
hinge.set_param(HingeJoint3D.PARAM_LIMIT_UPPER, 45.0) # +45°

# Moteur
hinge.set_flag(HingeJoint3D.FLAG_ENABLE_MOTOR, true)
hinge.set_param(HingeJoint3D.PARAM_MOTOR_TARGET_VELOCITY, 5.0) # rad/s
add_child(hinge)
```

Godot propose **cinq** types de joints 3D, tous héritant de `Joint3D` :

- **PinJoint3D** : point unique, rotation libre ($\approx$ spherical).

- **HingeJoint3D** : charnière ($\approx$ revolute).
- **SliderJoint3D** : translation le long d'un axe ($\approx$ prismatic).
- **ConeTwistJoint3D** : rotule avec limites de torsion (spherical + limites).
- **Generic6DOFJoint3D** : 6 degrés de liberté, chacun **lockable** ou **limitable**.

💡 Le Generic6DOFJoint3D — le couteau suisse

Le Generic6DOFJoint3D permet de **verrouiller ou limiter chaque axe** individuellement (3 translations + 3 rotations). Un revolute = on lock 5 axes, on en libère 1. Un prismatic = on lock 5, on libère 1 translation. Un spherical = on lock 3 translations, on libère 3 rotations. C'est l'outil **universel** — mais plus complexe à configurer. En pratique, on utilise les joints **spécialisés** quand ils existent, et le 6DOF pour les cas custom.

Moteurs en Godot — velocity vs position

Godot : moteur en vitesse uniquement

Contrairement à Rapier qui propose `configureMotorPosition` **et** `configureMotorVelocity`, Godot's `HingeJoint3D` n'a qu'un moteur en **vitesse** :

```
hinge.set_flag(HingeJoint3D.FLAG_ENABLE_MOTOR, true)
hinge.set_param(HingeJoint3D.PARAM_MOTOR_MAX_TORQUE, 100.0) # couple max
hinge.set_param(HingeJoint3D.PARAM_MOTOR_TARGET_VELOCITY, 5.0) # rad/s
```

Pour un **moteur en position** (servo), on utilise le `Generic6DOFJoint3D` avec `PARAM_ANGULAR_MOTOR_TARGET_VELOCITY` et un **PID maison** dans `_physics_process` : on ajuste la vitesse cible proportionnellement à l'écart angulaire. C'est plus de travail, mais c'est la flexibilité de Godot — on **code** l'asservissement au lieu de le configurer.

Partie 8 : Tableau comparatif des joints

Concept	Rapier (JS)	Godot 4 (GDScript)	Unity 6 (C#, PhysX)
Joint fixed	JointData.fixed(a1, f1, a2, f2)	FixedJoint3D (via 6DOF lock all)	FixedJoint
Joint revolute	JointData.revolute(a1, a2, axis)	HingeJoint3D	HingeJoint
Joint prismatic	JointData.prismatic(a1, a2, axis)	SliderJoint3D	SliderJoint
Joint spherical	JointData.spherical(a1, a2)	PinJoint3D / ConeTwistJoint3D	CharacterJoint
Joint 6DOF	— (composer plusieurs joints)	Generic6DOFJoint3D	ConfigurableJoint
Créer le joint	world.createImpulseJoint(desc, A, B)	add_child(joint); joint.node_a/b = ...	joint.connectedBody = rb
Ancres	Locales au body (Vector3)	joint.position (monde) + node_a/b	joint.anchor / connectedAnchor
Moteur vitesse	joint.configureMotorVelocity(velocity, factor)	PARAM_MOTOR_TARGET_VELOCITY + flag	joint.useMotor motor.targetVelocity
Moteur position	joint.configureMotorPosition(pos, k, c)	PID maison dans _physics_process	joint.springDamper (Configurable)
Limites	joint.setLimits(min, max)	PARAM_LIMIT_LOWER / UPPER + flag	joint.limits.min / max
Supprimer	world.removeImpulseJoint(joint)	joint.queue_free()	Destroy(joint)
Raycast	world.castRay(ray, max, solid)	PhysicsRayQueryParameters3D + space.intersect_ray	Physics.Raycast

Tableau 17. – Mêmes articulations, trois API. Rapier sépare **descripteur** et **création** ; Godot attache le joint comme **nœud de scène** ; Unity expose tout via le composant Joint. Les concepts sont identiques — seules les signatures diffèrent.

Partie 9 : Raycasts en Godot

PhysicsRayQueryParameters3D + intersect_ray

```
func _physics_process(_delta: float) -> void:
    var space := get_world_3d().direct_space_state
    var query := PhysicsRayQueryParameters3D.create(
        Vector3(0, 10, 0),    # from
        Vector3(0, -10, 0),   # to (POINT, pas direction !)
        0b0111                # collision mask
    )
    var hit := space.intersect_ray(query)
```

```

if hit:
    var point := hit.position      # Vector3
    var normal := hit.normal      # Vector3
    var collider := hit.collider  # le CollisionObject3D
    print("Touché à ", point)

```

Godot prend **deux points** (from, to) au lieu d'origine + direction + distance. Le `collision_mask` (bitmask) filtre quels colliders sont testés — c'est l'équivalent des **collision groups** de Rapier. Le résultat est un **dictionnaire** avec position, normal, collider, rid (le RID du collider).

💡 Le collision mask — filtrer les raycasts

En Godot, chaque body a une `collision_layer` et un `collision_mask`. Le raycast a aussi un mask : il ne touche que les bodies dont la layer **correspond** au mask du raycast. C'est essentiel pour ignorer le personnage lui-même (le raycast de détection du sol ne doit pas toucher le joueur). En Rapier, on filtre avec les **collision groups** (`setQueryFilterGroups`).

Synthèse

💬 Ce qu'il faut retenir

- **Un joint** est une **contrainte** qui relie deux rigid bodies et supprime des degrés de liberté. Le solveur la résout à chaque `step()` au même titre que les contacts.
- **Quatre types** : fixed (soudure), revolute (charnière — 1 rotation), prismatic (tiroir — 1 translation), spherical (rotule — 3 rotations). Choisir selon le **mouvement** qu'on veut autoriser.
- **Les moteurs** animent un joint activement : `configureMotorVelocity` (régulateur) ou `configureMotorPosition` (servo). Les paramètres `stiffness` et `damping` sont **le ressort de la Session 7** — mais résolu comme contrainte, donc stable.
- **Les limites** bornent la course d'un joint (`setLimits(min, max)`) — les butées mécaniques. Sans elles, un joint peut tourner indéfiniment.
- **Les raycasts** interrogent le monde sans le modifier : `castRay` retourne le premier collider touché, sa distance et sa normale. Outil de détection du sol, de visée, de capteurs.
- **Rapier vs Godot** : Rapier sépare descripteur (`JointData`) et création (`createImpulseJoint`) ; Godot attache le joint comme nœud (`Joint3D + node_a/b`). Rapier a moteur en position **et** vitesse ; Godot n'a que la vitesse (le servo se code à la main).

💡 La suite

Session 16 : **character controller** et **IK** (cinématique inverse). On pilotera un personnage — capsule cinématique, raycast au sol, glissement le long des murs — puis on résoudra la question : « quels angles faut-il donner aux articulations pour que le pied atteigne ce point ? » Puis le TP véhicules (S17) — où les joints de roue (revolute + moteur) feront avancer une voiture.

Session 16 : Character Controller — FK et IK

Objectifs de la session

- Comprendre pourquoi un personnage ne se simule **pas** comme un rigid body — et ce qu'est un **character controller**.
- Construire le contrôleur canonique : **capsule cinématique**, **raycast au sol**, glissement le long des obstacles.
- Passer de la **cinématique directe** (FK) à la **cinématique inverse** (IK) : donner la position de la main et retrouver les angles des articulations.
- **Implémenter** deux algorithmes d'IK : **CCD** (Cyclic Coordinate Descent) et **FABRIK** (Forward And Backward Reaching Inverse Kinematics), avec leurs compromis.
- Voir l'IK appliquée au personnage : **foot placement**, **look-at**, **hand grab**, **active ragdoll**.
- **Faire le pont** vers le TP Véhicules (S17) : le raycast vehicle est un **character controller à roues**.

💡 Le passage au personnage

Depuis la Session 14, on simule des **objets** : boîtes qui tombent, boulets qui percutent, catapultes qui s'articulent. Le moteur décide de tout — gravité, collisions, impulsions. Un **personnage**, c'est l'inverse : c'est le **code** qui décide (avancer, sauter, tourner), et le moteur qui **vérifie** (y a-t-il un mur ? un sol ? une pente ?). C'est le **character controller** — l'outil le plus important du gameplay. Et une fois qu'on contrôle un corps, il reste à contrôler ses **articulations** : c'est là que la cinématique inverse (IK) entre en jeu.

Partie 1 : Pourquoi pas un rigid body ?

Le piège du rigid body

L'intuition du débutant : « un personnage, c'est un corps physique — je lui mets une capsule de rigid body et ça marche ». Résultat :

- Il **trébuche** — le moindre contact le fait basculer (il a une masse et une inertie).
- Il **glisse** sur les pentes (la friction décide, pas le joueur).
- Il **rebondit** contre les murs (restitution) au lieu de s'arrêter net.
- Il **dérape** au moindre virage — impossible à diriger avec précision.

Un rigid body est **simulé** : le moteur décide. Un personnage doit être **contrôlé** : le code décide, le moteur vérifie.

Character controller

Un **character controller** est un corps **cinématique** piloté par le code. À chaque frame :

1. Le code calcule la **vitesse voulue** (input du joueur).
2. Le contrôleur **déplace** la capsule selon cette vitesse.
3. Le moteur **vérifie** les collisions sur la trajectoire :
 - **Mur** ? On **glisse le long** du mur (on garde la composante parallèle).
 - **Sol** ? On **reste dessus** — et on peut **sauter**.
 - **Pente trop raide** ? On ne **monte pas**.
 - **Marche basse** ? On **monte dessus** automatiquement (step offset).

Le personnage ne reçoit **jamais** d'impulsion, ne **tourne** jamais au contact, ne **tombe** jamais de fatigue. Il fait **exactement** ce que le code lui dit — dans les limites de la géométrie.

Partout dans l'industrie.

- **Quake / Source** : le contrôleur historique — capsule, glissement sur les murs, marches gravies automatiquement.
- **Unity** : composant `CharacterController (Move())`.
- **Godot** : nœud `CharacterBody3D (move_and_slide())`.
- **Unreal** : `ACharacter + CharacterMovementComponent`.
- Aucun n'utilise un rigid body pour le déplacement — tous sont **cinématiques**.

Partie 2 : Anatomie du contrôleur

La capsule — pas la boîte

La forme standard est la **capsule** (deux sphères + un cylindre) :

- **Pas de coins** — une boîte s'accroche sur les seuils de portes et les arêtes de marches ; la capsule **glisse** naturellement.
- **Section ronde** — elle se faufile dans les couloirs étroits sans se coincer.
- **Extrémités arrondies** — elle monte les petites marches sans « sauter » visuellement.

Le rayon et la hauteur de la capsule déterminent la **silhouette de collision** du personnage — souvent plus généreuse que son modèle visuel.

La boucle de contrôle

Godot – `CharacterBody3D` : la boucle canonique
`var velocity := Vector3.ZERO`

```
func _physics_process(delta):
    # 1. Input → vitesse voulue (le CODE décide)
    var input_dir := Input.get_vector("left", "right", "forward", "back")
    velocity.x = input_dir.x * SPEED
    velocity.z = input_dir.y * SPEED

    # 2. Gravité et saut – conditionnés au sol
    if not is_on_floor():
        velocity.y -= gravity * delta
    elif Input.is_action_just_pressed("jump"):
        velocity.y = JUMP_SPEED

    # 3. Déplacement + vérification (le MOTEUR vérifie)
    move_and_slide() # glisse le long des murs, détecte sol/pentes/marches
```

`move_and_slide()` fait tout le travail pénible : elle **déplace** la capsule, **détecte** les collisions, **projette** la vitesse restante le long des surfaces (glissement), et expose `is_on_floor()`, `is_on_wall()`, `get_slide_collision()`.

💡 Grounded — le raycast au sol

La question « suis-je au sol ? » est le **cœur** du contrôleur : elle conditionne le saut, la friction, l'animation. La réponse universelle : un **raycast** vers le bas, un peu plus long que la hauteur du pied :

```
// Rapier — suis-je au sol ?
const ray = new RAPIER.Ray(feetPosition, { x: 0, y: -1, z: 0 });
const hit = world.castRay(ray, 0.15, true); // 15 cm de tolérance
const grounded = hit !== null;
```

C'est exactement le raycast de la Session 15 — le même outil qui, la semaine prochaine, fera **rouler** votre véhicule.

💡 Et les contraintes dans tout ça ?

Un squelette, c'est une **chaîne de contraintes** — l'épaule, le coude, le poignet sont des revolute joints (S15). Jusqu'ici on a piloté ces chaînes **en FK** : on donne les angles, l'extrémité suit. On va maintenant les piloter **en IK** : on donne le point à atteindre, les angles suivent.

Partie 3 : Forward vs Inverse Kinematics

Forward Kinematics (FK)

En FK, on **connaît** les angles des articulations et on **calcule** la position de l'extrémité. C'est le **sens naturel** d'une chaîne articulée : on part de la base, on applique chaque rotation, et l'extrémité **suit**.

$$\vec{p} = f(\theta_1, \theta_2, \theta_3, \dots)$$

```
// FK — chaîne à 2 segments dans le plan
function forwardKinematics(angles, lengths) {
  let x = 0, y = 0, totalAngle = 0;
  const points = [{ x: 0, y: 0 }];
  for (let i = 0; i < angles.length; i++) {
    totalAngle += angles[i];
    x += lengths[i] * Math.cos(totalAngle);
    y += lengths[i] * Math.sin(totalAngle);
    points.push({ x, y });
  }
  return points; // positions de chaque articulation
}
```

La FK est **directe** : une seule solution, calculable en $O(n)$ pour n articulations. C'est ce qu'on fait depuis le début du cours — un bras de catapulte, un pendule, une roue.

Inverse Kinematics (IK)

En IK, on **connaît** la position cible de l'extrémité (la main, le pied) et on **cherche** les angles des articulations qui amènent l'extrémité à cette cible. C'est le **problème inverse** :

$$\theta_1, \theta_2, \theta_3, \dots = f^{-1}(\vec{p}_{\text{cible}})$$

Difficultés :

- **Plusieurs solutions** : un bras à 3 articulations peut atteindre la même cible avec des poses très différentes (coude en avant, coude en arrière, etc.).
- **Aucune solution** : si la cible est **hors de portée** (plus loin que la somme des longueurs), aucune pose ne fonctionne.
- **Pas de solution analytique simple** : pour $n \geq 4$ articulations, on ne peut pas **inverser** f directement — on utilise des **algorithmes itératifs**.

💡 **Quand utilise-t-on l'IK ?**

L'IK est partout où le **but** est plus important que le **moyen** :

- **Foot placement** : le pied doit toucher le sol à un point précis — l'IK calcule la pose de la jambe.
- **Hand grab** : la main doit atteindre un objet — l'IK calcule le bras.
- **Active ragdoll** : maintenir une pose (debout, assis) tout en subissant la physique.
- **Look-at** : la tête doit regarder une cible — l'IK calcule l'orientation du cou.

Partie 4 : CCD — Cyclic Coordinate Descent**CCD — principe**

Le CCD **ajuste les articulations une par une**, de l'extrémité vers la base. À chaque étape, on **tourne** une articulation pour **aligner** l'extrémité avec la cible, **vue depuis cette articulation**.

```
// CCD — une itération complète (de l'extrémité vers la base)
function ccdSolve(joints, target, iterations) {
    const n = joints.length;
    for (let iter = 0; iter < iterations; iter++) {
        // De l'avant-dernière articulation jusqu'à la base
        for (let i = n - 2; i >= 0; i--) {
            // Vecteur : articulation → extrémité
            const toEnd = sub(joints[n - 1], joints[i]);
            // Vecteur : articulation → cible
            const toGoal = sub(target, joints[i]);

            // Angle entre les deux vecteurs
            const angle = angleBetween(toEnd, toGoal);
            // Tourner toutes les articulations *après* i
            for (let j = i + 1; j < n; j++) {
                joints[j] = rotateAround(joints[j], joints[i], angle);
            }
        }
    }
}
```

Avantage : simple à implémenter, converge vite (souvent 5-20 itérations), gère naturellement les **limites d'angle** (on clamp la rotation à chaque étape).

Inconvénient : les mouvements sont **peu naturels** — l'extrémité se « tortille » car chaque articulation est ajustée **indépendamment**. Le résultat **fonctionne** mais ne ressemble pas à un vrai bras.

💡 CCD et les limites d'angle

Le CCD est **très** utilisé en pratique **grâce** à sa gestion des limites. Comme on ajuste chaque articulation **indépendamment**, on peut **clamp** la rotation de chaque joint à son amplitude maximale **immédiatement**. Les algorithmes **globaux** (FABRIK) gèrent les limites **après coup**, ce qui est plus complexe. C'est pourquoi le CCD reste populaire pour les **bras robotiques** (où chaque axe a une butée mécanique).

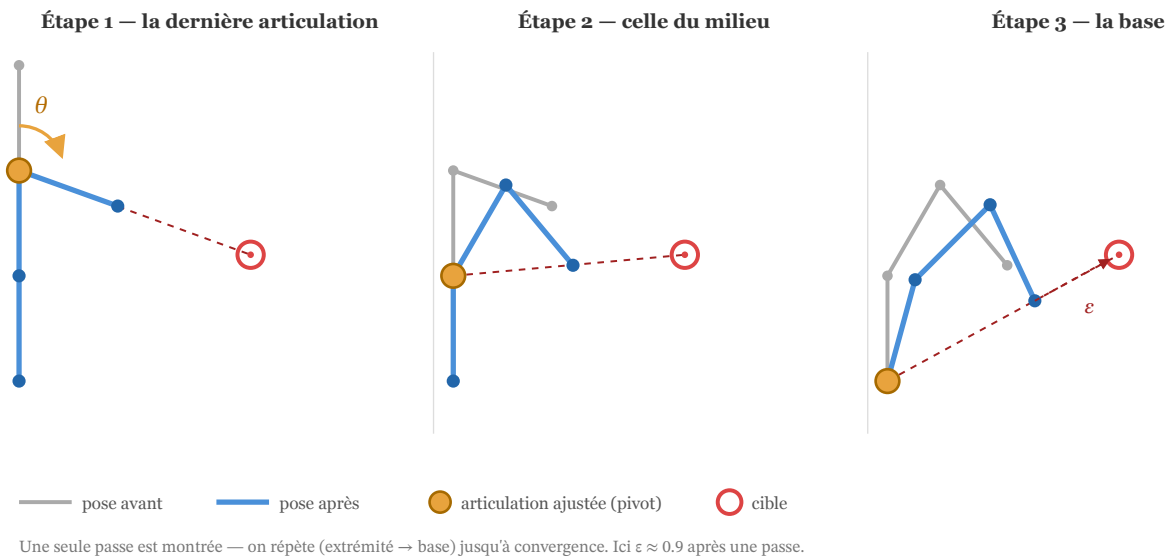


Fig. 35. — CCD : on ajuste les articulations **une par une**, de l'extrémité vers la base. À chaque étape, on fait **pivoter** toute la chaîne en aval pour aligner l'extrémité avec la cible, vue depuis l'articulation ajustée. Une seule passe est montrée — on répète jusqu'à convergence.

Partie 5 : FABRIK — Forward And Backward Reaching IK

FABRIK — principe

Le FABRIK ne manipule **pas** les angles — il manipule directement les **positions** des articulations. Deux passes par itération :

1. **Forward** : on **rapproche** l'extrémité de la cible, puis on **corrige** chaque articulation pour respecter les longueurs de segments — **en remontant** vers la base.
2. **Backward** : on **rapproche** la base de sa position d'origine (sinon le bras « flotte »), puis on **corrige** les longueurs **en descendant** vers l'extrémité.

```
// FABRIK — une itération
function fabrikSolve(joints, target, origin, lengths) {
  const n = joints.length;
  const distToTarget = distance(joints[0], target);
  const totalLength = lengths.reduce((a, b) => a + b, 0);

  // Cible hors de portée : étirer vers la cible
  if (distToTarget > totalLength) {
    for (let i = 0; i < n - 1; i++) {
      const r = distance(joints[i], target);
```

```

        const lambda = lengths[i] / r;
        joints[i + 1] = lerp(joints[i], target, lambda);
    }
    return;
}

// --- Phase FORWARD : cible → base ---
joints[n - 1] = target; // extrémité sur la cible
for (let i = n - 2; i >= 0; i--) {
    const r = distance(joints[i], joints[i + 1]);
    const lambda = lengths[i] / r;
    // Rapprocher joints[i] pour respecter la longueur
    joints[i] = lerp(joints[i + 1], joints[i], lambda);
}

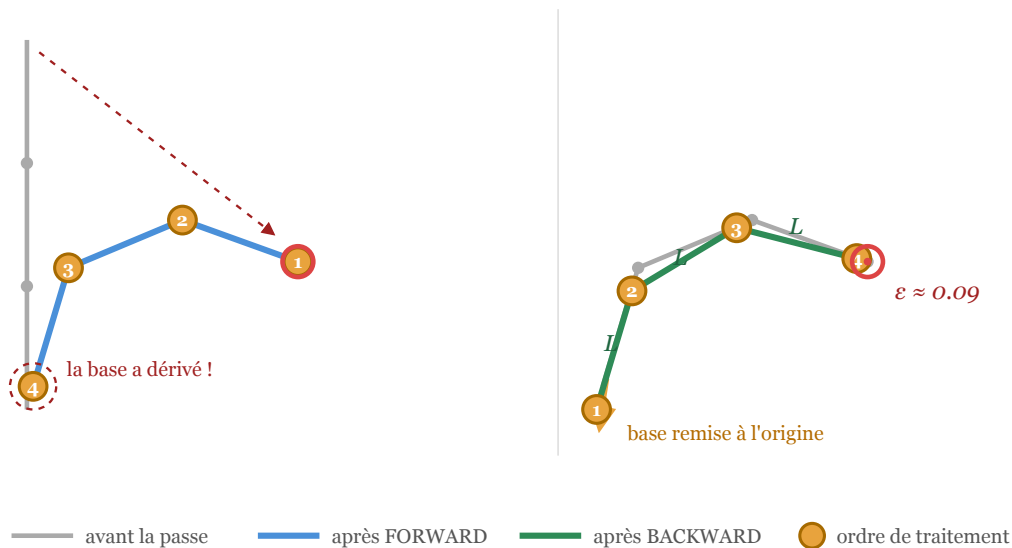
// --- Phase BACKWARD : base → extrémité ---
joints[0] = origin; // base remise à sa position
for (let i = 0; i < n - 1; i++) {
    const r = distance(joints[i], joints[i + 1]);
    const lambda = lengths[i] / r;
    joints[i + 1] = lerp(joints[i], joints[i + 1], lambda);
}
}

```

Avantage : mouvements **naturels** (toutes les articulations bougent **ensemble**), très rapide (souvent 10 itérations suffisent), pas de calcul d'angles.

Inconvénient : ne gère **pas** les limites d'angle directement — il faut un post-traitement pour **clamp** les orientations après chaque itération.

Passe FORWARD — de l'extrémité vers la base Passe BACKWARD — de la base vers l'extrémité



Chaque passe préserve les longueurs L des segments. Une itération complète (forward + backward) suffit ici à $\varepsilon \approx 0.09$.

Fig. 36. — FABRIK : la passe **FORWARD** place l'extrémité sur la cible puis repositionne les articulations en remontant vers la base — qui **dérive**. La passe **BACKWARD** ramène la base à l'origine puis repositionne en descendant vers l'extrémité. Les longueurs L des segments sont **toujours** préservées — une itération complète suffit ici à $\varepsilon \approx 0.09$.

⚠ FABRIK et les limites d'angle — le post-traitement

Le FABRIK pur ignore les limites articulaires : un coude peut se tordre à 360° . Pour respecter des limites, on **projette** chaque orientation de segment dans le **cône autorisé** après chaque itération. C'est plus complexe que le CCD (où on clamp directement), mais le résultat est plus **naturel**. En pratique, beaucoup de jeux utilisent FABRIK **sans** limites pour les **pieds** (où les limites sont peu contraignantes) et CCD **avec** limites pour les **bras** (où le coude a une amplitude limitée).

Partie 6 : Applications de l'IK

Cas d'usage concrets.

- **Foot placement (marche sur terrain)** : on raycast vers le bas depuis la hanche, on obtient le point de contact, et l'IK place le pied **sur** ce point — la jambe s'adapte à la pente, aux marches, aux cailloux.
- **Active ragdoll** : un personnage mort qui **essaie** de rester debout. L'IK maintient une pose cible (debout) pendant que la physique le pousse — le résultat est un corps qui **lutte** contre les chutes, comme un zombie.
- **Hand grab** : le joueur clique sur un objet, l'IK calcule la pose du bras pour que la main atteigne le point de préhension. Sans IK, il faudrait **animer** chaque prise à la main.
- **Look-at (tête/canon)** : la tête ou le canon d'une tourelle **poursuit** une cible. L'IK calcule l'orientation du cou ou de l'axe pour pointer vers la cible.
- **Procédural animation** : remplacer les animations **précalculées** par de l'IK en temps réel — le personnage **réagit** au monde au lieu de rejouer un clip.

Partie 7 : Le pont vers les véhicules

🔗 Le véhicule est un character controller à roues

Le TP de la Session 17 — le **Raycast Vehicle** — applique **exactement la philosophie** d'aujourd'hui : le code décide (throttle, direction), les raycasts vérifient (où est le sol sous chaque roue ?). Au lieu d'une capsule, un **châssis** ; au lieu d'un raycast, **quatre** — un par roue, chacun avec sa suspension :

- La **suspension** est un ressort vertical : on veut que la roue reste à une distance L du châssis, mais on **autorise** la compression.
- Le **grip** empêche la roue de **glisser** latéralement — sans quoi la voiture dérape à chaque virage.
- Le **raycast** est la **détection** : c'est le raycast **grounded** d'aujourd'hui, appliqué 4 fois par frame.

Aperçu du modèle Raycast Vehicle

Au lieu de simuler 4 roues comme des **rigid bodies** (complexe, instable), on traite le châssis comme un **seul** body et les roues comme des **raycasts** :

```
// Aperçu – suspension d'une roue (TP S17)
for (const wheel of wheels) {
  // 1. Raycast depuis la roue vers le bas
  const ray = new RAPIER.Ray(wheelPos, { x: 0, y: -1, z: 0 });
  const hit = world.castRay(ray, suspensionRest, true);

  if (hit) {
    // 2. Compression : 0 (étendu) → 1 (comprimé)
    const compression = 1 - hit.timeOfImpact / suspensionRest;
    // 3. Force de suspension (ressort + amortisseur)
    const springForce = suspensionK * compression;
    const damperForce = suspensionDamping * wheelVelocityY;
    const force = springForce - damperForce;
    // 4. Appliquer la force au châssis au point de contact
    chassisBody.addForceAtPoint({ x: 0, y: force, z: 0 }, wheelPos);
  }
}
```

La **suspension** est un **ressort** (Session 7) **résolu comme une contrainte** (aujourd'hui) **détekté par raycast** (S15). C'est la **synthèse** de trois sessions — c'est pourquoi le TP Véhicules est le **point d'orgue** du cours.

Partie 8 : Démonstration — le bras IK

🔗 L'exemple du cours

examples/session16_ik.html + session16_ik.js — un bras à 4 segments, une cible : **l'anneau suit la souris**, le bras le poursuit. La GUI bascule entre **CCD** et **FABRIK** — comparez le nombre d'itérations et la forme du bras sur la même trajectoire. L'anneau devient **rouge** quand la cible est hors de portée : le bras s'étire alors vers elle au lieu de l'atteindre.

Exercices intégrés

Exercice 1 — CCD sur le bras

Mission

Ouvrez `examples/session16_ik.html`. L'anneau suit la souris ; le bras le poursuit avec l'algorithme sélectionné dans la GUI. Pour cet exercice, on se concentre sur le **CCD**.

1. Dans `session16_ik.js`, trouvez la fonction `ccdSolve()`. Elle est **incomplète** — la rotation des articulations est manquante.
2. Implémentez la **rotation** : pour chaque articulation i (de l'extrémité vers la base), calculez l'angle entre (articulation $\rightarrow$ extrémité) et (articulation $\rightarrow$ cible), puis **tournez** toutes les articulations **après** i de cet angle. Attention — l'extrémité **bouge** après chaque rotation : relisez `joints[n-1]` à chaque étape.
3. Testez avec la cible à différentes positions. Combien d'itérations suffisent pour atteindre la cible à 0.01 près ?

Exercice 2 — FABRIK et la cible hors de portée

Mission

Toujours dans `session16_ik.js`, trouvez `fabrikSolve()`.

1. Le cas **hors de portée** est déjà géré (le bras s'étire vers la cible — l'anneau devient rouge). Vérifiez-le en plaçant la cible **au-delà** de la longueur totale.
2. Implémentez le cas **atteignable** : les deux passes (forward + backward). Attention — la phase **backward** doit **replacer la base** à son origine, sinon le bras **flotte**.
3. Basculez entre CCD et FABRIK dans la GUI : FABRIK converge-t-il plus vite que CCD sur la même cible ? (Comparez le compteur d'itérations.)

💡 Le fichier solution

Les solutions complètes sont dans `examples/session16_ik_solution.js`. **Essayez d'abord** — les exercices sont courts (10-20 lignes chacun), et c'est en les codant qu'on **comprend** les algorithmes. Ne regardez la solution qu'après avoir **vraiment** essayé.

Synthèse

💬 Ce qu'il faut retenir

- **Un personnage n'est pas un rigid body** : il trébuche, glisse, rebondit. On le contrôle avec un **character controller** — corps **cinématique** piloté par le code, collisions **vérifiées** par le moteur.
- **La boucle du contrôleur** : input $\rightarrow$ vitesse voulue $\rightarrow$ déplacement $\rightarrow$ **glissement** le long des obstacles. La question « suis-je au sol ? » se règle avec un **raycast vers le bas** — le même qui fera rouler le véhicule en S17.

- **FK** : angles → positions (direct, une solution). **IK** : cible → angles (inverse, plusieurs solutions, itératif).
- **CCD** : ajuste les articulations **une par une** (de l'extrémité vers la base). Simple, gère les limites, mouvements peu naturels.
- **FABRIK** : ajuste les **positions** en deux passes (forward/backward). Rapide, mouvements naturels, limites en post-traitement.
- **Applications** : foot placement, look-at, hand grab, active ragdoll — l'IK adapte le squelette au monde.

💡 La suite

Session 17 : le **TP Véhicules**. On assemble un châssis, 4 raycasts, une suspension, un moteur et une direction — **drivable**. C'est le **projet final** du cours, et il **réutilise tout** : rigid bodies (S14), joints et raycasts (S15), character controller et IK (aujourd'hui). Chargez vos batteries — à la prochaine session, on roule en numérique.

Session 17 : TP — Véhicules (Raycast Vehicle)

Objectifs du TP

- Comprendre le modèle **Raycast Vehicle** — et pourquoi on ne simule **jamais** les roues comme des rigid bodies.
- **Phase A** : implémenter la suspension **à la main** — raycast + ressort + amortisseur, roue par roue.
- Piloter le véhicule : **accélération, freinage, direction** — au clavier.
- **Phase B** : remplacer la suspension maison par le **vehicle controller** de Rapier (DynamicRayCastVehicleController) — et comparer.
- Comprendre la **tenue de route** : grip, slip angle, et pourquoi une voiture dérape.
- **Tout réutiliser** : rigid bodies (S14), raycasts (S15), la philosophie du character controller (S16), les ressorts (S7).

🔗 Le TP final

C'est la **convergence** du cours. Un véhicule, c'est un **character controller à roues** : le code décide (throttle, direction), les raycasts vérifient (où est le sol sous chaque roue ?). Entre les deux : une **suspension** — le ressort de la Session 7, piloté par le raycast de la Session 15, appliqué quatre fois par frame. Vous avez toutes les briques ; aujourd'hui, on les assemble.

Partie 1 : Le modèle Raycast Vehicle

Pourquoi pas 4 roues rigid bodies ?

L'intuition du débutant : « une roue = un rigid body cylindrique + un joint revolute ». Ça **marche** — et c'est un cauchemar :

- 4 joints à résoudre **à chaque frame** — le solveur lutte entre roue et châssis (oscillations).
- La moindre bosse **décolle** une roue — le joint s'étire, la voiture part en vrille.
- Le moindre virage **retourne** le châssis — il n'y a rien pour le tenir.

Le modèle des jeux **arcade** : les roues ne sont **pas des corps**. Ce sont des **raycasts** — quatre lignes invisibles qui interrogent le sol.

Raycast Vehicle

- Un châssis : rigid body dynamique (boîte).

Quatre roues « virtuelles » : un point d'attache sur le châssis + un **raycast** vers le bas.

- Si le rayon touche le sol : la roue **porte** — on calcule la force de suspension et on l'applique au châssis.
- Si le rayon ne touche rien : la roue est **en l'air** — pas de force, la roue **pend**.

Avantages : stable, performant, et les paramètres (raideur, amortissement, grip) se règlent **directement** — sans se battre contre le solveur.

🔗 Roues visuelles, roues physiques

Les roues que le joueur **voit** (meshes cylindriques) sont **découplées** des raycasts : chaque frame, on positionne le mesh de la roue au point d'impact du rayon (ou à la position « pendante » si en l'air), et on le fait tourner selon la vitesse. Le visuel **suit** la physique — il ne la **subit** pas.

Partie 2 : Phase A — la suspension à la main

La boucle par roue

Pour chaque roue, à chaque frame physique :

1. **Raycast** depuis le point d'attache, vers le bas, de longueur $L_{\text{repos}} + r_{\text{roue}}$.
2. Si on touche le sol à distance d : la **compression** vaut $c = 1 - \frac{d}{L_{\text{repos}}}$ (0 = détendue, 1 = écrasée).
3. **Force de ressort** : $F_k = k \cdot c \cdot L_{\text{repos}}$ — le ressort de la S7.
4. **Amortisseur** : $F_c = c_{\text{damp}} \cdot v_{\text{suspension}}$ — la vitesse **verticale** du point d'attache.
5. On applique $F = F_k - F_c$ **au point d'attache** : `addForceAtPoint`.

```
// Phase A — le cœur du TP (à implémenter)
for (const wheel of wheels) {
  const ray = new RAPIER.Ray(wheel.attachWorld(), { x: 0, y: -1, z: 0 });
  const hit = world.castRay(ray, wheel.restLength + wheel.radius, true);

  if (hit) {
    const compression = 1 - hit.timeOfImpact / wheel.restLength;
    const springF = wheel.k * compression * wheel.restLength;
    const damperF = wheel.damping * wheel.attachVerticalVelocity();
    chassisBody.addForceAtPoint(
      { x: 0, y: springF - damperF, z: 0 },
      wheel.attachWorld(), true
    );
    wheel.placeAt(ray.pointAt(hit.timeOfImpact)); // visuel
  } else {
    wheel.placeDropped(); // la roue pend
  }
}
```

🔗 Valeurs de départ

Châssis : cuboïde (0.9, 0.25, 1.2) (demi-tailles), densité 150 (≈ 324 kg).

Par roue : $k = 12,000$ N/m, $c_{\text{damp}} = 800$ N dot s/m, $L_{\text{repos}} = 0.5\text{m}$, $r_{\text{roue}} = 0.35\text{m}$.

Méthode de réglage : commencez sans amortisseur — la voiture **oscille** (c'est le symptôme).

Ajoutez c_{damp} jusqu'à ce qu'elle se stabilise en ≈ 1 rebond. Trop d'amortisseur = la voiture « plonge » et reste écrasée.

⚠ Le piège : appliquer la force au centre

`addForce` applique au **centre de masse** — la voiture ne peut plus **picher** ni **rouler**, elle glisse comme sur des rails. Il faut `addForceAtPoint` au point d'attaque de **chaque roue** : c'est ce qui fait pencher la voiture en virage et piquer au freinage. C'est le **moment** de la force (S_3) en action.

Partie 3 : Piloter

Accélération et direction

```
// Chaque frame – input au clavier (WASD / flèches)
const forward = new THREE.Vector3(0, 0, -1).applyQuaternion(chassisQuat);

// Accélération : force longitudinale sur les roues MOTRICES (arrière)
if (keyW) chassisBody.addForce(forward.clone().multiplyScalar(4000), true);
if (keyS) chassisBody.addForce(forward.clone().multiplyScalar(-3000), true); // frein/recul

// Direction : on *tourne les raycasts avant* + un couple léger pour assister
if (keyA) frontWheels.forEach(w => w.steer = 0.4); // rad
if (keyD) frontWheels.forEach(w => w.steer = -0.4);
```

La **direction** se fait en **inclinant les raycasts avant** : la force de suspension, appliquée dans l'axe incliné de la roue, pousse le châssis **de côté** — c'est ce qui fait tourner. On ajoute un couple d'assistance ($\vec{\tau}_y$) pour la réponse.

Grip et slip angle

- **Force longitudinale** : accélération / freinage — le long du plan de la roue.
- **Force latérale** : le **grip** — perpendiculaire au plan de la roue. Sans elle, la voiture glisse comme sur la glace.

Slip angle : l'angle entre la direction **du pneu** et la direction **réelle** du mouvement.

- Petit angle : la voiture tourne **proprement**.
- Grand angle : la voiture **dérage** (drift) — le grip « lâche ».

Modèle arcade simple : à chaque roue en contact, appliquer une force latérale proportionnelle à la vitesse latérale **locale** de la roue, plafonnée par μ (le grip max). Un μ élevé = karting, un μ faible = rallye sur neige.

Partie 4 : Phase B — le vehicle controller de Rapier

💡 Pourquoi changer d'outil ?

Vous venez de **sentir** chaque paramètre de la suspension — combien d'itérations pour stabiliser c_{damp} , comment un k trop mou fait toquer le châssis. C'était le but de la Phase A. En production, personne n'écrit cette boucle : le moteur la fournit. Rapier expose le

DynamicRayCastVehicleController — le modèle Bullet (`btRaycastVehicle`), intégré.

Le contrôleur, en une douzaine de lignes

```
// Phase B – remplacer toute la Phase A par ceci
const controller = world.createVehicleController(chassisBody);

// 4 roues : point d'attache, direction, axe, longueur repos, rayon
const wheelDefs = [
  { pos: { x: -0.8, y: 0.0, z: -1.2 }, front: true },
  { pos: { x: 0.8, y: 0.0, z: -1.2 }, front: true },
  { pos: { x: -0.8, y: 0.0, z: 1.2 }, front: false },
  { pos: { x: 0.8, y: 0.0, z: 1.2 }, front: false },
];
for (const w of wheelDefs) {
  const index = controller.addWheel(
    w.pos, { x: 0, y: -1, z: 0 }, // direction du rayon
    { x: -1, y: 0, z: 0 }, // axe de la roue
    0.5, 0.35 // repos, rayon
  );
  controller.setWheelSuspensionStiffness(index, 12 * 50);
  controller.setWheelSuspensionRelaxation(index, 4.5);
  controller.setWheelSuspensionCompression(index, 4.4);
  controller.setWheelFrictionSlip(index, 10);
}

// Chaque frame – pilotage par roue
for (let i = 0; i < 4; i++) {
  controller.setWheelSteering(i, isFront[i] ? steerInput : 0);
  controller.setWheelEngineForce(i, isRear[i] ? throttle * 4000 : 0);
}
controller.updateVehicle(dt); // avant world.step()
```

Lecture par roue : wheelIsInContact(i), wheelSuspensionLength(i), wheelRotation(i), currentVehicleSpeed — pour le visuel et le HUD.

La table de correspondance.

Votre Phase A	Contrôleur Rapier
$k \cdot c \cdot L_{\text{repos}}$	setWheelSuspensionStiffness
$c_{\text{damp}} \cdot v_{\text{susp}}$	setWheelSuspensionRelaxation / Compression
plafond de course	setWheelMaxSuspensionTravel
μ latéral	setWheelFrictionSlip
force moteur maison	setWheelEngineForce
inclinaison des raycasts	setWheelSteering

Mêmes concepts, autres noms — c'est le but : vous savez déjà ce que **chaque paramètre fait**, parce que vous venez de l'écrire.

Partie 5 : Missions

Phase A — la main sur la suspension

1. **M1 — Châssis** : boîte dynamique + sol + 4 **raycasts visibles** (lignes) + meshes de roues. La voiture tombe, les rayons s'affichent. *Check* : on voit les 4 rayons toucher le sol.
2. **M2 — Suspension** : ressort + amortisseur par roue (addForceAtPoint). *Check* : la voiture se pose et se stabilise — elle ne oscille plus, elle ne traverse pas.
3. **M3 — Pilotage** : WASD/flèches — accélération, freinage, direction. *Check* : on roule, on tourne, on ne se retourne pas à 20 km/h.
4. **M4 — Terrain** : une rampe et quelques bosses fixes. *Check* : on saute la rampe et on atterrit sans exploser.

Comptez 60-90 min de réglage — c'est normal, c'est le TP.

Phase B — l'outil du moteur

1. **M5 — Le contrôleur** : même véhicule, suspension remplacée par `DynamicRayCastVehicleController`. *Check* : comportement au moins aussi bon, en 15 lignes.
2. **M6 — Comparaison** : un essai par ligne — stabilité, code, réglage, feeling. *Check* : vous savez dire **pourquoi** le contrôleur est meilleur (ou pas).

Objectif : reconnaître chaque paramètre — si vous ne savez pas ce que fait `suspensionRelaxation`, relisez votre Phase A.

Défi — le circuit

Un petit circuit : rampe, virage serré, chicane. **Contrainte** : ne pas se retourner, ne pas dérailler. **Chrono** affiché au HUD. Le vainqueur du `frictionSlip` le plus bas sans sortir de piste gagne le respect éternel de la classe.

🔗 Le fichier solution

`examples/session17_vehicle_solution.js` — comme toujours : **essayez d'abord**. La Phase A est courte à écrire (≈ 40 lignes) mais longue à régler — c'est en réglant qu'on apprend.

Session 18 : Personnages

Objectifs de la session

- Comparer Dynamic Character Controller (DCC) et Kinematic Character Controller (KCC).
- Implémenter Move and Slide.
- Gérer la détection du sol, le Coyote Time et le Jump Buffering.
- Créer un contrôleur FPS avec Rapier.

1. DCC vs KCC

Dynamic Character Controller (DCC)

Le personnage est un Rigidbody standard.

- **Avantages** : interagit naturellement avec le monde (poussé, tombe, etc.).
- **Inconvénients** : difficile à contrôler, peut glisser, être poussé.

Kinematic Character Controller (KCC)

Le personnage est un Rigidbody kinematic. On contrôle sa position manuellement.

- **Avantages** : contrôle total, game feel précis.
- **Inconvénients** : il faut coder les collisions, la gravité, les pentes.

La norme dans l'industrie : KCC pour les jeux AAA.

2. Move and Slide

Algorithme

1. Calculer le mouvement souhaité (input + gravité + vitesse).
2. Tenter de déplacer le personnage.
3. Si on touche un mur, glisser le long du mur.

Projection et Rejection

Soit $\vec{v}$ la vitesse et $\vec{n}$ la normale du mur.

- Projection (à annuler) : $\vec{v}_{\text{proj}} = (\vec{v} \cdot \vec{n})\vec{n}$
- Rejection (à conserver) : $\vec{v}_{\text{rej}} = \vec{v} - \vec{v}_{\text{proj}}$
- Nouvelle vitesse : $\vec{v}' = \vec{v}_{\text{rej}}$

3. Corner Trap

Quand on glisse le long de deux murs (un coin), le personnage peut se bloquer. Solution : traiter une collision à la fois, trier par profondeur, résoudre la plus profonde, puis re-tenter le mouvement.

4. Détection du Sol

Méthodes

- **Raycast simple** : un rayon depuis le centre. Simple mais peut rater les bords.

- **Shapecast** : on projette la capsule entière. Plus précis.
- **Multi-raycast** : plusieurs rayons sous les pieds.

Coyote Time et Jump Buffering

Coyote Time : après avoir quitté une plateforme, le joueur peut encore sauter pendant 0.1s.

Jump Buffering : si le joueur appuie sur saut juste avant d'atterrir, le saut s'exécute dès l'atterrissage.

5. TP : FPS Controller avec Rapier

1. Créer un Rigidbody kinematic (capsule).
2. Lire les inputs (WASD + souris).
3. Calculer la vitesse souhaitée.
4. Utiliser `world.castRay` pour détecter le sol.
5. Implémenter le Move and Slide.
6. Ajouter le Coyote Time et le Jump Buffering.

Session 19 : Physique, GPU et IA

Objectifs de la session

- Découvrir comment l'IA combine physique pour générer du mouvement.
- Comprendre Reinforcement Learning et Neuroevolution.
- Voir les capteurs physiques comme inputs d'un réseau de neurones.
- Introduire l'optimisation des particules avec le GPU.

1. Physique et IA : Mouvement Organique

Pourquoi mélanger IA et physique ?

Les animations pré-enregistrées sont rigides. La physique + l'IA permet de générer du mouvement en temps réel, adapté au terrain.

2. Reinforcement Learning (RL)

La boucle RL

- **Agent** : le personnage.
- **Environment** : le monde physique (Rapier).
- **State** : ce que l'agent voit (capteurs, positions, vitesses).
- **Action** : forces ou torques appliqués.
- **Reward** : note de l'action.

Exemple : apprendre à marcher. Reward = distance parcourue. Pénalité = tomber.

3. Neuroevolution (Genetic Algorithms)

Principe

1. Créer une population d'agents avec des réseaux de neurones aléatoires.
2. Les faire évoluer dans le monde.
3. Évaluer le fitness (distance, score).
4. Sélectionner les meilleurs.
5. Crossover et mutation.
6. Nouvelle génération.

4. Capteurs Physiques comme Inputs

Inputs du réseau

- **Raycasts** : N rayons vers l'avant, distance au premier obstacle.
- **Vitesse** : vitesse actuelle.
- **Orientation** : angle par rapport à la cible.

Exemple : 5 raycasts + vitesse + orientation = 7 inputs. 8 neurones cachés. 2 outputs (accélération + direction).

5. Optimisation GPU pour les Particules

Pourquoi le GPU ?

- Les particules sont indépendantes, idéales pour le parallélisme.
- Le GPU possède des milliers de cœurs.
- Les données restent sur le GPU (pas de transfert CPU/GPU).

Compute Shaders

- Programme qui s'exécute sur le GPU mais n'est pas un shader de rendu.
- Les particules sont stockées dans un SSBO (Shader Storage Buffer Object).
- Le compute shader met à jour positions, vitesses, lifetime.

Pipeline : CPU envoie les paramètres -> Compute Shader update -> Vertex/Fragment Shaders rendent.

6. TP : Voiture Autonome

Objectif

Créer une voiture qui apprend à naviguer un circuit :

1. Créer un circuit (murs, checkpoints).
2. Créer N voitures avec un Raycast Vehicle.
3. Chaque voiture a un réseau de neurones simple.
4. Inputs : 5 raycasts + vitesse.
5. Outputs : accélération + direction.
6. Fitness : distance parcourue avant de crasher.
7. Implémenter sélection, crossover, mutation.

Session 20 : Résumé - Quiz final

Objectifs de la session

- Réviser les concepts clés du cours.
- Faire le point sur les compétences acquises.
- Passer un quiz final pour valider les apprentissages.

Session présentielle

Révisions et Quiz (20 points).

Concepts clés à retenir

Fondamentaux

- Les trois lois de Newton.
- Les forces de la nature (gravity, normale, friction, drag).
- L'intégration numérique (Euler, Verlet, RK4).
- L'impulsion et les collisions.

Détection de collision

- Broad Phase (Grille, Quadtree, SAP).
- Narrow Phase (SAT, GJK).
- Génération de contacts et résolution.

Moteurs et systèmes avancés

- RigidBodies, Colliders, Joints, Moteurs.
- Contraintes et IK (CCD, FABRIK).
- Particules, GPU, personnages, véhicules, IA.

Quiz final

Format

Le quiz porte sur l'ensemble des sessions. Il mélange questions théoriques et petits problèmes à résoudre.